University of Padua
Ph.D. in Brain, Mind & Computer Science
Curriculum in Computer Science and Innovation for Societal Challenges
Cycle XXXVIII

Reliable and Explainable Process Simulation Models: Discovery, Refinement and Applications

Ph.D. Candidate

Francesco Vinci

Supervisor

Prof. Massimiliano de Leoni

University of Padua

Coordinator

Prof. Anna Spagnolli

University of Padua

Co-supervisor

Dr. Silvia Gabrielli

Fondazione Bruno Kessler

Thesis written with the financial contribution of the European Union - Next Generation EU.

Contents

List of Figures

List of Tables

Abstract

Real-world systems are shaped by processes that govern how activities, decisions, and interactions evolve over time. In organizational settings, identifying and optimizing these processes is the central goal of Business Process Management (BPM), where Business Process Simulation (BPS) enables the experimentation of alternative scenarios (a. k. a. *what-if* analyses) in a safe and risk-free environment.

In recent years, the increasing availability of event data, together with advances in artificial intelligence and data science, has fostered the adoption of process mining techniques in organizational settings. Rather than relying on the subjectivity and assumptions of process analysts, these techniques extract objective, evidence-based insights directly from event data recorded by information systems. Building on this foundation, recent research has increasingly combined process mining with more sophisticated machine learning and deep learning techniques to support the automated discovery of business process simulation models. However, existing approaches present some limitations in terms of explainability, configuration, generalization, and adaptability.

This thesis advances the state-of-the-art in business process simulation. The first set of contributions focuses on the automated discovery and refinement of process simulation models. The proposed methods introduce simulation parameters that dynamically depend on the ongoing process state. In particular, it explores the use of probabilistic decision tree models to capture contextual and conditional dependencies between multiple process perspectives. This design improves simulation reliability while preserving explainability, enabling process experts to inspect and configure the parameters. The thesis then addresses the problem of model refinement in two different settings. The first focuses on refinement in presence of concept drifts, ensuring alignment with continuously evolving processes. The second aims at improving the simulation quality in stable settings (i.e. without concept drifts) by addressing the problem of repairing inaccuracies in process models.

The second part of the thesis investigates the application of simulation models within and beyond academic research. A methodology for optimizing resource allocation in healthcare settings is proposed and evaluated in a new case study concerning the Emergency Department of an Italian hospital. The results identify an optimal resource configuration capable of minimizing patient waiting times while maintaining cost sustainability. A second application regards the simulation of recommendations provided by Process-Aware Recommender Systems. This application can be particularly useful for researchers that aim to evaluate the efficacy of prescriptive techniques when ground truths are missing.

A substantial subset of the proposed techniques has been implemented in ProSiT, a full-fledged process simulation software that provides both a core library and an interactive graphical user interface. The ProSiT library offers

programmatic access to data-driven simulation discovery and simulation functionalities, while the graphical interface allows users to upload event logs, inspect the discovered models, and configure simulation parameters for *what-if* analyses. A structured study with potential end-users confirms the usability of the system and the practical workability for real-world decision-support scenarios.

Introduction

From natural phenomena to organizational workflows, the real world can be seen as a collection of diverse **processes**. A process captures how things evolve over time through sequences of actions, decisions, and interactions. Processes are everywhere, shaping environments that range from administrative procedures to complex customer-facing services. Recently, many organizations have adopted **Business Process Management** (BPM), a systematic discipline focused on identifying, analyzing, and redesign these end-to-end chains of events to improve organizational performance [60].

In recent years, the increasing availability of event data recorded by information systems, together with advances in **artificial intelligence** and **data science**, has paved the way for data-driven approaches. Rather than relying solely on manually designed models and expert assumptions, data-driven techniques exploit event logs to objectively discover and enhance process models. **Process mining** has emerged as a key discipline in this context, bridging process science and data science by extracting knowledge from event data to support *process discovery*, *conformance checking*, and *enhancement* [1].

Within the domain of process mining and, more generally, Business Process Management, **Business Process Simulation** (BPS) is gaining momentum to effectively analyze and optimize these processes. Business Process Simulation (BPS) provides a virtual environment where organizations can replicate their operations to understand current performance and, more importantly, explore potential changes. This makes simulation particularly valuable for ***what-if* analyses**, where alternative scenarios can be safely explored. Through simulation, process analysts can assess the potential impact of changes before their deployment, avoiding the high costs and risks associated with real-world experimentation.

The development of process mining and its data-driven methods have opened

new opportunities for building business process simulation models that are more accurate "digital twins" of the model under scrutiny, thus better suited for advanced analytics and improvements. In particular, the different components corresponding to the various perspectives of the process, such as control-flow, time, and resources, can be discovered directly from the event data by leveraging process mining techniques [164, 128, 42]. Moreover, recent research has increasingly incorporated more powerful machine learning and deep learning techniques into these approaches [41], thus enabling the modeling of complex dependencies and contextual factors that are difficult to capture with traditional methods alone. As a result, data-driven simulation models are becoming more expressive and accurate, strengthening their role as effective decision-support tools for analyzing and optimizing business processes.

Contribution

This thesis proposes several advances in the state of the art of business process simulation. First, it introduces novel techniques for the **discovery** of simulation models directly from event data. Second, it addresses the **refinement** of simulation models under various conditions. Beyond model construction and refinement, this thesis investigates the **application** of data-driven simulation models both within and beyond academic research settings. Finally, this thesis presents the implementation of a full-fledged process simulation **software** that integrates proposed techniques and makes them accessible to practitioners and researchers.

Discovery of Simulation Models. One core contribution of this thesis concerns the proposal of novel techniques for the **automatic discovery of business process simulation models** from event data. In contrast to traditional approaches that rely on static parameters, this thesis proposes simulation models whose parameters dynamically depend on the **ongoing process state**. Consider, e.g. branching probabilities, namely probabilities of executing determined activities when more are enabled, that typically are static but we aim at dynamic. To this end, the proposed techniques leverage **machine learning models** to capture how process behavior evolves as a function of contextual information observed during execution.

Specifically, the diverse perspective parameters (i.e. branching probabilities, activity durations, waiting times, arrival times, and resource assignments) are modeled as predictive functions of the current process state, which may include trace-level attributes and the execution history of the ongoing trace. By incorpo-

rating such state-dependent information, the discovered simulation models are able to represent complex behavioral patterns and conditional dependencies that are not possible to encode through static distributions. The results presented in this thesis show that enriching simulation models with these contextual dependencies leads to substantially more **reliable** simulations.

At the same time, this thesis explicitly addresses the importance of **explainability** of the predictive models used within the simulation models. In fact, explainable models not only increase trust and transparency, but also support the practical adoption of simulation models by allowing process experts to actively configure the discovered models to their specific goals, i.e. performing *what-if* analyses. To this end, this thesis proposes the use of *probabilistic decision tree models* to model state-dependent simulation parameters. These models provide an explicit and human-readable representation of the decision logic governing process behavior.

Beyond accuracy, this thesis emphasizes **generalization** as a key quality criterion for data-driven process simulation. Simulation models should not only reproduce observed behavior, but also generate variable executions under unseen conditions. The probabilistic nature of the proposed models is crucial: unlike deterministic approaches, which tend to overfit and produce rigid simulations, probabilistic models capture uncertainty and variability, leading to more robust and generalizable simulations for decision support and *what-if* analysis.

Refinement of Simulation Models. After model discovery, this thesis addresses the problem of **simulation model refinement**, acknowledging that simulation models must remain aligned with processes that evolve over time. The refinement techniques proposed in this thesis aim to ensure accurate simulated behavior by continuously or selectively adapting and repairing model components.

A first line of contribution focuses on refinement in the presence of **concept drift**. When the behavior recorded in event logs changes due to organizational or external factors, simulation models must be updated accordingly to reflect such evolution. This thesis proposes an online refinement approach in which, given a continuous stream of event data, the simulation model incrementally adapts to new observations. In particular, *incremental process discovery* [169] techniques are leveraged to update the control-flow model, while *online machine learning* [28] methods are employed to adapt the tree-based predictive components. This combination allows the simulation model to evolve with the real process, capturing new patterns while preserving stable and recurring process aspects.

A second refinement perspective targets the improvement of simulation qual-

ity in stable settings, where no explicit concept drift is present. In this context, the thesis addresses the problem of **process model repair** [64] by proposing a novel framework that combines simulation and *explainable artificial intelligence* techniques to pinpoint and repair inaccuracies in simulation models. This approach improves the balance between fitness and precision with respect to observed data, thus leading to more accurate simulations.

Applications Within and Beyond Academic Research. To demonstrate the practical relevance of the proposed methods, this thesis investigates the **application of simulation models** in two diverse settings with different scopes.

Some of our proposed methods were used in the **healthcare** domain to define a methodology to **optimize organizational processes**. This methodology includes healthcare staff in the discovery and validation of simulation models to ensure they are trustworthy and align with their operations. The methodology was applied to a new real-world case study concerning the Emergency Department of an Italian hospital, where it was used to identify resource configurations that minimize patient waiting times while maintaining cost sustainability.

The second application focuses on **evaluating recommendations** provided by *Process-Aware Recommender Systems*. The thesis proposes a framework where ongoing traces are aligned with a simulation model to reflect their current state, and recommendations provided by Process-Aware Recommender Systems are then injected to simulate potential outcomes. This application is of particular relevance in academia, where researchers seek to evaluate the efficacy of their prescriptive techniques but often lack a "ground truth" to determine what would have happened if a specific recommendation had been followed.

Event-log preparation. The employment of the contributions discussed above relies on the availability of high-quality, clean event logs containing complete information. However, in practice, event logs collected by information systems can often be incomplete or noisy, especially when dealing with complex contexts such as healthcare.

A particularly critical challenge is the presence of missing or inaccurate temporal information. Since temporal aspects are fundamental for discovering the time perspective parameters in simulation models, incomplete timestamps can compromise the application of simulation discovery techniques. To address this, this thesis proposes a novel framework for accurately **estimating missing timestamps**. This framework reconstructs the missing temporal data, providing event logs with complete information that can be effectively used for simulation modeling and analysis.

Software. Relevant data-driven techniques developed in this thesis have been implemented in a new **full-fledged process simulation software** named *ProSiT* (*Process Simulation Tool*).

The tool is composed of a core Python library, *ProSiT Library*, which provides the underlying computational engine to discover simulation parameters and perform simulation runs. Complementing the library, *ProSiT System* serves as a graphical interface that allows users to directly interact with the simulation models. Through this interface, users can upload event logs, visualize the discovered models, and interactively modify parameters to define and compare different *what-if* scenarios.

To validate the practical usability of *ProSiT*, a structured user assessment was conducted with participants of varying expertise in Business Process Management. The study demonstrated that participants could accurately complete simulation tasks, such as modifying tree-based parameters for *what-if* analysis, confirming that our techniques produce simulation models that are not only reliable but also effectively manageable to process analysts in real-world decision-support scenarios.

Thesis Outline

The structure of this thesis is illustrated in Figure 1, which highlights the chapters of this manuscript that discuss the thesis contributions. The figure ignores Chapter 1, which discusses the preliminary concepts, needed to comprehend the contributions thereafter. The thesis is divided into three parts:

Part I: Process Simulation Discovery and Refinement. It focuses on the core techniques for discovering and refining simulation models. This part begins with Chapter 2, which addresses the critical stage of event **log preparation**, specifically focusing on the estimation of missing timestamps. Chapter 3 and Chapter 4 introduce the techniques for **discovering** process-state-dependent and explainable simulation models from these logs. To ensure that these models remain relevant over time, Chapter 5 presents the method for **adapting** models to concept drift, while Chapter 6 details a framework for **repairing** inaccuracies in process models.

Part II: Process Simulation Applications. This part explores the utility of the proposed techniques in diverse contexts. Chapter 7 reports on the application of simulation for **healthcare optimization**, using a case study of an Emergency Department to find optimal resource allocations. Chapter 8 investigate the use of simulation models to **evaluate the potential impact**

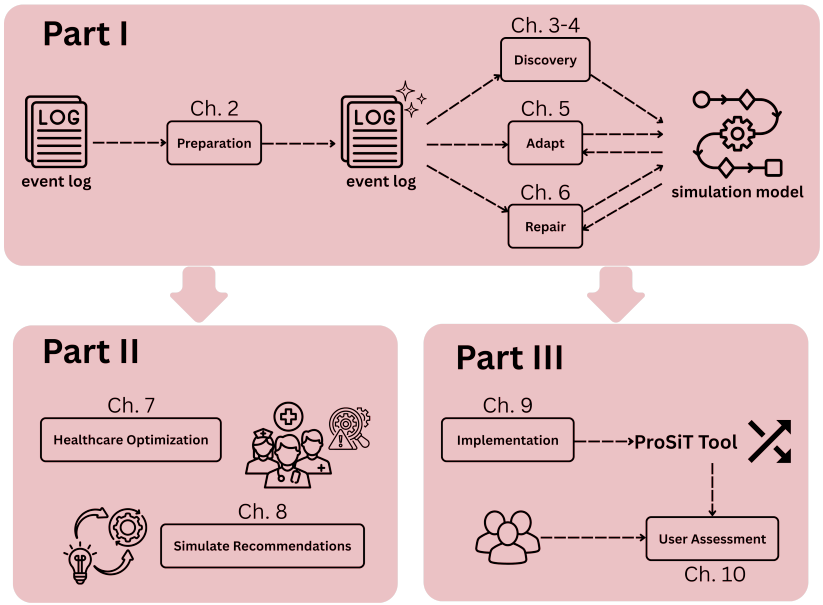


Figure 1: Conceptual map of the thesis structure, depicting the progression from event log preparation and the discovery and refinement of simulation models (Part I) to their practical application in optimization and recommendation tasks (Part II), concluding with the implementation and user asssessment of the ProSiT software (Part III).

of recommendations generated by process-aware recommender systems.

Part III: Software and User Interactions. It presents the implementation and evaluation of the ***ProSiT* software**. Chapter 9 describes the implementation and functionalities of the tool. Chapter 10 finally reports on the conducted **user assessment** for evaluating the usability of ProSiT.

Publications

The research contributions presented in this thesis are supported by the following peer-reviewed international conference and journal publications:[1]

[C.1] Massimiliano de Leoni, **Francesco Vinci**, Sander Leemans, Felix Mannhardt (2023). **Investigating the Influence of Data-Aware Process States on Activity Probabilities in Simulation Models: Does Accuracy Improve?**. In *Proceedings of International Conference on Business Process Management*, pp. 129–145. Springer Nature Switzerland. September 11–15, 2023, Utrecht,

[1]Publications are categorized as **C** (international conference proceedings) and **J** (international journal articles). Each publication is discussed within the corresponding chapters of this thesis.

The Netherlands. DOI: 10.1007/978-3-031-41620-0_8. The contribution of this paper is discussed in Chapter 3.

[C.2] **Francesco Vinci**, Massimiliano de Leoni (2024). **Repairing Process Models Through Simulation and Explainable AI**. In *Proceedings of International Conference on Business Process Management*, pp. 129–145. Springer Nature Switzerland. September 1–6, 2024, Krakow, Poland. DOI: 10.1007/978-3-031-70396-6_8. The contribution of this paper is discussed in Chapter 6.

[C.3] Alessandro Padella, Felix Mannhardt, **Francesco Vinci**, Massimiliano de Leoni, Irene Vanderfeesten (2024). **Experience-Based Resource Allocation for Remaining Time Optimization**. In *Proceedings of International Conference on Business Process Management*, pp. 345–362. Springer Nature Switzerland. September 1–6, 2024, Krakow, Poland. DOI: 10.1007/978-3-031-70396-6_20. The contribution of this paper is discussed in Chapter 8.

[J.1] **Francesco Vinci**, Massimiliano de Leoni (2025). **Balancing Fitness and Precision in Process Model Repair: Framework Formalization, Assessment, and Benefits for Simulation**. In *Process Science*, Vol. 2, No. 3, Best Process Science Conference Papers 2024 Collection. Springer Nature. DOI: 10.1007/s44311-025-00007-7. The contribution of this paper is discussed in Chapter 6.

[C.4] **Francesco Vinci**, Gyunam Park, Wil van der Aalst, Massimiliano de Leoni (2025). **Online Discovery of Simulation Models for Evolving Business Processes**. In *Proceedings of International Conference on Business Process Management*, pp. 451–468. Springer Nature Switzerland. August 31–September 5, 2025, Seville, Spain. DOI: 10.1007/978-3-032-02867-9_27. The contribution of this paper is discussed in Chapter 5.

[C.5] Ngoc-Diem Le, Alessandro Padella, **Francesco Vinci**, Massimiliano de Leoni (2025). **Leveraging Counterfactuals for Prescriptive Process Analytics**. In *Proceedings of Business Process Management: Responsible BPM Forum, Process Technology Forum, Educators Forum. BPM 2025*. Springer Nature Switzerland. August 31–September 5, 2025, Seville, Spain. DOI: 10.1007/978-3-032-02936-2_15. The contribution of this paper is discussed in Chapter 8.

[C.6] **Francesco Vinci**, Gyunam Park, Wil van der Aalst, Massimiliano de Leoni (2026). **Reliable and Configurable Process Simulations via Probabilistic White-Box Models**. In *Proceedings of International Conference on Service*

Oriented Computing, pp. 337–352. Springer Nature Singapore. December 1-4, 2025, Shenzhen, China. DOI: 10.1007/978-981-95-5015-9_24. The contribution of this paper is discussed in Chapter 4.

[C.7] **Francesco Vinci**, Gyunam Park, Wil van der Aalst, Massimiliano de Leoni (2026). **ProSiT: A Tool for Interactive and Transparent Process Simulations**. In *Proceedings of International Conference on Service Oriented Computing: Demonstrations and Resources*. December 1-4, 2025, Shenzhen, China. **Accepted**. The contribution of this paper is discussed in Chapter 9.

[J.2] **Francesco Vinci**, Davide Aloini, Elisabetta Benevento, Alessandro Stefanini, Francesca Zen, Massimiliano de Leoni (2026). **Improving Organizational Processes in Healthcare through Simulation-Driven Resource Allocation: Methodology and Real-World Case Study**. In *Artificial Intelligence in Medicine*. Vol. 176. pp. 103391. Elsevier. DOI: 10.1016/j.artmed.2026.103391. The contribution of this paper is discussed in Chapter 2 (for the timestamp estimation) and Chapter 7 (for healthcare optimization).

1 Preliminaries

This chapter establishes the theoretical and methodological foundation of this thesis by introducing the key concepts and mathematical formalisms that support the approaches and analyses developed throughout the work. These preliminaries provide a rigorous framework for the representation, analysis, and manipulation of business processes and their associated data. A solid understanding of these notions is essential to comprehend the more advanced techniques presented in the subsequent chapters.

After an introduction of basic mathematical concepts in Section 1.1, the chapter starts with an overview of **Business Process Management** (BPM) (cf. Section 1.2), describing its lifecycle and the fundamental principles that guide the modeling, analysis, and optimization of business processes. It then introduces the basics of **Process Mining** (cf. Section 1.3), focusing on event data and process models that enable data-driven insights into operational behavior. Next, Section 1.4 outlines key concepts from **Machine Learning**, including predictive modeling, online learning, and **explainable artificial intelligence** (XAI), which are leveraged in different parts of this thesis. Section 1.5 discusses the inclusion of these in Process-aware Recommender (PAR) systems, introducing predictive and prescriptive process analytics. Then, Section 1.6 presents the foundations of **Business Process Simulation**, illustrating how complete process models can be employed to reproduce and analyze system dynamics under varying conditions. A final section describes the datasets used for the assessment of the techniques introduced in this thesis (cf. Section 1.7).

1.1 Basic Mathematical Concepts and Notations

This section introduces basic mathematical concepts and notations used throughout the thesis to formally define the techniques and methods presented in the following chapters.

A **set** X is a collection of distinct elements. We denote the *cardinality* of a set X, i.e. the number of its elements, by $|X|$. The power set of X, denoted with 2^X, is the set of all subsets of X, i.e. $2^X = \{X' \mid X' \subseteq X\}$. Given n sets $X_1, \ldots, X_n$ we define the n-ary *Cartesian product* of these n sets as $X_1 \times \cdots \times X_n = \{(x_1, \ldots, x_n) \mid x_i \in X_i \; \forall i = 1, \ldots, n\}$. An element of a Cartesian product is called a **tuple**. In the reminder, we refer to a tuple of two and three elements with *pair* and *triple*, respectively.

In this thesis, we additionally use the concept of **multiset** that extends the sets allowing multiple occurrences of the same element. A multiset is denoted using square brackets, e.g., $[x, x, y]$. The set of all multisets over a set X is denoted by $\mathbb{B}(X)$. The *multiset union* operator is denoted by $\uplus$, where the multiplicities of common elements are added.

A **sequence** is an ordered collection of elements from a set. Given a set X, we denote with $\sigma = \langle x_1, \ldots, x_n \rangle \in X^*$ a finite sequence of n elements $x_i \in X \; \forall i = 1, \ldots, n$, with X^* indicating the set of all possible sequences over X. We indicate with $\langle\rangle$ the empty sequence.

A **function** f from a set X (the *domain*) to a set Y (the *codomain*) is a relation $f \subseteq X \times Y$ such that for every element $x \in X$ there exists exactly one element $y \in Y$ with $(x, y) \in f$. This is denoted by $f : X \to Y$. The set of all functions from X to Y is denoted by $(X \to Y)$. We use the notation $x \mapsto y$ to explicitly indicate that a function maps an element $x \in X$ to an element $y \in Y$, when the mapping is clear from the context.

1.2 Business Process Management

Business Process Management (BPM) is the art and science of organizing and executing work within an organization to ensure efficiency, consistency, and value creation [60]. The lion's share of attention of BPM is the end-to-end chains of events, activities, and decisions, called **processes**, that ultimately deliver outcomes to customers and stakeholders. Its goals can vary, ranging from reducing costs, execution time, and errors, to fostering innovation and gaining competitive advantage. BPM involves a lifecycle of activities, including identifying, analyzing, redesigning, implementing, and monitoring processes, with the aim of achieving continuous improvement [60]. A business process can be under-

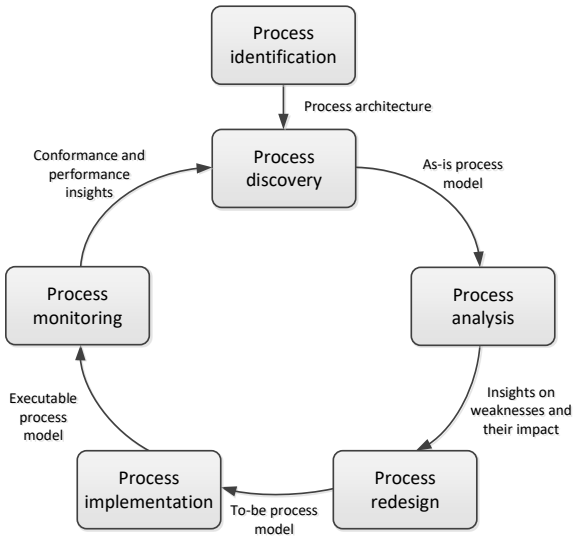


Figure 1.1: The BPM lifecycle. Source: [60].

stood as a collection of activities that, taken together, achieve a specific organizational objective, typically by delivering value to a customer or stakeholder. BPM provides organizations with the methodologies, techniques, and tools needed to align processes with strategic goals, enhance efficiency, and improve overall performance.

At the core of BPM lies the **BPM lifecycle**, which structures the set of steps required to manage and optimize processes in a systematic way (see Figure 1.1). The lifecycle encompasses several phases, each of which addresses a distinct aspect of process management, while also creating outputs that serve as inputs for subsequent stages. The main phases can be summarized as follows:

1. **Process identification**: In this phase, processes are recognized and prioritized with respect to organizational strategy. The outcome is a process architecture that provides an overview of the organization's key processes and their interrelations.

2. **Process discovery**: Selected processes are captured and documented, typically through process modeling, to obtain an *as-is* representation. This results in an *as-is* process model, which provides transparency on how work is currently performed.

3. **Process analysis**: The *as-is* models are analyzed with the aim of identifying weaknesses, inefficiencies, or bottlenecks. Analytical techniques may range

from qualitative assessments (e.g., stakeholder interviews) to quantitative methods (e.g., process mining, simulation). The output consists of insights into process issues and their impacts.

4. **Process redesign**: Based on the analysis, alternative process designs are proposed. The goal is to define *to-be* process models that overcome the shortcomings of the *as-is* processes while aligning with business goals. These can be supported by *process simulation* to assess the expected performance of redesign alternatives before implementation.

5. **Process implementation**: The redesigned processes are translated into practice. This may involve organizational change, training, or the deployment of process-aware information systems. Where applicable, executable process models are used to configure workflow engines or other process automation technologies.

6. **Process monitoring**: Executed processes are continuously monitored to ensure compliance with the intended design and to measure performance indicators. Monitoring provides conformance and performance insights, and may further support *predictive and prescriptive analytics* to recommend or trigger corrective actions during process execution, which in turn feed back into the identification and discovery phases, enabling continuous improvement.

This lifecycle highlights the iterative nature of BPM: processes are not managed in isolation, but rather through a cycle of discovery, evaluation, change, and control. The integration of these phases allows organizations to move from *as-is* to *to-be* models in a structured manner, thereby creating an evidence-based foundation for process improvement and innovation.

1.3 Process Modeling and Mining

Process Mining builds on the BPM introduction by using event data from information systems to reveal how processes actually run. The novelty of Process Mining compared with the traditional process intelligence lays into leveraging on the objectivity of the the event data in place of human subjectivity (e.g. retrieved via questionnaires, interviews, etc.) [1]. This explains why Process Mining is considered as **the missing link between Process and Data science** (see Figure 1.2). The discipline connects BPM domains such as workflow management, automation, and improvement with data-centric areas like statistics, machine learning, and data mining, supported by adjacent fields including stochastics,

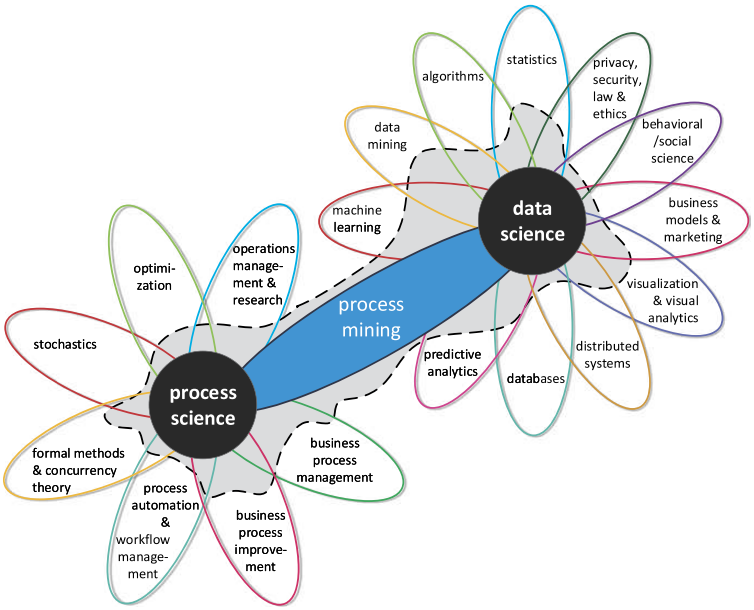


Figure 1.2: Process mining as the bridge between data science and process science. Source: [1].

optimization, formal methods, visualization, distributed systems, and privacy and ethics, which together enable rigorous, compliant, and explainable analyses.

Today's information systems record vast amounts of data about the activities being executed. Workflow management systems, BPM systems, ERP systems, PDM systems, CRM systems, middleware, and hospital information systems all generate detailed events of operational processes. These records, referred to as event logs (see Figure 1.3), provide the basis for process mining. Event logs enable three primary types of process mining:

Discovery Discovery techniques automatically construct a process model directly from an event log, without relying on any prior information. For example, the α-algorithm [5] takes an event log and generates a Petri net that captures the observed behavior without using additional knowledge. Discovery can also reveal additional perspectives, such as organizational structures or social networks, showing how people collaborate.

Conformance Conformance checking compares an existing process model with an event log to assess whether recorded executions align with the modeled behavior. This helps identify deviations, such as violations of business rules. Conformance techniques can locate, explain, and quantify deviations, providing insight into compliance and potential fraud. An example is the conformance checking algorithm described in [163]. Given a process

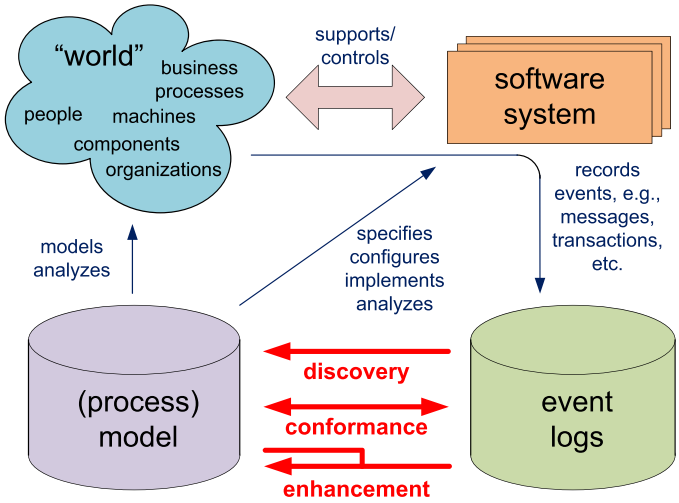


Figure 1.3: Positioning of the three main types of process mining: discovery, conformance, and enhancement. Source: [1].

model and a corresponding event log, this algorithm can quantify and diagnose deviations.

Enhancement Enhancement techniques enrich or improve an existing process model using information from event logs. Process enhancement is divided into process extension and process improvement [110]. Process extension aims to incorporate additional perspectives, such as data, decisions, resources, and time, into the model. Process improvement focuses on creating an "improved" version of the model that better reflects reality or achieves superior performance. This can involve repairing the model to better reflect reality, and the strategic modification of the process to ensure it yields better Key Performance Indicators (KPIs). By identifying and disallowing behaviors that lead to unsatisfactory outcomes, such as high costs or bottlenecks, and promoting paths that correlate with positive results, process improvement ensures the model helps the business run more efficiently.

In summary, discovery, conformance, and enhancement constitute the three foundational pillars of process mining, each addressing a different dimension on how processes are captured, validated, and improved through event data. Throughout this thesis, all these types of process mining are considered, ensuring that analyses remain data-driven, rigorous, and aligned with the methodological foundations of the discipline.

Table 1.1: Event log example based on an emergency department process. Each row represents an event with its attributes. The sequence of events sharing the same *Case Id* is a trace, i.e. the sequence of events of a single patient during the process.

Case Id	Activity	Timestamp	Lifecycle	Age	Access Type	Urgency Code	Pathology
...	...	...	...	...	...	...	...
002082	Access	Mar 15, 00:17	complete	49	Autonomous	Blue	Skin Rash
002082	Triage	Mar 15, 00:17	complete	49	Autonomous	Blue	Skin Rash
002082	Visit	Mar 15, 01:26	complete	49	Autonomous	Blue	Skin Rash
002082	Discharge	Mar 15, 01:27	complete	49	Autonomous	Blue	Skin Rash
002082	Exit	Mar 15, 01:28	complete	49	Autonomous	Blue	Skin Rash
002083	Access	Mar 15, 00:48	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	Triage	Mar 15, 00:48	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	Visit	Mar 15, 01:26	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	Blood Sampling	Mar 15, 01:30	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	Lab Service	Mar 15, 01:30	start	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	CT Request	Mar 15, 01:30	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	CT Accept	Mar 15, 01:32	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	CT Execution	Mar 15, 01:55	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	CT Reporting	Mar 15, 02:06	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	Lab Service	Mar 15, 02:20	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	Medium Short Stay Unit	Mar 15, 02:22	start	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	Medium Short Stay Unit	Mar 15, 11:10	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	Discharge	Mar 15, 11:10	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
002083	Exit	Mar 15, 11:48	complete	72	Ambulance (no doctor)	Yellow	Mild TBI
...	...	...	...	...	...	...	...

1.3.1 Event Logs

Process' event data are represented as **event logs**. Table 1.1 shows a tabular representation of an event log: each row represents an *event*, which captures a specific execution of an activity within a *case* (i.e., an instance of the process). Columns are the attributes associated with the events. Mandatory attributes are the *Case Identifier*, which allows grouping events belonging to the same cases, the *Timestamp* when the event occurred, which allows ordering events within cases, and the *Activity* that was executed. Other typical attributes are the *Resource* that executed the activity and *Transactional information* indicating the activity's state. Using transactional information one can determine the duration of the activities. As an example, the execution of activity *Lab Service* for case 002083 started at 1:30 on 15 March, and completed at 2:20, and hence, took 50 minutes.

Event logs can be formally defined in terms of **events**, **traces**, and their collections. In particular, an event log can be seen as a set of traces, where each trace records a single process execution as a temporally ordered sequence of events [1]. We first formalize the notion of an event.

Definition 1 (Event). *Let $\mathcal{I}$, $\mathcal{A}$, and $\mathcal{R}$ denote the sets of all possible case identifiers, activities, and resources, respectively. An event e is a record with the following projections:*

- $case(e) \in \mathcal{I}$, *the case identifier to which e belongs,*
- $act(e) \in \mathcal{A}$, *the activity executed in e,*

- $res(e) \in \mathcal{R}$, *the resource involved in* e,
- $time(e) \in \mathbb{N}$, *the timestamp of* e, *inducing the order of events within a trace,*
- $life(e) \in \{start, complete\}$, *indicating whether* e *marks the start or completion of the activity,*

Building on the definition of events, we can now define traces and event logs.

Definition 2 (Traces & Event Logs)**.** *Let* $\mathcal{E}$ *denote the universe of all possible events, with* $\mathcal{I}$ *the set of all possible case identifiers. A trace* $\sigma = \langle e_1, e_2, \ldots, e_n \rangle \in \mathcal{E}^*$ *is a finite sequence of events that belong to the same case, i.e.,* $case(e_1) = \cdots = case(e_n) \in \mathcal{I}$*, and are ordered by their timestamps, i.e.,* $time(e_i) \leq time(e_{i+1})\ \forall i = 1, \ldots, n-1$*. An event log* $\mathcal{L}$ *is a finite set of traces, where each trace corresponds to a unique case. Let* $\mathcal{V}$ *and* $\mathcal{U}$ *denote the sets of attribute names and attribute values. The function* $attr_{\mathcal{L}} : \mathcal{L} \to (\mathcal{V} \to \mathcal{U})$ *assigns trace attributes to a trace in the event log* $\mathcal{L}$*.*

Note that each event e has exactly one timestamp, $time(e)$. However, in the remainder, we assume a lifecycle semantics in which each activity execution is represented by a pair of events: a start and a corresponding completion event. Given an event e in a trace σ, we use $t_{start}(e)$ and $t_{complete}(e)$ to denote respectively, the timestamps of the start event and the completion event of the same activity execution. If e is a start event, i.e. $life(e) = start$, then $t_{start}(e) = time(e)$ and $t_{complete}(e)$ refers to the timestamp of the first subsequent event in the same trace with the same activity and resource and lifecycle value $complete$. Conversely, if e is a completion event, i.e. $life(e) = complete$, then $t_{complete}(e) = time(e)$ and $t_{start}(e)$ refers to the timestamp of the corresponding preceding start event.

We also use the notation $e \in \mathcal{L}$ to indicate that there exists a trace $\sigma \in \mathcal{L}$ such that $e \in \sigma$.

Note that in this thesis, we consider only trace attributes, i.e., attributes associated with case identifiers and shared by all events of the corresponding trace, and we do not consider event-level attributes. For example, in the event log shown in Table 1.1, attributes such as age, access type, urgency code, and pathology take the same value for all events with the same *Case Id*, as they describe properties of the patient visit as a whole rather than of individual activity executions.

1.3.2 Process Models

Aside from event logs, another typical artifact of process mining techniques is a **process model** (see Figure 1.3). A process model is a description of how a process ought to be executed. Compared with a textual description in natural language, a

Figure 1.4: Example process model in BPMN notation, which represents the same activities as presented in the event log in Table 1.1.

process model encodes (or should encode) the set of allowed executions in a more formal way. Using a formal process modeling notation has several advantages, i.e., it allows us to assess various quality properties of the model, e.g., absence of deadlocks. It also allows software tools to automatically reason about the modeled process behavior.

Two widely used formalisms are **Business Process Model and Notation (BPMN)** [60] and **Petri nets** [1]. BPMN is particularly common in practice because of its intuitive and visual representation, whereas Petri nets provide a mathematically precise foundation that enables formal analysis. In this thesis, we rely on Petri nets as the underlying formalism, while occasionally illustrating concepts through BPMN diagrams for their readability.

Figure 1.4 presents a typical structure of a BPMN model, which represents the same behavior as described in the event log in Table 1.1. The model describes a few fundamental constructs, often used when modeling a process. The green circle represents the starting point of the process, whereas the orange circle represents the end point. Each rounded rectangle represents an activity that belongs to the process, e.g., the *visit* activity. The arcs represent the general behavioral flow of the model, e.g., the first activity is *access*.

The diamond-shaped operators represent behavioral control-flow constructs. The diamonds containing a $\times$-symbol represent an *exclusive choice*, i.e., either one of the outgoing/incoming arcs is followed. The diamonds containing a $+$-symbol represent *concurrency*: all outgoing/incoming arc (and branches) are followed, and, executed completely independently/concurrently. Control-flow constructs can act as either *splits* (one incoming and multiple outgoing arcs, dividing the flow into alternative or concurrent branches) or *joins* (multiple incoming and one outgoing arc, merging branches by selection or synchronization). For a more detailed overview of BPMN constructs, see [60].

Although BPMN models are widespread in the industry for their relative ease of understanding, Petri nets offer a simpler mathematical representation and are supported by a rich body of formal theory. Petri nets are then often preferred for formal analysis, providing a rigorous mathematical model for describing and analyzing process behavior.

Definition 3 (Petri Net)**.** *A Petri net is a triple* (P, T, F)*, where* P *is a finite set of*

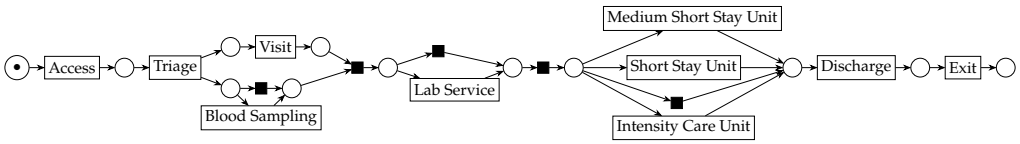


Figure 1.5: Petri net representation of the model in Figure 1.4. Circles represents places. Rectangles with associated labels denote visible transitions. Black rectangles denote invisible transitions. The • symbol indicates the initial marking.

places, T is a finite set of transitions, and $F \subseteq (P \times T) \cup (T \times P)$ is a set of arcs.

In the remainder, for any $t \in T$, $t^\bullet = \{p | (t,p) \in F\}$ and ${}^\bullet t = \{p | (p,t) \in F\}$ are the set of places with arcs coming from or going to t, respectively. The state of a Petri net is identified by its current marking:

Definition 4 (Marking). *A marking m of a Petri net (P,T,F) is a function $m : P \to \mathbb{N}$ which assigns for each place $p \in P$ the number $m(p)$ of tokens at this place. A Petri net system (P,T,F,m^I,m^F) is a Petri net with an initial marking m^I and a final marking m^F.*

A transition $t \in T$ is **enabled** in a marking m if each place $p \in {}^\bullet t$ contains at least one token, i.e., $m(p) \geq 1$. An enabled transition may **fire**, consuming one token from each input place $p \in {}^\bullet t$ and producing one token in each output place $p \in t^\bullet$, thus yielding a new marking m'.

A **labelled Petri net system** enriches the classical Petri net formalism by associating transitions with observable activities (or invisible actions).

Definition 5 (Labelled Petri Net System). *A labelled Petri net system $N = (P,T,F, \lambda, m^I, m^F)$ consists of a Petri net (P,T,F), a labelling function $\lambda : T \to \mathcal{A} \cup \{\tau\}$ where $\mathcal{A}$ is a set of activities and τ denotes the label of invisible transitions, m^I the initial marking and m^F the final marking.*

Hereafter, we use the term Petri net to indicate a (labelled) Petri net system, without causing ambiguities.

When Petri nets are used to model business processes, we assume that they are represented as **workflow nets** [4]. In a workflow net, there exist one and only one input place $i \in P$ and one and only one output place $o \in P$ that have no incoming or outgoing arcs, respectively, as well as every node is in a path between i and o. Figure 1.5 illustrates an example of a Petri net as a workflow net, representing the same process model shown in BPMN notation in Figure 1.4. Places are depicted as circles and transitions as rectangles, with invisible transitions shown in black. The initial marking is indicated by a bullet placed inside the corresponding place.

The behavior of a Petri net can be represented as a so-called **reachability graph**, which is a way of representing the states of Petri nets.

Definition 6 (Reachability Graph). *A reachability graph of a Petri net* $N = (P, T, F, m^I, m^F)$ *is a directed graph* $G_N = (V, E)$*, where:*

- V *is the set of nodes where each node* $v \in V$ *represents a reachable marking (from the initial* m^I*);*
- E *is the set of edges, each of which is a triple* (m_i, t, m_j)*, which indicates that* t *can fire at marking* m_i*, leading to marking* m_j*.*

In the remainder, given two activities $a, b \in \mathcal{A}$, of a labelled Petri net N with labelling function $\lambda \colon T \to \mathcal{A} \cup \{\tau\}$ and reachability graph (V, E), $a >_N b$ is used to indicate that, according to N, activity b can be immediately executed after a, namely E contains a set of edges $\{(m_1, t_1, m_2), \ldots, (m_{n-1}, t_{n-1}, m_n)\}$ such that $\lambda(t_1) = a$, $\lambda(t_{n-1}) = b$ and, for all $1 < k < n - 1$, $\lambda(t_k) = \tau$.

A process model is considered valid if it properly represents the intended process behavior without deadlocks or unintended terminations. Since we focus on workflow net, we base on the **soundness** criteria introduced in [4].

Definition 7 (Soundness). *A Petri net* $\left(P, T, F, m^I, m^F\right)$ *is sound iff:*

- Option to complete*: For any reachable marking* m*, it is possible to reach marking* m^F*.*
- Proper completion*: The only reachable marking that contains a token in the final place* o *is the marking* m^F*.*
- Absence of dead transitions*: There are no dead transitions, i.e. it does not exist a transition that is not enabled at any reachable marking.*

These three requirements ensure that the modeled process can always run to completion, terminates in a well-defined state, and allows all modeled behavior to be exercised.

In Petri nets, when multiple transitions are enabled at a certain marking, there is no determinism or preference on which is going to fire. In reality, not all runs are equally probable, and some are more likely than others. To model these probabilities, **stochastic Petri nets** have been proposed where each transition $t \in T$ is associated with a weight $w(t) \in \mathbb{R}^+$ [127]. We employ the concept of **Stochastic Labelled Data Petri Net System**, where transition weights are functions dependent on data states $x \in \Delta \subset \mathbb{R}^n$, that is an assignment to numeric variables [123].

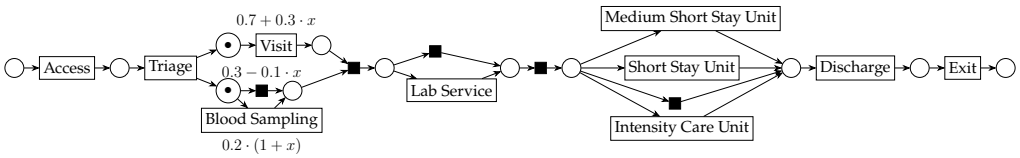


Figure 1.6: Example of a Stochastic Labelled Petri net. The black dots (•) represent the current marking. Enabled transitions are annotated with weights depending on the variable x representing a binary data state assigning value 1 for high priority cases, otherwise 0.

Definition 8 (Stochastic Labelled Data Petri Net System)**.** *Let $\Delta \subset \mathbb{R}^n$ be the set of all possible data states. A stochastic labelled data Petri net system $(P, T, F, \lambda, m^I, m^F, w)$ is a tuple, such that $(P, T, F, \lambda, m^I, m^F)$ is a labelled Petri net system, with $\lambda : T \to \mathcal{A} \cup \{\tau\}$, and $w \colon T \to (\Delta \to \mathbb{R}^+)$ is a weight function.*

Hereafter, we use the term stochastic Petri net to indicate a stochastic labelled data Petri net system, without causing ambiguities.

In a stochastic Petri net, given a marking m and data state $x \in \Delta$, the probability of firing an enabled transition $t \in E(m)$ is defined as:

$$P(t \mid m, x) = \frac{w(t)(x)}{\sum_{t' \in E(m)} w(t')(x)} \tag{1.1}$$

where $E(m) \subseteq T$ denotes the set of transitions enabled at marking m. Note that $\sum_{t \in E(m)} P(t \mid m, x) = 1$.

Example: Let N be a Petri net as shown in Figure 1.6, with the current marking m highlighted with black dots, and weight functions on the enabled transitions, i.e. *Visit*, *Blood Sampling* and the silent transition skipping *Blood Sampling*. The weights depend on the data state $x \in \{0, 1\}$, which is assigned value 1 for cases with *high* urgency and 0 otherwise. The probability of firing the transition Blood Sampling is then computed as $\frac{0.2 \cdot (1+x)}{(0.7+0.3 \cdot x)+(0.3-0.1 \cdot x)+(0.2 \cdot (1+x))} = \frac{0.2 \cdot (1+x)}{1.2+0.4 \cdot x}$, i.e. $1/4$ when $x = 1$ (*high* urgency cases), and $1/6$ otherwise.

1.3.3 Process Discovery

While Process Mining can be used to assess the compliance of process executions w.r.t. to a set of norms and protocols or to provide run-time operational support, the lion's share of attention has certainly been about **process discovery** [1].

Process discovery aims to discover a process model from an event log, e.g.

in form of a Petri net or BPMN model. Several process-discovery algorithms exist in literature [1], where each algorithm has its representational bias: some algorithms can only discover models with certain constructs, and thus they may not necessarily be able to discover a model that can represent all the behavior observed in the event log. Furthermore, certain algorithms may have a more or less extensive capabilities to separate legitimate from noisy/rare behavior, aiming to only incorporate the former into the discovered models.

The most widespread discovery algorithms are certainly Inductive Miner [105] and Split Miner [15]. These mining algorithms ensure that the discovered process models do not allow for behavior that can lead to deadlocks: they provide guarantee that, whatever activity is performed as next, there will be an option to complete and reach a final state. If these models are meant to be used for simulation, the absence of deadlock is of utmost importance to ensure each simulation run can reach the final stage.

Traditional process discovery algorithms focus primarily on control-flow structures and do not capture the stochastic behavior inherent in real-world processes. For process simulation purposes, stochastic process discovery algorithms become essential as they learn both the structural and probabilistic aspects of processes from event logs. Notable approaches include techniques for discovering generalized stochastic Petri nets [39, 38], methods for learning data-aware stochastic models [123], and algorithms that incorporate timing information into discovered process models [160]. These stochastic discovery methods enable realistic simulation by capturing the probabilistic decision points, timing distributions, and data dependencies that characterize actual process executions.

1.3.4 Log-Model Alignments & Conformance Checking

To evaluate how well a process model represents the observed behavior recorded in an event log, we need a mechanism to relate traces from the log with executions of the model. Event logs consist of traces, i.e., sequences of events labeled with the activities that occurred during process executions. Conversely, replaying a Petri net yields firing sequences of transitions that describe the behavior allowed by the model.

One of the earliest and most widely adopted techniques to relate event logs and Petri net models was **token replay** [163]. In token replay, a log trace is replayed on the Petri net by sequentially firing the transitions corresponding to the observed activities, while tracking the flow of tokens through the places of the net. Deviations are identified by missing tokens (when a transition is not enabled but needs to be fired) or remaining tokens (when tokens are left behind after replaying the trace), which are then used to quantify conformance, most

notably fitness.

While token replay is computationally efficient and intuitive, it has well-known limitations [45]. In particular, it is sensitive to the chosen firing strategy, and then models may get flooded with tokens in highly non-conforming executions, enabling unwanted parts of the process model. These limitations motivated the development of **alignment** techniques, which generalize token replay by explicitly allowing and optimizing over deviations between log behavior and model behavior.

An alignment relates *moves in log* and to *moves in model* as explained in the following definition. Here, we explicitly indicate *no move* with $\gg$. We also indicate with $\sigma|_{\mathcal{A}}$ the projection of the trace into the set of activities $\mathcal{A}$, i.e. if $\sigma = \langle e_1, \ldots, e_n \rangle$, then $\sigma|_{\mathcal{A}} = \langle act(e_1), \ldots, act(e_n) \rangle$.

Definition 9 (Alignment). *Let $N = \left(P, T, F, \lambda, m^I, m^F\right)$ Labelled Petri net system with $\lambda : T \to \mathcal{A} \cup \{\tau\}$. Let us denote $S_L = \mathcal{A} \cup \{\gg\}$ and $S_M = T \cup \{\gg\}$. A pair $(s_L, s_M) \in (S_L \times S_M) \setminus \{(\gg, \gg)\}$ is*

- *a* move in log *if $s_L \in \mathcal{A}$ and $s_M = \gg$,*
- *a* move in model *if $s_L = \gg$ and $s_M \neq \gg$,*
- *a* synchronous move *if $s_L \neq \gg$, $s_M \neq \gg$, and $\lambda(s_M) = s_L$.*

$\Sigma = (S_L \times S_M) \setminus \{(\gg, \gg)\}$ is the set of the legal moves. *The* alignment *of a log trace σ and a Petri net N is a sequence $\gamma \in \Sigma^*$ such that the projection on the first element (ignoring $\gg$) yields $\sigma|_{\mathcal{A}} \in \mathcal{A}^*$, and the projection on the second element yields a valid firing sequence of transitions of N.*

An alignment of the event log $\mathcal{L}$ and a Petri net N is a multiset $\mathcal{S} \in \mathcal{B}(\Sigma^*)$ of alignments such that, for each log trace $\sigma \in \mathcal{L}$, there exists an alignment $\gamma \in \mathcal{S}$ of σ and N. Note that $\mathcal{S}$ is a multiset because the projection of event log traces onto sequences of activities yields to a multiset of sequences of activities, and identical activity sequences may therefore correspond to identical alignments occurring multiple times.

In order to quantify the severity of a deviation, a cost function $\kappa \in (\Sigma \to \mathbb{R}_0^+)$ needs to be defined over the legal moves. This cost function can be generally customized, as a deviation severity depends on the process' domain. With a cost function at hand, the cost of an alignment γ is defined as the sum of the costs of the individual moves in the alignment, i.e., $\mathcal{K}(\gamma) = \sum_{(s_L, s_M) \in \gamma} \kappa(s_L, s_M)$.

It may be defined differently for individual processes, since, generally speaking, the costs depend on the specific characteristics of the process. In most scenarios, a **standard cost function** is applicable: given $\forall\, (s_L, s_M) \in \Sigma$, $\kappa^{std}(s_L, s_M) = 1$

if $s_L = \gg$ and $\lambda(s_M) \neq \tau$, or $s_M = \gg$; otherwise, $\kappa^{std}(s_L, s_M) = 0$. The cost of an alignment γ is defined as the sum of the costs of the individual moves in the alignment, i.e., $\mathcal{K}(\gamma) = \sum_{(s_L, s_M) \in \gamma} \kappa(s_L, s_M)$. In the remainder, if not stated otherwise, we use the *standard cost function*.

In general, the number of potential alignments is infinite, but we are not interested in any, but in an alignment with the lowest cost, also known as **optimal alignment**. Note that there may be multiple alignments that are optimal: without further information for discriminating, any of alternative optimal alignment can be considered. In [181], van der Aalst et al. propose techniques to compute optimal alignments.

Example: Consider the process of an emergency department represented by the Petri net N in Figure 1.5 and a trace σ with sequence of activities $\langle$Access, Triage, Visit, Exit$\rangle$. An optimal alignment γ of σ and the model N is:

Log	Access	Triage	Visit	$\gg$	$\gg$	$\gg$	$\gg$	$\gg$	Exit
Model	Access	Triage	Visit	τ_1	τ_2	τ_3	τ_4	Discharge	Exit

with τ_i, $i = 1, \ldots, 4$, denoting the invisible transitions. Using the standard cost function, the total cost of this alignment is $\kappa(\gamma) = 1$, where all the (legal) moves have cost 0, except for ($\gg$, Discharge), which has cost 1.

For some applications (see, e.g., Chapter 6), it is necessary to map each alignment prefix γ' with the corresponding marking reached after firing the transition sequence in the process projection of γ'. These alignments with associated markings are hereafter named **extended alignment**:

Definition 10 (Extended Alignment). *Let $\gamma = \langle (s_L^1, s_M^1), \ldots, (s_L^n, s_M^n) \rangle$ be the alignment of a Labelled Petri net system $N = \left(P, T, F, \lambda, m^I, m^F\right)$ with a trace σ. The extended alignment γ^{ext} of γ is a sequence of triple $\langle (s_L^1, s_M^1, m_1), \ldots, (s_L^n, s_M^n, m_n) \rangle$ where, for all $1 \leq j \leq n$, m_j is the marking that is reached from m^I after firing the sequence of transitions $\langle s_M^1, \ldots s_M^i \rangle$, ignoring the instances of $\gg$.*

In fact, when it is clear from the discussion, we use the term alignment to refer to the extended alignment, avoiding any ambiguities.

Optimal alignments can then be used to *repair traces* that cannot be replayed on a Petri net model by minimally modifying them to ensure compliance with the model. Given a trace σ, we define a **Petri net run** of σ as a sequence of transitions with event characteristics (resource, timestamp, and lifecycle) induced by an optimal alignment between σ and the Petri net model. In particular, the run is

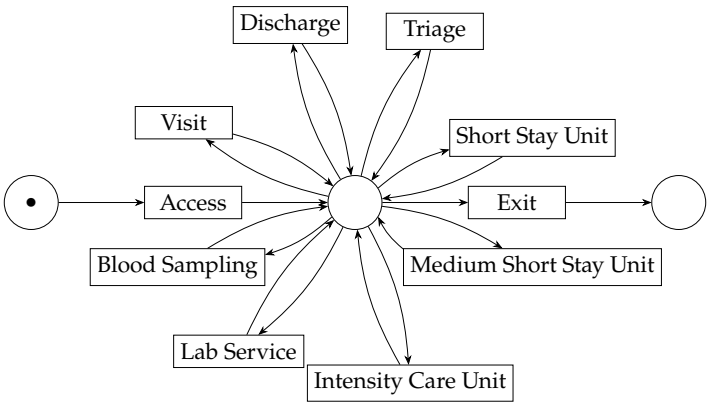


Figure 1.7: Petri net "flower model" with same labelled transitions of the Petri net in Figure 1.5.

constructed considering the sequence of model moves of an optimal alignment of σ, i.e., the transitions fired in the Petri net (ignoring $\gg$). For synchronous moves, the corresponding elements inherit all characteristics (resource, timestamp, and lifecycle) of the aligned events in the original log trace σ. For non-synchronous model moves, corresponding characteristics are derived from the most recent preceding synchronous move. This notion is particularly useful in the following chapters of this thesis, where event logs need to be mapped onto a process model.

Alignments thus provide a fine-grained comparison between event data and process models, and form the foundation for **conformance checking** and repair techniques. To evaluate the quality of a process model with respect to an event log, four main conformance dimensions are typically considered: **fitness**, **precision**, **simplicity**, and **generalization** [1]. Both of metrics are assigned a value between zero and one, with one being the perfect value.

Fitness captures how well the model can reproduce the observed behavior. Using an alignment-based approach, fitness is measured by comparing each trace in the log with its optimal alignment in the model, counting deviations such as missing or extra activities. A model achieves perfect fitness (value 1) when all traces can be replayed without deviations. Precision evaluates the extent to which a model avoids allowing behavior not seen in the log. While fitness penalizes missing behavior, precision penalizes extra, unobserved behavior. Alignment-based precision, as proposed by Adriansyah et al. [8], compares the transitions enabled by the model at each visited marking with those actually observed in the log. A highly precise model only allows continuations supported by the log, whereas overly permissive models, such as the "flower model" (see Figure 1.7), score poorly. Simplicity favors models that are easier to understand and maintain. Among models with similar fitness and precision,

simpler structures are preferred. In this thesis, we adopt the arc degree simplicity measure [184], which penalizes highly connected nodes and rewards models with balanced, minimally complex routing. Finally, generalization reflects the model's ability to capture plausible behavior beyond the observed log, addressing the fact that logs usually represent only a sample of all possible executions. Alignment-based and anti-alignment techniques can quantify generalization by assessing whether the model can accommodate unobserved but feasible behavior [92, 59]. Generalization complements precision by preventing overfitting to the log while still representing likely unseen behavior.

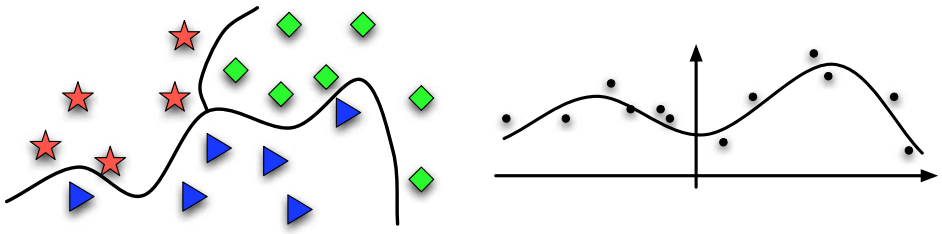
1.4 Machine Learning

Machine Learning (ML) is a subfield of Artificial Intelligence and Computer Science that focuses on developing algorithms and models that enable computers to learn from data and improve their performance over time. Rather than relying solely on explicit programming, machine learning systems identify patterns, make predictions, and adapt their behavior based on experience or observed data [166]. The learning process typically begins with a set of examples or observations, from which the system extracts statistical regularities and structures to make informed decisions or predictions on new, unseen data. Machine learning provides a framework for building probabilistic models that capture uncertainty in data, allowing systems to generalize from finite observations and make robust predictions [29].

In the context of process mining, machine learning has opened up new frontiers beyond the traditional descriptive analytics of process discovery, conformance checking, and model enhancement. It provides the foundation for predictive and prescriptive capabilities, enabling organizations to move from a reactive to a proactive approach in managing their business processes. By leveraging historical event log data, machine learning models can be trained to forecast future process behavior. This is the core of **predictive process monitoring**, where predictions can be made about the outcome of a running process instance, its remaining time, or the next likely activity [121]. More recently, ML has also been integrated with process simulation, augmenting it with realistic, data-driven models that can predict the impact of changes before they are applied [41, 136].

1.4.1 Machine Learning Predictors

Machine learning algorithms are commonly categorized into two main paradigms. In **supervised learning**, the algorithm is trained on a labeled dataset $D = \{(x_i, y_i)\}_{i=1}^{n}$, where each input vector $x_i \in \mathcal{X}$ is paired with a corresponding



(a) (b)

Figure 1.8: Illustration of supervised learning tasks: a Classification – separating data points into categories by learning decision boundaries; b Regression – fitting a predictive curve to estimate continuous output values from input data. Source: [174].

output label $y_i \in \mathcal{Y}$. The objective is to learn a mapping $\psi : \mathcal{X} \to \mathcal{Y}$ that accurately predicts the label y for new unseen inputs x, generalizing from the training data. Conversely, **unsupervised learning** works with unlabeled data, aiming to discover inherent patterns, clusters, or association rules within the input data without explicit output labels [29]. Techniques such as clustering and dimensionality reduction exemplify unsupervised learning.

This thesis primarily emphasizes supervised learning algorithms due to their applicability in process simulation techniques, where machine learning predictors can be integrated to enhance predictive accuracies. Supervised learning algorithms are generally divided into two main categories: **classification** and **regression**. Classification algorithms aim to assign inputs to discrete categories where data points are separated by decision boundaries (see Figure 1.8a). Regression algorithms, on the other hand, model the relationship between input features and a continuous output variable where the model fits a smooth curve through observed data points to predict numeric values (see Figure 1.8b).

Several well-established models form the basis of many machine learning applications. **Linear regression** is one of the simplest regression algorithms. for modeling the relationship between a continuous dependent variable y and one or more independent variables $x = (x_1, \ldots, x_n)$. The model assumes a linear form:

$$y = \beta_0 + \sum_{j=1}^{n} \beta_j x_j + \epsilon \tag{1.2}$$

where β_j are coefficients to be learned, and ϵ is the error term. The goal is to find the coefficient vector β that minimizes the residual sum of squares between observed and predicted values. Linear regression is primarily used for regression

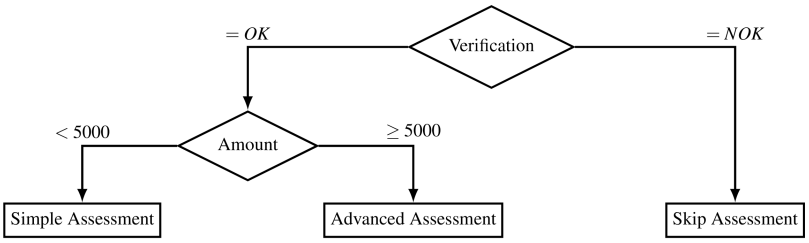


Figure 1.9: Decision tree example used for assessment decisions. Source: [112].

tasks, such as predicting process durations or costs.

While linear regression is suited for continuous outputs, **logistic regression** is designed for binary classification problems, where the outcome $y \in 0, 1$. Instead of predicting the value of y directly, it estimates the probability that $y = 1$ given the input x. This is achieved using the logistic function:

$$P(y = 1|x) = \frac{1}{1 + e^{-\left(\beta_0 + \sum_{j=1}^{n} \beta_j x_j\right)}} \tag{1.3}$$

The output is a probability between 0 and 1, which can then be thresholded (e.g., at 0.5) to produce a class label. The parameters β are learned by maximizing the likelihood of the observed data. Logistic regression is widely used in process mining for tasks such as outcome or activity occurrence prediction.

Decision trees are another fundamental class of supervised learning algorithms that can be used for both classification and regression tasks. They work by recursively partitioning the feature space into subsets based on feature value thresholds, creating a tree-like model of decisions. Each internal node represents a test on an attribute, each branch corresponds to an outcome of that test, and each leaf node represents a predicted class label (in classification) or a numerical value (in regression). The tree is built using algorithms such as ID3, C4.5, or CART, which select the best feature splits based on criteria like information gain, Gini impurity, or variance reduction. One of the main advantages of decision trees lies in their interpretability: the resulting model can be visualized and easily understood as a series of simple decision rules. However, they are prone to overfitting if not properly pruned or regularized, especially when the tree grows too deep.

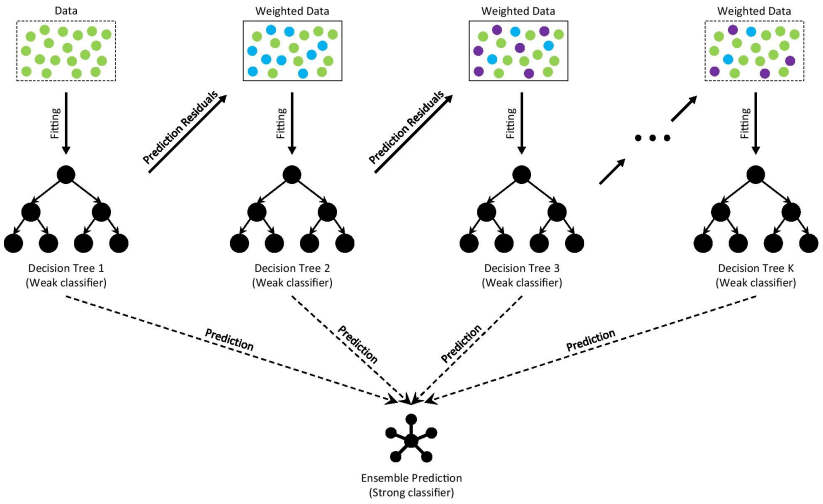


Figure 1.10: Illustration of the gradient boosting process, where multiple weak learners (decision trees) are trained on the residuals of previous models to form a strong predictive ensemble. Source: [55].

Example: Figure 1.9 illustrates a simple decision tree used for an approval assessment process. The process begins with a *Verification* step: if the verification result is *NOK* (not OK), the process directly skips the assessment. If the result is *OK*, the next decision node evaluates the *Amount*. Transactions below 5000 proceed to a *Simple Assessment*, while those equal to or above 5000 trigger an *Advanced Assessment*. This example shows how a decision tree can encode complex decision logic in an interpretable structure.

Ensemble methods build upon the idea of combining multiple weak learners to form a stronger and more accurate predictive model. Among the most popular ensemble techniques are bagging, random forests, and boosting. While bagging methods like Random Forest train several trees independently on bootstrap samples and average their predictions to reduce variance, boosting takes an iterative approach that focuses on correcting the errors of previous models.

Gradient boosting is a powerful boosting technique that constructs an additive model in an iterative manner (see Figure 1.10). Starting from an initial weak model, subsequent trees are trained to predict the residual errors made by the ensemble so far. Each new tree $h_m(x)$ is added to the model with a learning rate η controlling its contribution:

$$\psi_m(x) = \psi_{m-1}(x) + \eta h_m(x) \tag{1.4}$$

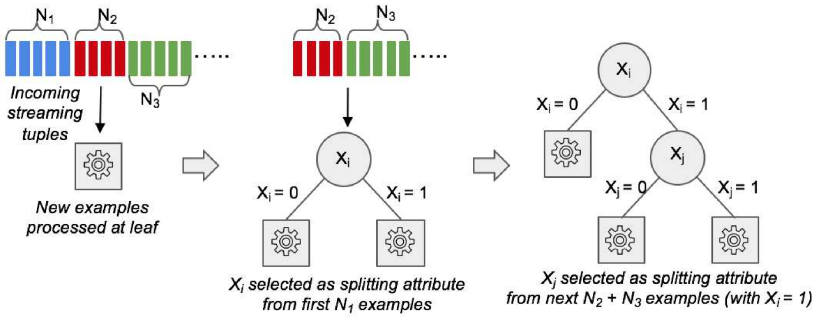


Figure 1.11: Online decision tree construction. Source: [53].

Here, each tree is fitted to the negative gradient of the loss function with respect to the predictions of the current model. This sequential learning process enables gradient boosting to efficiently minimize the overall loss function, producing highly accurate models.

Different parts of this thesis leverage **CatBoost** (Categorical Boosting) [151], a high performance open source framework for gradient boosting in decision trees, designed to efficiently handle categorical features. It builds strong predictors by iteratively combining weak models in a greedy manner. Unlike traditional approaches requiring one-hot encoding, CatBoost uses an *ordered target statistics* (i.e. on the dependent variable) method to encode categorical variables. In this approach, each categorical value is replaced by a statistic (e.g., a mean target value) computed only from previously seen samples according to a random permutation of the training data. By ensuring that the target value of the current sample is never used in its own encoding, this strategy effectively prevents target leakage. At each iteration, it performs a random permutation of the data and constructs trees using combined categorical features from previous splits. With its *ordered boosting* scheme and symmetric tree structures, CatBoost mitigates overfitting and achieves high accuracy with minimal tuning, outperforming existing gradient boosting implementations.

1.4.2 Online Machine Learning

Traditional machine learning models are typically trained in a *batch* setting, where the entire dataset is available beforehand and processed multiple times to optimize model parameters. However, in many real-world scenarios such as process monitoring or predictive process mining, data arrives sequentially in a continuous stream. In these contexts, models must learn incrementally and adapt to new information as it becomes available. **Online machine learning** addresses this need by updating models continuously, one instance (or a small

batch) at a time, without retraining from scratch [70]. This enables models to efficiently adapt to evolving data distributions and handle *concept drift*, i.e., changes in the underlying patterns over time [71].

Building on this paradigm, **online decision trees** extend the concept of traditional decision trees to streaming data scenarios. Instead of constructing the entire tree from a static dataset, these models incrementally grow and refine their structure as new instances arrive [58]. Each node maintains sufficient statistics about observed data (e.g., counts, sums, variances) and periodically evaluates potential splits using statistical tests to ensure that decisions are made with high confidence. This allows the model to evolve efficiently without revisiting past data (see Figure 1.11).

A prominent example of this approach is the **Hoeffding Tree**, also known as the *Very Fast Decision Tree (VFDT)* [58]. It relies on the *Hoeffding bound* [82] to decide, with a specified confidence level, when enough data has been observed to select the best attribute for splitting at a node. The Hoeffding bound provides an upper limit on the difference between the true mean of a random variable, denoted by its expected value $\mathbb{E}[r]$ and the empirical mean $\bar{r}$ computed on n samples. Formally, with probability at least δ,

$$|\mathbb{E}[r] - \bar{r}| \leq \epsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}}, \tag{1.5}$$

where R is the range of r. In VFDT, this bound guarantees that when the difference in information gain (or Gini reduction) between the two best attributes exceeds ϵ, the top attribute can be safely chosen for splitting. This allows the tree to grow incrementally from streaming data while ensuring statistically sound decisions [58].

While the Hoeffding Tree effectively handles data streams under stationary conditions, it does not inherently manage changes in the data distribution. To address this limitation, the **Hoeffding Adaptive Tree (HAT)** [28] extends the VFDT by integrating a statistical mechanism to detect and adapt to *concept drift*. HAT associates each leaf node with an adaptive estimator, typically an *ADWIN* (ADaptive WINdowing) detector, that monitors the error rate of predictions over time. ADWIN maintains a variable-length sliding window W of recent observations and continuously tests whether the average error in two subwindows, W_1 and W_2, differs significantly. When a drift is detected at a node, HAT creates a *background subtree* trained on the new data distribution. If this subtree later outperforms the existing branch (based on predictive accuracy or error rate), it replaces the old one. This adaptive replacement strategy enables HAT to dynamically adjust its structure in response to evolving data, ensuring robust

performance in non-stationary environments. Consequently, HAT is particularly suitable for applications such as continuous process prediction and online process mining, where underlying process dynamics may change over time.

1.4.3 Deep Learning

Deep learning is a class of machine learning methods that employs multi-layered artificial neural networks to automatically learn hierarchical representations from raw data, enabling models to capture complex patterns and dependencies with minimal feature engineering [103]. Architectures such as convolutional neural networks (CNNs) [104] and recurrent neural networks (RNNs) [62] are particularly effective at extracting features from high-dimensional or sequential data streams.

In the context of process mining, deep learning provides the ability to model intricate temporal and contextual dependencies from large-scale event logs, going beyond the capabilities of traditional machine learning predictors [67]. A prominent example is the **Long Short-Term Memory** (LSTM) network [81], a variant of RNNs designed to overcome the vanishing gradient problem in long sequences. LSTMs leverage memory cells and gating mechanisms (input, output, and forget gates) to selectively store, update, and propagate information over time. This makes them particularly suited for predictive process monitoring tasks, such as forecasting the next activity, estimating remaining execution time, or predicting process outcomes [44, 143].

Despite their predictive power, deep learning models suffer from a major limitation: they are inherently difficult to interpret. Unlike transparent models such as decision trees or linear predictors, LSTMs and other deep networks operate as "black boxes", making it challenging to understand the reasoning behind their predictions [138]. This lack of explainability can hinder trust and adoption in critical process mining applications, where interpretability is often as important as accuracy [43].

1.4.4 Explainable Machine Learning

In many machine learning applications, model interpretability is crucial for understanding, trusting, and effectively deploying predictive systems. Some models are inherently interpretable: for example, decision trees (see Figure 1.9) can be visualized as a series of simple decision rules, and linear or logistic regression models provide direct insight into feature contributions through their learned coefficients. These models allow practitioners to trace predictions back to input features and explain why a certain decision was made. However, more

complex models such as ensembles (e.g., gradient boosting) or deep neural networks (see Section 1.4.3) often achieve higher accuracy at the cost of transparency, acting as so-called "black boxes" that obscure the relationships they have learned.

Several approaches have been introduced to address the problem of the accuracy-interpretability trade-off. One of the most widespread is based on computing Shapley values, which can be efficiently computed via the method **SHAP (SHapley Additive exPlanations)** [120]. Shapley values are computed for each feature of each potential element $\vec{x} = (x_1, \ldots, x_n) \in X_1 \times \ldots \times X_n$: given a feature f_i, the Shapley value $\phi_i^{\vec{x}}$ intuitively expresses how much the feature value x_i contributes to the model prediction when the input is $\vec{x}$. The contribution is computed as the average prediction difference with respect to a certain base value, when feature f_i takes on value x_i and each other feature is either discarded or retained. The base value is computed as the average of the predictions when a given support multiset $\mathcal{X} \in \mathbb{B}(X_1 \times \cdots \times X_m)$ is used as input (usually the model's training set).

Shapley values are computed for individual input vectors and individual features. However, we need to obtain a general contribution value for each individual feature, fairly independent of the specific input. We thus introduce the following explanation function, used thereafter in the thesis, that returns the average of the Shapley values when a feature assumes a certain value.

Definition 11 (Explanation function)**.** *Let $\mathcal{F} = \{f_1, \ldots, f_m\}$ be the set of all the features, with X_j be the domain of f_j. Let $\psi : X_1 \times \cdots \times X_m \to Y$ be a regressor or a classificator model. Let $\mathcal{X} \in \mathbb{B}(X_1 \times \cdots \times X_m)$ be a multiset of elements in $X_1 \times \cdots \times X_m$. Let us consider any $f_i \in \mathcal{F}$. Given a value $x \in X_i$, we define*

$$\phi_{f_i}(x, \mathcal{X}) = \underset{\substack{\vec{x}=(x_1,\ldots,x_m)\in\mathcal{X} \\ x_i=x}}{avg} \left(\phi_i^{\vec{x}}\right) \tag{1.6}$$

as the average of the Shapley values $\phi_i^{\vec{x}}$ computed over the set of input values $\vec{x} \in \mathcal{X}$, when the feature f_i assumes values $x \in X_i$.

Intuitively, $\phi_{f_i}(x, \mathcal{X})$ expresses the expected impact of feature f_i taking on value x, i.e. how much the fact $f_i = x$ influences the prediction.

SHAP values have been increasingly applied in process mining, where explainability is particularly important for supporting human decision-making and ensuring trust in predictive models [145, 68]. By providing transparent, feature-level explanations, SHAP enables organizations to balance predictive accuracy with interpretability, fostering more reliable and actionable insights in complex business processes.

1.5 Predictive and Prescriptive Process Analytics

Building upon the machine learning foundations introduced in the previous section, process mining has progressively evolved from descriptive analysis toward more advanced forms of **predictive** and **prescriptive** analytics. This evolution enables organizations to move beyond understanding past behavior and instead anticipate future process developments and actively influence them. In this context, **Process-aware Recommender (PAR) systems** represent a specialized class of information systems designed to support operational decision-making during process execution [111].

PAR systems combine predictive models with decision logic to forecast how ongoing process instances are likely to evolve and to proactively recommend corrective actions aimed at improving process outcomes. These outcomes are typically defined in terms of process-specific *Key Performance Indicators (KPIs)*, such as execution time, operational costs, service-level compliance, or customer satisfaction. Conceptually, a PAR system can be decomposed into two blocks: **predictive process monitoring**, which focuses on forecasting future process behavior, and **prescriptive process analytics**, which focuses on identifying and recommending suitable interventions based on these predictions.

1.5.1 Predictive Process Monitoring

Predictive process monitoring is a branch of process mining concerned with forecasting the future state of running process instances based on historical event data [67, 121]. Given a partial trace (i.e., a **prefix**) of an ongoing case, predictive techniques estimate future outcomes of that case by exploiting patterns learned from completed executions recorded in event logs. Typical prediction tasks include forecasting the *remaining execution time* of a case, predicting the *next activity* to be executed, or estimating the *final outcome* of the process instance.

Predictive process analytics typically relies on supervised machine learning models (see Section 1.4.1), trained on labeled traces derived from historical logs. Each running case is encoded into a feature representation capturing control-flow information, temporal aspects, data attributes, and resource-related features. These representations are then fed into classification or regression models, ranging from interpretable predictors such as decision trees to more complex models such as gradient boosting ensembles or deep neural networks, to produce forecasts for ongoing cases.

Beyond predictive accuracy, recent research has increasingly emphasized the importance of *explainability* in predictive process analytics [68]. Since predictions are often used to support or trigger human decisions, understanding *why* a

certain outcome is predicted is crucial for trust and adoption.

1.5.2 Prescriptive Process Analytics

Prescriptive analytics techniques go beyond predictive methods by not only forecasting future states, often undesired, but also **recommending** or prescribing interventions to prevent a case from reaching an undesired future state [100, 111].

Prescriptive process analytics techniques map ongoing cases to a set of recommended or prescribed actions, such as executing additional activities or reallocating resources, that, if applied, are expected to improve the predicted KPI values. The effectiveness of a prescription is often evaluated through a cost or utility function that balances multiple factors, including the reduction in the number of negative outcomes, the cost of executing the intervention, and the timing at which the intervention is applied.

A key challenge in prescriptive process analytics is ensuring that recommendations are understandable and actionable. To address this issue, recent approaches exploit explainability techniques to justify prescribed interventions by linking them to the underlying predictive model. For example, Padella et al. [145] use SHAP values to explain why certain actions are expected to improve the predicted outcome, thus improving transparency and trust in prescriptive recommendations.

1.6 Business Process Simulation

Business Process Simulation (BPS) is a well established technique for quantitatively analyzing, evaluating, and improving business processes [60]. By combining information on control-flow, task durations, and resource allocation, BPS provides a probabilistic representation of process behavior. This allows organizations to perform *what-if* analyses, testing alternative process configurations, redesigns, or resource allocations, without disrupting real operations. Through simulation, analysts can anticipate bottlenecks, evaluate system performance under varying conditions, and make data-driven decisions to enhance efficiency and throughput.

1.6.1 Business Process Simulation Models

Classical process discovery algorithms, such as those discussed in Section 1.3.2, primarily aim to derive a process model that captures the control-flow perspective, i.e., the allowed sequences of activities within a process. However, real-world processes also involve other essential perspectives, including resources,

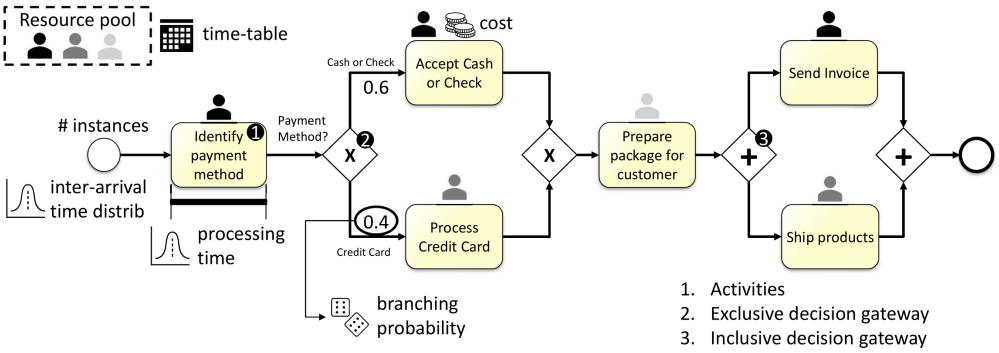


Figure 1.12: BPS model components example for a purchase-to-pay process. Source: [40].

decisions, and time. These additional dimensions can also be discovered by analyzing event logs. **Process enhancement** techniques aim to incorporate the process perspectives on data, decision, resources and time into the model: their inclusion in process models enable designers to fine-tune the model specifications, thus obtaining models with higher levels of precision [110]. The integration of multiple perspectives results in richer, more complete process models that can also serve as the foundation for **process simulation**.

A **BPS model** is typically defined by combining a process model, such as a Petri net or BPMN diagram, with a set of quantitative and contextual parameters (see Figure 1.12). These parameters collectively determine the stochastic and operational behavior of the simulated process. A comprehensive simulation model usually includes the following components:

- **Branching probabilities:** Define the likelihood of following each alternative path at decision points (e.g., XOR-splits), enabling the simulator to reproduce realistic control-flow variability.
- **Activity duration distributions:** Capture the variability in task execution times (e.g., normal, exponential, or log-normal), allowing the simulation to reflect realistic temporal dynamics.
- **Inter-arrival time distributions:** Specify the temporal spacing between the beginning of two consecutive process instances, supporting the simulation of realistic workload patterns and input rates.
- **Waiting time distributions:** Represent delays caused by resource contention, dependencies, or external factors, capturing the temporal gaps between activity readiness and execution.
- **Resource assignments and roles:** Define which resources (e.g., employees,

machines, or departments) can perform specific activities, shaping task execution and workload distribution.

- **Resource multitasking capabilities:** Indicate whether resources can handle multiple activities concurrently or must process them sequentially, affecting overall throughput and utilization.
- **Working schedules and availability constraints:** Specify resource availability (e.g., shifts or office hours), allowing simulations to account for non-continuous operations.
- **Other attribute distributions:** Capture additional case-related characteristics, such as case type, priority, or customer category, that may influence control-flow, timing, or resource allocation.

Together, these components enable a simulation model to represent not only the structural control-flow of a process but also its temporal, organizational, and contextual dimensions, providing a robust foundation for simulation analysis and *what-if* experiments.

In this thesis, we formally define a business process simulation model as follows.

Definition 12 (Business Process Simulation Model)**.** *A business process simulation model is a tuple (N, D, K), where:*

- $N = \big(P, T, F, \lambda, m^I, m^F, w\big)$ *is a* ***stochastic Petri net*** *with $\lambda \colon T \to \mathcal{A} \cup \{\tau\}$, $w \colon T \to (\Delta \to \mathbb{R}^+)$, and $\Delta \subset \mathbb{R}^n$, $n \geq 1$, the set of possible data states.*
- $D = (\mathcal{R}, C^{\mathcal{R}}, MT^{\mathcal{R}}, d^{\mathcal{V}})$ *represents the* ***descriptive parameters*** *of the model, consisting of a set $\mathcal{R}$ of resources; a function $C^{\mathcal{R}} \colon \mathcal{R} \to \{0,1\}^{7\times 24}$ mapping each resource to its weekly work schedule, represented as a 7×24 binary matrix, where $C^{\mathcal{R}}_{w,h} = 1$ indicates that the resource is available at hour h on weekday w; a set $MT^{\mathcal{R}} \subseteq \mathcal{R}$ denoting the subset of resources that are allowed to perform multiple activities concurrently; and a function $d^{\mathcal{V}} \colon \mathcal{V} \to \mathcal{P}$ mapping each trace attribute to a parametric distribution $d \in \mathcal{P}$.*
- $K = (p^R, p^{ET}, p^{WT}, p^{AT})$ *defines the* ***predictive parameters*** *of the model. The function $p^R \colon \mathcal{R} \to (\Delta \to \mathbb{R}^+)$ assigns to each resource a data state–dependent weight representing the probability of its allocation. The function $p^{ET} \colon \mathcal{A} \to (\Delta \to \mathcal{P})$ assigns to each activity a data state–dependent parametric distribution modeling its execution time. The function $p^{WT} \colon \mathcal{R} \to (\Delta \to \mathcal{P})$ assigns to each resource a data state–dependent parametric distribution predicting waiting times for events performed by that resource. Finally, $p^{AT} \in (\Delta \to \mathcal{P})$ is a function that*

returns a data state–dependent parametric distribution modeling the inter-arrival times of new process instances.

The formalization of each component in (N, D, K) enables the generation of complete synthetic event logs that capture both the control-flow and behavioral dynamics of a business process. The stochastic Petri net N governs the execution logic of activities, while the descriptive parameters D define the static characteristics of the process environment, such as resource availability and working calendars. The predictive parameters K determine the stochastic and data-driven aspects of the simulation, specifying how timing, resource allocation, and case arrivals evolve throughout simulated executions. This definition allows the integration of these components for the runtime generation of events following the approach in [136].

Notably, both the predictive components in K and the weight function w in the stochastic Petri net are data state-dependent, meaning that their values are functions of the current data states in $\Delta \subset \mathbb{R}^n$, $n \geq 1$. This dependency allows the simulation model to dynamically adjust its behavior based on contextual information available during process execution, such as trace attributes, resource allocations, or temporal features. To capture this notion formally, we define the concept of a **process state**, which specifies how the current data state is derived from the process history.

Definition 13 (Process State)**.** *Let $\mathcal{I}$ be the set of case identifiers. Let T the set of transitions of N, $\mathcal{R}$ the set of possible resources, and $\mathcal{V}$ and $\mathcal{U}$, respectively, the set of trace attributes and the corresponding possible values. A **process state function** is a mapping*

$$\mathcal{S}_\Delta : (\mathcal{I} \times T \times \mathcal{R} \times \mathbb{N} \times \mathbb{N} \times (\mathcal{V} \to \mathcal{U}))^* \to \Delta \tag{1.7}$$

which assigns the sequence of past moves in the model within case identifier, resource, start and complete timestamps, and trace attributes, to a data state vector $\delta \in \Delta \subset \mathbb{R}^n$, reflecting the current values of process variables.

The definition of the set of data state Δ is flexible and can be configured according to the specific dependencies to be modeled. Examples of data states can capture a wide range of information reflecting the current context of the process. They may include the history of executed activities for the current case, such as which activities have already been completed and in what order, along with the values of trace attributes like *access type*, *urgency code*, or *pathology* in an emergency department process. Data states can also encode temporal context, including the current hour, or the day of the week. Resource-related information can be incorporated, such as which resources have previously worked on the case, or their current workload.

The predictive parameters K and the weight function w can be instantiated through machine learning models trained on historical event data. This approach gives rise to a hybrid process simulation model (cf. Section 1.6.2), where the structural and stochastic definitions of the Petri net are combined with data-driven predictive components, thereby enabling realistic, adaptive, and context-sensitive process simulations.

1.6.2 Discovery of Business Process Simulation Models

Traditionally, the construction of process simulation models has relied on manual effort from domain experts. This approach is both time-consuming and error-prone, as it relies heavily on human interpretation and incomplete process documentation. To overcome these limitations, Rozinat et al. [164] pioneered the use of process mining techniques for the (semi-)automatic discovery of simulation models from event logs. This data-driven approach leverages event data not only to reconstruct the control-flow but also to extract empirical information about timing and resource behavior, enabling the creation of realistic and evidence-based simulation models with minimal human intervention.

More recently, automated discovery of BPS models has gained growing attention within the process mining field. These approaches aim to derive complete, data-driven simulation models directly from event logs, minimizing the need for manual configuration. Building on the early work of Rozinat et al. [164], which introduced semi-automated techniques to extract simulation parameters such as activity durations and resource behavior, more recent methods leverage advanced discovery algorithms to automatically infer a wider range of simulation parameters [42]. These include not only control-flow structures, but also distributions for task durations, arrival rates, resource assignments, and decision probabilities. By relying on empirical data rather than human intuition, automated discovery techniques enable the generation of simulation models that more accurately reflect real operational dynamics, reducing modeling effort, and improving scalability across different processes and domains.

The increasing availability of large-scale event data has led to the adoption of deep learning techniques in business process simulation. With abundant and high-quality process execution data, researchers have explored neural architectures to learn complex temporal and behavioral dependencies directly from logs [43]. It has been shown that such deep learning approaches can produce more accurate and realistic event logs than traditional methods. In particular, deep learning models are able to capture intricate correlations among activities, resources, and time-related attributes that are difficult to encode explicitly in classical process models [43].

Nevertheless, these deep learning approaches suffer from a major limitation: they often behave as "black boxes". The internal mechanisms by which neural models make predictions remain largely opaque, and the absence of explicit process structure hinders interpretability and control. This opacity makes it challenging for analysts to perform *what-if* analyses [43].

To address these issues, recent research has proposed **hybrid simulation models** that combine the structural interpretability of process mining techniques with the predictive power of deep learning [41, 136]. These hybrid approaches typically rely on a process model, such as a Petri net or BPMN model, to represent the control flow, while neural predictors are embedded to estimate specific parameters like activity durations and waiting times. By integrating learned temporal or behavioral patterns into an interpretable process structure, hybrid methods enhance the fidelity of simulation without completely sacrificing explainability.

Despite these advantages, a trade-off persists between predictive performance and interpretability. While hybrid models can better capture nonlinear dependencies and subtle patterns in real-world processes, the inclusion of neural components can still limit the analyst's ability to inspect or manipulate simulation parameters explicitly [43]. Consequently, while hybrid methods represent a promising direction for increasing realism in simulations, they still face the challenge of reconciling predictive accuracy with explainability.

1.6.3 Measuring the Accuracy of Business Process Simulation Models

Business process simulation (BPS) models are only as reliable as their ability to reproduce observed behavior; thus, **simulation accuracy** determines whether scenario analyses and *what-if* results can be trusted in practice [48]. The core challenge, however, lies in establishing a non-ambiguous measure of accuracy that captures the complexity inherent in business processes, which are defined by their structural flow, the time-related distribution of tasks, and the aggregate performance outcomes. Without a clear set of metrics, reliably comparing two alternative BPS models or determining a model's fitness for practical use becomes difficult.

To address this challenge and provide a robust framework for model quality assessment, Chapela Campa et al. [47] introduced a collection of measures that define BPS accuracy across multiple critical perspectives. These measures work by computing distances between a real, ground-truth event log $\mathcal{L}^{real}$, usually used for discovering the BPS model, and a simulated event log $\mathcal{L}^{sim}$ generated by the BPS model. The work in [47] divide measure across different perspectives:

Control-Flow

- **Control-Flow Log Distance (CFLD):** This measure provides a precise, trace-level comparison. It computes the effort required to transform the set of traces in $\mathcal{L}^{sim}$ into the set of traces in $\mathcal{L}^{real}$. It computes the Damerau-Levenshtein [204] distance between each pair of cases belonging to $\mathcal{L}^{real}$ and $\mathcal{L}^{sim}$, respectively, normalizing them by the maximum of their lengths (obtaining a value in $[0, 1]$). Subsequently, it computes the matching between the cases of both logs minimizing the sum of distances using the Hungarian algorithm for optimal alignment. The CFLD is the average of the normalized distance values.

- **N-Gram Dinstance (NGD)** This is a more computationally efficient alternative to CFLD. It abstracts each log into a histogram representing the frequency of all unique n-grams (sequences of n consecutive activities). The distance is then the sum of absolute errors between the normalized frequency histograms of the two logs.

Temporal Perspective

- **Absolute Event Distribution (AED):** This measure compares the overall trend and volume of events over the entire timeline. It converts each log into a time series by binning all event start and end timestamps into discrete intervals (e.g., hourly). The distance between the two resulting time series is then calculated using the Earth Mover's Distance (EMD) [107], which measures the minimum "work" needed to transform one distribution into the other, accounting for both the difference in volume and the temporal distance between bins.

- **Circadian Event Distribution (CED)** This measure assesses the model's ability to capture weekly seasonality. It partitions each log by the day of the week (Monday, Tuesday, etc.) and creates an hourly time series of event counts for each day. The final distance is the average EMD computed between the corresponding days of the week from each log.

- **Relative Event Distribution (RED)** Moving from absolute time, RED focuses on the internal timing of cases. For each event, it calculates the elapsed time since the start of its own case. These relative durations are then binned into a histogram. The EMD between the histograms from $\mathcal{L}^{sim}$ and $\mathcal{L}^{real}$ reveals how well the model reproduces the internal process cadence, such as processing times and waiting times between activities.

Workforce Perspective

- **Circadian Workforce Distribution (CWD):** Similar to CED, this measure analyzes weekly patterns but focuses on resource activity instead of event counts. For each day of the week, it creates an hourly time series representing the number of unique active resources. The distance is the average EMD across the corresponding days, providing insight into whether the model accurately simulates resource availability and workload distribution.

Congestion Perspective

- **Case Arrival Rate (CAR):** This measure isolates the very start of the process: the arrival of new cases. It filters each log to retain only the first event of every case, treating its start time as the case arrival time. These arrival timestamps are binned into a time series, and the EMD is used to compare the arrival patterns of the simulated log against the real log.
- **Cycle Time Distribution (CTD):** It compares the end-to-end duration of cases. It computes the cycle time for every case in each log and creates a histogram of these durations. The CTD is then the EMD distance between the two histograms.

This comprehensive set of measures provides a powerful diagnostic toolkit, enabling analysts to pinpoint whether a BPS model's inaccuracies stem from its control-flow logic, temporal dynamics, or its handling of resources and system congestion.

Recognizing the critical role of accuracy, recent research has increasingly focused on improving the accuracy of discovered simulation models by considering multiple perspectives of the process [118, 49, 116].

1.6.4 Measuring the Generalization of Business Process Simulation Models

Beyond accuracy, another important metric for assessing the quality of Business Process Simulation (BPS) models is **generalization**. While accuracy evaluates how closely simulated behavior matches observed data, generalization captures the ability of a simulation model to reproduce the inherent variability of the real process, rather than overfitting specific observed executions. A simulation model with poor generalization may achieve high accuracy on known traces but fail to generate diverse and realistic behaviors when exposed to unseen scenarios.

To quantify generalization, **entropy** has been widely used in the literature as an indicator of variability and generalization capacity of process data [18]. Intu-

itively, higher entropy values indicate a richer and more diverse set of simulated behaviors, which is desirable for realistic and robust process simulation.

In this thesis, we focused on the **entropy of activity execution times**. Particularly, since execution times are continuous variables, we approximated their entropy by discretizing the observed simulated values into bins $b_1, \ldots, b_k$. The number of bins is determined using Sturges' rule, which defines the optimal number of bins as $k = \lceil \log_2(n) + 1 \rceil$, where n is the number of observed samples. Given this discretization, the entropy of the probability distribution p^a of execution times for an activity a is computed as

$$H(p^a) = -\sum_{i=1}^{k} p_i^a \log(p_i^a) \tag{1.8}$$

where p_i^a expresses the probability of a value being in the bin b_i.

Finally, to obtain an overall measure of generalization across the entire process execution times, we compute the average execution time entropy over all activities. Let $\mathcal{A}$ denote the set of activities in the process; the average entropy is then given by

$$\frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} H(p^a) \tag{1.9}$$

This aggregated entropy measure reflects the extent to which the BPS model is capable of generating diverse execution times across activities, and thus serves as an indicator of its generalization capability.

1.7 Event Data Used in this Thesis

Process Mining is a data-driven research field (cf. Section 1.3) and, as such, the availability of event data is a fundamental prerequisite for both methodological development and empirical evaluation. Open-source datasets have always represented a cornerstone of the field, enabling reproducibility and fair assessment of proposed techniques. Nevertheless, the availability of real-world event logs has historically been limited. In recent years, advances in information technologies and efforts from companies and public institutions have led to the release of few real-world event log datasets.

Beyond availability, the assessment of many process mining techniques requires event data to exhibit specific characteristics. Different analysis and modeling tasks impose different data requirements. For instance, the modeling of activity execution times requires the presence of both start and completion events, while the analysis of organizational perspectives relies on the availability

of resource-related attributes.

These requirements become particularly strict in the context of Business Process Simulation, where the goal is to learn multi-perspective process models from event logs. In particular, the development and evaluation of simulation techniques require event logs to contain both start and end timestamps to accurately model execution and waiting times, as well as resource information to capture resource allocations. Additional attributes, such as trace attributes, can also be employed for the modeling of predictive parameters when such information is available.

This section introduces the datasets used in this thesis to experimentally evaluate the proposed techniques. We considered eleven distinct business processes for which corresponding datasets are available and event logs can be extracted. In the following, we briefly describe each process and its associated event log:

Purchasing to Pay (P2P) It is a realistic purchasing example process with a synthetic event log provided by Fluxicon.[1] The event log contains both start and complete events for each executed activity and corresponding involved resources. It has been used for the discovery of complete simulation models (cf. Chapter 2, Chapter 4, Chapter 5, and Chapter 6)

Production (MP) This dataset represents a realistic manufacturing process.[2] The corresponding event log is used in Chapter 2 to evaluate the proposed start timestamp estimation technique.

Sepsis This dataset refers to a real-life event log extracted from an Enterprise Resource Planning (ERP) system of a regional hospital in the Netherlands.[3] A normative process model is provided in [122] and is illustrated as a Petri net in Figure 1.13. The event log and the corresponding normative model makes this dataset particularly suitable for the evaluation of control-flow process discovery and process model repair techniques (cf. Chapter 3 and Chapter 6).

Road Traffic Fines (RTF) This event log describes the process of managing road traffic fines handled by a local police force in Italy.[4] A normative process model for this process is provided in [122] and is illustrated as a Petri net in Figure 1.14. The availability of a reference control-flow model makes

[1] https://fluxicon.com/academic/material/

[2] http://doi.org/10.5281/zenodo.4699983

[3] https://doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460

[4] https://doi.org/10.4121/uuid:270fd440-1057-4fb9-89a9-b699b47990f5

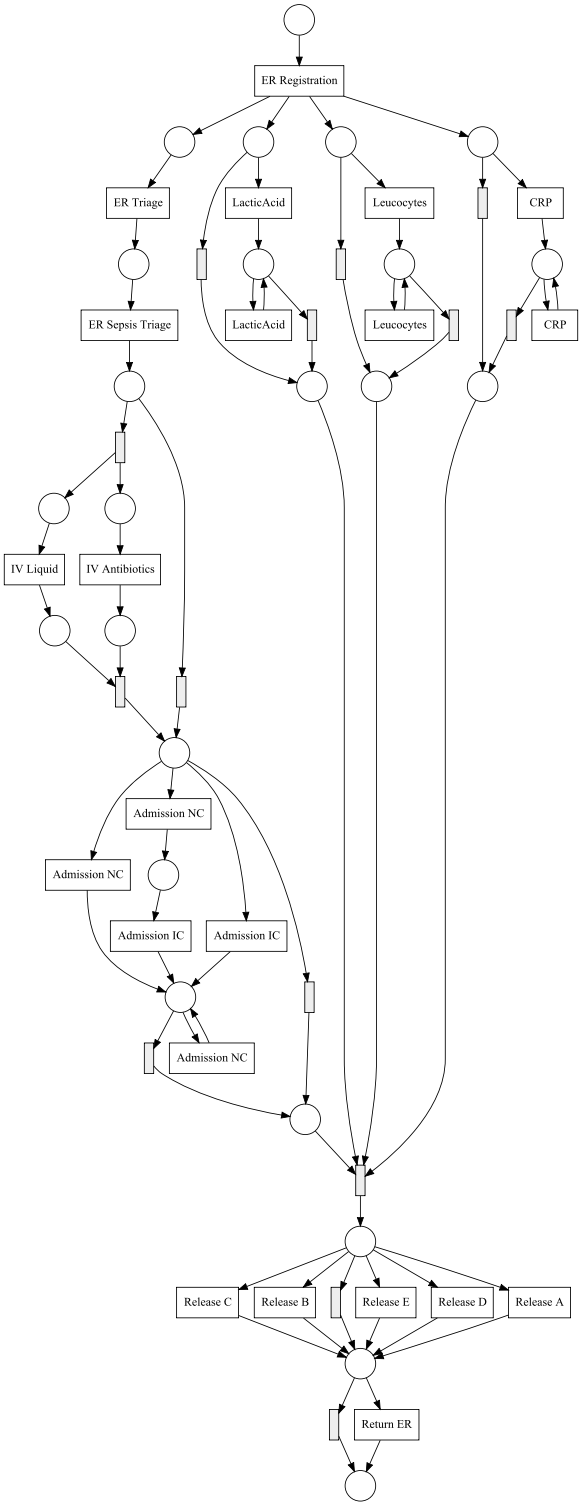


Figure 1.13: Sepsis normative Petri net model.

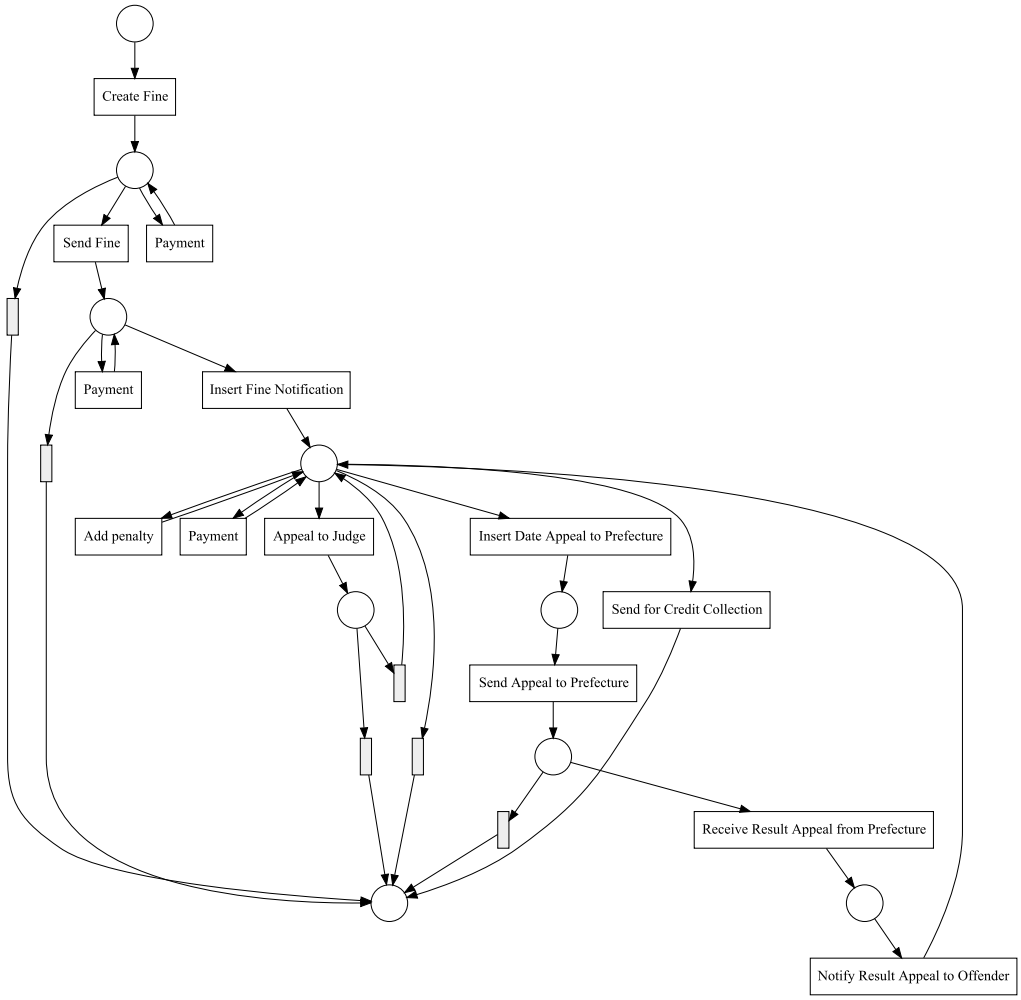


Figure 1.14: Road-Traffic Fines normative Petri net model.

this dataset well suited for the evaluation of control-flow discovery and repair techniques (cf. Chapter 3 and Chapter 6).

Hospital Billing It describes the billing process for medical services in a regional hospital in The Netherlands.[5] The normative process model from [122], depicted in Figure 1.15, and its corresponing event log have been used in Chapter 3 for the modeling of the control-flow perspective.

CVS Retail Pharmacy (CVS) This process focuses on a realistic retail pharmacy process with a synthetic event log.[6] The dataset contains both start and

[5] https://doi.org/10.4121/uuid:76c46b83-c930-4798-a1c9-4be94dfeb741

[6] http://doi.org/10.5281/zenodo.4699983

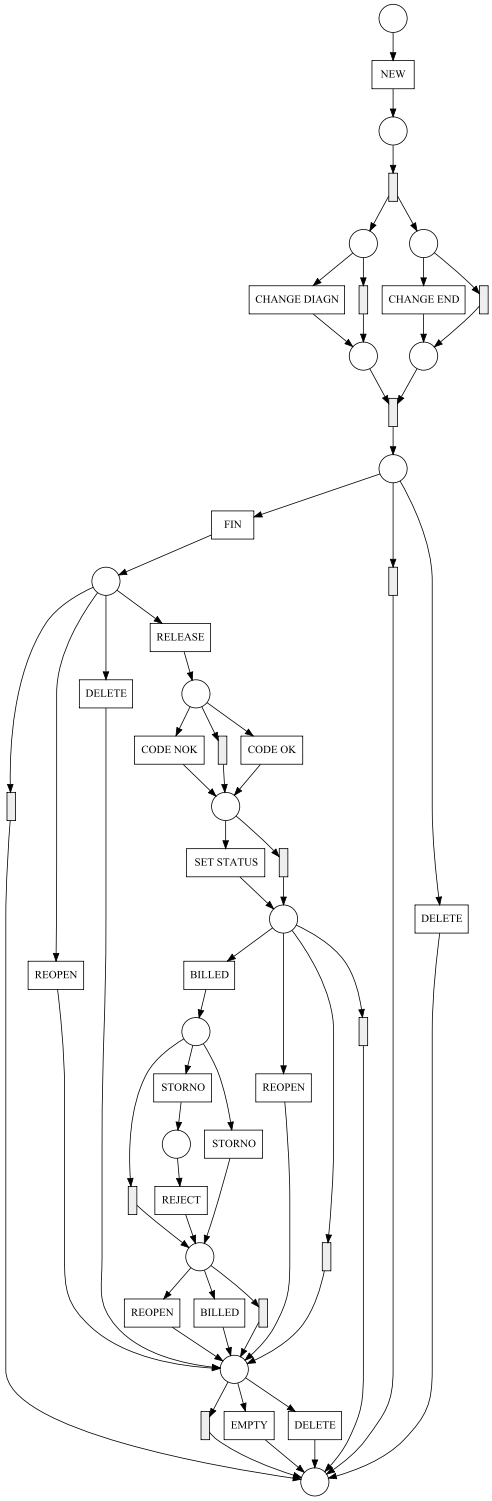


Figure 1.15: Hospital Billing normative Petri net model.

complete events for each activity, and resource information making it suitable for the discovery of complete simulation models in Chapter 4 and Chapter 5.

Academic Credential Recognition (ACR) (Consulta) This event log records the executions process at the University of Los Andes in Colombia.[7] The event log contains all necessary information for the discovery of simulation models (cf. Chapter 4). Moreover, this dataset is used for predictive and prescriptive tasks, and we used it for simulating recommendations provided by recommender systems in Chapter 8.

BPIC12 This dataset originates from the BPI Challenge 2012 and describes a real-life loan application process from a Dutch financial institution.[8] In our experiments, we considered only the subset of *workflow* relevant activities (BPIC12W). This filtered event log contains both start and complete lifecycle events and includes organizational information, making it suitable for the discovery and evaluation of complete multi-perspective simulation models (cf. Chapter 4, Chapter 5, and Chapter 6).

BPIC17 It refers to the same sub-process as BPC12W, but related to the executions in 2016, when the process underwent some significant changes.[9] For the discovery and evaluation of complete simulation models (cf. Chapter 4, Chapter 5, and Chapter 6), we employed the event log containing only the *workflow* relevant activities (BPIC17W), which provides start and complete events together with resource information. For prescriptive and recommendation experiments in Chapter 8, we used the event log filtered on the *application* (A) and *offer* (O) activities (BPIC17AO). Moreover, following the concept drift analysis reported in [7], where a an increase in the workload of resources at week 22 led to a decrease in the service times at week 28, the BPIC17AO event log was split into two temporal segments: BPIC17AO-before log containing traces up to week 22, and an BPIC17AO-after log containing traces starting from week 28. This permits a fair evaluation of techniques that do not assume concept-drift in the event log.

BPIC13 This dataset refers to the BPI Challenge 2013 and describes an incident management process from the IT department of Volvo Belgium.[10] This

[7] http://doi.org/10.5281/zenodo.4699983

[8] https://doi.org/10.4121/uuid:3926db30-f712-4394-aebc-75976070e91f

[9] https://doi.org/10.4121/uuid:5f3067df-f10b-45da-b98b-86ae4c7a310b

[10] https://doi.org/10.4121/uuid:500573e6-accc-4b0c-9576-aa5468b10cee

Table 1.2: Structural characteristics of the datasets used in the experimental evaluation, including the number of distinct activities and resources, the availability of start and completion lifecycle events, and the presence of trace-level attributes.

Dataset	N. Activities	N. Resources	Start	Trace Attributes
P2P	21	27	✓	-
MP	26	48	✓	-
Sepsis	16	-		16
RTF	11	149		5
Hospital Billing	18	1151		5
CVS	15	6	✓	-
ACR	18	561	✓	-
BPIC12W	7	60	✓	1
BPIC17W	8	149	✓	3
BPIC17AO-before	18	98		3
BPIC17AO-after	18	123		3
BPIC13	4	1440		4
BAC	15	690	✓	3

event log is commonly used for the assessment of predictive and prescriptive techniques, and to this aim we used it for simulating recommendations effects experiments in Chapter 8.

Bank Account Closure (BAC) It refers to a process of an Italian bank institution that deals with the closures of bank accounts.[11] The dataset includes a process mining log and three task mining logs; in this thesis, we relied exclusively on the process mining log. It dataset has been employed in different prescriptive analytics experimentation, and we used it for simulating process recommendations in Chapter 8.

Table 1.2 summarizes the main structural characteristics of the datasets, including the number of distinct activities and resources, the availability of start and completion timestamps for events, and the presence of trace-level attributes. These properties are crucial for assessing the suitability of each dataset for the different process mining and simulation tasks addressed in this thesis. Finally, Table 1.3 reports descriptive statistics of the corresponding event logs, providing an overview of their size and variability in terms of traces and events, as well as the distribution of the number of events per trace.

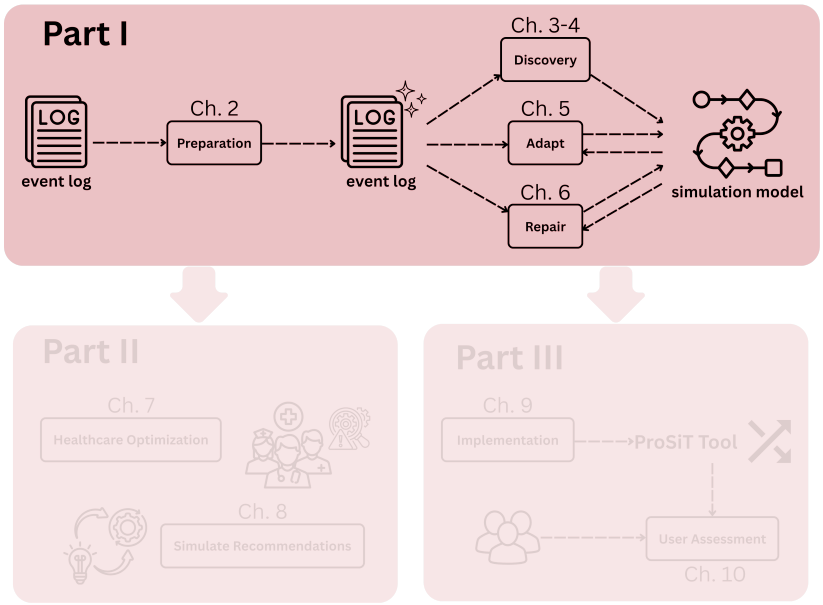
[11] https://github.com/IBM/processmining/tree/main/Datasets_usecases

Table 1.3: Descriptive statistics of the event logs, reporting the number of traces and events, together with the median, minimum, and maximum number of events per trace.

Dataset	Traces	Events	Events per trace		
			Median	Min	Max
P2P	608	18238	36	4	88
MP	225	9906	32	6	354
Sepsis	1050	15214	13	3	185
RTF	150370	561470	5	2	20
Hospital Billing	100000	451359	5	1	217
CVS	10000	103906	10	6	16
ACR	954	13740	12	6	46
BPIC12W	9658	170107	14	2	156
BPIC17W	31509	768823	21	3	151
BPIC17AO-before	9792	132767	13	7	46
BPIC17AO-after	15479	212109	13	7	50
BPIC13	7554	65533	6	1	123
BAC	116566	730336	7	1	26

Part I

Process Simulation Discovery and Refinement



The first part of this thesis focuses on the **discovery and refinement** of process simulation models. It introduces contributions that build towards more reliable, explainable, and adaptive simulations, moving from data preparation to model construction and repair.

Chapter 2 It presents a framework for **preparing event logs for process simulation discovery**, with a specific focus on repairing event timestamps. The proposed approach automatically estimates timestamps when they are missing in the event log. This lack is frequently encountered in real-world event logs, where only completion events are often recorded. However, both start and completion timestamps are essential for modeling relevant temporal aspects in process simulation, such as activity durations and waiting times. The proposed method has been applied to the case study that refers to the emergency department of an Italian hospital (see Chapter 7), published in [187].

Chapter 3 It studies the impact of different **process-state** abstractions on machine learning models for capturing the control-flow perspective of processes. The chapter illustrates how transition weights of data-aware stochastic Petri nets can be determined. It explores the influence of different abstractions of process states using interpretable models such as logistic regression and decision trees, showing that data- and history-aware abstractions can substantially increase the accuracy of simulated control-flow behavior. This contribution has been published in [113].

Chapter 4 Building on the results achieved in Chapter 3, this chapter introduces white-box **tree-based simulation models** beyond the control-flow perspective, enabling the discovery of probabilistic machine learning models. In this chapter, **transparent and configurable** machine learning models, specifically decision trees, are employed to estimate probabilities and distributions across multiple perspectives, including temporal and resource-related dimensions. Comparative experiments against state-of-the-art approaches show that this method leads to more reliable simulations preserving interpretability. The findings of this study have been published in [192].

Chapter 5 This chapter focuses on a setting in which the process behavior continuously **evolve** as new event data becomes available, effectively handling process changes and **concept drifts**. The chapter introduces an **online** methodology that combines Incremental Process Discovery techniques with online machine learning, specifically Hoeffding Adaptive Trees. The proposed approach ensures simulation models to remain accurate and stable over time, where traditional discovery techniques fail. This contribution has been published in [190].

Chapter 6 The backbone of BPS models is a Petri net. If the Petri net does not accurately model the real process, simulations are of little use because they reflect realistic process behaviors. This chapter presents a novel framework for **process model repair** that leverages process simulation and explainable AI. The framework identifies inaccuracies via explainable AI method, specifically SHAP, applied to a machine learning model trained to discriminate between real and simulated traces obtained from the input process model. Experiments show that the repaired models yield more accurate simulations. This work has been published in [188], which extends the previous contribution in [189].

Together, these chapters establish a coherent path from data preparation to adaptive and configurable simulation modeling. The works contribute to the development of accurate, interpretable, and evolving process simulation models, providing the foundation for their practical applications.

2 Repairing Event Log Timestamps for Process Simulation

In this chapter, we address the problem of **estimating missing start timestamps** in event logs, which are required by several business process simulation techniques. Building on the framework proposed by Fracca et al. [66], we introduce an alternative heuristic for tuning simulation parameters that relies solely on information available in the original event log. The proposed approach is both more accurate and significantly more computationally efficient. Experimental results on real event logs with known start timestamps show that our method outperforms the existing approach in terms of estimation quality and efficiency.

2.1 Introduction

The availability of both timestamps is essential to estimate execution and waiting times in business process simulation, enabling the construction of accurate simulation models. However, in practice, information systems often record only the completion of activities, omitting the start times. This limitation hinders the direct application of a wide range of process mining techniques that depend on complete lifecycle information [129, 63, 42].

A naive approach to reconstruct missing start timestamps assumes that each activity begins as soon as all its preceding activities are completed and the required resources are available. Such an assumption implies negligible waiting times, which is unrealistic in real-world settings where resources multitask, take breaks, or perform additional unlogged duties. As a result, estimating start timestamps remains an open challenge in process mining research.

Fracca et al. [66] proposed a framework for estimating missing start timestamps by leveraging process simulation. Their technique builds a simulation model from the available event log, generates simulated event logs by varying activity

duration parameters, and selects the simulation model whose output is closest to the original log according to specific statistical properties. The corresponding parameters are then used to estimate the start timestamps in the original log. While this method provided a solid foundation, it suffers from limitations in both computational efficiency and the precision of the estimated timestamps.

In this chapter, we extend the work of Fracca et al. [66] by introducing a new algorithm for estimating start timestamps that improves both accuracy and efficiency. This design allows the algorithm to reach better local optima and to adapt to the variability of activity behaviors across the process.

The effectiveness of the proposed technique has been evaluated using two real-life event logs containing both start and completion timestamps. By removing the start events and attempting to rediscover them, we demonstrate that our approach achieves more accurate and computationally efficient estimations than the technique of Fracca et al. [66].

2.2 Related Work

Several techniques rely on the availability of the start activity timestamps, namely the timestamps of the start events. In business process simulation complete and precise event logs, including information about completion and start of activities, is highly valuable and necessary as the starting point for many simulation parameters discovery algorithms, such as techniques for multi-perspective information extraction. For example, Martin et al. [129] present a discover technique for the resource availability calendars and in [63] a technique to detect the presence of multitasking. These techniques start from event log assuming both start and complete life-cycle transitions of activities. A complete event log is necessary as input also for certain techniques for automated discovery of business process models, such as in [105], to better distinguish between true concurrency and interleaving. The construction of a business simulation model that accurately model resources also requires the presence of the timestamps of start and completion events [42, 164]. In particular, Carmargo et al. [42] confirm the difficulty to find real-life event logs with both start and completion timestamps, thus limiting the applicability of their simulation model construction. The same applies to techniques to discover the resource availability calendars [129] or the detection of multitasking [63].

The problem of estimating the start activity timestamps is somehow related to queue mining, namely assessing the queue lengths and waiting times of process activities [24, 172]. However, queue mining uses stochastic approaches to provide probability distributions, confidence intervals, etc., without focusing

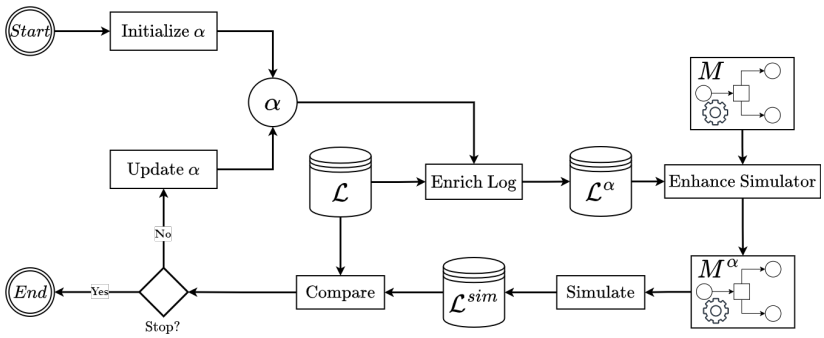


Figure 2.1: Start Timestamps Estimator schema. It starts by initializing an alpha parameter for each activity label. Then they are used for enriching the event log $\mathcal{L}$ obtaining an event log $\mathcal{L}^{\alpha}$ with start timestamps. A simulation model M is enhanced using the temporal information derived from $\mathcal{L}^{\alpha}$, and then it generates a simulated event log $\mathcal{L}^{sim}$. The simulated event log is compared with the actual one, and finally a stopping criterion determines whether to stop the procedure or update the alpha parameters and proceed in an iterative way.

on computing punctual values for each start event. Rogge-Solti and Weske use stochastic Petri nets to determine the probability distribution of the duration of process instances, but their goal is not to estimate the start-event timestamps [161]. Techniques to repair, clean, and restore event data before analysis have been suggested in other works: classical trace alignment algorithms are used to restore missing events but without restoring their timestamps[45]. In [56], Denisov et al. propose an alternative technique to restoring missing timestamps, under the strong assumption, which does not generally hold, that the model is acyclic and contains no parallelism constructions (e.g., AND splits). Pegoraro et al. have proposed a repertoire of process mining techniques over missing event data, but they have aimed at process discovery and conformance checking [147, 148], without focusing on repairing event logs and to adding the missing start events and their respective timestamps.

2.3 Method

The starting points of the method are an event log $\mathcal{L}$ defined over a set of activities $\mathcal{A}$, and a simulation model M. To simplify the discussion, we assume that all start timestamps are missing. The application is naturally straightforward if certain timestamps are present.

To estimate the starting timestamp of an event $e \in \mathcal{L}$, we formulate the problem as finding a value $\alpha\,(a) \in [0, 1]$, where $a = act(e)$. Particularly, the starting event timestamp $t_{start}(e) \in \mathcal{T}$, where $\mathcal{T}$ is the set of timestamps, can be parameterized

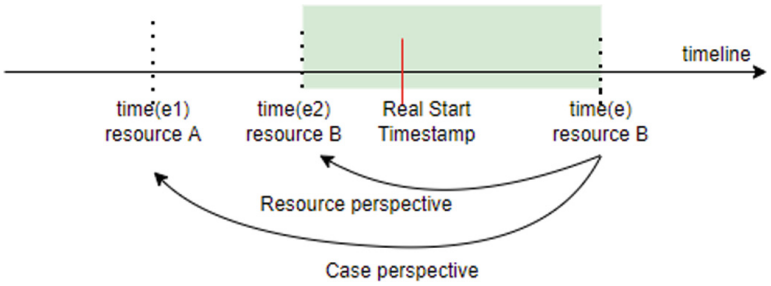


Figure 2.2: Schema representing the concepts of previous event in a trace by control-flow and previous event performed by a resource. For the event e, $time(e)$ is the timestamp related to the event e, and $res(e) = resource\ B$ the resource performing the activity $act(e)$. Looking at the trace perspective, e_1 is the previous event in the trace by control-flow, and e_2 the previous event performed by $resource\ B$. The green box represents the timeline range in which the event e could be started. Source: [66].

as:

$$t_{start}(e) = \alpha\,(a) \cdot mintime\,(e) + (1 - \alpha\,(a)) \cdot time\,(e) \tag{2.1}$$

where $a = act(e)$, and $mintime(e)$ is the minimum timestamp when event e could have started. We estimate the minimum starting timestamp $mintime(e)$ for an event e as follows:

Definition 14 (Minimum Starting Timestamp)**.** *Let $\mathcal{L}$ be an event log. Given a trace $\sigma = \langle e_1, \ldots, e_n \rangle \in \mathcal{L}$, the minimum starting timestamp of e_i with $1 \leq i \leq n$ is defined as*

$$mintime_{\mathcal{L}}\,(e_i) = \begin{cases} time^{\mathcal{L}}_{prev}\,(e_i) & \text{if } i = 1 \\ \max(time(e_{i-1}), time^{\mathcal{L}}_{prev}(e_i)) & \text{otherwise} \end{cases} \tag{2.2}$$

where $time^{\mathcal{L}}_{prev}\,(e)$ denotes the timestamp of the most recent event in $\mathcal{L}$, preceding e, that was performed by the same resource $res(e)$:

$$time^{\mathcal{L}}_{prev}\,(e) = \max_{\bar{e} \in \mathcal{L}} \{time(\bar{e}) \mid time(\bar{e}) < time(e) \wedge res(\bar{e}) = res(e)\} \tag{2.3}$$

The intuition is to consider both the control flow and the resource perspective. We hypothesize that an event can only start after the previous one has completed and that the resource performing the activity cannot be engaged in multiple activities simultaneously. These assumptions are legitimate for many processes, for example in an healthcare process, patients and resources (such as doctors and nurses) hardly perform activities in parallel (cf. Chapter 7). Figure 2.2 illustrates this hypothesis.

The goal is to determine the optimal set of parameters $\alpha\,(a)$ for all $a \in \mathcal{A}$. The method is summarized by the schema illustrated in Figure 2.1. It starts with an

initial configuration $\alpha = \{\alpha(a) \mid a \in \mathcal{A}\}$ enriching the event log $\mathcal{L}$ with estimated start events, given using Equation 2.1, resulting in an event log $\mathcal{L}^{\alpha}$. This is then used to enhance the given process simulation model M with activity execution time distributions. The simulator is subsequently run to generate a simulated event log $\mathcal{L}^{sim}$. Finally, $\mathcal{L}^{sim}$ is compared with $\mathcal{L}$, and the set α is adjusted to minimize defined errors. The procedure is repeated for a number of iterations or until a given stopping criterion is reached.

Specifically, the comparison between the simulated and real event log, is based on the inter-event time distributions, i.e. the time between two consecutive events with complete timestamps.

Definition 15 (Inter-Event Time)**.** *Let $\mathcal{L}$ be an event log, with $\mathcal{E}$ the set of all events in $\mathcal{L}$, i.e. $e \in \mathcal{E}$ iff $e \in \mathcal{L}$. We define the inter-event time function $\Delta_I : \mathcal{E} \to \mathbb{R}^+$ s.t. where given a trace $\sigma = \langle e_1, \ldots, e_n \rangle \in \mathcal{L}$, for all $1 \leq i \leq n$:*

$$\Delta_I(e_i) = \begin{cases} 0 & \text{if } i = 1 \\ time(e_i) - time(e_{i-1}) & \text{otherwise} \end{cases} \tag{2.4}$$

where $time(e_i) - time(e_{i-1})$ represents the duration time between event e_i and e_{i-1}.

These durations are used to estimate the following distribution functions for each activity in an event log:

Definition 16 (Activity Inter-Event Time Distribution)**.** *Let $\mathcal{L}$ be an event log defined over a set of activities $\mathcal{A}$. We define the inter-event time distribution function $\Phi_{\mathcal{L}} : \mathcal{A} \to (\mathbb{R}^+ \to [0, 1])$ where $\Phi_{\mathcal{L}}(a)$ indicates distribution of inter-event times $\Delta_I(e)$ where $e \in \sigma$ s.t. $act(e) = a$, for all $\sigma \in \mathcal{L}$.*

In practice, $\Phi_{\mathcal{L}}(a)$ is not assumed to be known in a closed form. Instead, it is empirically estimated from the finite set of observed inter-event times $\Delta_I(e)$ associated with activity a. Notably, we decided to rely on this metric since it can be computed directly on the original event log, in the absence of start timestamps. This allows for a coherent comparison between the simulated and real event logs from a temporal perspective.

We then define the errors to minimize the Wasserstein distance [154] between the activity inter-event time distributions (see Definition 16) of the original event log and the simulated one for each activity $a \in \mathcal{A}$:

$$\epsilon_a(\mathcal{L}, \mathcal{L}^{sim}) = \int_0^{+\infty} \left| \Phi_{\mathcal{L}}(a)(x) - \Phi_{\mathcal{L}^{sim}}(a)(x) \right| \, dx \tag{2.5}$$

The Wasserstein distance quantifies the minimum amount of "mass" that must be transported to transform one probability distribution into another, taking

Algorithm 1 Start Timestamp Estimator

Require: $\mathcal{L}$: event log; M: simulator model
1: **for** $a \in \mathcal{A}$ **do**
2: $l[a] \leftarrow 0$
3: $\alpha[a] \leftarrow l[a]$
4: **end for**
5: $\mathcal{L}^{sim} \leftarrow \texttt{simulate}(M, \mathcal{L}, \alpha)$
6: **for** $a \in \mathcal{A}$ **do**
7: $err[a][l] \leftarrow \epsilon_a(\mathcal{L}, \mathcal{L}^{sim})$
8: **end for**
9: **for** $a \in \mathcal{A}$ **do**
10: $r[a] \leftarrow 1$
11: $\alpha[a] \leftarrow r[a]$
12: **end for**
13: $\mathcal{L}^{sim} \leftarrow \texttt{simulate}(M, \mathcal{L}, \alpha)$
14: **for** $a \in \mathcal{A}$ **do**
15: $err[a][r] \leftarrow \epsilon_a(\mathcal{L}, \mathcal{L}^{sim})$
16: **end for**
17: **while** `StoppingCriterion` **do**
18: **for** $a \in \mathcal{A}$ **do**
19: $\alpha[a] \leftarrow (l[a] + r[a])/2$
20: **end for**
21: $\mathcal{L}^{sim} \leftarrow \texttt{simulate}(M, \mathcal{L}, \alpha)$
22: **for** $a \in \mathcal{A}$ **do**
23: $\overline{\alpha} \leftarrow argmin(err[a][l], err[a][r])$
24: **if** $\overline{\alpha} = l[a]$ **then**
25: $r[a] \leftarrow \alpha[a]$
26: $err[a][r] \leftarrow \epsilon_a(\mathcal{L}, \mathcal{L}^{sim})$
27: **else**
28: $l[a] \leftarrow \alpha[a]$
29: $err[a][l] \leftarrow \epsilon_a(\mathcal{L}, \mathcal{L}^{sim})$
30: **end if**
31: **end for**
32: **end while**
33: **return** α

into account both the magnitude and the location of the differences between distributions. In practice, this measure is computed from the finite samples of observed inter-event times.

To find the optimal set of α parameters, we employ a local search-based algorithm, as detailed in Algorithm 1. The algorithm iteratively refines the interval $[0, 1]$ for each $\alpha(a)$, progressively halving it and moving towards the extremity that corresponds to the minimum error until a certain stopping criterion, such as a number of iterations or until the errors do not improve.

In Algorithm 1, for each activity a, the left and right bounds of the search interval, denoted by $l[a]$ and $r[a]$ are initialized to 0 and 1, respectively (see lines

1-4 and 9-12), respectively. The function `simulate`$(M, \mathcal{L}, \alpha)$ is first invoked using the initial configurations corresponding to the left and right bounds (lines 5 and 13), producing simulated event logs that are compared with the original log $\mathcal{L}$. The resulting errors, computed using $\epsilon_a(\mathcal{L}, \mathcal{L}^{sim})$, are stored in $err[a][l]$ and $err[a][r]$ (lines 6-8 and 14-16).

The algorithm then iteratively refines the interval (lines 17-32). At each iteration, the midpoint $(l[a] + r[a])/2$ is selected as the current estimate for $\alpha[a]$ (lines 18-20), and a new simulated log is generated (line 21). For each activity, the function $argmin(err[a][l], err[a][r])$ identifies the bound associated with the smaller error (line 23). The interval is subsequently updated by replacing the opposite bound with the midpoint, and the corresponding error value is recomputed (lines 24–30).

The procedure terminates when the stopping criterion is satisfied, e.g., when the improvement between successive iterations falls below a given threshold, and returns the resulting parameter configuration α (lines 31–33).

The novel contribution of our approach with respect to the method proposed by Fracca et al. [66] lies in both the formulation of the optimization problem and the search strategy employed to estimate the parameters α. Specifically, the key distinction of our method lies in the optimization function (see Equation 2.5), which is defined at the activity level, thus tailoring the estimation process to the temporal behavior of each activity. Moreover, unlike the approach of Fracca et al. [66], where the parameters $\alpha(a)$ are optimized sequentially, i.e., one activity at a time, our method updates them in parallel. Moreover, our search strategy improves exploration of the parameter space by avoiding the dependence on a defined granularity parameter, which constrained the search around a fixed starting point in [66]. Instead, our algorithm progressively halves the search interval $[0, 1]$ for each activity parameter, following a binary search strategy. This allows for a more efficient exploration of the search space and facilitates convergence towards better local optima.

2.4 Evaluation

In this section, we assess the quality of the approach, evaluating it on two case studies for which start timestamps are present in the event logs. In particular, we removed the start events in the event-logs, and we used our technique to generate them. The evaluation is based on comparing the actual ones with those obtained using our technique.

We used Prosimos [117] as a simulator engine, integrating it into our imple-

Table 2.1: Start Timestamp Estimator results for each case study and method. The column "MAE (mins)" indicates the Mean Absolute Error (in minutes) between the estimated and the actual timestamps. The column "Comp. Time (mins)" refers to the computational time. Best results are highlighted in bold.

Case Study	Method	MAE (mins.)	Comp. Time (mins)
P2P	Our	**184.24**	**2.03**
	[66]	316.14	8.18
MP	Our	**218.64**	**1.39**
	[66]	374.74	8.54

mentation.[1] The method has been developed in Python and is available in an online repository.[2]

The evaluation has been conducted on two distinct case studies with available event logs: a synthetic purchasing example process (P2P) and a manufacturing production process (MP) (cf. Section 1.7 for further details).

We compared the starting timestamps obtained by our framework with the actual ones computing the Mean Absolute Errors (MAE) in time units between all the events. To mitigate the stochasticity inherent in simulations, we ran the experiments 30 times and averaged the results. Table 2.1 reports the outcomes obtained using our approach comparing them with those obtained by Fracca et al. [66].

In addition to MAE, we also evaluated the computational time. Both methodologies were executed on the same machine to ensure a fair comparison, showing that our methodology yields better results and significantly shorter computing time. Specifically, we used an equipment with an AMD Ryzen 7 PRO 3700U w/ Radeon Vega Mobile Gfx 2.30 GHz and 16 GB RAM.

In the P2P case study, our method achieved a 41.72% reduction in MAE and a 75.18% decrease in computational time. Similarly, MP reveals a decrease in MAE of 41.66% and an improvement in computational time of 83.72%. These results suggest the consistency and efficiency of our method.

2.5 Conclusion

This chapter presented a novel approach for estimating missing start timestamps in event logs, extending the framework introduced by Fracca et al. [66]. The accurate reconstruction of start timestamps is essential for process simulation, as well as for other process mining techniques, where both start and completion

[1] https://github.com/AutomatedProcessImprovement/Prosimos

[2] https://github.com/franvinci/StartTimestampEstimator

times are required to compute activity durations and waiting times. Our method addresses the limitations of existing techniques that assume activities start immediately after their predecessors complete, which often leads to inflating the execution times.

The proposed approach leverages a simulation-based optimization framework to iteratively refine the estimation of start timestamps. Each activity is associated with a parameter $\alpha(a)$ that regulates the interpolation between its earliest possible starting time and its recorded completion time. These parameters are optimized by minimizing the discrepancy between the real and simulated event logs, measured through the Wasserstein distance computed over the inter-event time distributions.

Our method extends the original approach by Fracca et al. [66]. First, the optimization function is defined at the activity level, capturing activity-specific temporal patterns and improving estimation accuracy. Second, the parameters are optimized in parallel rather than sequentially, thus reducing computational time. Finally, the search for optimal parameters employs a binary search strategy that iteratively halves the search interval, ensuring efficient convergence toward better local optima.

The efficacy and accuracy of our approach suggest its potential usability for enriching event logs with missing timestamps, a common challenge in many real-world information systems. The current proposed framework relies on the simplifying assumption that activities occur sequentially without considering external delays or multitasking effects. While this assumption remains reasonable in many structured processes, such as in organizational healthcare processes (cf. Chapter 7), more complex temporal dependencies may play a significant role in other domains. Future research could therefore extend the framework by integrating advanced simulation techniques capable of explicitly capturing temporal dependencies and external delays between activities, enabling a more general and realistic reconstruction of event timestamps. Future work could also study the sensitivity of the method to the accuracy of the simulation model. Alternative optimization strategies could also be explored; for example, genetic algorithms [195] may provide a robust approach to estimating activity-specific parameters. Furthermore, the evaluation was conducted on only two event logs, which limits the generalizability of the results; validating the approach on a broader and more diverse set of real-world event logs remains an avenue for future work.

3 History- and Data-Aware Control-Flow Discovery

In this chapter, we address the problem of how activity choices should be modeled in business process simulation in order to faithfully reproduce real process behavior. In particular, we focus on the problem of determining which activity fires next when multiple alternatives are possible. The chapter introduces and empirically evaluates a simulation approach in which **activity firing probabilities** are conditioned on the current execution context of a case. The proposed approach is assessed on multiple real-life processes, demonstrating its impact on the accuracy of simulation outcomes.

3.1 Introduction

A successful application of business process simulation for process improvement relies on a simulation model that reflects the real process behavior; conclusions drawn on an unrealistic simulation model lead to process redesigns that may not yield improvements, or even may worsen the performances (cf. Section 1.6.3).

To produce meaningful insights, the simulation must capture various run-time aspects of the process. A critical aspect is how the next activity is determined when multiple activities are enabled. Relying solely on fixed branching probabilities introduces a coarse and static decision logic that does not account for the variability observed in real executions. In practice, the likelihood of activities to occur may vary as function of the characteristics of the process instances, the activities that previously occurred in the same process instance, time-related information, etc. Consider, for example, a loan application process: the probability of executing an activity about a thorough assessment grows, e.g., with the amount of the requested loan, and decreases with annual salary of the applicant. Also, the probability of rejecting an application may grow with the number of

requests to the applicant of providing further documents.

The concept of **process state** (cf. Definition 13) provides a faithful abstraction of a process case. In this chapter, we investigate how an appropriate definition of this abstraction can improve the estimation of activity firing probabilities in simulation models, going beyond simple branching probabilities. A more informative and well-defined process state enables more accurate modeling of control-flow behavior, ultimately leading to simulation models that better reproduce the dynamics observed in real processes.

In order to answer this research question, the work builds upon Petri nets, and discovers a so-called **weight function** for each transition (cf. Section 1.3.2), which is defined over a process state, which, e.g., can include *process variables*, and/or the *transition-firing history*. Then, for the cases that are simulated, the current process state is computed and used to evaluate the weight function of the transitions that are enabled at that point. The probability to fire a transition is thus obtained as the ratio of its weight and the sum of the weights of all enabled transitions. It follows that the higher is the weight of a transition, the higher is the probability of that transition to fire. Since the case characteristics that may influence the probability may depend on the specific process that is being simulated, we propose a framework where the process-state abstraction can be customized to include or exclude certain characteristics.

The weight function can be discovered using several machine learning algorithms. In this work, we focus on logistic regression and decision trees, as these models provide a high degree of interpretability. However, the framework is customizable and can incorporate alternative models depending on the desired trade-off between accuracy, interpretability, and computational effort.

The research question is finally answered by applying the aforementioned framework to three real-life processes, each with a real-life event log. For each process, five different definitions of process states have been considered to compute weights of the Petri net models of the two processes to be simulated. The results show that the simulation using models with transition probabilities based on process states allows obtaining simulation results that are more accurate, if compared with simulation models based on simple frequency-based probabilities.

3.2 Related Work

The determination of the activity probabilities in simulation model is related to **stochastic process discovery** (cf. Section 1.3.3). However, these works [39, 160, 123] do not focus on building simulation models, nor do they verify whether

or not their approaches are beneficial to increase the accuracy of business simulation models. In particular, Burke et al. use a similar concept of weights to determine probabilities [39], but, instead of using a customizable definition of process state, they only consider the occurrences of previous activities as process state, which our experiments show to not always be beneficial. Mannhardt et al. also use weights to determine probabilities [123], but they only focus on process attributes, which is similarly not always beneficial for more accurate simulation models. This chapter generalizes the works by Burket et al. [39] and by Mannhardt et al. [123], allowing both process-state abstractions, their combination, and also richer abstraction (e.g., including resource and time information).

Related work also focuses on Markov-based stochastic models [19, 161]. This class of models is, however, unable to feature concurrency, data and silent transitions, which are all crucial aspects of business process models, including when used for simulation.

On the other hand, several studies in the simulation domain have addressed the discovery and enhancement of simulation models. More recently, Meneghello et al. [136] and Orlenys et al. [118] have introduced techniques to derive decision rules for branching conditions in process models. However, these approaches exclusively rely on decision trees and only consider process attributes to be leveraged when discovering such rules. Moreover, these approaches determine decision rules that are only local to exclusive-choice constructs, i.e. XOR splits, while we aim to general rules that can also prioritize transitions in parallel branches, namely giving higher likelihood to certain inter-leavings of parallel transitions (see, e.g., Figure 3.1).

3.3 Framework for Determination of Transition Weights

This section introduces a framework that, given an event log $\mathcal{L}$ and a Petri net $N = \big(P, T, F, \lambda, m^I, m^F\big)$, can be used to determine the weights $w : T \to (\Delta \to \mathbb{R}^+)$ of the transitions, with $\Delta \subset \mathbb{R}^n$, thus transforming the Petri net into a stochastic Petri net $\big(P, T, F, \lambda, m^I, m^F, w\big)$.

The framework uses the concept of **process state** as defined in Definition 13. A process-state function can be customized according to the specific domain. For instance, if one wants to account for the trace attributes in the set $\mathcal{V}$, the set $\Delta \subset \mathbb{R}^n$ of possible process states consists of tuples $(x_1, \ldots, x_n)$ where x_i is the value assigned to variable $v_i \in \mathcal{V}$.

The framework can be instantiated for a process-state function $\mathcal{S}_\Delta$, generalizing the proposal in [123], and a parameterized weight function w, and consists of three steps:

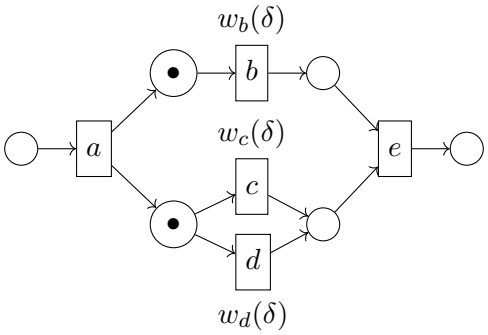


Figure 3.1: Example of a Stochastic Labelled Petri net. The black dots (•) represent the current marking. Transitions of the enabled transitions are annotated with weight functions $w_t = w(t) \colon \Delta \to \mathbb{R}^+$ depending on the current data state $\delta \in \Delta$.

Step 1 For each trace $\sigma \in \mathcal{L}$, we repair σ with respect to the model N, thus obtaining a Petri net run $\langle e_1, \ldots, e_m \rangle$ (cf. Section 1.3.4). For reliability, it is indeed necessary to ensure replayability.

Step 2 For each prefix $\langle e_1, \ldots, e_k \rangle$, with $1 \leq k \leq m-1$, of each Petri net run, we compute the corresponding process state $\delta = \mathcal{S}_\Delta(\langle e_1, \ldots, e_k \rangle)$. We then add δ to the set Δt^+ associated with the transition $t \in T$ fired by event e_{k+1}. Moreover, for every transition $t \in T$ that is enabled after the prefix but not fired, we add δ to the corresponding set Δ_{t^-}. From these, for each transition $t \in T$, we construct a correspongint training set of labeled samples:

$$\mathcal{T}_t^\Delta = \biguplus_{\delta \in \Delta_{t^+}} [(\delta, 1)] \cup \biguplus_{\delta \in \Delta_{t^-}} [(\delta, 0)]$$

Step 3 For each transition $t \in T$, we train a machine learning model on the corresponding labeled training set $\mathcal{T}_t^\Delta$ to learn its parameterized weight function $w(t)$. Given a process state δ, the learned model returns a probability estimating the likelihood that transition t fires in state δ. The training objective is to assign higher probabilities to states in which t was observed to fire (positive samples) and lower probabilities to states in which t was enabled but not fired (negative samples). Since the weights in a stochastic Petri net are not required to sum to 1 across enabled transitions, each transition-specific model can be trained independently of the others.

Example: As an example, we instantiate our framework with a process state function $\mathcal{S}_\Delta$ that takes into account (i) the trace attribute values and (ii) the history of the activity occurrences. Given a Petri net run $\langle e_1, \ldots, e_n \rangle$ with trace attributes $\mathcal{V}$ assuming values in $\mathcal{U} \subset \mathbb{R}^m$, we define $\mathcal{S}_\Delta$ as:

$$\mathcal{S}_\Delta(\langle e_1, \ldots, e_n \rangle) = (v \mapsto u, [\lambda(t_1), \ldots, \lambda(t_k)]).$$

with $t_i, \ldots, t_k$, $k \leq n$, being the non-silent transitions in the run σ. The set of data state is thus defined as $\Delta = (\mathcal{V} \to \mathcal{U}) \times \mathbb{B}(\mathcal{A})$.

Assume the example trace σ, with sequence of activities $\langle a, d, d, b, e \rangle$ and one single trace attribute v taking value 3. Aligning σ in Step 1 to the model shown in Figure 3.1, we obtain:

Log	a	d	d	b	e
Model	a	d	$\gg$	b	e

In Step 2, we then construct the observations:

$$\begin{aligned}
\Delta_{a^+} &= [(v \mapsto 3, [])] & \Delta_{a^-} &= [] \\
\Delta_{b^+} &= [(v \mapsto 3, [a, d])] & \Delta_{b^-} &= [(v \mapsto 3, [a])] \\
\Delta_{c^+} &= [] & \Delta_{c^-} &= [(v \mapsto 3, [a])] \\
\Delta_{d^+} &= [(v \mapsto 3, [a])] & \Delta_{d^-} &= [] \\
\Delta_{e^+} &= [(v \mapsto 3, [a, b, d])] & \Delta_{e^-} &= []
\end{aligned}$$

thus obtaining training sets:

$$\begin{aligned}
\mathcal{T}_a^\Delta &= [((v \mapsto 3, []), 1)] \\
\mathcal{T}_b^\Delta &= [((v \mapsto 3, [a, d]), 1), ((v \mapsto 3, [a]), 0)] \\
\mathcal{T}_c^\Delta &= [((v \mapsto 3, [a]), 0)] \\
\mathcal{T}_d^\Delta &= [((v \mapsto 3, [a]), 1)] \\
\mathcal{T}_e^\Delta &= [((v \mapsto 3, [a, b, d]), 1)]
\end{aligned}$$

Finally, in Step 3, after collecting the complete training sets $\mathcal{T}_t^\Delta$ for all transitions $t \in T$ by aggregating observations from all traces in the event log, we train parametric machine learning models over these sets to obtain the corresponding weight functions $w_t = w(t) \colon \Delta \to \mathbb{R}^+$.

In this work, we employ **logistic regression** and **decision tree** models, as they provide white-box explanations that can be seamlessly integrated as weights. Since most machine learning models require numerical input, we first transform the multiset representation of activity occurrences into a numeric feature vector by introducing one variable per activity in the process model. Likewise, categorical attributes are encoded using one-hot encoding. Once this feature transformation is applied, we fit the models and extract their parameters, such as coefficients in the case of logistic regression or decision rules for decision trees. These learned parameters are then used to instantiate the stochastic Petri net with the derived weight functions.

3.4 Experiments

The goal of the experiments is to assess how closely the event logs generated through simulation align with the original logs. The proposed framework is implemented in Python as part of the ProSiT library (cf. Chapter 9). We conducted experiments on three real-life case studies, each associated with a normative Petri net model, and evaluated five different configurations of process state representations, combining process attribute values with activity history.

We compare our approach against a baseline that relies on fixed branching probabilities, and we study the impact of different machine learning techniques by comparing logistic regression with decision tree models as parameterized weight functions.

3.4.1 Experimental Setup

The approach has been evaluated in three different case studies for which a public event log and a normative model is available: Sepsis, Road Traffic Fines (RTF), and Hospital Billing (see Section 1.7 for further information).

The assessment is based on comparing the similarity of the original event logs with those obtained via simulation. The simulation models are constructed using five different characterization of the process state: with *data* only, namely using the values of the process attributes; with *history* only, namely using the number of occurrences of process activities; and with *data and history*, as well as with combinations of so-called *binary history*. History is also delineated in the binary version, where we only consider whether or not an activity has occurred irrespectively of the number of occurrences.

As a baseline of comparison, we built the simulation model using branching probabilities looking at the frequency of transition firings. The comparison of the original event log and those obtained via simulation is computed through

the 2GD and 3GD (cf. Section 1.6.3). These measures consider the stochastic characteristics of the event log control-flow: which activities are executed in which order, and how often a particular order of the execution of activities is observed.

To evaluate our approach, we divided the original logs into training and test sets using a temporal split: 80% for the training set and 20% for the test set. The training set was used to compute the weights of the corresponding models and also the branching probabilities of the comparison baseline. Moreover, during simulation, we sampled process attributes from those observed in the training set. By comparing the results, we can determine the effectiveness of our approach and assess the impact of different processing techniques on performance.

For decision tree models, we performed a k-fold cross-validation on the training set to determine the optimal tree depth within the range of 1 to 5. This procedure ensures that the selected model achieves a good balance between predictive performance and generalization. In particular, decision trees with depth greater than 5 may become too complex and difficult to interpret.

The test set is used to compute the evaluation metrics, namely 2GD and 3GD, for the different configurations. For each configuration, we perform five independent simulations, each generating the same number of traces as the test set, ensuring a fair and robust comparison. We then report the average and standard deviation of the results across the five runs, providing a comprehensive assessment of both the accuracy and reliability of the simulation performance.

3.4.2 Experimental Results

For each case study, we have then computed 11 simulation models, using probabilities based on simple frequencies and on the weights with five different process-state abstractions, and two possible machine learning models, i.e. logistic regression and decision tree. Each simulation model has been run five times, to mitigate the stochasticity of simulation.

Table 3.1 reports the results obtained for each case study and configuration. Specifically, it shows the average distance between the simulated and real behavior, along with the standard deviation, measured using the 2GD and 3GD metrics. Lower values indicate a higher fidelity of the simulated logs with respect to the real process. The best results are highlighted in bold with a green background, while the worst results are shown with a red background. We also report on the baseline configuration, which relies solely on simple branching probabilities.

For the Sepsis case study, the best-performing configuration (*Data & History* with *Decision Tree*) achieves a 2GD of 0.1983 and a 3GD of 0.2974, representing

Table 3.1: Average ($\pm$ standard deviation) 2GD and 3GD distances between simulated and real behavior across different process-state abstractions and machine learning models for each case study. Best and worst results are highlighted in bold with green and red backgrounds, respectively. The last row of each case study reports the baseline using simple branching probabilities.

Case Study	Configuration	Model	2GD	3GD
Sepsis	Data	Logistic Regression	0.3396 ± 0.0099	0.4728 ± 0.0141
		Decision Tree	0.3414 ± 0.0067	0.4690 ± 0.0126
	Binary History	Logistic Regression	0.2651 ± 0.0055	0.3836 ± 0.0075
		Decision Tree	0.2496 ± 0.0021	0.3592 ± 0.0049
	History	Logistic Regression	–	–
		Decision Tree	0.2375 ± 0.0126	0.3555 ± 0.0159
	Data & Binary History	Logistic Regression	0.2449 ± 0.0073	0.3508 ± 0.0089
		Decision Tree	0.2338 ± 0.0069	0.3319 ± 0.0114
	Data & History	Logistic Regression	0.2302 ± 0.0094	0.3423 ± 0.0126
		Decision Tree	$\mathbf{0.1983 \pm 0.0026}$	$\mathbf{0.2974 \pm 0.0068}$
	Frequencies		0.3558 ± 0.0112	0.4907 ± 0.0124
RTF	Data	Logistic Regression	0.2675 ± 0.0015	0.3663 ± 0.0018
		Decision Tree	0.2682 ± 0.0007	0.3679 ± 0.0012
	Binary History	Logistic Regression	0.0897 ± 0.0015	0.1039 ± 0.0015
		Decision Tree	0.0920 ± 0.0014	0.1072 ± 0.0017
	History	Logistic Regression	$\mathbf{0.0863 \pm 0.0014}$	$\mathbf{0.1010 \pm 0.0013}$
		Decision Tree	0.0898 ± 0.0012	0.1074 ± 0.0013
	Data & Binary History	Logistic Regression	0.0896 ± 0.0021	0.1039 ± 0.0022
		Decision Tree	0.0911 ± 0.0028	0.1043 ± 0.0032
	Data & History	Logistic Regression	0.0868 ± 0.0021	0.1016 ± 0.0019
		Decision Tree	0.0886 ± 0.0012	0.1054 ± 0.0012
	Frequencies		0.2703 ± 0.0007	0.3714 ± 0.0009
Hospital Billing	Data	Logistic Regression	0.1583 ± 0.0013	0.1813 ± 0.0015
		Decision Tree	0.1614 ± 0.0012	0.1837 ± 0.0012
	Binary History	Logistic Regression	$\mathbf{0.1553 \pm 0.0023}$	$\mathbf{0.1765 \pm 0.0025}$
		Decision Tree	0.1577 ± 0.0018	0.1789 ± 0.0020
	History	Logistic Regression	0.1580 ± 0.0022	0.1792 ± 0.0023
		Decision Tree	0.1576 ± 0.0015	0.1787 ± 0.0013
	Data & Binary History	Logistic Regression	0.1572 ± 0.0019	0.1796 ± 0.0019
		Decision Tree	0.1574 ± 0.0009	0.1786 ± 0.0011
	Data & History	Logistic Regression	0.1573 ± 0.0031	0.1793 ± 0.0031
		Decision Tree	0.1574 ± 0.0007	0.1790 ± 0.0007
	Frequencies		0.1803 ± 0.0017	0.2176 ± 0.0017

a relative reduction of approximately 44% and 39%, respectively, compared to the baseline using simple frequencies. Models incorporating *Binary History* or combinations of *Data* and *History* consistently outperform the solely data-based configurations, confirming the importance of capturing sequential dependencies. *Logistic Regression* performs slightly worse than *Decision Trees* in most configurations, likely due to its limited capacity to model non-linear dependencies between process attributes and historical context. Notably, results for the *History* configuration with *Logistic Regression* are not available. This occurs because, for certain transitions, the model assigns probabilities approaching one as

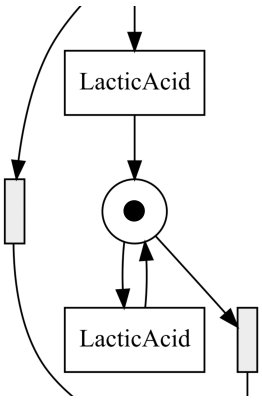


Figure 3.2: Example of a livelock in the normative Sepsis Petri net model (see Figure 1.13) when using logistic regression weight functions. As the execution history of *LacticAcid* grows, the logistic regression model assigns increasingly high firing probabilities to this transition, eventually leading the simulation towards repeatedly this behavior and preventing reaching the final marking.

the history of that transition grows, effectively creating a self-loop that prevents the simulation from progressing. This is due to the monotonic nature of logistic regression: for certain transitions, the predicted firing probability increases monotonically when the history of that transition increases and can approach 1. As a consequence, the simulation may enter a form of livelock, repeatedly favoring the same enabled behavior and preventing the execution of alternative transitions required to reach the final marking. Figure 3.2 shows an example of this situation in the Sepsis Petri net. Decision trees do not exhibit this behavior, as their discrete, rule-based structure produces probability assignments that remain fixed within a given rule, avoiding the monotonic effects observed with logistic regression.

For the RTF case study, the process data provide limited additional information for improving performance. The best results are achieved with the *History* configuration using *Logistic Regression*, corresponding to a reduction of approximately 68% and 73%, respectively, compared to the baseline with simple frequencies. *Logistic Regression* slightly outperforms *Decision Trees*, although the differences are small and the overall results are comparable across the two models. Configurations that combine data attributes with history do not yield substantial additional improvements.

For the Hospital Billing case study, all configurations achieve comparable results. Even using only data or only history yields results similar to those obtained with combined configurations, suggesting possible correlations between data attributes and historical information. *Logistic Regression* and *Decision Tree* models

also achieve similar performance, implying that linear models are sufficient to capture the dependencies in this process. The best configuration is achieved with the *Binary History* abstraction using *Logistic Regression*, reaching an improvement of approximately 14% and 19%, respectively, over the frequency-based baseline.

Overall, the results clearly demonstrate the advantage of simulation models that estimate transition probabilities based on enriched process states incorporating process attributes and execution history. Across all datasets, these data- and/or history-aware configurations consistently outperform the frequency-based baseline. This confirms that accounting for the evolving process state, rather than treating each decision point in isolation, leads to simulations that more accurately reproduce the observed process behavior.

Moreover, overall results suggest that, often, using only historical information is sufficient to achieve strong simulation performance. This suggests that trace attributes do not often influence the probabilities, possibly because their effects are already implicitly captured through correlations with the activity history, with the Sepsis case study representing a notable exception.

However, logistic regression and decision tree models generally yield comparable results, suggesting that the underlying transition probabilities often exhibit linear dependencies, which can be captured adequately by both models. However, decision trees offer practical advantages. Decision trees can effectively prevent the livelock issue observed with logistic regression in the Sepsis case study. Additionally, they provide transparent and interpretable decision rules, allowing users to easily understand how transition probabilities are determined. This interpretability is particularly valuable for configuring simulations and performing *what-if* analyses, as users can directly modify rules or thresholds to explore different scenarios (see a more general framework discussed in Chapter 4). In contrast, adjusting logistic regression coefficients to influence simulation behavior is very likely less intuitive and often challenging. Overall, decision trees combine reliable performance with clarity and usability, making them a preferred choice for practical applications.

3.5 Conclusion

Accurate business process simulation is essential for deriving reliable insights and guiding effective process improvements. Traditional approaches relying on fixed branching probabilities oversimplify decision-making and fail to capture the variability inherent in real world processes. In practice, the likelihood of an activity occurring depends on the evolving state of each process instance, including case-specific attributes and previously executed activities. Ignoring

these dependencies can result in simulations that misrepresent the process, leading to inaccurate results.

This work investigated the impact of process-state abstractions on the accuracy of business process simulation. Using these abstractions, we defined a weight function for each Petri net transition, which allows computing transition probabilities dynamically based on the current process state. The framework supports customizable process state definitions and allows the use of machine learning models, such as logistic regression and decision trees, to discover the weight functions.

Our experimental evaluation on three real-life case studies — Sepsis, Road Traffic Fines, and Hospital Billing — showed that simulation models using data- and history-aware transition probabilities consistently outperform the frequency based baseline. Overall, the results confirm that capturing the evolving process state, rather than treating each decision point in isolation, leads to more accurate simulation models. The proposed framework provides a flexible, interpretable, and effective approach to compute transition probabilities based on process states, allowing practitioners to simulate processes with higher fidelity and to derive more reliable insights for process improvement initiatives.

In future work, we aim to extend the process state definition to incorporate additional process features and perspectives, including resource allocation and temporal dependencies, to further enhance the fidelity and applicability of simulation models.

4 Probabilistic White-Box Simulation Models

This chapter investigates how **explainable** and **configurable** predictors can be used to improve the realism of business process simulation. We build on the prior results in Chapter 3 showing the impact of process-state abstractions on control-flow simulation accuracy and include them to additional run-time perspectives by adopting probabilistic decision trees. These **white-box models** preserve high predictive accuracy while explicitly representing uncertainty and decision logic, enabling process analysts to inspect, modify, and explore *what-if* scenarios. Our results show that probabilistic decision trees provide a flexible and reliable foundation for simulation models that balance accuracy, interpretability, and variability, addressing key limitations of existing black-box and deterministic approaches.

4.1 Introduction

Current state-of-the-art works show that integrating deep learning models into BPS can lead to more realistic run-time characterization [41]. However, deep learning models are black-box: the rules guiding these predictors are not explicitly provided as analytic expressions, thus not being intelligible. As such, it becomes impossible to alter these rules to define *what-if* scenarios and test the consequences on process performance via BPS [43]. Moreover, since deep learning models usually require a large amount of data, deep learning-based BPS models cannot be constructed when the real process data are limited or missing. In addition, these models are often deterministic, which reduces their ability to generalize across different scenarios and limits their applicability in dynamic or uncertain environments.

This chapter addresses these limitations by proposing a simulation approach,

where the run-time characterization is based on probabilistic decision tree models. Building on the framework introduced in Chapter 3, this chapter investigates how different process state abstractions affect simulation accuracy across multiple perspectives. These white-box models essentially consist of decision rules that process analysts can inspect and alter to test *what-if* scenarios involving different process configurations. We introduce **tree-based Business Process Simulation models**, which are basically process models extended with the run-time characterization of BPS based on decision trees. Our technique discovers decision trees from process data, using machine learning to capture selection rules and dependencies across various features (e.g., process attributes, timing, workload). We also present a method for interacting with these decision trees to define *what-if* scenarios, enabling process architects to evaluate the impact of process changes or new service compositions.

We evaluated our approach in five case studies to assess its accuracy in reproducing the behavior of the real process. The results demonstrate that our tree-based approach consistently outperforms the state-of-the-art white-box simulation methods, namely Simod [50] and AgentSimulator [93], and achieves competitive accuracy compared to black-box approaches [41, 136] while offering superior interpretability for process management decisions. Additionally, our method demonstrates greater realism and variability in simulation outcomes, making it particularly suitable for dynamic environments that demand transparent, configurable, and realistic behavioral modeling for effective process orchestration and composition management.

4.2 Related Work

The increasing availability of data and the advancement of new machine and deep learning techniques led to the integration of these into traditional process simulation methods. Camargo et al. [43] investigated the benefits of employing deep learning techniques compared to the traditional methods, referred to as "data-driven". The authors found that deep learning techniques produce more accurate simulations, although their technique does not enable *what-if* analyses.

Camargo et al. [41] and Meneghello et al. [136] propose two hybrid approaches, respectively named **DSim** and **RIMS**, where a process model is discovered to model the process' control-flow perspective, which is extended with deep learning models for the run-time characterization of the time perspectives. However, the use of black-box models makes it impossible to conduct specific *what-if* analyses. Additionally, deep learning models are usually "data greedy", making these approaches less suitable in case of scarcity of process data for one or more

Table 4.1: Comparison of various simulation approaches across different perspectives (Control-Flow, Temporal, and Resource). A ○ symbol indicates that the perspective is modeled using a white-box approach, while • denotes a black-box approach. **P** and **D** indicate if the models are probabilistic or deterministic, respectively. A gray box indicates the use of machine/deep learning techniques for the underlying rules.

Approach	Process Perspective		
	Control-Flow	Temporal	Resource
Rozinat et al. [164]	○**D**	○**P**	○**P**
Simod [50]	○**P**	○**P**	○**P**
DSim [41]	○**P**	•**D**	○**P**
RIMS [136]	○**P**	•**D**	○**P**
Agent Simulator [93]	○**P**	○**P**	○**P**
Our	○**P**	○**P**	○**P**

perspectives, or even impossible when no process data is available for different perspectives (e.g. event logs without resource information). Moreover, these models are inherently deterministic, always producing the same output for a given input. This lack of variability introduces a limitation in business process simulation, where incorporating noise and stochasticity is essential to generate realistic and insightful simulation results. As a consequence, their deterministic nature reduces their ability to generalize across different scenarios, limiting their applicability in dynamic or uncertain service environments.

Some body of research exists that focuses on defining white-box models to characterize the run-time BPS aspects. RIMS and **Simod** [50], adopt rule-aware models to discover and represent data-dependent branching probabilities. However, these approaches mainly concentrate on control-flow and data perspectives, without explicitly modeling rules for resource and temporal perspectives. Recently, Kirchdorfer et al. [93] have proposed **Agent Simulator**, an agent-based approach for discovering BPS from an event log, providing high interpretability and adaptability. This approach reverts the center of simulation attention from the control-flow to the resource perspective, using resource models as backbone for BPS models: it follows that can be challenging to perform *what-if* analyses heavily focused on the control-flow perspective.

Table 4.1 provides a comparative summary of state-of-the-art process simulation approaches across the key process perspectives (control-flow, temporal, resource), highlighting the nature of the modeling techniques employed. The last row refers to the approach proposed in this chapter, which - along with Simod and Agent Simulator - employs probabilistic methods to model the run-time characterization of the process' perspectives on control-flow, time and resource.

As mentioned, a probabilistic characterization allows simulations to be more accurate and more generalized, while white-box methods are preferable over black-box because the former makes it extremely easier to configure process simulation models for alternative *what-if* analyses. Among the white-box, probabilistic approaches, Simod and Agent Simulator use simple distributions and probabilities to characterize the temporal and resource perspective, which naturally do not provide outcomes that are as accurate as those provided by machine learning approaches. In fact, our approach focuses on probabilistic decision trees, highly intuitive and easily configurable. In summary, Table 4.1 allows one to conclude that our approach is more likely to accurately capture real-world uncertainties, incorporating noise and stochasticity for greater realism, while easily enabling *what-if* analyses.

Note that every approach discussed so far, including those reported Table 4.1 and ours, uses methods that capture correlations. These correlations are not used directly for root-cause analysis and process improvements but are merely used to guide the simulation, which is conversely used for root-cause analysis and improvement. There is a large body of research on causal reasoning methods, such as by Alaee et al. [10], which certainly provides more rigorous methods for causal inference but, on the other hand, do not easily enable *what-if* analyses.

4.3 Probabilistic Decision Trees

The technique discussed in this chapter leverages **probabilistic decision tree** models for their interpretability, explainability, and ease of user interaction. Their white-box, interpretable nature makes them very suitable to enable *what-if* analyses, where deep learning models fail. Other white-box models, such as logistic regression, rely on counterintuitive coefficient, making them less practical for domain analysts who aim to define *what-if* analyses. Probabilistic decision trees make predictions by hierarchically splitting the feature space into distinct regions based on specific decision rules. In these models, internal nodes and arcs define the decision criteria based on features in $\mathcal{F}$, while leaf nodes represent probability distributions. The use of probabilistic distributions make them be different from traditional decision tree, where leafs are associated to single static values.

In the BPS domain, stochasticity plays a crucial role. Traditional deterministic decision tree models may be inadequate, as they always produce the same output without accounting for noises or variability. Through stochastic decision tree, the final output is then generated by sampling from the distribution associated to the interested leaf.

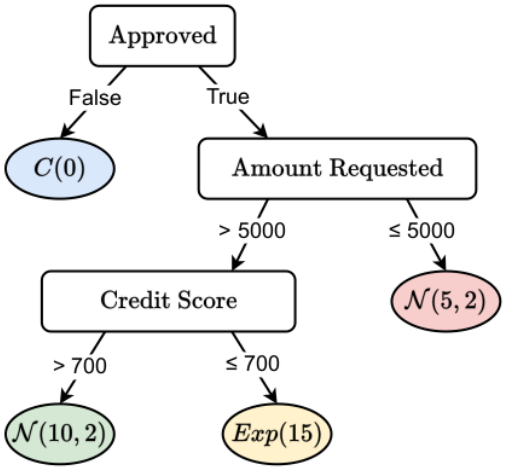


Figure 4.1: Example of a stochastic decision tree that models the execution time of the activity *Close Application* in a credit loan process. $C(t)$ indicates a constant distribution with value t, $Exp(t)$ the exponential distribution with mean t, $\mathcal{N}(\mu, \sigma)$ the normal distribution with mean μ and standard deviation σ. Numbers at leaf values are in minutes.

Example: In a loan application process, service times of certain activities may depend on rules based on previous activities and/or on process attributes. Figure 4.1 illustrates an example of a decision tree modeling the execution time of the closure application activity. If the loan has not been approved the application closes immediately. Otherwise, for requested amounts below 5000, the duration follows a normal distribution with mean 5 and standard deviation 2. For higher amounts, service time increases and depends on the applicant's credit score. For example, for higher requested amount from customers with a low score the closure time follows an Exponential distribution with average 15.

Given a set of feature values Δ, a deterministic decision tree is defined as a function $\psi \in (\Delta \to \mathbb{R})$, where, for a given input $x \in \Delta$, representing a numerical values for each feature, $\psi(x)$ represents the decision tree prediction. A probabilistic decision tree is a function $\psi_{\mathcal{P}} \in (\Delta \to \mathcal{P})$, where $\mathcal{P}$ denotes the universe of probability distributions. For a given input $x \in \Delta$, $\psi_{\mathcal{P}}(x)$ is a probability distribution of potential output values.

Since our simulation models are defined via (probabilistic) decision trees, such analyses are based on their modifications, which are of three types:

1. **Change leaves values**, namely distribution parameters in a certain leaf are

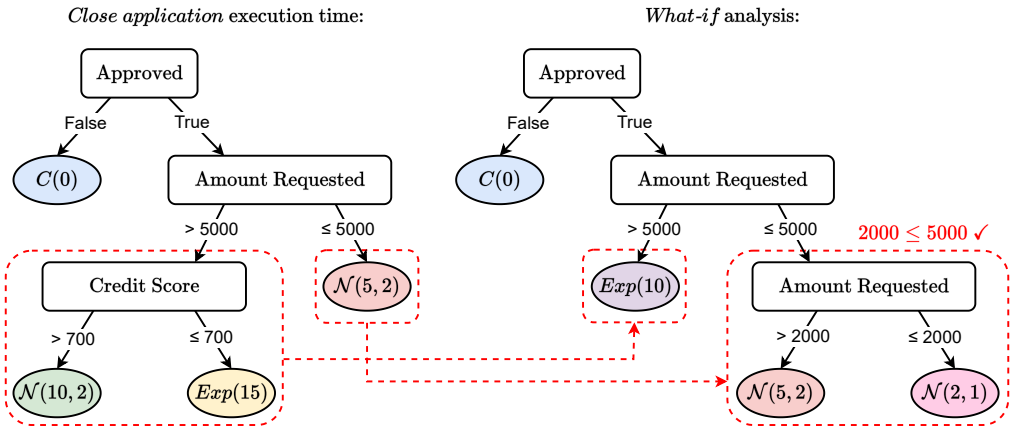


Figure 4.2: User interaction example on the decision tree illustrated in Figure 4.1. $C(t)$ indicates a constant distribution with value t, $Exp(t)$ the exponential distribution with mean t, $\mathcal{N}(\mu, \sigma)$ the normal distribution with mean μ and standard deviation σ. Rules are modified by removing a decision tree and adding a new one in a different branch. The new rule satisfies the constraint of the parent node ($2000 < 5000$).

altered.

2. **Remove subtrees**, namely a subtree is replaced by a single leaf.

3. **Add subtrees**, namely a composite tree replaces a single leaf.

Note that these interactions can also be combined, and the user can also define new decision tree models (e.g. when including more resources or new activities in the process models). Alternatively, users can also reuse existing decision trees from other resources to mimic the same behaviors. Also, when including a new activity, i.e., new transitions in the process model with a new activity label, the user can similarly introduce new decision trees for each new transition defining the probability of firing that transition when enabled.

Example: Figure 4.2 illustrates an example of interacting with the decision tree from Figure 4.3 to perform a *what-if* analysis. In this new scenario, the service execution time of *Closure* is no longer dependent on the *Credit Score* for requested amounts exceeding 5000. Additionally, for requested amounts below 2000, the service time is reduced. This adjustment allows evaluating how the process performance is affected by these changes.

4.4 Discovery of Tree-Based Business Process Simulation Models

In this section, we present a methodology for discovering **tree-based Business Process Simulation models**, where decision tree models govern control-flow, temporal, and resource simulation parameters based on process simulation states, from an input event log $\mathcal{L}$. The objective is to identify process rule patterns within the event log data for building decision tree models.

The Petri net model $(P, T, F, \lambda, m^I, m^F)$ can be obtained using process discovery algorithms (cf. Section 1.3.3). The descriptive parameters $D = (\mathcal{R}, C^{\mathcal{R}}, MT^{\mathcal{R}}, d^{\mathcal{V}})$ are discovered analyzing events of the event logs as follows.

- $\mathcal{R}$ is the set of observed resources in the event log, i.e., $\mathcal{R} = \{res(e) \mid e \in \mathcal{L}\}$.
- The calendars $C^{\mathcal{R}}(r)$ of each resource $r \in \mathcal{R}$ are obtained by looking at the timestamp of events performed by r, specifically $C^{\mathcal{R}}(r)_{h,r} = 1$ iff $\exists\, e \in \mathcal{L}\ s.t.\ res(e) = r,\ hour(time(e)) = h,\ weekday(time(e)) = w$, otherwise $C^{\mathcal{R}}(r)_{h,r} = 0$, with $hour$ and $weekday$ functions returning the hour $h \in \{1, \ldots, 24\}$ and the weekday $w \in \{1, \ldots, 7\}$ of the corresponding timestamp respectively.
- $MT^{\mathcal{R}}$ is the subset of resources able to perform multitasking, specifically, $r \in MT^{\mathcal{R}}$ iff $\exists\, e_1, e_2 \in \mathcal{L}$ with $res(e_1) = res(e_2) = r$ such that $t_{start}(e_1) < t_{complete}(e_2) \ \wedge t_{start}(e_2) < t_{complete}(e_1)$.
- For each trace attribute $e \in \mathcal{V}$, the distribution $d^{\mathcal{V}}(v)$ is estimated from the multiset of observed values $[attr_{\mathcal{L}}(\sigma) \mid \sigma \in \mathcal{L}]$. The best-fitting parametric distribution is selected following the approach proposed by Villani et al. [186], which chooses the distribution minimizing the Wasserstein distance.

To discover the transition weight function w and the predictive parameters $K = (p^R, p^{ET}, p^{WT}, p^{AT})$ we build on the framework proposed in Chapter 3. First, the traces on the event log $\mathcal{L}$ are repaired accordingly to the process model $(P, T, F, \lambda, m^I, m^F)$, thus obtaining Petri net runs. The weight function w is discovered through decision trees as described in Chapter 3. Then each predictive parameter is discovered as follows.

Resource Assignment Each prefix of each Petri net run is mapped to a data state $\delta \in \mathcal{A} \times \mathbb{B}(\mathcal{R}) \times (\mathcal{V} \to \mathcal{U})$, capturing the activity to be executed, the history of previous resources, and trace attributes. Similarly as with transition weights, we build a training set of observations $(\delta, class)$ for each resource

r, with $class$ taking the value 1 if the corresponding resource has been employed in the last event of the state of the correspondent process, 0 if it was available and not used. Specifically, we say that a resource r is available if it is not performing other events at the corresponding timestamp or if $r \in MT^{\mathcal{R}}$. Finally, a decision tree classifier is trained to predict values, returning in the leaves values in the range from 0 to 1 representing the probability of choosing the corresponding resource when available.

Execution Time Each prefix $\langle e_1, \ldots, e_k \rangle$ of each Petri net run is mapped to a data state $\delta \in \mathcal{R} \times \mathbb{B}(\mathcal{A}) \times (\mathcal{V} \to \mathcal{U})$, representing the selected resource, the history of activities, and the trace attributes. For each activity $a \in \mathcal{A}$, we build a training set of observations (δ, τ_{et}), where τ_{et} denotes the execution time of the current event e_k with $act(e_k) = a$, computed as $\tau_{et} = t_{complete}(e_k) - t_{start}(e_k)$. A decision tree regressor is trained for each activity. Finally, for each leaf node, a best fitting distribution is determined from a set of predefined ones by filtering the data based on the corresponding decision rules. The distributions are discovered following the procedure in [42], which aims to minimize the Wasserstein distance [186].

Waiting Time Similarly to execution time, for each prefix $\langle e_1, \ldots, e_k \rangle$, with $res(e_k) = r$, we derive observations (δ, τ_{wt}), where $\delta \in \mathbb{N} \times \mathcal{A} \times (\mathcal{V} \to \mathcal{U})$ represents the data state composed of the workload of resource r (i.e., the number of events being executed at the corresponding timestamp by r), the current activity, and the trace attributes. The waiting time is computed as $\tau_{wt} = t_{start}(e_k) - t_{complete}(e_j)$ with e_j the event that enables the transition observed in e_k. Decision tree regressors are trained for each resource, and best fitting distributions for each leaf are retrieved as done for execution times. This results in a probabilistic decision tree $p^{WT}(r)$ for each resource $r \in \mathcal{R}$ that modeling its waiting time distribution.

Arrival Time The arrival time model is learned from a training dataset constructed as follows. For each trace $\sigma \in \mathcal{L}$, we collect observations (δ, τ_{at}), where $\delta \in \{1, \ldots, 7\} \times \{1, \ldots, 24\}$ encodes the weekday and hour of arrival, and τ_{at} denotes the time elapsed between the start times of two consecutive traces. A decision tree regressor is trained to estimate arrival times, and, as for execution and waiting times, a probabilistic decision tree p^{AT} is obtained by fitting distributions at each leaf.

4.5 Experiments

While Section 4.3 has shown how probabilistic decision trees can be leveraged to perform *what-if* analyses, this section provides an extensive evaluation of their simulation performance in terms of accuracy, demonstrating that our approach not only outperforms alternative white-box methods, but also achieves results comparable to state-of-the-art black-box methods (cf. Section 4.2).

Accuracy evaluation focuses on computing distances between the distributions observed in the simulated event logs and the real ones [47]. We compared our results with those obtained by state-of-the-art methods (cf. Section 4.2), showing that our approach outperforms the white-box simulation models Simod [50] and AgentSimulator [93], and often achieves competitive or superior performance compared to the black-box methods [41, 136].

Beyond the overall simulation accuracy, our method also demonstrates high precision in reproducing decision behavior, thanks to explicit modeling of decision rules. Furthermore, our simulations exhibit greater generalization and variability, key indicators of realistic behavior, as evidenced by the entropy analysis.

4.5.1 Experimental Setup

The simulator has been developed in the ProSiT tool (cf. Chapter 9). To assess the accuracy of our approach, we discovered the simulation models using available event logs from five different business processes: Purchase to Pay (P2P), CVS retail pharmacy (CVS), Academic Credentials Recognition (ACR), BPIC12W, and BPIC17W (see Section 1.7 for further details).

We temporally split each event log into train and test sets with respectively 80% and 20% of the total traces. The train set is used to discover the simulation parameters as described in Section 4.4, while the test set is employed for the final evaluation.

To ensure a fair evaluation of the different methods, we used the same control-flow model as the backbone of the simulation models of the different methods. Specifically, we employed models mined via Split Miner [15], used in [41] and [136].

Regarding the configuration of Simod [50], we adopted the complete configuration example provided by the authors, which has multitasking disabled.[1] We note that, since our approach supports multitasking (cf. Section 4.4), part of the accuracy gains over Simod may stem from this difference in modeling capability.

[1] https://github.com/AutomatedProcessImprovement/Simod/blob/master/resources/config/complete_configuration.yml

Decision trees have been discovered using the CART algorithm. To maximize accuracy, we performed a 3-fold cross-validation with hyperparameter optimization. Specifically, we defined the hyperparameter search space for the maximum tree depth by varying its values in the range 1 to 5. In fact, the more terminal nodes and the deeper the tree, the more difficult it becomes to understand the decision rules of a tree [138].

4.5.2 Evaluation of the Level of Accuracy

The goal of the accuracy evaluation is to assess how realistic the simulation conducted with our method is and to compare it with Simod [50] and AgentSimulator (ASim) [94], which also use white-box models, as well as with RIMS [136] and DeepSimulator (DSim) [41], which are hybrid BPS methods that leverage deep learning techniques. For ASim [94], we specifically employed the version with the process model, in order to ensure a fair and comparable evaluation across all methods.

To assess the accuracy of our proposed method, we conducted simulations and computed distances between the event logs obtained by the different simulation methods and the original ones, which were used to build the BPS models. The assessment was based on three metric, discussed in Section 1.6.3, that aligns with the focus of our approach: **N-Gram Distance** (NGD), with $N = 3$ for measuring the control-flow quality; **Cycle Time Distribution distance** (CTD) to measure the ability of the model for capturing the times of process instances; **Case Arrival Rate distance** (CAR) to computes the difference in the temporal distribution of case arrivals throughout the process timeline. In fact, these metrics focus on evaluating and comparing the perspectives where we introduced (probabilistic) decision trees.

For each case study, we ran five simulations to obtain event logs containing the same number of traces as the real ones, and computed the average distances between the obtained event logs. Table 4.2 presents the results of simulation distances across five case studies, comparing our approach with both white-box (Simod [50], ASim [94]) and black-box (DSim [41], RIMS [136]) methods. Across nearly all metrics and datasets, our consistently outperforms Simod [50] and ASim [94], except for four metrics, establishing it as the most effective white-box method. Notably, our method even matches or comes close to the performance of state-of-the-art black-box methods RIMS and DSim.

Although both Simod [50] and RIMS [136] incorporate decision models within the control-flow, our approach demonstrates superior effectiveness. This can be attributed to two key factors: (1) unlike Simod [50] and RIMS [136], which rely solely on event attributes, our method also leverages process history as a predic-

Table 4.2: Simulation distance results. For each case study, metric results are reported for each method: our approach, ASim [94], Simod [50], and the black-box methods DSim [41] and RIMS [136]. The best results are highlighted in bold. Cells highlighted in green indicate the best results among the white-box methods (i.e., Our, ASim, and Simod). The last rows, denoted with *Avg.*, indicate average results for each metric and method across the case studies.

Case Study	Metric	White-Box			Black-Box	
		Our	ASim [94]	Simod [50]	DSim [41]	RIMS [136]
P2P	NGD	**0.41**	0.57	0.42	0.42	0.42
	CAR	**571.50**	821.97	794.38	899.14	903.90
	CTD	**465.30**	542.10	469.82	584.27	578.74
ACR	NGD	**0.28**	0.47	0.37	0.37	0.37
	CAR	**232.80**	354.09	264.93	247.49	244.25
	CTD	372.11	427.47	399.00	59.03	**45.77**
CVS	NGD	**0.03**	0.41	0.08	0.08	0.08
	CAR	20.84	**5.79**	10.30	20.37	20.26
	CTD	**9.17**	95.00	83.54	52.43	99.25
BPIC12W	NGD	**0.31**	0.35	0.36	0.40	0.35
	CAR	98.62	39.03	**24.80**	31.47	27.08
	CTD	**64.72**	91.78	67.99	148.05	85.17
BPIC17W	NGD	**0.27**	0.31	**0.27**	0.28	0.28
	CAR	85.51	154.01	129.66	**26.60**	84.79
	CTD	37.40	30.61	**16.13**	114.20	36.43
Avg.	NGD	**0.26**	0.42	0.30	0.31	0.30
	CAR	**201.85**	274.98	244.814	245.01	256.06
	CTD	189.74	237.39	207.30	191.60	**169.07**

Table 4.3: Rule-based distance results. Each row reports a case study. Results are grouped by metric (Rule-CFD, Rule-HRD, Rule-ATD, Rule-ETD, Rule-WTD, Rule-HRD), with our approach and Simod [50]. The best results are highlighted in bold. Averages are reported in the last row.

Case Study	Rule-CFD		Rule-HRD		Rule-ATD		Rule-ETD		Rule-WTD	
	Our	Simod	Our	Simod	Our	Simod	Our	Simod	Our	Simod
P2P	**0.06**	0.11	**0.05**	0.06	**783.26**	900.86	**14.56**	78.12	**843.30**	946.04
ACR	**0.15**	0.19	**0.04**	0.11	**141.61**	202.68	24.77	**7.46**	594.50	**384.48**
CVS	**0.01**	0.24	**0.01**	0.01	**0.06**	1.11	**0.86**	1.87	**377.01**	989.86
BPIC12W	**0.10**	0.17	**0.04**	0.06	**2.55**	6.17	**7.07**	7.96	**200.13**	232.99
BPIC17W	**0.11**	0.14	**0.02**	0.03	**1.47**	4.10	**18.01**	20.74	**366.55**	514.79
Avg.	**0.08**	0.17	**0.03**	0.05	**185.79**	222.98	**13.05**	23.23	**476.30**	613.63

tive feature; and (2) we model transition behavior using weighted probabilities on transitions, allowing us to capture priority in concurrent executions (see Figure 1.6), while Simod [50] and RIMS [136] apply decision trees only at XOR gateways.

To further evaluate how accurately our method models decision rules, we

Table 4.4: Average entropy of execution time distributions. For each case study, correspondent results are reported for each method: our approach, ASim [94], Simod [50], DSim [41], RIMS [136], and our deterministic approach. Best results are highlighted in bold. Red background is used to indicate the worst results. Last row, denoted with *Avg.*, indicate average results for each metric and method across the case studies.

Case Study	Probabilistic			(Partly) Deterministic		
	Our	ASim [94]	Simod [50]	DSim [41]	RIMS [136]	Our-det
P2P	**1.38**	0.25	1.33	1.16	1.20	0.56
ACR	**1.15**	0.63	1.01	0.74	0.59	0.85
CVS	0.36	**0.72**	0.64	0.57	0.57	0.08
BPIC12W	**2.25**	2.13	1.65	0.56	0.56	0.60
BPIC17W	2.21	**2.26**	1.97	0.61	0.61	0.76
Avg.	**1.47**	1.19	1.32	0.73	0.71	0.57

computed distances between simulated and real event data by analyzing their behavior at the level of the leaves induced by the discovered decision rules. Specifically, for each discovered rule (i.e., decision tree leaf), we filtered both the real and simulated event logs to retain only the events satisfying that rule, and then computed distances between the corresponding filtered event logs.

For time-related decision trees (i.e., arrival time, execution time, and waiting time), we compared the corresponding empirical distributions using the Wasserstein distance [154], which quantifies the minimal cost of transforming one probability distribution into another (respectively indicated with Rule-ATD, Rule-ETD, Rule-WTD). For decision trees modeling transition and resource weights, we computed the absolute difference in transition firing frequencies (Rule-CFD) and resource-choice frequencies (Rule-HRD). We compared our approach with Simod [50], as ASim [94], DSim [41] and RIMS [136] either do not produce resource information, used as features in some decision trees, or lack the necessary trace attribute (cf. Section 4.4). Table 4.2 summarizes the results across the five rule-based metrics. Our approach outperforms Simod [50] in the vast majority of cases, achieving the lowest average distances in all four metrics. In particular, they show substantial improvements in Rule-CFD and Rule-ETD, indicating more accurate modeling of control-flow decisions and execution time distributions. These results confirm the enhanced precision of our simulations in replicating the decision logic within the process.

4.5.3 Evaluation of the Level of Generalization

We attribute the superior and competitive performance of our method compared to black-box ones to its higher degree of generalization, which is related to its inherently probabilistic nature. While RIMS [136] and DSim [41] rely on

deterministic deep learning models to generate execution times, often resulting in rigid, overfitted behavior with limited temporal variability, our method leverages probabilistic decision trees to effectively capture uncertainty and reflect the natural variability observed in real-world processes. To further support this claim, we computed the entropy of execution time distributions across all activities on the simulated event logs (cf. Section 1.6.4).

Table 4.4 reports the average execution time entropy values, with methods grouped into *Probabilistic* and *Deterministic* categories. We also include a deterministic variant of our method that replaces probabilistic decision trees with deterministic ones. As expected, all deterministic approaches exhibit substantially lower entropy, with an exception in a case study of ASim [94], confirming their limited ability to capture temporal variability. This statement is particularly confirmed if we compare the entropy values of our approach with our deterministic: the simple replacement in the latter of probabilistic decision trees with deterministic ones causes a significant drop in the entropy values, further pointing out that the improvement is certainly related to its probabilistic nature. Our approach achieves higher entropy on average and across all case studies, except for CVS. The CVS dataset warrants special attention: it is synthetically generated, likely without attempting to increase behavioral variability. Consequently, an use of probabilistic models in place of deterministic one does not provide significant improvements.

These findings confirm the effectiveness of our method in generating behaviorally accurate, realistic, and diverse simulations while maintaining full interpretability. We consistently outperform the white-box methods Simod [50] and AgentSimulator [94], not only in overall simulation accuracy but also in faithfully modeling decision rules. Moreover, our method achieves competitive accuracy when compared to black-box state-of-the-art approaches, while offering greater realism by capturing a higher degree of behavioral variability, as demonstrated by the entropy analysis.

4.6 Conclusion

The simulation of business processes that involve human resources and/or services is based on a model that describes how activities are executed, along with a characterization of how the perspectives on data, resource, and time are simulated. This characterization refers to new process-instance arrival times, activity durations, resource assignment, data-dependent probabilistic guards, etc.

While several approaches exist for discovering business process simulation models and characterizing their run-time behavior, most rely on deep learning

and/or deterministic models. Deep learning models are typically black-boxes and do not expose their decision logic in an interpretable, analytical form. This limits their configurability and hinders the ability to perform meaningful *what-if* analyses. Deterministic models, in turn, fail to capture the inherent stochasticity and uncertainty of real-world processes.

This chapter discusses a novel approach that uses probabilistic decision trees for run-time characterization. These decision models are both white-box, allowing for easy interpretation and configurability, and stochastic, enabling the simulation of natural process variability.

Our approach has been compared with existing related techniques that are based on black-box and/or deterministic models, as well as with state-of-the-art white-box and stochastic models. Five different processes were used in the evaluation, and the results outline that our method enables running business process simulations that are more realistic than and generalize more than these existing techniques.

We evaluated the usability of the proposed probabilistic decision trees for *what-if* analyses through a dedicated user study. Chapter 10 provides a detailed description of the evaluation methodology and the developed tool. The usability assessment demonstrates a high level of usability, indicating that process analysts can effectively leverage probabilistic decision trees to explore alternative scenarios in process simulation (cf. tasks T3 and T4 in Chapter 10).

In future work, we plan to investigate the use of more complex machine learning models for run-time characterization while preserving the benefits of interpretability and configurability. In particular, we aim to explore hybrid approaches combining high performing predictive models with explainability techniques, such as model distillation [80].

5 Online Discovery of Process Simulation Models

In this chapter, we address the challenge of discovering process simulation models in the presence of evolving process behavior. We propose a **streaming process simulation discovery** technique that continuously updates simulation models by integrating **Incremental Process Discovery** with **Online Machine Learning**. Building on tree-based business process simulation models (see Chapter 4), we employ Hoeffding Adaptive Trees (see Section 1.4.2) to learn decision components that can dynamically adapt to changes while retaining relevant historical knowledge.

An extensive experimental evaluation on multiple event logs shows that prioritizing recent behavior, without discarding past information, leads to more stable and accurate simulations. The results highlight the robustness of the proposed technique in the presence of concept drift, demonstrating its suitability for dynamic, real-world business environments.

5.1 Introduction

Dealing with evolving processes and with their dynamic behavior remains one of the major challenges in Business Process Management. Organizations continuously adapt their processes in response to internal policy changes or external factors. Traditional techniques for discovery of simulation models fail to capture these ongoing changes because they do not feature possibilities to update the discovered models as changes are observed in the process' behavior. This affects the process when a sudden concept drift occurs, as the event log treats both older and newer behavior with equal importance in process simulation model discovery. As a result, the discovered model would simulate behavior from both before and after the drift as if it were future behavior, likely leading to invalid

conclusions.

In this chapter, we propose a novel **streaming process simulation discovery** technique that integrates Incremental Process Discovery [169] with Online Machine Learning methods (cf. Section 1.4.2). Our technique continuously updates the simulation model by incorporating new event behaviors while preserving historical knowledge. Specifically, we employ Hoeffding Adaptive Trees (see Section 1.4.2) that can be easily integrated into tree-based process simulation models (cf. Chapter 4) where predictive components consist of decision trees. These predictors are well-suited for evolving data streams and can dynamically adapt to process changes over time. By prioritizing recent data without discarding valuable past information, our technique enhances the accuracy and stability of simulation models.

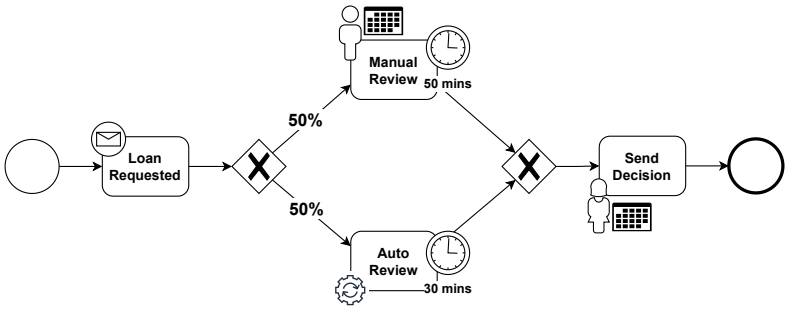
The evaluation compares our streaming discovery technique with two baselines. A first baseline considers all historical event data, including those before any process' behavioral drifts; a second only uses the recent data, which would not mix process variants with different behavior. This comparison highlights the effectiveness of our technique and determines when all historical data are necessary. The results show that our technique produces more reliable simulations and effectively adapts to evolving processes.

The rest of the chapter is structured as follows: Section 5.2 illustrates a motivational example, Section 5.3 reviews related work on process simulation and online learning techniques. Section 5.4 details our proposed technique, outlining how it integrates business process simulation discovery with adaptive learning. Section 5.5 presents the experimental setup and evaluation results. Finally, Section 5.6 concludes the chapter.

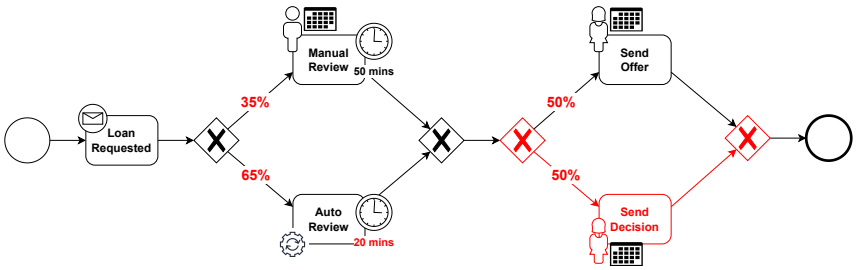
5.2 Motivating Example

In this section, we present an example that highlights the motivation for proposing an online process simulation discovery technique. Consider a loan application process within a financial institution. Initially, when a customer submits a *loan request*, the application is processed through one of two paths: either a *manual review* by an expert (50% of probability, with 50 minutes duration) or an *automated approval* by a system (50% of probability, with 30 minutes duration). The process concludes with the *notification of the decision*. Figure 5.1a depicts a simulation model that a process simulation discovery technique would generate based on historical data.

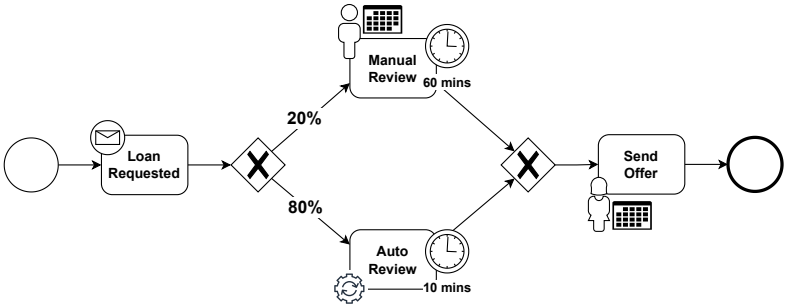
However, due to internal decisions to optimize the process, the institute decided to enhance its automated system, increasing the probability of automated



(a) Simulation model before the concept-drift.



(b) Simulation model discovered after the concept-drift by using traditional process simulation discovery approaches. Red parts denote inaccuracies.



(c) Simulation model discovered after the concept-drift by using online process simulation discovery techniques.

Figure 5.1: Loan application process model examples. The control-flow process models is illustrated using BPMN notation (cf. Section 1.3.2). Percentages in the arcs represent path probabilities after the decision points. Clock indicates activities durations in minutes. Resources, where involved, are represented with icons and calendars.

reviews to 80% while reducing its processing time to 10 minutes. Moreover, rather than simply concluding with an approval or rejection, the process outcome now includes a *loan offer*.

When discovering simulation models using traditional process simulation discovery approaches, we discover the model depicted in Figure 5.1b. As we incorporate all past event data (i.e., with pre- and post-drift behaviors), we fail to

accurately reflect the changes. Instead, if we properly prioritize the recent data, we can discover the simulation model described in Figure 5.1c. At the same time, previous data information are crucial for capturing additional observations, such as resource calendars and other event attributes. In this chapter, we propose a technique for incrementally updating previously discovered simulation models, ensuring they adapt to process changes while maintaining high accuracy.

5.3 Related Work

Early research in business process simulation investigated how to automatically discover simulation models from event logs. These approaches aimed to construct comprehensive simulation models that reflect observed process behavior. Over time, research has increasingly focused on improving the accuracy and realism of the discovered models, by also proposing hybrid approaches intergrating machine and deep learning technique (cf Section 1.6.2).

These discovery techniques aim to generate a process simulation model from an input event log. However, they assume that the process remains stable over time, analyzing patterns by averaging past and recent behaviors. In reality, processes may be dynamic and evolve over time due to external factors or internal process optimizations. This phenomenon, known as concept drift, occurs when the process behavior changes over time, potentially making previously discovered models inaccurate. To address this, several works proposed approaches for detecting, localizing and dealing concept drifts [31, 167]. Recent studies have introduced techniques and methodologies for detecting concept drift across multiple process perspectives, including control-flow, resources, and performance, showing how processes can evolve in complex and multifaceted ways [7, 95, 98].

Other works proposed approaches for detecting concept drifts from event streams [119]. Process mining techniques applied to the analysis of data streams are referred as *streaming process mining* [35]. These techniques can be used to dynamically updating existing process models [36, 46, 201]. Navarin et al. [142] presented a technique for discovering declarative process models from event streams that incorporates both control-flow dependencies and data conditions. In [168] Scheibel and Rinderle-Ma presented an online decision mining technique using an adaptive window technique (ADWIN) [28] to detect changes. However, these works have primarily focused on the control-flow and decision mining perspectives, without incorporating multiple perspectives, e.g. organizational and temporal, essential for BPS models.

In the domain of predictive process monitoring, Rizzi et al. [159] evaluated three different strategies that support either the periodic rediscovery or the in-

cremental construction of predictive models, thereby allowing models to stay up-to-date with new data. However, these approaches are not specifically designed for the discovery or updating of simulation models.

5.4 Online Process Simulation Discovery

Traditional process discovery techniques rely on analyzing historical event data in a single batch to derive a process model. These methods assume that the process remains static over time, resulting in models that may become outdated as the process evolves dynamically. This section introduces a novel technique for discovering business process simulation models from event data, designed to remain robust despite changes over time.

In this chapter, we propose a technique for incrementally discovering process simulation models. Process simulation models are here defined as tree-based BPS, following the definition given in Section 4.4, with the exception of the predictive parameter of resource assignment, which is simply defined with $\Delta^R = \mathcal{A}$, where, for each resource, a corresponding function assigns the probability of assigning that resource to a specific activity.

The starting point of the technique is a simulation model $M_{t_0} = (N_{t_0}, D_{t_0}, K_{t_0})$, which represents a process based on information available up to timestamp t_0. Given a new set of events at timestamp $t > t_0$, the goal is to produce an updated simulation model $M_t = (N_t, D_t, K_t)$ integrating newly observed behaviors while refining the existing model.

Let $\mathcal{E}$ be a universe of events, and $\mathcal{M}$ be a universe of simulation models. This problem can be formulated as synthesizing a function $\Phi : \mathcal{M} \times \mathcal{E}^* \to \mathcal{M}$, such that given a process simulation model $M \in \mathcal{M}$ and a sequence of events $\mathcal{S} \in \mathcal{E}^*$, $\Phi(M, \mathcal{S})$ returns an update simulation model that considers the sequence of events in $\mathcal{S}$.

The procedure for updating the existing simulation model can be divided into four key steps. First, data preparation is performed to select and structure the event sequence for model updating (cf. Section 5.4.1) . Next, the control-flow model is refined using Incremental Process Discovery techniques (cf. Section 5.4.2) . Then, the descriptive parameters are updated (cf. Section 5.4.3). Finally, the predictive parameters (models) are adjusted using Online Machine Learning techniques (cf. Section 5.4.4).

5.4.1 Step 0: Event Data Preparation

Traditionally, Process Mining techniques take as input an entire set of completed traces and use it to derive a conclusion (cf. Section 1.3). In this paper, we assume

to deal with **event streams**, i.e. a sequence of unique events [200].

Definition 17 (Event Stream). *Let $\mathcal{E}$ be the universe of events. An event stream $\mathcal{S} = \langle \dots, e_i, e_{i+1}, \dots \rangle \in \mathcal{E}^*$ is an infinite sequence of events such that, for any $i \geq 1$, $time(e_i) \leq time(e_{i+1})$.*

Several techniques have been proposed to extract finite sets of events from event streams for use in Process Mining [200]. In this work, we adopt the concept of **sliding window**, but note that the technique is generalizable to any other extraction approaches.

Definition 18 (Sliding Window). *Let $\mathcal{S}$ an event stream. Given a timestamp $t \in \mathbb{N}$, and the window size $w \in \mathbb{N}^{>0}$. We define the sliding window of size w at timestamp t as $\mathcal{S}_{w,t} = \langle e \in \mathcal{S} \mid time(e) \in [t - w, t] \rangle$*

The rationale is that a process analyst would update the simulation model at regular time intervals defined by a predefined window size w. This means that a new model is generated every w time units. For example, if w is set to one week, the simulation model is updated weekly to reflect the latest observed process changes during that week. This periodic update mechanism can be formalized as iteratively refining an initial process simulation model M_{t_0} at a timestamp t_0. The updated process simulation model at a timestamp t is given by $M_t = M_{t_i} = \Phi\left(M_{t_{i-1}}, \mathcal{S}_{w,t_i}\right)$, where $i > 0, t_i \leq t < t_{i+1}, t_i = t_{i-1} + w$ and $\mathcal{S}_{w,t_i}$ represents the stream of events occurred within the most recent time window of length w. Figure 5.2 illustrates this discussion, showing how the simulation models are obtained over time for a time window of size w. This approach defines a sequence of simulation models, each corresponding to a specific time window, where one extends the previous.

5.4.2 Step 1: Control-Flow Model Update

The previous process model $N_{t_{i-1}}$ is incrementally updated to incorporate new observed behaviors from the last observed data in the sliding window $\mathcal{S}_{w,t_i} \in \mathcal{E}^*$. We employ Incremental Process Discovery techniques to refine the existing process model, resulting in an updated model N_{t_i} able to capture the new observations [169]. $\mathcal{S}_{w,t_i}$ may include incomplete traces - prefixes, infixes or postfixes - in addition to complete traces (see Figure 5.2). To effectively handle this, we build upon the work by Schuster et al. [170] designed for applying an Incremental Process Discover technique using trace fragments. Trace fragments can be easily reconstructed from the stream of events by projecting on the case identifiers. Specifically, prefixes refer to cases that start within the window but are not yet completed, postfixes to cases that complete in the window but started

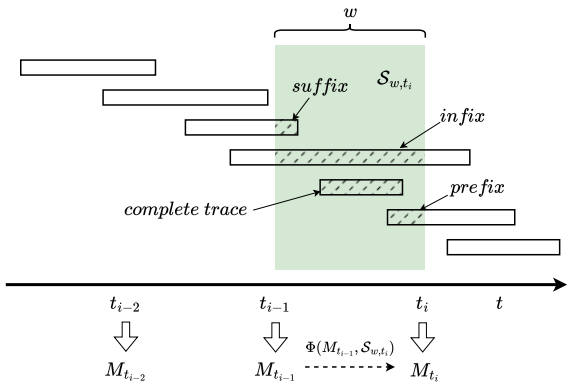


Figure 5.2: Iterative procedure for online process simulation discovery at each timestamp t_i using window size w. Rectangles represent traces over time, with suffixes, infixes, prefix and complete traces in the window highlighted with dashed rectangles. The green part represents the time window containing the events in $\mathcal{S}_{w,t_i}$ used for incrementally updating the simulation model at time t_i.

earlier, and infixes are fragments of ongoing cases that both started before and continue beyond the current window. We assume that domain knowledge can be used to determine when cases reach a completion. This knowledge can be given in form of possible activities that mark the end of process executions, or alternatively by setting a timeout, namely a case is assumed to be completed if no new activity is observed within a time threshold [201].

We acknowledge that this can limit the applicability in certain settings. However, this is plausible in other settings. For example, in a loan application process, a case ends when the offer is either accepted or rejected; in a purchase process, upon completion of payment; and in a pharmacy retail setting, when the prescription is fulfilled (see Section 5.5.1).

5.4.3 Step 2: Descriptive Parameters Update

The descriptive parameters $D_{t_{i-1}} = (\mathcal{R}_{t_{i-1}}, C^{\mathcal{R}}_{t_{i-1}}, MT^{\mathcal{R}}_{t_{i-1}}, d^{\mathcal{V}}_{t_{i-1}})$ are updated by including the new behaviors in the sliding window $\mathcal{S}_{w,t_i}$:

- When previously unobserved resources $\{r^1_{t_i}, \ldots, r^n_{t_i}\}$ appear in $\mathcal{S}_{w,t_i}$, they are integrated in the set of resources producing a new set of resources $\mathcal{R}_{t_i} = \mathcal{R}_{t_{i-1}} \cup \{r^1_{t_i}, \ldots, r^n_{t_i}\}$.
- Then the resource calendars are redefined with $C^{\mathcal{R}}_{t_i} \colon \mathcal{R}_{t_i} \to \{0,1\}^{7\times 24}$ computing working schedules for new resources observed in $\mathcal{S}_{w,t_i}$, and

recomputing those for resources observed in previous windows.

- The set of multi-tasking resources is updated with the new resources in $\{r^1_{t_i}, \ldots, r^n_{t_i}\}$ able to perform multiple tasks in a same time period.

- The event data distributions are recomputed using the events in $\mathcal{S}_{w,t_i}$.

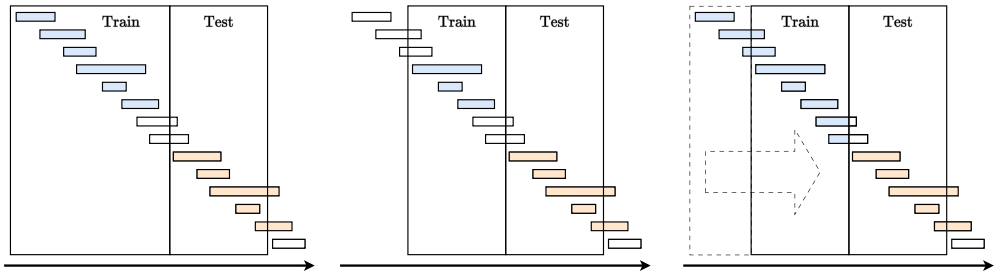
This results in an updated set of descriptive parameters $D_{t_i} = (\mathcal{R}_{t_i}, C^{\mathcal{R}}_{t_i}, MT^{\mathcal{R}}_{t_i}, d^{\mathcal{V}}_{t_i})$.

5.4.4 Step 3: Predictive Models & Transition Weights Update

Finally, the predictive models in $K_{t_{i-1}} = (p^R_{t_{i-1}}, p^{ET}_{t_{i-1}}, p^{WT}_{t_{i-1}}, p^{AT}_{t_{i-1}})$ and the transition weight function $w_{t_{i-1}}$, are updated to reflect changes in the process observed within the sliding window $\mathcal{S}_{w,t_i}$. These updates occur in two ways:

- The updated process model N_{t_i} and parameter set D_{t_i} may introduce previously unseen elements, such as new resources $\{r^1_{t_i}, \ldots, r^n_{t_i}\}$, new activities $\{a^1_{t_i}, \ldots, a^m_{t_i}\}$, or new pathways, and hence new transitions $\{t^1_{t_i}, \ldots, t^k_{t_i}\}$ in N_{t_i} that were not present before. In such cases, new predictive models and transition weights are defined. First, for any new transition $t \in \{t^1_{t_i}, \ldots, t^k_{t_i}\}$, we train a new decision tree modeling its weight $w_{t_i}(t)$, following the framework presented in Chapter 3. Then for each new resource $r \in \{r^1_{t_i}, \ldots, r^n_{t_i}\}$ and activity $a \in \mathcal{A}_{t_i}$, we define a probability of selecting that resource given that activity, looking at the frequencies in the new sliding window $\mathcal{S}_{w,t_i}$, thus defining $p^R_{t_i}(r)(a)$. For any new activity $a \in \{a^1_{t_i}, \ldots, a^m_{t_i}\}$, we train a new probabilistic decision tree to estimate its execution time $p^{ET}_{t_i}(a)$. Finally, for any new resource $r \in \{r^1_{t_i}, \ldots, r^n_{t_i}\}$, we train a new probabilistic decision tree to estimate the waiting times $p^{WT}_{t_i}(r)$.

- Existing predictive models in $K_{t_{i-1}}$ are continuously refined using Online Machine Learning techniques. These methods enable incremental updates, allowing models to evolve with the latest observed behaviors without discarding previous knowledge. Several Online Machine Learning techniques exist in the literature; however, we chose Hoeffding Adaptive Trees [28] due to their explainability and ability to adaptively learn from data streams that evolve over time, making them robust to concept drift (cf. Section 1.4.2). These trees incrementally update their structure based on incoming data, using statistical tests to determine when to split nodes, enabling efficient and adaptive learning. Moreover, Hoeffding Adaptive Trees incorporate drift detection mechanisms, selectively updating branches of the tree when significant changes in the data distribution are detected.



(a) Single large batch. **(b)** Last small batch. **(c)** Online.

Figure 5.3: Illustration of the evaluation methodology. Rectangles depict traces over time. The blue color indicates the events used for training the simulation model. The orange indicates the traces used for testing it.

Through these updates, the set of predictive models evolves into an updated set K_{t_i}, contributing to the refinement of the overall simulation model. The final resulting simulation model $M_t = M_{t_i} = (N_{t_i}, D_{t_i}, K_{t_i})$ incrementally incorporates the behaviors observed in the most recent data window $\mathcal{S}_{w,t_i}$, thus being representative for the upcoming period.

5.5 Experiments

In this section, we present the experiments conducted to evaluate the performance and applicability of our discovery technique. The goal is to show its ability to produce accurate process simulations over time. To this aim, we monitored the accuracy by dividing the temporal dimension into multiple windows and computing accuracy metrics for each.

We compared our results using three different discovery techniques: (a) the **single large batch** technique, which uses all previous data, (b) the **last small batch** technique, which considers only the data from the most recent time window, and (c) our proposed **online** discovery technique. Specifically, (a) and (b) serve as baseline techniques for comparison. This evaluation aims to assess the importance of prioritizing the most recent data (online/last vs single batch), and preserving historical information (online/single vs last batch). Figure 5.3 illustrates the three cases. Note that compared to other approaches, our online technique allows the use of trace fragments. This comparison examines whether assigning greater weight to recent data improves the accuracy of simulation models for predicting the future. At the same time, we also explore whether retaining historical data is essential for better estimating simulation parameters.

5.5.1 Experimental Setup

The online discovery technique of business process simulation model has been implemented as part of ProSiT (see Chapter 9). We used the implementation of [170] in the Cortado library [171].[1] Additionally, we integrated the River library [139] which implements Hoeffding Adaptive Trees.[2]

We conducted experiments on four different event logs, each representing a different process: BPIC17W, BPIC12W, Purchase to Pay (P2P), and CVS retail pharmacy (see Section 1.7).

The temporal dimensions of the event logs have been divided into 10 temporal windows as follows: we counted the number of weeks from the earliest to the latest timestamp, and split the weeks in 10 obtaining the windows $W_1, \ldots, W_{10}$. The choice of 10 windows aims to balance the significance of each window, on the one hand, and the satisfactory frequency of model updates. Using more than 10 windows would reduce the data per window to less than 10% of the total, potentially compromising the statistical reliability of each update. On the other hand, using fewer than 10 windows would result in infrequent model updates.

Denote the latest timestamp in W_i with t_i, we consider process simulation models at timestamps $t_1, \ldots, t_{10}$, which we discovered via two baseline techniques and ours. In particular, the simulation model at timestamp t_i is discovered as follows for two baseline techniques, namely technique (a) and (b), and our technique proposal, i.e. technique (c):

(a) We employed traditional, not incremental, discovery techniques for process simulation models (see below), using the traces contained in $W_1, \ldots, W_i$.

(b) We employed traditional discovery techniques for process simulation models, using the traces in the only window W_i.

(c) Our online discovery technique began by creating an initial simulation model using completed traces from the first window W_1. The simulation model is incrementally updated using events in $W_2, \ldots, W_i$.

The experimental results at timestamp t_i are those measured against the test set containing completed traces that started in the following window, i.e. W_{i+1}. Specifically, the obtained simulation model at time t_i is used for generating an event log that is then compared with the event log containing traces started in window W_{i+1}. The comparison is based on the distances proposed by Chapela-Campa et al. in [47], which assess the accuracy of the simulation discovery

[1] https://github.com/cortado-tool/cortado-core

[2] https://riverml.xyz/

technique from various perspectives (see Section 1.6.3). Notably, for the P2P process, no trace was in W_{10}, preventing the assessment of the model obtained at timestamp t_9, which is thus not considered in the results. The same has happened for the CVS process where four windows $W_7, \ldots, W_{10}$ were empty.

For the techniques (a) and (b) in the list above, control-flow process models have been discovered using Inductive Miner [105], while the decision trees modeling the predictive models are obtained via the CART algorithm, and imposing a maximum depth of 5 for maintaining explainability and avoid overfitting.

For our technique, i.e, technique (c) in the list above, the control-flow model discovered in the first time window W_1, is also obtained via Inductive Miner [105], while the control-flow model was subsequently adapted, as per Step 1 discussed in Section 5.4. For the other perspectives, we leverage on Hoeffding Adaptive Trees, for which we set a maximum depth of 5, as done for techniques (a) and (b). Moreover, we experimented with various *grace periods* (100, 500, 1000, 5000, 10000, 50000), which determine the number of instances observed before considering a split at a node, and using in the simulation model the one with best accuracy. Particularly, lower grace periods facilitate rapid adaption to sudden concept drifts, while higher grace periods reduce the risk of overfitting and promote more stable decisions (cf. Section A.2).

5.5.2 Results

To assess the accuracy of our proposed technique, we ran simulations for each time window and computed the distances between the event logs obtained via the different simulation techniques, and the original ones. Distances were computed using the metrics proposed in [47]. Specifically, CFLD and 3GD assess control-flow related distances, AED and RED measure number of events distributions over time, in absolute or relative terms, respectively. CED and CWD evaluate the simulator's ability to accurately replicate events within the circadian dimension (day and hour of the week). CAR quantifies the accuracy of case arrivals, and CTD measures the difference in cycle time distributions, implicitly capturing the accuracy of execution waiting times [47].

For each use case, we conducted five simulations and computed the average distances between the obtained event logs for each time window. The final results are presented in Table 5.1, where the column with the green background highlights our technique. The reported values represent the average distance across all time windows, with standard deviations provided in parentheses. Since they are distances, lower average values indicate better performances. Additionally, lower standard deviation values suggest greater stability and consistency in accuracy across the temporal dimension.

Table 5.1: Average results of simulations between time windows per each technique. Numbers in parentheses indicate standard deviations. The online technique results are highlighted with a green background.

Measure	Event Log	Single Batch	Last Batch	Online
CFLD	BPIC17W	**0.16** (0.05)	0.17 (0.04)	0.18 (**0.02**)
	BPIC12W	0.25 (0.08)	0.42 (0.07)	**0.17** (**0.05**)
	P2P	**0.2** (**0.03**)	0.39 (0.18)	**0.2** (**0.03**)
	CVS	**0.02** (**0.0**)	0.07 (0.08)	**0.02** (**0.0**)
	Avg.	0.16 (0.04)	0.26 (0.09)	**0.14** (**0.03**)
3GD	BPIC17W	**0.18** (0.07)	0.19 (0.05)	0.19 (**0.04**)
	BPIC12W	0.26 (0.11)	0.54 (**0.06**)	**0.25** (0.09)
	P2P	**0.22** (**0.02**)	0.4 (0.19)	**0.22** (**0.02**)
	CVS	**0.03** (**0.0**)	0.1 (0.1)	**0.03** (**0.0**)
	Avg.	**0.17** (0.05)	0.31 (0.1)	**0.17** (**0.04**)
AED	BPIC17W	1205.86 (643.01)	1260.28 (**579.05**)	**1100.72** (680.09)
	BPIC12W	736.2 (530.88)	717.86 (548.07)	**687.44** (**518.25**)
	P2P	1475 (622.17)	**869.08** (**504.36**)	1132.41 (626.99)
	CVS	60.63 (24.5)	5711.09 (10952.39)	**38.54** (**16.89**)
	Avg.	869.42 (**455.14**)	2139.58 (3145.97)	**739.78** (460.56)
RED	BPIC17W	41.52 (21.68)	83.74 (22.57)	**24.01** (**11.11**)
	BPIC12W	72.96 (28.24)	156.32 (41.07)	**44.61** (**22.36**)
	P2P	616.65 (270.55)	676.77 (351.08)	**535.81** (**252.49**)
	CVS	51.95 (13.74)	47.4 (10.0)	**20.19** (**3.0**)
	Avg.	195.77 (83.55)	241.06 (106.18)	**156.16** (**72.24**)
CED	BPIC17W	1.38 (0.9)	1.42 (1.22)	**0.82** (**0.61**)
	BPIC12W	2.89 (1.22)	2.45 (1.55)	**2.45** (**1.5**)
	P2P	**0.88** (0.22)	2.34 (0.86)	1.06 (**0.15**)
	CVS	0.27 (0.15)	1.06 (0.6)	**0.13** (**0.03**)
	Avg.	1.36 (0.62)	1.82 (1.06)	**1.12** (**0.57**)
CWD	BPIC17W	1.25 (0.87)	1.33 (1.18)	**0.69** (**0.39**)
	BPIC12W	2.77 (**1.16**)	**2.27** (1.53)	2.37 (1.51)
	P2P	**0.85** (**0.13**)	2.13 (0.89)	0.96 (0.14)
	CVS	0.22 (0.06)	0.71 (0.54)	**0.19** (**0.05**)
	Avg.	1.27 (0.56)	1.61 (1.04)	**1.05** (**0.52**)
CAR	BPIC17W	1262.21 (678.0)	1272.08 (**574.62**)	**1241.19** (761.5)
	BPIC12W	791.45 (628.59)	**718.37** (**524.64**)	778.33 (590.46)
	P2P	1025.01 (**431.99**)	**617.11** (478.91)	799.21 (452.99)
	CVS	25.28 (18.66)	5759.53 (10963.3)	**18.51** (**13.66**)
	Avg.	775.99 (**439.31**)	2091.77 (3135.37)	**709.31** (454.65)
CTD	BPIC17W	63.6 (41.46)	99.07 (34.12)	**36.82** (**17.78**)
	BPIC12W	94.26 (41.86)	194.03 (37.21)	**60.39** (**23.75**)
	P2P	729.88 (357.63)	900.86 (504.84)	**647.69** (**294.34**)
	CVS	95.75 (27.28)	85.95 (23.21)	**41.07** (**3.66**)
	Avg.	245.87 (117.06)	319.98 (149.85)	**196.49** (**84.88**)

The results demonstrate that in general our technique outperforms the baselines. Indeed, we achieve equal or superior average results across all metrics, highlighting its effectiveness. Notably, the last batch technique consistently yields poor average results, emphasizing the importance of not just ignoring older portions of event-data sets. Looking at the control-flow measures (CFLD

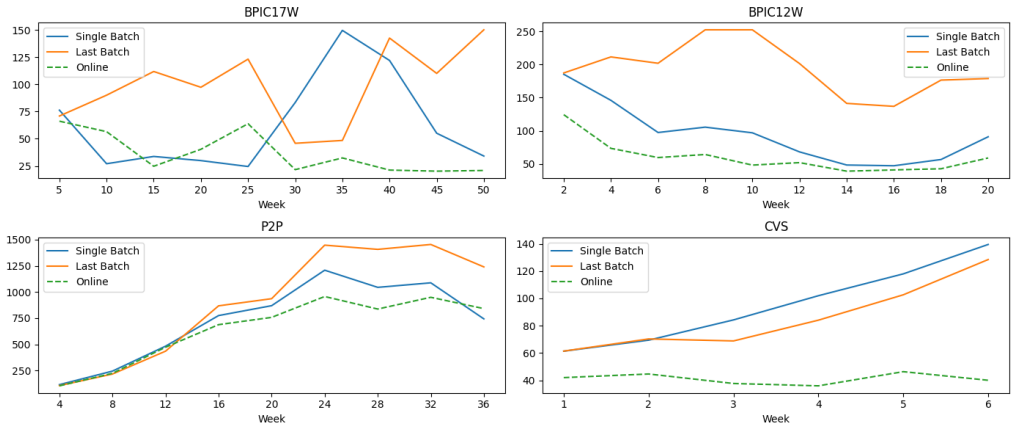


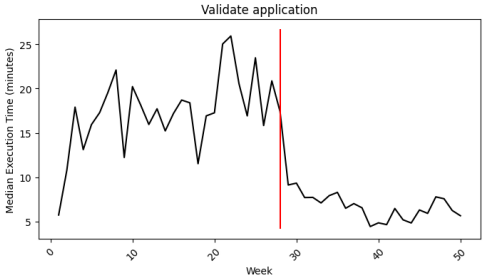
Figure 5.4: Cycle Time Distribution (CTD) distance over time for each use case.

and 3GD), we can notice that they are very close to those obtained using the single batch technique. This suggests that no significant process changes were detected in these control-flow metrics. Very good results are achieved with the Cycle Time Distribution (CTD) and the Relative Event Distribution (RED) distances, consistently outperforming the other techniques. These results indicate that our technique effectively adapts to temporal perspective changes.

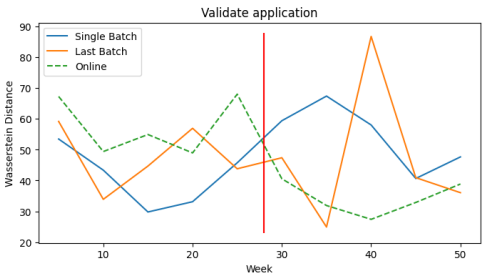
Analyzing individual event logs and metrics, our technique achieves the best results in 24 out of 32 cases (i.e. 75% of the cases), compared to 8 cases (25%) for the single batch technique and only 4 cases (12.5%) for the last batch technique. Note that there are 4 cases where our technique and the single batch technique yield identical average results. Standard deviation analysis further supports the robustness of our technique. In 24 out of 32 cases, our technique demonstrates greater stability and consistency over time compared to the other two techniques.

We also visually explored the results through time windows. Figure 5.4 illustrates the Cycle Time Distribution (CTD) distance over time for each use case. Our technique consistently outperforms the others in BPIC12W and CVS. In some cases, such as BPIC17W and CVS, the last batch technique yields better results than the single batch technique. The results over time windows for each metric are illustrated in Appendix A, also showing results with fixed grace periods. Overall, our technique demonstrates greater stability, producing more consistent results over time.

The BPIC17W use case was one for which our technique has most remarkably outperformed the baseline, especially in the comparison metrics related to the time and resource perspectives. Previous work has indeed highlighted a presence of significant drift related to the increase of the resource workload [7]. For this case study, we conducted a more thorough assessment related to activ-



(a) Median duration over weeks (in minutes) of the *Validate Application* activity.



(b) Wasserstein distances between the actual distribution of durations and those obtained via simulations over weeks.

Figure 5.5: *Validate application* execution time results. The vertical red lines indicate when the concept drift has been detected in [7].

ity *Validate Application*: the average duration of the activity shows a reduction of 38% at week 28 (see Figure 5.5a). In particular, we computed the Wasserstein distance [154] between the simulated and real execution time distributions for *Validate application*. Figure 5.5b shows these distances over weeks, where one could clearly see that our technique certainly leads to lower Wasserstein distances after the concept drift, except for two weeks, if compared with the baseline techniques. This also implicitly contributes to better CTD results after week 28 (see Figure 5.4).

5.6 Conclusion

Traditional Business Process Simulation discovery techniques rely on analyzing finite sets of historical traces. These methods assume the process does not change over time, treating both past and recent event behaviors equally. However, real-world business process can evolve over time due to internal policies changes aimed at process improvement or external influencing factors. In this paper, we proposed a simulation discovery technique able to adapt to evolving processes.

Chapter 4 has demonstrated how tree-based process simulation models can produce more accurate simulations. To maintain high accuracy, we combined Incrementally Process Discovery [169] with Online Machine Learning [28] techniques. As new process instances are executed, the simulation model is updated. The control-flow perspective of the simulation model is updated using the technique proposed by Schuster et al. [170], while the other perspectives rely on predictive models that are based on Hoeffding Adaptive Trees [28], which can adaptively learn from data streams that evolve over time.

The conducted experiments (cf. Section 5.5) reveal the potential effectiveness of our technique. The evaluation uses four distinct processes and their associated event logs. The results show that our technique can potentially lead to more accurate results across various perspectives, and it is more stable over time.

We acknowledge that the work by Schuster et al. [170] does not explicitly remove unobserved behaviors from the process model, and this is an open challenge in the fields of Incremental Process Discovery and Repair. However, this does not pose major challenges in our studies, since we rely on predictive models to estimate branching probabilities at decision points, rare or unobserved paths are naturally assigned near-zero probabilities. However, we acknowledge that the readability of the models would be improved, if those "unused" parts were removed from the model. We plan to work on this as an avenue of future work.

6 Repair of Process Simulation Models

This chapter addresses the problem of **repairing process models** used as back-bone of business process simulation models. The goal is to improve the quality of these models so that they provide a realistic representation of observed process behavior and support reliable simulation-based analysis. To this end, we present a novel model-repair framework that integrates process simulation, Machine Learning, and Explainable AI to systematically identify and reduce discrepancies between simulated and real executions, with a particular focus on balancing fitness and precision from the control-flow perspective.

6.1 Introduction

Process models are used to engage stakeholders in discussions about how processes should be executed, or alternatively serve as input for Process-aware Information Systems (PAIS) to support or automate process execution [1]. In the context of business process simulation, they are the *back-bone of simulation models* (cf. Section 1.6) and describe the behavior allowed in simulation.

Desirable models need to be precise and only allow legitimate behavior, while they should be able to represent the valid executions that have been observed (high fitness). Conversely, e.g., an imprecise process model would lead to a business simulation model that enables executions that would not be observed in reality, while a low-fitness model would ultimately cause some legitimate behavior to never be simulated: in both cases, process analysts would gain insight on the basis of unrealistic simulations.

When process models may fail to achieve satisfactory level of fitness and precision, they need to be repaired. However, fitness and precision are often contrasting: improving one may worsen the other.

This chapter focuses on the control-flow perspective and reports on a novel model-repair framework that tries to improve the harmonic means of fitness and precision, so as to provide a better balance between these two dimensions. The core idea is to repeatedly simulate executions from the given process model, obtaining a simulated event log, and to systematically compare this log with the corresponding real event log. To enable this comparison, a Machine Learning–based discriminator is trained to distinguish between simulated and real traces. SHAP (cf. Section 1.4.4) is then used to interpret the predictions of the discriminator by identifying the most discriminating features. Based on these insights, the process model is automatically repaired in an iterative manner, progressively reducing the behavioral gap between simulated and real executions.

To demonstrate the effectiveness of our proposed technique, we conducted evaluations on five distinct real-life processes and various models, which were literature hand-made or mined models. The results showcase the technique's ability to enhance the original process models, improving on both fitness and precision. The evaluation also compares the resulting repaired models with respect to those obtained by the technique of Fahland et al. [64], which is a de-facto reference technique in model repair. The comparison again highlights how the technique proposed in this thesis achieves a better balance between fitness and precision.

To showcase the benefits of better balancing fitness and precision in business process simulation, the repaired models are used as backbone of simulation models. The evaluation of the same case studies illustrate that the resulting simulation models are more realistic: this is testified by the fact that the simulation models produce sets of process execution, in fact simulated event logs, which are closer to the original real-life event logs, compared with the simulated models built upon the original model and upon the model repaired by the technique by Fahland et al. [64].

The rest of the chapter is organized as follows. Section 6.2 introduces the framework, Section 6.3 outlines the evaluation and results. Section 6.4 reports the related work. Section 6.5 discusses how our model repair technique, which is currently focused on the control-flow perspective, can be extended to incorporate the other perspectives, namely data, resource and time, thus enabling tighter integration with process simulation; in particular, a proposal towards multi-perspective process repair is proposed, along with delineating open challenges. Finally, Section 6.6 concludes the discussion.

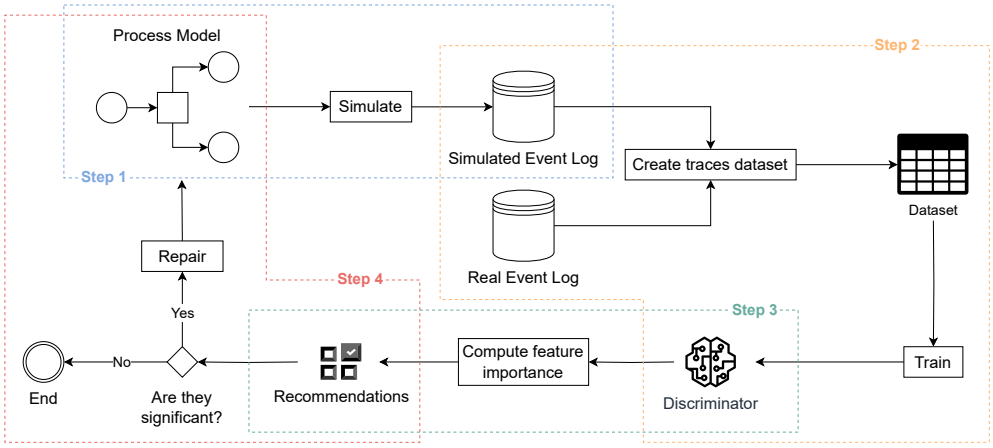


Figure 6.1: The schema of the proposed framework. Step 1 provides a simulated event log; in Step 2 a dataset of real and simulated traces is created for training a Machine Learning discriminator model; Step 3 regards the computation of the feature importance, providing recommendations for repairing the model; in Step 4 the recommendations are used for automating process model repairs.

6.2 Framework

The framework can be summarized by the schema illustrated in Figure 6.1. The key idea is to leverage on Business Process Simulation: the original process model, namely before any repair, is used to generate a set of simulated traces. These traces are then compared with those of a real event log to highlight the differences. This comparison is based on training a Machine Learning (ML) model that, given an execution trace, is capable to classify whether it is from the real or simulated event log. The rationale is that, if the process model is characterized by high fitness and precision, the ML model is not capable to determine whether a trace belongs to the real or simulated event log. In other words, the lower is the harmonic means of fitness and precision in a process model, the higher is the accuracy of the ML model. When the ML model accuracy is high, it is possible to pinpoint which trace characteristics are used to discriminate whether the trace is from the real or simulated event log. In this chapter, we use the SHAP (SHapley Additive exPlanations) method to pinpoint the differences (cf. Section 1.4.4). These trace characteristics are then employed to automatically improve the process model, aiming to achieve a process model that, once simulated, would produce traces that are indistinguishable from the real event log.

The framework includes a *Repair* step (cf. Figure 6.1), in which the identified

recommendations for process repair are operationalized to modify the process model accordingly. We propose three types of repair strategies: a **basic** strategy that applies recommendations directly, a **greedy** strategy that iteratively selects the most beneficial recommendation, and an **alignment-based** strategy that leverages log-model alignments (cf. Section 1.3.4). While the basic and greedy strategies assume each activity to only occur in a single transition of the model, the alignment-based lifts this assumption.

Section 6.2.1 to 6.2.4 describe the details of the four steps of the framework. Particularly, Section 6.2.4 introduces three kinds of possible approaches for repairing the process model.

6.2.1 Step 1: Process Simulation

The framework takes as input an event log $\mathcal{L}^r$ over an activity set $\mathcal{A}$, together with a Petri net $N = \left(P, T, F, \lambda, m^I, m^F\right)$ with labeling function $\lambda : T \to \mathcal{A} \cup \{\tau\}$ (cf. Section 1.3.2).

The event log is temporally split into training and validation sets: the first part of the traces is allocated to the training set and the remaining traces to the validation set. The former is used to train the model and discover the discriminatory features, while the latter to validate the tuning of hyper-parameters, aiming to avoid underfitting and overfitting of the discriminator model.

Starting from the event log $\mathcal{L}^r$, a *simulation model* is discovered and subsequently used to generate simulated behavior. Since in this work we focus exclusively on the control-flow perspective, only control-flow–related parameters, namely the transition firing probabilities, are learned. These probabilities are estimated by analyzing the frequencies of transition firings in the optimal alignments between the log and the Petri net.

The simulation is then executed for a number of process instances equal to the number of traces in the original event log $\mathcal{L}^r$. This generates a simulated event log $\mathcal{L}^s$, which is subsequently divided into a train and a validation set, with the same number of traces of the splitting in the real event log. This balanced distribution facilitates a fair comparison and evaluation of the framework performance against real data.

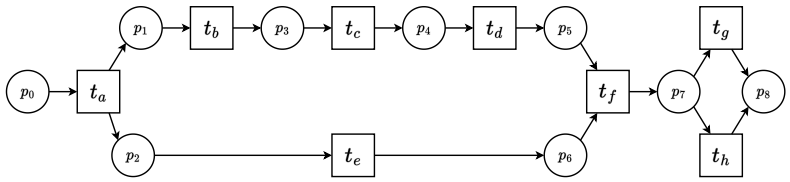


Figure 6.2: Petri net example. Circles and squares represents respectively places and transitions. We indicate visible transition with label l with t_l.

Example: Let $\mathcal{L}^r$ be an event log:

$$\mathcal{L}^r = \{\langle a,b,d,i,f,g\rangle, \langle a,e,b,c,d,f,g\rangle, \langle a,b,c,d,e,f,g\rangle, \langle a,b,c,d,e,i,f,h\rangle\}$$

and N the Petri net illustrated in Figure 6.2. Each transition of the Petri net is assigned a weight based on the frequency of its occurrence in the optimal alignments. Then, given a marking, the probability of firing a certain transition t is computed as the ratio between the weight of t and the sum of the weights of all enabled transitions at that marking. For example, the probability of firing the transition labelled with g, when it is enabled, is 0.75. The Stochastic Petri net can then be used for obtaining a simulated event log:

$$\mathcal{L}^s = \{\langle a,b,c,e,d,f,g\rangle, \langle a,b,c,d,e,f,g\rangle, \langle a,b,e,c,d,f,g\rangle, \langle a,e,b,c,d,f,h\rangle\}$$

6.2.2 Step 2: Detection of Inaccuracies of Simulation Models

In the second phase, the real and simulated event logs $\mathcal{L}^r$ and $\mathcal{L}^s$, are used to create a training set to be used for the machine learning discrimination model.

Given a trace $\sigma = \langle e_1, \ldots, e_n\rangle$, we define the directly-follow relation $>_\sigma \subset \mathcal{A} \cup \{\bot\} \times \mathcal{A} \cup \{\bot\}$, such that $(a,b) \in >_\sigma$ iff $\exists\, i = 1, \ldots, n-1$ such that $act(e_i) = a \wedge act(e_{i+1}) = b$ or $act(e_1) = b \wedge a = \bot$ or $act(e_n) = a \wedge b = \bot$, and we write $a >_\sigma b$. We then populate the directly-follow feature set $\mathcal{F}^{>}_{\mathcal{A}} = \{f_{a>b} \mid (a,b) \in \mathcal{A} \cup \{\bot\} \times \mathcal{A} \cup \{\bot\}\}$. The creation and population of these features are in accordance to the following mapping function:

Definition 19 (Trace-to-Feature Mapping Function). *Let $\mathcal{L}$ be an event log over a set of activity labels $\mathcal{A}$. Let $\mathcal{F}^{>}_{\mathcal{A}}$ be the set of directly-follow features. We define a trace-to-feature mapping function $\rho^{\mathcal{L}} : \mathcal{L} \to \left(\mathcal{F}^{>}_{\mathcal{A}} \to \{0,1\}\right)$ s.t. $\rho^{\mathcal{L}}(\sigma) = \nu^\sigma$ where,*

given $\sigma \in \mathcal{L}$, $\forall f_{a>b} \in \mathcal{F}^{>}_{\mathcal{A}}$:

$$\nu^{\sigma}(f_{a>b}) = \begin{cases} 1 & \text{if } a >_{\sigma} b \\ 0 & \text{otherwise} \end{cases} \tag{6.1}$$

With these concepts at hand, the training dataset $T_{\mathcal{L}^r,\mathcal{L}^s}$ of the discriminator model is constructed as follows:

$$T_{\mathcal{L}^r,\mathcal{L}^s} = \biguplus_{\sigma \in \mathcal{L}^r} (\rho^{\mathcal{L}^r}(\sigma), 1) \uplus \biguplus_{\sigma \in \mathcal{L}^s} (\rho^{\mathcal{L}^s}(\sigma), 0) \tag{6.2}$$

where 1 and 0 indicate that the trace belongs to the real and simulated event logs respectively.

Example: Given the real event log

$$\mathcal{L}^r = \{\langle a,b,d,i,f,g\rangle, \langle a,e,b,c,d,f,g\rangle, \langle a,b,c,d,e,f,g\rangle, \langle a,b,c,d,e,i,f,h\rangle\}$$

and the simulated one

$$\mathcal{L}^s = \{\langle a,b,c,e,d,f,g\rangle, \langle a,b,c,d,e,f,g\rangle, \langle a,b,e,c,d,f,g\rangle, \langle a,e,b,c,d,f,h\rangle\}$$

Table 6.1 describes the training dataset: each row represents the encoding of a trace, where the last column indicates if the trace belongs to the real or simulated event log (1 if `real`, 0 otherwise), and the others the correspondent features. For example, considering the trace $\sigma = \langle a,b,d,i,f,g\rangle$, the value of the features $f_{a>b}$, $f_{b>d}$, $f_{d>i}$, $f_{i>f}$ and $f_{f>g}$ are 1 since $a >_{\sigma} b$, $b >_{\sigma} d$, $d >_{\sigma} i$, $i >_{\sigma} f$ and $f >_{\sigma} g$ while the other features take value 0.

The constructed dataset may have variables with a high correlation, leading to collinearity. When collinearity exists, it can be a challenge to interpret the individual effects of these variables on the target variable. In fact, it becomes difficult to discern whether a feature has a significant impact on the target variable independently or whether its effect is captured by other correlated features in the model. To overcome these issues, a feature selection is performed, considering the correlation between the features: the framework iterates across the features, and for each one, it considers the set of the features with an absolute correlation greater than a given threshold, only retaining one for each of these correlation groups, whose choice can be arbitrarily done.[1] Furthermore, the features that are associated with a variance lower than a fixed threshold are removed. The

[1] Usually, suitable thresholds are above 0.9. In our evaluation, we set it to 0.99.

Table 6.1: Training dataset example: each row represents the encoding of a trace, *target* is equal to 1 if the trace is `real`, 0 if `simulated`. For space reasons, we only show some features. The others follow the same reasoning.

	$f_{a>b}$	$f_{a>e}$	$f_{b>c}$	$f_{b>d}$	...	$f_{e>f}$	$f_{e>i}$	$f_{i>f}$	*target*
$\langle a,b,d,i,f,g\rangle$	1	0	0	1	...	0	0	1	1
$\langle a,e,b,c,d,f,g\rangle$	0	1	1	0	...	0	0	0	1
$\langle a,b,c,d,e,f,g\rangle$	1	0	1	0	...	1	0	0	1
$\langle a,b,c,d,e,i,f,h\rangle$	1	0	1	0	...	0	1	1	1
$\langle a,b,c,e,d,f,g\rangle$	1	0	1	0	...	0	0	0	0
$\langle a,b,c,d,e,f,g\rangle$	1	0	1	0	...	1	0	0	0
$\langle a,b,e,c,d,f,g\rangle$	1	0	0	0	...	0	0	0	0
$\langle a,e,b,c,d,f,h\rangle$	0	1	1	0	...	0	0	0	0

validation set is then constructed following the same procedure and using the same features selected by the feature selection phase described above.

Then a ML model is developed to predict if traces are real or simulated: the output of the model has to be intended as the probability that the input trace is real. The model is trained using the training set $T_{\mathcal{L}^r,\mathcal{L}^s}$, defined in Equation 6.2, and the validation set is used to validate the procedure and fine-tune the hyper-parameters of the model.

6.2.3 Step 3: Identification of the Discriminating Features

Once the model is trained, we create a multiset $\mathcal{X}^{true}$ that is obtained from the training-set instances that the ML model has correctly classified, projected over the independent features (i.e., removing the target variable).[2] This is done in order to understand the set of features that enable a correct discrimination. We compute the impact of the features in the output for the samples in $\mathcal{X}^{true}$ to provide explanations of certain classifications (cf. Section 1.4.4). For each activity pair $a, b \in \mathcal{A}$:

- If the feature $f_{a>b}$ taking value 1 (or 0) increases (decreases, resp.) the probability for a trace to belong to the real event log, i.e. $\phi_{f_{a>b}}(1, \mathcal{X}^{true}) > 0$ ($\phi_{f_{a>b}}(0, \mathcal{X}^{true}) < 0$, resp.), it means that the model does not sufficiently allow for a being followed by b. Therefore, the model needs to enable this.

- If $\phi_{f_{a>b}}(1, \mathcal{X}^{true}) < 0$ (or $\phi_{f_{a>b}}(0, \mathcal{X}^{true}) > 0$), the feature $f_{a>b}$ taking value 1 (0, resp.) impacts the prediction to classify a trace as belonging to the

[2]The ML model returns a probability value between 0 and 1. Therefore, an instance with a true value of 1 is considered as correctly classified if the probability value is larger than 0.5. Similarly, an instance with a true value of 0 is correctly classified if the value is smaller than 0.5.

simulated (real, resp.) event log. This means that the real event log does not show that behavior.

In the remainder, R and $\overline{R}$ are respectively the set of features related to behavior that needs to be included into or excluded from the model, namely the first or second case in the list above. At this stage, the features that were removed because they were strongly correlated are considered again: any feature exhibiting a strongly positive correlation with another within the set R is incorporated into R. Similarly, if any feature is negatively correlated to any other in $\overline{R}$, then it is added to R.

Example: Let us assume that a Machine Learning model M has been trained on the basis of the data set in Table 6.1, which encodes the traces in both real and simulated event logs. When the same dataset, excluding the *target* variable, is used for testing, a value between 0 and 1 is predicted, as shown in the column *prediction value* in Table 6.2: values below 0.5 are going to be treated as if the *predicted* target variable takes on value 0 (trace classified as belonging to the `real` trace), whereas values above 0.5 are going to be treated as if it takes on value 1 (`simulated` trace).

In Table 6.1, one can observe that the predicted value of the second and third trace, highlighted in red, are far below 0.5, thus predicting these traces belong to the real event log: however, those traces are wrongly predicted, because they belong to the simulated event logs. After removing the vectors for those two traces and filtering out the *target* feature/column, the resulting set forms the support set $\mathcal{X}^{true}$ required by SHAP (cf. Section 1.4.4). Now, the Shapley values can be computed. Suppose to have $\phi_{f_{b>d}}(0, \mathcal{X}^{true}) = -0.25$. This means that when the feature $f_{b>d}$ takes value 0 in a trace, it causes a decrease of 0.25 in the model's prediction output, thereby reducing the probability to be classified as `real` (and increasing the probability of it being classified as `simulated`). This means that traces in which activity b is not followed by d are more likely being part of original event log than part of the simulated event log, which indicates that the original model should be repaired so as to allow activity d to follow b. Similar reasoning can be done because $\phi_{f_{i>f}}(1, \mathcal{X}^{true}) = 0.2$, meaning that when the feature $f_{i>f}$ takes value 1 in a trace, it causes the increasing of 0.2 in the prediction output: the possibility for activity f to follow i should be incorporated into the repaired model.

Table 6.2: Predictions of the Machine Learning model example. Column *actual target value* represent if the trace is real or simulated (1 and 0 respectively), while column *predicted value* the output of the Machine Learning model. The output has to be intended as the probability of the trace to be real. Green values represent correct predictions (i.e. if the value is closest to the target value), red values wrong predictions.

	$f_{a>b}$	...	$f_{i>f}$	*actual target value*	*predicted value*
$\langle a, b, d, i, f, g \rangle$	1	...	1	1	0.99
$\langle a, e, b, c, d, f, g \rangle$	0	...	0	1	0.10
$\langle a, b, c, d, e, f, g \rangle$	1	...	0	1	0.45
$\langle a, b, c, d, e, i, f, h \rangle$	1	...	1	1	0.90
$\langle a, b, c, e, d, f, g \rangle$	1	...	0	0	0.15
$\langle a, b, c, d, e, f, g \rangle$	1	...	0	0	0.45
$\langle a, b, e, c, d, f, g \rangle$	1	...	0	0	0.10
$\langle a, e, b, c, d, f, h \rangle$	0	...	0	0	0.10

6.2.4 Step 4: Process Model Repair

In this final step, the set of features R is used for repairing the Petri net $N = (P, T, F, \lambda, m^I, m^F)$, with $\lambda : T \to \mathcal{A} \cup \{\tau\}$, where $\mathcal{A}$ is a set of activity labels, and $G_N = (V, E)$ is the reachability graph. We assume the Petri net N to be sound and safe (cf. Section 1.3.2). This is not very limiting: several process-discovery algorithms, such as Inductive Miner [105], produce Petri nets for which safety holds; also BPMN models with the typical constructs (AND and XOR splits and joins) can be modeled as a safe Petri net. Assuming soundness is also very reasonable: unsound process models cannot be used to enact processes, making them impractical and essentially futile. Repairing models to ensure soundness is orthogonal to what proposed here, and may be a preliminary step before applying our framework.

The remainder of this section proposes three different approaches to repair the process model to incorporate the direct-following relation $a > b$ for each discriminating feature $f_{a>b} \in R$. It reports on a first **basic approach**, which requires each visible transition to be associated to a different label, namely two different visible transitions can be associated to the same label. The basic approach repairs the Petri net by introducing several new transitions, which can be slow to perform, on the one hand, and can potentially hamper the net's precision and simplicity, on the other hand. The section then introduces a **greedy approach** that implements some heuristics that enables introducing a smaller subset of the changes, if compared with the basic approach, although this might negatively affect the quality of the repairing model. Last,it proposes an alternative **approach based on log-model alignments**, which allows multiple

visible transitions to be associated to the same activity labels.

Note that, since $\perp$ is included for defining the set of features $\mathcal{F}_{\mathcal{A}}^{>}$ (cf. Section 6.2.2), a transition with such a label could be added in the model when a new "start" or "end" transitions need to be defined. When this happens, this transition can be deleted within its ingoing and outgoing arcs at the end of the repair steps, without compromising the proposed repair approaches.

The Basic Approach

As mentioned above, the basic technique requires that each visible transition be associated with a different label. We indicate with t_l the transition with label $l \in \mathcal{A}$, i.e. $\lambda(t_l) = l$.

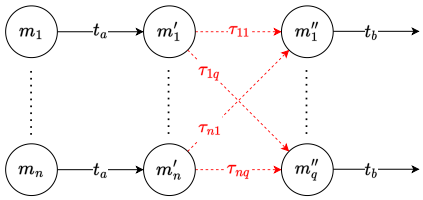
For any feature $f_{a>b} \in R$ we should consider four categories: (i) when there exist net transitions with labels a and b in the original Petri net N, (ii) and (iii) when one of them is missing, and (iv) when both are missing. The model is repaired differently for each category:

(i) If $a \not>_N b$ and $t_a, t_b \in T$, we alter the Petri net such that the corresponding reachability graph contains one edge (m'_i, τ_{ij}, m''_j), for each marking $m'_1, \ldots, m'_n \in V$ reached after firing transition t_a and for each marking $m''_1, \ldots, m''_q \in V$ in which t_b is enabled. This is pictorially represented in Figure 6.3a. The transitions τ_{ij} with $1 \leq i \leq n$ and $1 \leq j \leq q$ are invisible (i.e. $\lambda(\tau_{ij})$ is set to τ) and added to the Petri net. Figure 6.3b illustrates the resulting Petri net, in a case in which $n = 2$ and $q = 2$, where the inserted invisible transitions are shown in red. The above-mentioned changes in the reachability graph correspond to these specific changes in the Petri net because the Petri net is safe.

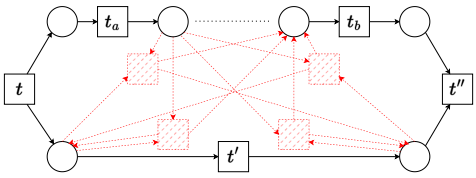
(ii) If $t_b \notin T$, we alter the Petri net as follows:

- for each $p \in t^\bullet$, we remove the arc (t_a, p) from F;
- we introduce a visible transition t_b with $\lambda(t_b)$ set to b, an invisible $\overline{\tau}$, and a place $\overline{p}$;
- for each $p \in t^\bullet$, we add the arcs (t_b, p) and $(\overline{\tau}, p)$, along with the arcs $(t_a, \overline{p}), (\overline{p}, t_b), (\overline{p}, \overline{\tau})$.

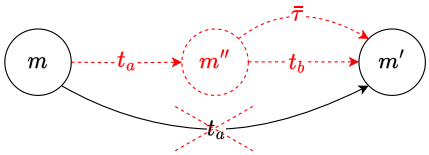
These changes are visually depicted in Figure 6.3d, which refers to a case where the outgoing places from t_a are two. Note the category (ii) is directly elaborated with respect to the Petri net, because the discussion is simpler. Of course, these changes in the Petri net imply modifications in the reachability graph, as illustrated in Figure 6.3c.



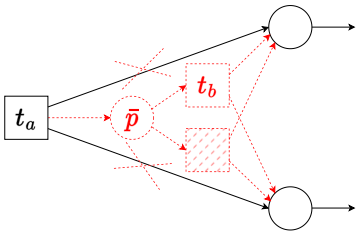
(a) Reachability graph: $t_a, t_b \in T$.



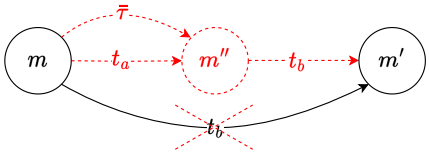
(b) Petri net: $t_a, t_b \in T$.



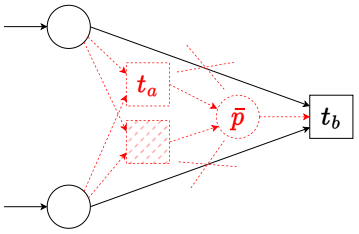
(c) Reachability graph: $t_a \in T, t_b \notin T$.



(d) Petri net: $t_a \in T, t_b \notin T$.



(e) Reachability graph: $t_a \notin T, t_b \in T$.



(f) Petri net: $t_a \notin T, t_b \in T$.

Figure 6.3: Petri net repairs. Red and dashed parts denote repairs. We indicate visible transition with label l with t_l, i.e. $\lambda^{-1}(l) = t_l$ if $l \neq \tau$. We assume, without loss of generality, that in category (i) (Figure 6.3b), there is only one parallelism involving a single transition. For categories (ii) and (iii) (Figure 6.3d-6.3f), we consider places to be disjoint. Any differences are negligible.

(iii) If $t_a \notin T$ the procedure is similar to the category (ii) where now we consider the places in ${}^\bullet t_b$. Figure 6.3f and 6.3e ought to be sufficient for a full understandability.

(iv) If $t_a, t_b \notin T$, then no repair is carried out. However, if $f_{a>b}$ is a discriminatory feature, there is surely a feature $f_{d>a}$ (or $f_{b>d}$, resp.) for some activity d.[3] The model would then be repaired wrt. $f_{d>a}$ (or $f_{b>d}$, resp.), namely the category *(ii)* or *(iii)* would be applied. This would incorporate transition t_a (or t_b) in the model, bringing this repair to category *(ii)* or *(iii)*, which will be incorporated through the subsequent framework iteration.

[3]It might unlikely be the case that d is not present, either. However, this would not invalidate the reasoning, because one can reiterate it until one transition is found.

The proposed repairs guarantee the soundness and safeness of the repaired Petri net if the original one is sound and safe:

Theorem 1. *Let $N = (P, T, F, m^I, m^F)$ be a sound and safe Petri net system, then the Petri net system N' obtained after applying all repair changes related to every category, namely (i), (ii) or (iii), is sound and safe.*

Proof. The safeness of N' can be easily proven: the only new markings are m_i'' in categories (ii) and (iii), then it is safe by construction.

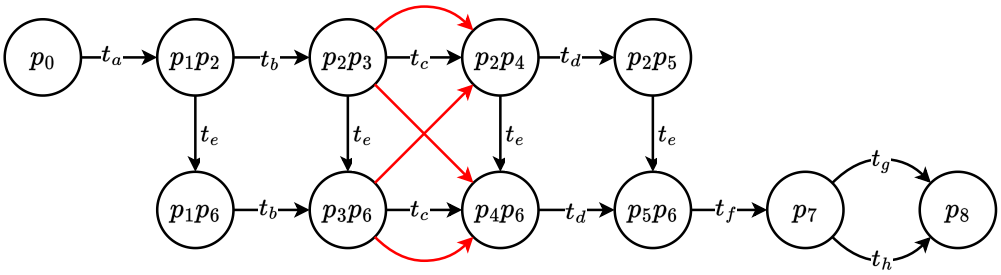
Regarding soundness, it is sufficient to prove the soundness properties (cf. Section 1.3.2) for every category.

Option to Complete. For category (i), the reachability graph of the repaired Petri net does not include new states/markings and no arcs are removed: only new arcs are inserted. Therefore, if every marking m of the original reachability graph allowed reaching the final marking m^F, then the same marking m still allows m^F to be reached, at least using the same path. For categories (ii) and (iii), the reachability graph is changed similarly to what depicted in Figures 6.3c and 6.3e, respectively. From the newly-introduced marking m'', it is possible to reach marking m', which allowed reaching the final marking in the original Petri net, and so like in the repaired Petri net. Similarly, the original marking m now enables arriving at marking m'', from which, as just mentioned, it is possible to reach the final marking: therefore, it is possible to reach the final marking from m.

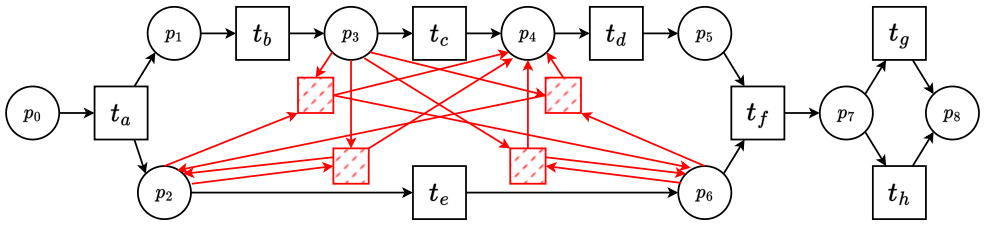
Proper completion. Let us define $o \in P$ as the final place, namely $m^F(o) = 1$ and, $m^F(p) = 0$ for all $p \in P \setminus \{o\}$. For category (i), this is straightforward, as no new markings are included and the initial Petri net is sound. For category (ii), we need to prove that $m''(o) = 0$. Let us assume $m''(o) > 0$ by contradiction. Since the repaired Petri nets is safe, there must be $m''(o) = 1$. There are two cases: $m' \neq m^F$ and $m' = m^F$. If $m' \neq m^F$, $m'(o) = 0$ (cf. Figure 6.3c). Since $m''(o) = 1$, when firing, transitions t_b and $\overline{\tau}$ consume the token in o. However, by construction (cf. Figure 6.3d), these transitions only consume one token from $\overline{p}$: then, $m''(o) = 0$, contradicting the hypothesis. If $m' = m^F$, $m'(o) = 1$. Since, by hypothesis, $m''(o) = 1$ and transitions t_b and $\overline{\tau}$, when firing, produce one token in o, hence $m'(o) = 2$, which contradicts the safeness properties. Category (iii) is analogous.

Absence of dead transitions. Every transition is present in the reachability graph of the repaired Petri net, which implies that no transition is dead. □

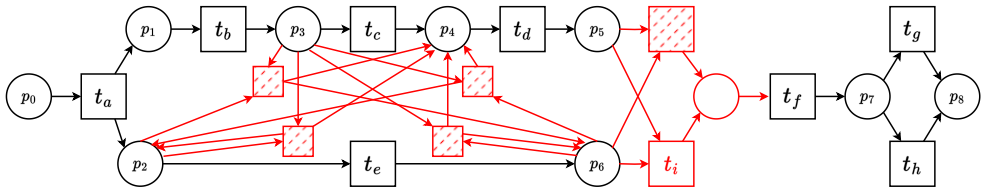
Note that, since the repaired Petri net is kept sound and safe after each change, the different changes can be applied in order. Also, after applying all changes, the entire four phases of our framework can be reiterated. Indeed, during each



(a) Reachabilility graph after including $b >_N d$.



(b) The repaired Petri net for including $b >_N d$.



(c) The repaired Petri net for including $i >_N f$.

Figure 6.4: Petri net repairs example. Red parts denote repairs. Markings in the reachability graph are indicated as multisets. We indicate visible transitions with label l with t_l, i.e. $\lambda^{-1}(l) = t_l$ if $l \neq \tau$.

iteration, the Machine Learning model may uncover new discriminating features, as the training set adapts in response to the evolving process model.

Example: Consider the Petri net N illustrated in Figure 6.2. To include the relation $b >_N d$, the reachability graph is updated, as depicted in Figure 6.4a, resulting in the Petri net shown in Figure 6.4b. If we also want to incorporate the relation $i >_N f$, the final Petri net is as shown in Figure 6.4c.

Greedy Repair

Category (i) introduced in the basic approach triggers the introduction of several arcs between markings in the reachability graph $G_N = (V, E)$, and consequently brings the introduction of multiple invisible transitions in the original Petri net $N = \big(P, T, F, \lambda, m^I, m^F\big)$. The allowance of new behavior may negatively affect the resulting Petri net models, due to the reduction of model simplicity.

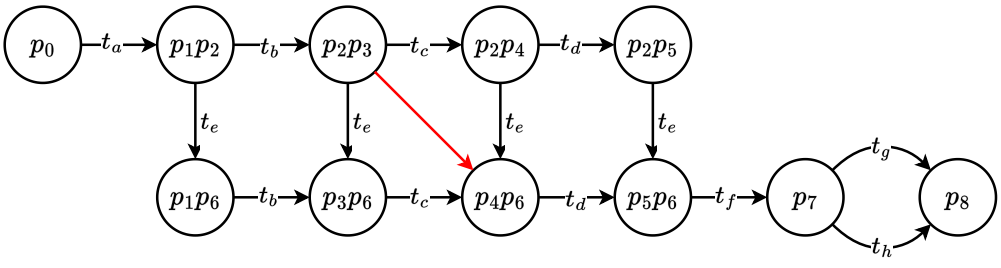
To tackle this challenge, we propose a greedy approach to include relation $a >_N b$ between two transitions $t_a, t_b \in T$. This greedy approach comes from the observation that, given any marking $m_a \in G_N$ reached after firing t_a and any marking $m_b \in G_N$ where t_b is enabled, it is sufficient to add arc (m_a, τ, m_b) to formally incorporate $a >_N b$ into the Petri net. In particular, the approach picks the marking m_a with the shortest path in G_N from the initial marking m^I, along with marking m_b with the short path to the final marking m^F. The approach is motivated by the fact that, by selecting such these two nodes, fewer paths are included in the reachability graph: this leads to a minimization of the behaviour additionally included into the Petri net model, thereby reducing the negative impacts on the precision.

Example: Given the Petri net N in Figure 6.2, if we want to introduce the relation $b >_N d$ using the greedy approach, only one invisible transition is added to the Petri net model which corresponds to the new arc in the reachability graph highlighted in red in Figure 6.5a.

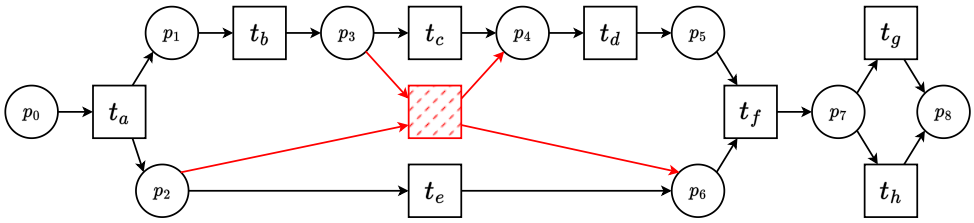
Of course, we acknowledge that this approach only introduces a subset of the potential changes needed to maximize fitness, and, even if we want to introduce a single invisible transition, the choice does not guarantee to be the best to repair the model. This explains why the approach is indeed labelled as greedy.

Moreover, the greedy repair approach also leads to a significant reduction in computational costs compared to the basic repair method. Specifically, the basic repair is characterized by a computational complexity of $\mathcal{O}(|E| + |V|^2)$. This includes $\mathcal{O}(|E|)$ for identifying the nodes to include in the repair by searching across the arcs, and $\mathcal{O}(|V|^2)$ for adding the new arcs that connect nodes in V to each other. In contrast, the greedy repair is more efficient, with a computational complexity of $\mathcal{O}(|E|+|V|)$. This cost is derived from $\mathcal{O}(|E|+|V|)$ for locating the two nodes in the reachability graph via the BFS algorithm and $\mathcal{O}(1)$ for adding the new arc.

Note that Theorem 1 is still valid for the greedy approach, because we apply a subset of the changes of the basic approach, and each change guarantees soundness and safety, if taken in isolation.



(a) The reachabilility graph after greedy repair.



(b) The repaired Petri net.

Figure 6.5: Petri net greedy repairs. Red parts denote repairs. Markings in the reachability graph are indicated as multisets. We indicate visible transition with label l with t_l, i.e. $\lambda^{-1}(l) = t_l$ if $l \neq \tau$.

Repair via Log-Model Alignments

This section proposes an alternative technique to repair process models by associating the activity labels of the discriminating features to the correspondent transitions to be repaired.

This section supports that the initial Petri net $N = \left(P, T, F, \lambda, m^I, m^F\right)$, with $\lambda : T \to \mathcal{A} \cup \{\tau\}$, may contain multiple visible transitions for for same label, i.e. $\forall t_1, t_2 \in T, \lambda(t_1) = l \wedge \lambda(t_2) = l \wedge l \neq \tau \nRightarrow t_1 = t_2$.

The general idea is to use alignments to map the discriminating features $a >_N b$, identified in Step 3, to valid markings that can be used for repairing the Petri net model as described in the previous sections.

Let us suppose that the relation $a >_N b$ needs to be incorporated into the model, and net contains transitions for each label a and b, i.e. we are in the category (i) of repair as described in the basic approach. The technique to repair the model by leveraging on log-model is here applied as follows:

1. We create an event log $\mathcal{L}^r_{a>b}$ by filtering out the traces that do not contain

an event for activity a followed by one for activity b.

2. For each trace $\sigma \in \mathcal{L}^r_{a>b}$, we build the following:
   (a) We compute the extended alignment $\gamma = \langle (s^1_L, s^1_M, m_1), \ldots, (s^n_L, s^n_M, m_n) \rangle$ of σ and N.
   (b) For each synchronous move $(s^i_L, s^i_M, m_i) \in \gamma$ such that $\lambda(s^i_M) = a$, we add marking m_i to a marking multiset M^a.
   (c) For each synchronous move $(s^j_L, s^j_M, m_j) \in \gamma$ such that $\lambda(s^j_M) = b$ and $j > 1$, we add the marking m_{j-1}, i.e. reached before firing s^j_M, to a marking multiset M^b.

When every trace in $\mathcal{L}^r_{a>b}$ is considered, we consider the most frequent markings in the multiset M^a and m_b in M^b, denoted as m_a and m_b in M^b, respectively. These markings are ultimately used to repair the Petri net model, as described in the greedy approach.

For category (ii), namely at least one transition for activity a is present in N, but no transition exists for activity b, a similar approach is used, focusing only on the most frequent marking m_a in M^a. This marking can then be used for repairing the Petri net as shown in Figure 6.3c. Category (iii), namely a transition for activity b is present in N, but no transition exists for activity a, follows an analogous reasoning as for category (ii).

Example: Given an event log $\mathcal{L} = \{\langle a, c, c \rangle^{10}, \langle a, b, c \rangle^{10}, \langle a, b, c, c \rangle^{10}, \langle a, c \rangle^2\}$ and a Petri net N shown in black in Figure 6.6. Suppose we want to incorporate the relation $a >_N c$ into N. Filtering the event log to retain only the traces σ s.t. $a >_\sigma c$ yields $\mathcal{L}_{a>c} = \{\langle a, c, c \rangle^{10}, \langle a, c \rangle^2\}$. The optimal alignment of $\langle a, c, c \rangle$ is as follows:

Log	a	$\gg$	c	c
Model	t^1_a	t_b	t^1_c	t^2_c
Marking	$[p_2]$	$[p_3]$	$[p_4]$	$[p_6]$

Similarly, the optimal alignment for $\langle a, c \rangle$ is:

Log	a	$\gg$	c
Model	t^2_a	t_b	t^2_c
Marking	$[p_5]$	$[p_4]$	$[p_6]$

The two most frequent markings m_a and m_c are $[p_2]$ and $[p_3]$, respectively. Finally, the correspondent repair to the Petri net is highlighted in red in Figure 6.6.

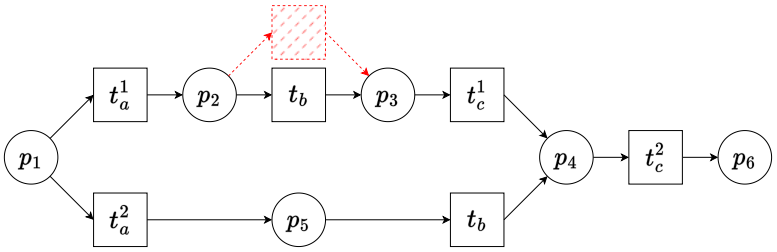


Figure 6.6: Petri net log-model alignment repair. Red and dashed parts denote repairs. We indicate visible transition with label l with t_l.

Note that selecting specific markings for repairs prevents the inclusion of multiple arcs in the reachability graph, leading to simpler Petri net models similarly to the greedy approach. However, using alignments may increase computational costs (cf. Section 6.3.6).

Theorem 1 remains valid also in this case, following a reasoning similarly to what expressed the greedy approach.

6.3 Evaluation

In this section, we present the evaluation of our framework, showing our approach is capable of improving the overall accuracy of process models.

Section 6.3.1 presents the case studies and the models used for the experiments, while Section 6.3.2 describes the experimental setup. In Section 6.3.3, we discuss the results in terms of fitness, precision and their balance, comparing them with the state of the art [64]. Section 6.3.4 provides a comparison of model simplicity between our approach and [64]. Additionally, Section 6.3.5 shows the method's effectiveness in producing models that generate more accurate simulations compared to those created using the initial process model. In Section 6.3.6 we compared the efficiency of the different approaches and the state of the art. Finally, in Section 6.3.7 we analyzed the scalability of the framework with input of increasing complexity.

6.3.1 Case Studies

To assess the accuracy and effectiveness of our framework, we evaluated it on five different case studies, each representing different business processes, with an event log provided for each case: Purchase to Pay (P2P), BPIC12W, BPIC17W,

Road Traffic Fines (RTF), and Sepsis (cf. Section 1.7 for further details).

The original Petri net models, i.e. before repairs, for the first three case studies were generated by converting the BPMN models publicly available in the repository of the reproducibility package of [43], which were discovered through Split Miner [15]. For all case studies, we also employed the Inductive Miner [105] to mine additional models. To gain insights into the framework's impact on the initial fitness and precision of the models, we experimented with four different noise thresholds $(0.25, 0.5, 0.75, 1)$. It's worth noting that we could obtain the same models using varying noise thresholds. We also performed experiments with a noise threshold equal to 0: with fitness already equal to 1, our techniques did not alter the model, and neither did the technique by [64]. In other words, when the models had fitness equal to 1, all repair techniques returned the original models without any change.

To evaluate the effectiveness of the log-model alignments approach in repairing models with multiple transitions sharing the same label, we also included the normative models for the Road Traffic Fines and Sepsis case studies (see Figure 1.14 and Figure 1.13 in Section 1.7). However, these models exhibit an high level of fitness, thus capturing all the observed behaviors. We then modified the normative models by removing the least frequent 10% of transitions, obtained by looking at the frequencies of transition firings in optimal alignments, along with their ingoing and outgoing arcs, ensuring that the resulting models remain sound and retain transitions with same labels.

6.3.2 Experimental Setup

The framework has been implemented in Python: it takes in input an event log in XES format, and a Petri net in Petri Net Markup Language (PNML). [4]

The event log is temporally split into train, validation, and test sets: with respectively 60%, 20% and 20% of the total traces. The training and the validation logs are used by the framework as described in Section 6.2, while the test log is employed for evaluation. In Step 2, CatBoost is used as discriminator model (cf. Section 1.4). SHAP values are computed using the Python's implementation.[5] The validation set/log were also used to perform hyper-parameter optimization during each framework iteration to avoid overfitting and/or underfitting.

During the experiments, we monitored the accuracies in the training and validation sets of the discriminator model during each framework iteration in order to avoid overfitting and/or underfitting. In general, we found optimal

[4] https://github.com/franvinci/MLProcessModelRepair

[5] https://shap.readthedocs.io

Table 6.3: Experimental results using the models obtained via [64], and through our basic (RB), greedy (RG) and log-model alignments approach (RA). Results are compared through fitness, precision and their harmonic mean. Column O illustrates the values for the original models, namely before repairing, which were either mined through process-discovery techniques or present in literature. The best harmonic mean for each model is highlighted in bold.

Case Study	Input Model	Fitness					Precision					H. Mean				
		O	[64]	RB	RG	RA	O	[64]	RB	RG	RA	O	[64]	RB	RG	RA
P2P	Ref.	0.64	1.00	0.95	0.95	0.95	0.99	0.13	0.82	0.85	0.82	0.77	0.23	0.88	**0.89**	0.88
	IM(0.25)	0.63	1.00	1.00	1.00	1.00	1.00	0.13	0.81	0.81	0.81	0.77	0.23	**0.89**	**0.89**	**0.89**
	IM(0.5,0.75,1)	0.63	1.00	1.00	1.00	1.00	1.00	0.13	0.81	0.81	0.81	0.78	0.23	**0.89**	**0.89**	**0.89**
BPIC12W	Ref.	0.68	1.00	1.00	1.00	1.00	0.91	0.38	0.49	0.49	0.49	**0.78**	0.55	0.65	0.65	0.65
	IM(0.25)	0.84	1.00	1.00	1.00	1.00	0.91	0.42	0.65	0.65	0.65	**0.87**	0.60	0.79	0.79	0.79
	IM(0.5,0.75)	0.62	1.00	0.91	0.91	0.91	0.65	0.36	0.49	0.49	0.49	0.63	0.53	**0.64**	**0.64**	**0.64**
	IM(1)	0.61	1.00	1.00	1.00	1.00	0.55	0.36	0.36	0.36	0.36	**0.58**	0.53	0.53	0.53	0.53
BPIC17W	Ref.	0.73	1.00	1.00	1.00	1.00	1.00	0.54	0.73	0.73	0.73	**0.84**	0.70	**0.84**	**0.84**	**0.84**
	IM(0.25)	0.85	1.00	1.00	0.98	1.00	0.74	0.46	0.70	0.79	0.77	0.79	0.63	0.82	**0.87**	**0.87**
	IM(0.5)	0.57	1.00	0.91	0.79	1.00	0.71	0.42	0.61	0.79	0.69	0.63	0.59	0.73	0.79	**0.81**
	IM(0.75)	0.58	1.00	0.78	0.67	0.74	0.63	0.42	0.57	0.67	0.63	0.61	0.59	0.66	0.67	**0.68**
	IM(1)	0.46	1.00	0.79	0.78	0.74	0.36	0.42	0.50	0.55	0.56	0.40	0.59	0.61	**0.65**	0.64
RTF	N	1.00	1.00	-	-	1.00	0.81	0.56	-	-	0.81	**0.90**	0.72	-	-	**0.90**
	N*	0.95	0.99	-	-	0.98	0.86	0.35	-	-	0.87	0.90	0.51	-	-	**0.92**
	IM(0.25)	0.96	1.00	0.99	0.99	0.99	0.52	0.44	0.54	0.55	0.57	0.68	0.61	0.70	0.70	**0.72**
	IM(0.5)	0.89	1.00	1.00	0.98	0.96	0.67	0.45	0.45	0.70	0.68	0.76	0.62	0.62	**0.81**	0.80
	IM(0.75)	0.93	1.00	0.98	0.98	0.99	0.76	0.51	0.78	0.78	0.71	0.84	0.68	**0.87**	**0.87**	0.83
	IM(1)	0.61	1.00	0.99	0.99	0.64	0.75	0.43	0.78	0.75	0.86	0.67	0.60	**0.87**	0.86	0.74
Sepsis	N	0.99	1.00	-	-	0.99	0.26	0.25	-	-	0.26	**0.41**	0.40	-	-	**0.41**
	N*	0.89	0.96	-	-	0.92	0.59	0.21	-	-	0.55	**0.71**	0.34	-	-	0.69
	IM(0.25)	0.87	1.00	0.93	0.91	0.91	0.49	0.20	0.49	0.49	0.49	0.63	0.33	**0.64**	**0.64**	**0.64**
	IM(0.5)	0.58	1.00	0.95	0.76	0.77	0.78	0.20	0.30	0.73	0.64	0.67	0.34	0.45	**0.74**	0.70
	IM(0.75,1)	0.52	1.00	0.83	0.55	0.62	1.00	0.82	0.44	0.78	0.75	0.63	0.34	0.57	0.65	**0.68**
Avg.		0.69	1.00	0.95	0.91	0.92	0.76	0.38	0.60	0.67	0.65	0.70	0.50	0.72	**0.76**	0.75

results using depth equal to 3, a learning rate of 0.01, and training for 150 iterations.

6.3.3 Experimental Results: Fitness & Precision

In this section, the quality of the obtained models is measured as harmonic means of fitness and precision [181]. The harmonic means aims to quantify the improvement in fitness and/or precision, and their balance. Harmonic means is preferable over arithmetic means because the former yields more penalization than the latter when fitness or precision is significantly lower than the other (i.e., balancing less). Separately, we also report the simplicity of the models as introduced in [184]. Results have finally been also compared with the approach by [64].

The results are reported in Table 6.3, where fitness, precision and their harmonic mean are reported for the original model (label O), namely before the repair, our framework (columns with gray background) using the basic approach (label RB), using greedy repairs (label RG) and using alignments repair (label RA), and the best result by the work of [64]. Each row indicates a dif-

ferent model. Rows *IM(0.25)*,. . .,*IM(1)* denote the original models discovered via Inductive Miner with noise threshold $0.25, \ldots, 1$. *Ref.* refers to the original "reference" models from the reproducibility package of [43], i.e. obtained by using Split Miner, row *N* indicates the normative models from [122], while *N** the modified models obtained as described in Section 6.3.1. Note that results for the models from [122] using basic and greedy approaches are not present since they contain transitions with same labels.

The reported results are computed on the test event log, namely the part that was not used to apply our framework. All our approaches revealed themselves to be effective for almost all models employed in our experiments: compared with 19 out of the original 23 models (83%), they provide models that improve the harmonic means of fitness and precision, while consistently outperforming the method presented by Fahland et al. [64] (except for one case, where the harmonic means is the same). In fact, the improvement with respect to [64] is unsurprising, since the method in [64] maximizes fitness by incorporating all the behaviour of the event log into the Petri net model, leading to models with perfect fitness but low precision, whereas our method only adds the most discriminating behaviour without penalizing precision, yielding to a balanced process model.

Considering the four original models where we do not improve on the harmonic means, they are already characterized by a high means, which does not allow further improvement. In fact, in an attempt to improve, we worsen the quality of those models: this clearly does not invalidate our technique, because one can easily check whether or not the repaired is actually a better model, and can discard the changes when they are counterproductive.

Comparing our approaches between them, we observe that the greedy produces repaired models with higher or similar harmonic means of fitness and precision. Moreover, the log-model alignments approach produces performances similar to the greedy one and it shows its effectiveness in his usage also in models that present transitions with same labels.

6.3.4 Experimental Results: Simplicity

We also compared the level of simplicity of the original and repaired models. Table 6.4 reports the results. Clearly, since our approaches incorporate additional behavior, the repaired models are inevitably less simple. However, comparing the simplicity of the models generated by [64] and by our approaches, we observe that, on average, our greedy and log-model alignments approaches produce simpler repaired models: the average simplicity is 0.55 for the approach by Fahland et al. [64] while it is 0.58 for our greedy approach and 0.57 for the alignments

Table 6.4: Simplicity results using the models obtained via [64], and through our basic (RB), greedy (RG) and log-model alignments approach (RA). Column O illustrates the values for the original models, namely before repairing, which were either mined through process-discovery techniques or present in literature.

Case Study	Input Model	Simplicity				
		O	[64]	RB	RG	RA
P2P	Ref.	0.92	0.63	0.78	0.81	0.78
	IM(0.25)	0.96	0.64	0.75	0.75	0.75
	IM(0.5,0.75,1)	1.00	0.64	0.75	0.75	0.75
BPIC12W	Ref.	0.82	0.55	0.60	0.62	0.60
	IM(0.25)	0.71	0.59	0.61	0.61	0.61
	IM(0.5,0.75)	0.78	0.53	0.60	0.60	0.60
	IM(1)	0.68	0.48	0.55	0.55	0.53
BPIC17W	Ref.	0.79	0.59	0.64	0.64	0.64
	IM(0.25)	0.68	0.55	0.48	0.56	0.56
	IM(0.5)	0.73	0.51	0.35	0.44	0.44
	IM(0.75)	0.67	0.51	0.32	0.47	0.47
	IM(1)	0.71	0.49	0.29	0.42	0.42
RTF	N	0.63	0.56	-	-	0.61
	N*	0.82	0.53	-	-	0.73
	IM(0.25)	0.70	0.55	0.36	0.58	0.58
	IM(0.5)	0.73	0.57	0.17	0.52	0.52
	IM(0.75)	0.77	0.54	0.59	0.66	0.58
	IM(1)	1.00	0.52	0.49	0.57	0.83
Sepsis	N	0.62	0.56	-	-	0.54
	N*	0.80	0.53	-	-	0.56
	IM(0.25)	0.67	0.56	0.05	0.56	0.35
	IM(0.5)	0.75	0.51	0.11	0.33	0.32
	IM(0.75,1)	0.77	0.50	0.11	0.54	0.36
Avg.		0.78	0.55	0.45	0.58	0.57

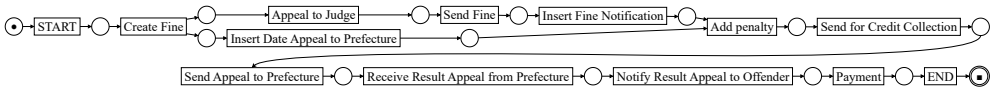
one. A visual inspection of the models repaired via our greedy approach shows that the models are clearly readable (see Figure 6.7).

In general, it follows that *our greedy and log-model alignments approaches are able to produce simpler models with a higher harmonics means of fitness and precision, if compared with [64].*

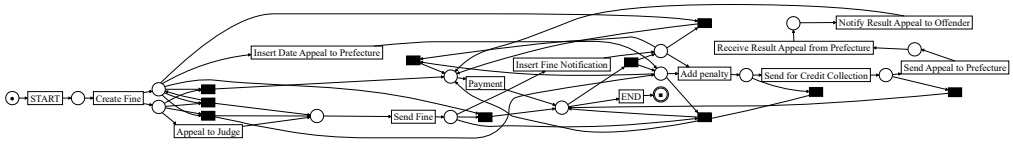
6.3.5 Experimental Results: Simulations Quality

In this section we show the ability of the proposed framework in repairing the input process models for obtaining more accurate simulated event logs. In fact, the framework is built to find inaccuracies between a simulated event log using the starting model and the real event log. The inaccuracies are then used for repairing the model to obtain a simulated event log which presents less deviations from the real one.

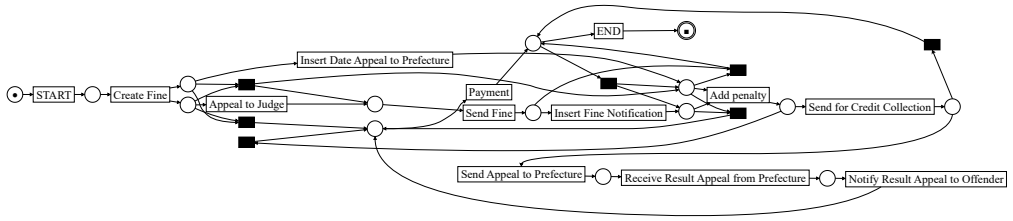
To assess simulation performances, we generated simulated event logs from the process model, as described in Section 6.2.1, and compared them to the actual event log using the control-flow measurements proposed in [47]: Control-flow Log Distance (CFLD) and N-Gram Distance (NGD).



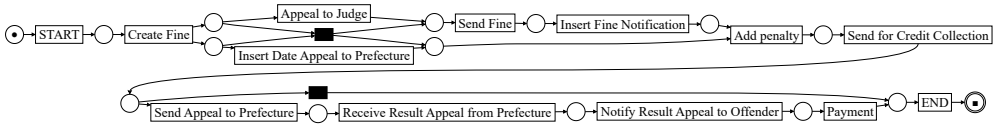
(a) RTF IM(1) original Petri net. Simplicity: 1.00. Fitness-Precision Harmonic Mean: 0.67.



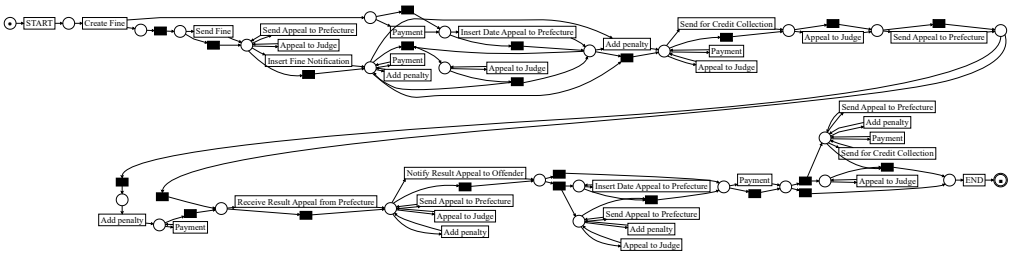
(b) RTF IM(1) repaired Petri net using the basic approach. Simplicity: 0.49. Fitness-Precision Harmonic Mean: 0.87.



(c) RTF IM(1) repaired Petri net using the greedy approach. Simplicity: 0.57. Fitness-Precision Harmonic Mean: 0.86.



(d) RTF IM(1) repaired Petri net using the log-model alignment approach. Simplicity: 0.83. Fitness-Precision Harmonic Mean: 0.74.



(e) RTF IM(1) repaired Petri net using [64]. Simplicity: 0.52. Fitness-Precision Harmonic Mean: 0.60.

Figure 6.7: Road Traffic Fine (RTF) Petri net mined using Indective Miner with theshold equal to 1 (IM(1)) and the Petri nets obtained via repairs.

In our evaluation, we compared the results obtained from the original models, with those from the *best* repaired process model. Here, we refer to the *best* process model as the one that achieves the highest harmonic mean of fitness and precision across the repaired models generated using our approaches. To mitigate the inherent stochasticity of simulations, we generated 10 event logs for each

Table 6.5: Simulation results using the original models (O) and our best results from our framework (*b*R).

Case Study	Input Model	CFLD		3GD		4GD	
		O	*b*R	O	*b*R	O	*b*R
P2P	Ref.	0.425(±0.004)	**0.156(±0.009)**	0.316(±0.004)	**0.074(±0.009)**	0.346(±0.005)	**0.096(±0.010)**
	IM(0.25)	0.421(±0.002)	**0.156(±0.021)**	0.291(±0.004)	**0.092(±0.015)**	0.316(±0.006)	**0.116(±0.015)**
	IM(0.5,0.75,1)	0.431(±0.003)	**0.120(±0.026)**	0.334(±0.004)	**0.090(±0.016)**	0.370(±0.004)	**0.116(±0.016)**
BPIC12W	Ref.	0.427(±0.002)	**0.274(±0.003)**	0.543(±0.002)	**0.433(±0.004)**	0.584(±0.002)	**0.485(±0.004)**
	IM(0.25)	0.280(±0.003)	**0.136(±0.004)**	0.308(±0.004)	**0.107(±0.004)**	0.373(±0.003)	**0.141(±0.003)**
	IM(0.5,0.75)	0.505(±0.001)	**0.334(±0.003)**	0.602(±0.001)	**0.538(±0.004)**	0.642(±0.001)	**0.604(±0.005)**
	IM(1)	0.515(±0.001)	**0.359(±0.002)**	**0.598(±0.001)**	0.605(±0.003)	**0.639(±0.001)**	0.671(±0.003)
BPIC17W	Ref.	0.343(±0.001)	**0.154(±0.002)**	0.473(±0.001)	**0.261(±0.003)**	0.5595(±0.001)	**0.315(±0.003)**
	IM(0.25)	0.267(±0.003)	**0.211(±0.002)**	0.340(±0.002)	**0.281(±0.002)**	0.388(±0.002)	**0.319(±0.002)**
	IM(0.5)	0.481(±0.001)	**0.219(±0.003)**	0.482(±0.001)	**0.290(±0.002)**	0.559(±0.001)	**0.361(±0.002)**
	IM(0.75)	0.491(±0.001)	**0.439(±0.002)**	0.532(±0.001)	**0.494(±0.003)**	0.575(±0.001)	**0.529(±0.003)**
	IM(1)	0.631(±0.001)	**0.407(±0.003)**	0.670(±0.001)	**0.463(±0.002)**	0.706(±0.001)	**0.513(±0.002)**
RTF	N	0.102(±0.002)	0.103(±0.003)	0.171(±0.003)	0.172(±0.002)	0.213(±0.004)	0.214(±0.002)
	N*	0.104(±0.001)	**0.094(±0.002)**	0.182(±0.002)	**0.172(±0.003)**	0.223(±0.002)	**0.212(±0.003)**
	IM(0.25)	0.163(±0.002)	0.162(±0.002)	0.239(±0.003)	**0.234(±0.002)**	0.303(±0.003)	**0.296(±0.003)**
	IM(0.5)	0.207(±0.001)	**0.186(±0.002)**	0.198(±0.003)	0.202(±0.002)	0.236(±0.004)	0.237(±0.003)
	IM(0.75)	0.045(±0.001)	0.046(±0.001)	0.084(±0.001)	0.082(±0.001)	0.097(±0.001)	**0.093(±0.002)**
	IM(1)	0.545(±0.001)	**0.056(±0.002)**	0.609(±0.001)	**0.058(±0.003)**	0.685(±0.001)	**0.081(±0.003)**
Sepsis	N	0.443(±0.005)	0.447(±0.005)	0.529(±0.007)	0.533(±0.008)	0.635(±0.007)	0.640(±0.007)
	N*	0.472(±0.006)	0.469(±0.006)	0.562(±0.007)	0.547(±0.010)	0.676(±0.006)	**0.659(±0.008)**
	IM(0.25)	0.565(±0.008)	0.578(±0.008)	0.653(±0.010)	**0.609(±0.009)**	0.742(±0.009)	**0.687(±0.005)**
	IM(0.5)	**0.533(±0.004)**	0.554(±0.004)	0.749(±0.005)	**0.721(±0.004)**	0.818(±0.004)	0.816(±0.004)
	IM(0.75,1)	0.580(±0.005)	**0.512(±0.004)**	0.781(±0.002)	**0.720(±0.004)**	0.844(±0.002)	**0.784(±0.003)**
Avg.		0.390	**0.267**	0.445	**0.338**	0.501	**0.391**

process model. Then, using a *t-test* with a 90% confidence level, we computed confidence intervals for these results. Particularly, we found that 10 simulations were sufficient as they produced a standard deviation of the results close to zero. For the N-Gram Distance we set N equal to 3 and 4, finding that larger values did not produce additional changes. Table 6.5 presents the results, where columns O and *b*R indicate the original and the *best* repaired process model respectively, with the best-performing values highlighted in bold. Specifically, we conclude that the repaired process model outperforms the original if its confidence interval is entirely above that of the original model. If the intervals overlap, we cannot draw a definitive conclusion, while a confidence interval entirely below the original indicates poorer performance than the initial model.

The results prove that our framework repairs the process models such that the simulation models built on them guarantee simulations that are more accurate representation of the real behavior. For the CFLD we achieved a reduction in distance in 16 out of 23 cases, with only one case showing a worst performance. Overall, we achieved a general improvement of 31.54% compared to the original models. For the 3GD and 4GD metrics we obtained similar results, with average improvements of 24.04% and 21.96%, respectively. Moreover, the 3GD and 4GD measures highlight the framework capability to improve the handling of long-term dependencies.

Table 6.6: Computational times (in *seconds*) per each case study and correspondent models, using the approach proposed by Fahland et al. [64] versus our our basic approach (RB), our greedy approach (RG) and the approach based on log-model alignments (RA).

Case Study	Input Model	[64]	RB	RG	RA
P2P	Ref.	4	16	11	20
	IM(0.25)	6	18	15	32
	IM(0.5,0.75,1)	6	21	16	27
BPIC12W	Ref.	13	147	146	202
	IM(0.25)	13	73	68	130
	IM(0.5,0.75)	14	240	236	283
	IM(1)	17	167	151	175
BPIC17W	Ref.	4226	350	344	573
	IM(0.25)	67	409	304	558
	IM(0.5)	728	447	282	559
	IM(0.75)	71	455	450	537
	IM(1)	74	402	398	732
RTF	N	20	-	-	69
	N*	320	-	-	60
	IM(0.25)	23	142	135	191
	IM(0.5)	58	3920	180	259
	IM(0.75)	14	138	131	165
	IM(1)	53	172	171	189
Sepsis	N	157	-	-	587
	N*	386	-	-	79
	IM(0.25)	94	6781	235	686
	IM(0.5)	44	2824	275	1041
	IM(0.75,1)	32	1296	95	111
Avg.		280	948	**192**	316
Std.		857	1701	**124**	275
Median		**44**	240	171	191

6.3.6 Experimental Results: Computational-Time Comparison with the State of the Art

We monitored the computational times using the three approaches, running the experiments on the same machine equipped with an Intel Core i5-8300H CPU @ 2.30GHz and 16GB RAM. Table 6.6 summarizes key statistics of computational times across the experiments. Our findings indicate that the method by Fahland et al. [64] is in general faster than ours, but there are some cases where it exhibits significantly higher computational times. This is due to the generation of complex intermediate models that make it hard to compute model-to-trace alignments, thereby increasing the computational times. Moreover, we observed that the greedy method is the fastest among those we proposed. In fact, the basic approach introduces multiple arcs into the reachability graph (cf. Section 6.2.4), increasing the complexity of intermediate models. Also, the log-model alignments method requires the computation of alignments, which is computationally intensive.

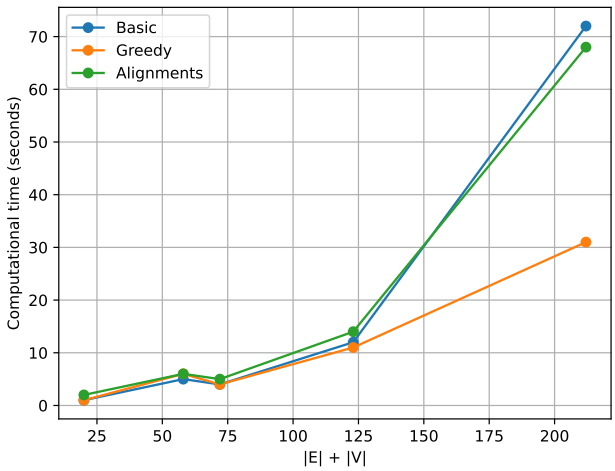


Figure 6.8: Computation times (in seconds) in relation to process model complexity for each proposed approach. Model complexity is quantified by the total number of edges E and nodes V of the process model reachability graph.

6.3.7 Experimental Results: Noise Robustness and Scalability

This section elaborates on two further aspects that are relevant for the practical applicability of the framework: robustness to noise, and scalability. Noise robustness refers to the capability of our repair techniques to distinguish genuine deviations and noise. Genuine deviations are those that tend to repeat with a certain degree of frequency, which is clearly different from noise, which is conversely characterized by random deviations that do not repeat with a consistent pattern. Scalability conversely focuses on the capability of our techniques to repair models of large size.

These two problems have been investigated by generating five Petri nets of larger and larger sizes and accordant event logs with 500 traces with and without noise employing the PLG2 tool by [34].

For the scalability analysis, we then randomly swapped two consecutive transitions in the Petri nets. We conducted experiments on these altered process models, monitoring the computational time for each of our repair approaches. Figure 6.8 reports the computation time (y axis) for Petri nets of increasing size (x axis). A model's size is here measured as the sum of the number of arcs and nodes of its reachability graph. Indeed, in the greedy approach, the theoretical complexity of the repair is correlated to the size of the reachability graph, which is certainly correlated to the number of places and transitions of the Petri nets but better characterizes the complexity of the model from an algorithmical view-

point (cf. a Petri net with ten transitions in parallel versus another with the same ten transition in a sequence). The results show that the computational times of the greedy method grow linearly with model complexity, while the basic approach would follow a nearly-quadratic trend. These observation align with the computational cost analysis reported in the greedy approach (cf. Section 6.2.4). Regarding the alignment-based model-repair approach, it seems to practically follow a trend similar to the basic approach, although it is based on computing alignments, which is at least an NP-hard problem.

To assess the noise tolerance, we injected random noise by randomly swapping 10%, 20%, and 30% of trace events in the logs, aiming to assess whether any noise level caused unwanted model repairs. We used the five Petri nets that PLG directly generated. When attempting to repair the Petri nets with these three noisy event logs, our framework did not make any change to the model. Recall that the noisy-free event logs were fully compliant with the model, and hence no changes are expected to occur after applying our repair technique. This showed that the noise was identified as such, and ignored. This is related to the ability of the Machine-Learning classifier to detect the noise and not overfit the data. Consequently, SHAP did not identify any significant features influencing the outputs (i.e. whether a trace is real or simulated), thereby no repairs were applied to the original process model. In our experiments, we employed CatBoost, but the management of noise and outliers is a feature present in the vast majority of classifiers.

6.4 Related Work

The work by Fahland et al. [64] is among the first ones that address the model-repair problem to enhance fitness, but is not the only one. Polyvyanyy et al. [149] put forward methods to define process model repair as an optimization problem, where costs are assigned to various repair actions. However, this work aims to maximizing the fitness, as authors also clearly state, relegating precision as second-class citizen.

Mitsyuk et al. [137] propose a model-repair approach able to achieve a high level of fitness, but, as authors themselves admit, at a cost of dropping precision. Other works address the model repair and try to improve the model precision [91, 156].

In a recent work, Genga et al. [74] address the challenge of balancing fitness and precision by proposing a repair methodology based on anomalous local instance graphs. However, their approach assumes the presence of a user that chooses among a set of anomalous instances the one to be used for repairing.

We then decided to compare our framework with [64] since it is the one, with an available implementation, proposing the problem of repairing process models similarly as our approach.

Existing literature also proposes interactive frameworks for process model repair, in which possible differences between the model and the log are displayed to the user who manually repairs the process model [14, 150]. These research works do not aim at an automated model repair, which serves a different purpose: repairing the model, using feedback from humans.

Repairing a process model is also linked to *concept drift* [157]: one wants to repair the model to align it with the behaviour after, i.e. when the process changes over time and then the model does not align the reality anymore. Revoredo [157] framed process model repair as a theory revision problem: the event log and the process model are partitioned to only repair changeable parts of the model that are manually selected by practitioners, considering only traces representing acceptable behaviour. In case of concept drift one can select the part of the model subject of the process changes, and repair that.

Incremental process discovery (see, e.g. [170]) is also related to model repair, but the goal is different as the former does not aim to obtain models that are as close as possible to a reference original process model.

Meneghello et al. [134] proposed a framework that uses decision-tree models to discriminate the traces observed in the event log from those allowed by a simulation model. The resulting decision trees should be used as input to subsequently change the simulation model in multiple iterations, so that ultimately the traces of the event log and those obtained from simulation cannot be distinguished, namely the resulting decision tree has very low accuracy. However, the changes need to be manually implemented by process analysts, a step that is very likely difficult because the decision trees may return ambiguous advices for changes.

Note that repairing a model can be regarded as ensuring soundness of the model [72, 115], which is a relevant, yet orthogonal problem.

6.5 Towards Multi-perspective Process Model Repair

An avenue of future work is to not limit the repair to the control-flow perspective, but to extend it to consider other perspectives as well. This can be achieved by including features in the discriminator that capture information from the data, resource, and time perspectives, thereby enabling a more comprehensive understanding of process behavior. Incorporating these perspectives would also facilitate integration with process simulation, allowing the repair of complete

simulation models. The following outlines possible directions for extending the repair to each perspective:

Resource Perspective. For each transition T, a resource categorical feature can be incorporated, which takes on a value equal to the name of the resource that performed A. If transition T is performed multiple times, one needs to approximate it as, e.g., the last resource performing A. Adding a feature for each pair of transition and resource would likely generate too many features, with a high risk of leading to overfitting. If a discrimination is found on the feature for, e.g., T, the set of resources allowed to perform T is changed. Namely, if resource R performs T, then the trace is real or, resp., from the simulated log, then R is added or, resp., removed from the set of resources allowed to perform T.

Time Perspective. For each transition T, a numerical feature can be added to indicate the execution duration of instances of T (or, average if executed multiple times). If a discrimination is found on the feature for a specific transition, the modelling of the distribution of T's duration is considered incorrect, which thus needs to be (re-)discovered from data, as, e.g., indicated in [42]. Currently, directly-follow features in Definition 19 are boolean-typed, to which true and false values are assigned depending on whether or not the sequence of two transitions is observed in a trace (cf. Equation 6.1), respectively. The boolean value can potentially be replaced by the time elapsed between two subsequent transitions, thus being able to discriminate on the basis of the transition's waiting times. However, waiting time might be related to exogenous conditions and events that are outside the "universe" of the process model, or alternatively to the unavailability of resources. The latter case is clearly related to the resource perspective, more than to the time. It is thus challenging to (semi)automatically repair the model to accommodate the correct waiting times. In a spirit of separation of concerns, one could firstly fix the resource perspective: this would mean that the resulting waiting time is related to exogenous conditions, annotating the arc with waiting-time characterization (simple averages, distributions, etc.).

Data Perspective. The relevant aspects related to the data perspective of a process model are largely focused on the data-driven decisions that determine how process instances are executed. These decisions are typically modeled via guards attached to transitions [110]. For each transition T with a guard G, a feature can be included that indicate whether G evaluated true when T was performed. Similarly to the time perspective, if a discrimination is

found of a guard-related feature, that is an indication that the guard was likely wrong and would be required to be rediscovered.

6.6 Conclusion

A process model is one of the key ingredients for process simulation, as it provides the structural foundation on which the dynamic control-flow behavior of a process can be analyzed and reproduced. Beyond simulation, process models are also essential for techniques supporting the analysis, improvement, and enactment of business processes, and serve as a communication tool to engage stakeholders in discussions about how processes are executed and how they should ideally operate.

For any of these purposes, a process model must capture all legitimate behavior (high fitness) while avoiding undesired or spurious behavior (high precision). However, models, whether manually designed or automatically discovered from event logs, often fail to achieve a good balance between fitness and precision. In fact, improving one often comes at the cost of the other.

This chapter discussed a model-repair framework designed to improve an initial process model by maximizing the harmonic mean of fitness and precision, thereby ensuring a balanced enhancement of both dimensions. This focus on balance distinguishes the approach from much of the existing research, which tends to prioritize either fitness or precision individually.

The framework takes as input a process model, represented as a Petri net, and a corresponding event log. The process model is used to generate a simulated event log, which is compared with the real event log to identify discrepancies. These discrepancies are detected using a machine learning discriminator, whose performance indicates the similarity between simulated and real logs. When the discriminator successfully distinguishes between the two, explainable AI techniques, specifically SHAP, are applied to reveal the features driving the discrimination. These features are then translated into actionable repair recommendations, which are used to automatically improve the process model.

We proposed three repair strategies: a basic approach that directly applies repair recommendations, a greedy approach that iteratively selects the most beneficial changes, and an alignment-based approach that leverages log-model alignments to handle models with repeated activities. This last approach enables the repair of complex models that were previously difficult to handle, such as those containing multiple visible transitions with the same label.

The evaluation, conducted on several real-life and synthetic case studies, demonstrates that the proposed framework produces models with significantly

improved harmonic means of fitness and precision compared to both the original models and alternative repair techniques. Importantly, the repaired models also prove to be more effective foundations for process simulation: simulation models derived from repaired processes better reproduce realistic execution patterns, showing improved alignment with observed system behavior.

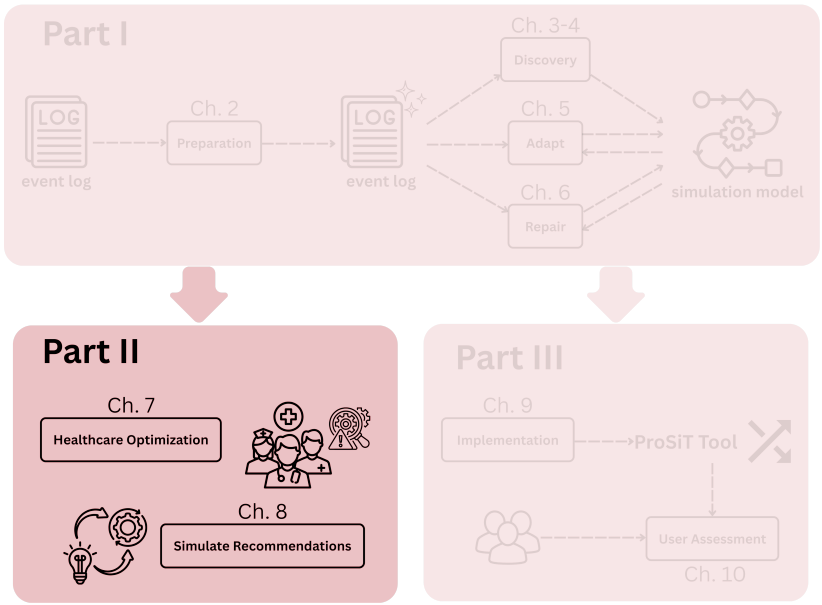
Beyond these results, our experiments confirm the robustness of the repair approach to noise and its scalability to larger models. Furthermore, to ensure the reliability of simulation-based evaluations, multiple simulation runs and statistical tests (t-tests) were performed to account for stochastic variability and assess the representativeness of the outcomes.

In this chapter, our focus was primarily on repairing process models from the control-flow perspective, as this dimension defines the foundational structure of a process. Nevertheless, real-world process execution depends on the interaction of multiple perspectives, including resources, time, and data. As future work, we aim to extend the proposed framework to support the repair of complete process simulation models, incorporating these complementary dimensions, thus ensuring that the resulting simulation models not only exhibit correct control-flow behavior but also accurately reflect real-world characterization in the different perspectives.

Finally, we acknowledge that our current framework assumes that simulated event logs are nearly complete with respect to the process model [199]. This assumption may affect validity, and future work would focus on evaluating log completeness and enabling repair operations that can also restrict behavior, when this leads to a more optimal balance between fitness and precision. Overall, the proposed approach paves the way for a tighter integration between process model repair and process simulation, enabling the automated creation of more accurate and realistic simulation models.

Part II

Process Simulation Applications



This second part of the thesis explores two different **applications** of business process simulation. A first contribution addresses the optimization of **healthcare processes** through simulation-based resource allocation. Then, we also investigate the **simulation of the effects of recommendations** provided by Process-Aware Recommender Systems.

Chapter 7 It proposes a methodology for optimizing resource allocation via process simulation. We first investigated the process mining literature in the healthcare domain, with a focus on simulation applications, identifying the gaps and domain challenges discussed in [141]. We then addressed these challenges by proposing a methodology that explicitly takes them into account. The methodology also involves healthcare staff to improve the quality of the discovered simulation model, thereby creating a simulation model that is trustworthy for healthcare stakeholders. Finally, the model can be used to optimize resource utilization in the process according to specific KPIs. We applied the methodology to a new case study concerning the Emergency Department of an Italian hospital, showing how the obtained resource configuration improves the time–cost trade-off, even in more urgent settings characterized by higher patient arrival rates. This conrtibution has been published in [187].

Chapter 8 It presents a framework for assessing the effects of process recommendations provided by Process-Aware Recommender Systems (cf. Section 1.5). While prescriptive techniques can generate actionable recommen-

dations, their evaluation remains challenging due to the lack of a ground truth describing what would have happened with the recommendation. To address this issue, we leverage business process simulation as a controlled environment for evaluating prescriptive strategies. The framework aligns a set of ongoing traces to a given simulation model that accurately represents reality. This alignment results in a "frozen" simulation model at the corresponding process state. Recommendations can then be injected into the simulation model, and simulation runs are performed to produce possible outcomes. The framework has been applied to two prescriptive analytics techniques aimed at reducing total process time: one, presented in [146], where the goal of the recommendations is to achieve a more balanced resource utilization while improving the remaining time; and another, published in [102], which leverages counterfactual explanations to provide process recommendations.

Overall, these chapters demonstrate how process simulation models can be effectively exploited in concrete application scenarios, showing their practical value for decision support, optimization, and prescriptive analytics in real-world settings.

7 Healthcare Process Optimization via Simulations

This chapter introduces a methodology that uses process simulation to improve **organizational healthcare processes** through data-driven resource allocation. Simulation models support process improvement through the evaluation of alternative *what-if* scenarios, which can be tested without disrupting live operations. The methodology guides the discovery and validation of simulation models, involving healthcare personnel. These models can then be used to automate optimization procedures by allocating additional resources where bottlenecks are identified.

The methodology has been applied to the emergency department of an Italian hospital, focusing on reducing patient waiting times. Simulations involving a careful addition of medical staff demonstrated a potential substantial reduction in waiting times (88%) with a modest cost increase (7-8%). Furthermore, the improved process exhibited enhanced resilience to surges in patient arrivals, highlighting how improved processes guarantee higher preparedness in front of emergencies.

7.1 Introduction

Recent reports, e.g. from World Health Organization (WHO) [197] and from the World Bank [196], are clearly indicating that every nation is witnessing a rise in both the number and proportion of older adults within their populations. By 2030, one in six people globally will be aged 60 or older. The population in this age group is projected to grow from 1 billion in 2020 to 1.4 billion. Looking ahead to 2050, the number of people aged 60 and over is expected to double, reaching 2.1 billion. Additionally, the population of those aged 80 and above is anticipated to triple, increasing to 426 million by mid-century. This trend has originally been

seen in western world but has recently been observed world-wide.

"All countries face major challenges to ensure that their health and social systems are ready to make the most of this demographic shift" [197]: the healthcare costs are destined to steadily grow, and a smaller share of population can be employed to provide healthcare support.

Healthcare administrations deliver services through processes that codify rules and regulations, and actions need to be taken on these processes to improve their efficiency and reduce the costs to tackle with the aging-population challenge. The improvement of healthcare processes would also reduce the errors, and thus enhance the patient outcomes, with better diagnosis, treatment, and overall care, reducing complications and mortality rates. This improvement would also positively affect the workforce well-being, with reduction of stress and burnout among healthcare professionals. Last but very importantly, improving healthcare processes would increase preparedness and resilience for emergencies, whose importance has been pointed out by the COVID-19 pandemic: the management of crises (pandemics, natural disasters, mass casualties, etc.) can be better dealt with through improved processes.

These processes, particularly in clinical settings, are often complex and can vary accordingly to several factors, situations, and patient characteristics. Therefore, their improvement is not straightforward.

The goal is to provide a methodology in which processes are improved, and the extent of this improvement is aimed to be quantified. Therefore, we exclude the use of qualitative-analysis techniques, such as Value-Added and Waste Analysis, Root-Cause Analysis, or Pareto Analysis [60]. Instead, we aim at quantitative analyses, which can be based on the applications of queuing theory or business process simulation [101]. Queuing-based analyses are excellent for capacity problems, which are common and a key driver of process redesign. However, a number of limitations exist, as pointed out by Dumas et al. [60]. They can be used to analyze waiting times (and hence processing times) but cannot focus on cost or quality measures. It is certainly suitable to analyze a single activity at a time that can be performed by resources in one single pool, but fails when aiming at analyzing end-to-end processes consisting of multiple activities performed by multiple resource pools. **Process simulation** provides a means of overcoming these limitations [60, 101, 128].

This chapter brings forward a methodology that leverages the discovery of an accurate simulation model (cf. Section 1.6) on the basis of event data and uses this to automate the optimization of the process. The methodology takes into account several characteristics of the healthcare process, facing open and relevant challenges (cf. discussion in Section 7.3).

The methodology was concretely applied to the emergency department of an Italian hospital, with the aim of improving patient waiting times both before and during treatment. The methodology includes a validation phase to assess a sufficient level of realism of the simulation model, which was positively confirmed for the case study in question. The improvement attempt was carried out by simulating various *what-if* scenarios based on the addition of resources in different roles—such as specialists, nurses, and laboratory technicians. Since hiring resources for certain roles is more expensive than for others, the potential benefits must be weighed against the associated costs. These benefits are primarily reflected in the reduction of waiting times for activities requiring specific roles. Ultimately, this means that a trade-off exists that must be carefully balanced.

The results of the application of the methodology on the case study show that the waiting time can be potentially improved by approximately 88% compared to the original time, with a limited cost's increase of around 7-8%. A simulation-driven analysis of the waiting times in case of various increases of arrival rate confirms that healthcare process improvement is also beneficial for the preparedness in case of emergencies: the patient's waiting time increases less than half when the patient arrival rate doubles, if compared with the original process.

The methodology presented is particularly well suited for **organizational healthcare processes**, i.e., processes that are structured, standardized, and primarily concerned with resource management and routing of patients through organizational units (as opposed to highly flexible, knowledge-intensive clinical workflows) [108]. For such processes our data-driven simulation discovery approach effectively captures control-flow, temporal, and resource perspectives from event logs, supporting realistic *what-if* analyses and resource optimization.

The structure of this chapter is as follows. Section 7.2 reviews relevant literature on the application of process mining and Process simulation in the healthcare domain. Section 7.3 then discusses open process mining challenges in the healthcare domain, positioning our research with respect to the current state of the art. Section 7.4 presents the methodology in a step-by-step manner. Section 7.5 describes the successful application of the methodology to the emergency department of an Italian hospital. Finally, Section 7.6 summarizes the main findings.

7.2 Related Work

The healthcare sector increasingly leverages data-driven methodologies to gain insights into complex processes and identify opportunities for improvement. In

this section, we first review the application of process mining techniques for analyzing and understanding healthcare workflows using event data (cf. Section 7.2.1). Next, we discuss the literature on process simulation approaches for modeling, evaluating, and improving healthcare processes (cf. Section 7.2.2).

7.2.1 Process Mining in Healthcare

Process execution data stored in Healthcare Information Systems can be used through process mining techniques to support the management and optimization of healthcare processes. Extensive research has explored the use of data and process mining techniques for analyzing these processes using event data, revealing distinguishing characteristics and associated challenges to be addressed [141, 162, 132, 73, 78, 131, 125].

A significant part of works focused on analyzing and modeling processes to draw conclusions and provide recommendations for improvement. Augusto et al. [16] used process data to analyze the impact of the COVID-19 pandemic in an Australian hospital. Mans et al. [124] used process mining to gain insight into the care flow of a Dutch hospital from three different perspectives. Leemans et al. [106] introduced a novel model incorporating process-based micro-costing estimations, which can be used in a decision analytics context. In [75] the authors proposed a method to model surgical processes that involve experts in the healthcare domain. Rabbi et al. [153] analyzed an emergency department process using process mining algorithms.

Interpretability and understandability pose significant challenges in the healthcare sector [131, 51]. Although process mining techniques are recognized as white-box, many models remain complex and difficult to interpret. Other works focused on enhancing the understandability of process models in the Healthcare domain. Li et al. [114] proposed a novel process model miner to construct interpretable process models for complex medical processes. Furthermore, [23] demonstrated the benefits of the interaction of a domain expert in improving the quality and understandability of process models.

Healthcare processes tend to exhibit substantial variability [141], for example, certain process paths can be taken only by some patients with specific characteristics. Considering the variability of process actors is essential for effective process management. Andrews et al. [12] developed an approach to detect process variations between various groups of patients. In [183] the authors introduced a process mining methodology to explore the disease-specific care process. Mazhar et al. [133] proposed an automatic method incorporating the stochastic perspective to determine the likelihood of a particular pathway for certain patient cohorts. Bakhshi et al. [20] applied process mining algorithms to

find decision rules based on features available in the event logs, revealing patient trajectories patterns.

Another addressed key challenge is to deal with low quality data. Real life data from Healthcare Information System often lack precision and are improperly collected, resulting in inaccurate data analysis and process modeling leading to not effective recommendations and optimizations. Several studies proposed alternatives for addressing these issues in the Healthcare domain. Vanbrabant et al. [182] proposed a framework to categorize possible data quality problems in emergency department electronic healthcare records. In [30], 27 possible classes of quality issues are subdivided into four main categories: missing, incorrect, imprecise, irrelevant data. Suriadi et al. [177] document a collection of typical quality problems in event logs; Leonardi et al. [109] introduced an explainable methodology using Deep Learning models for process trace quality assessment.

Van Hulzen et al. [85] analyzed the effects of data quality issues on simulation outcomes, showing the importance of domain knowledge in addressing this challenge using a case study in the radiology department of a hospital. Benevento et al. [22] validated the suitability of Interactive Process Discovery in handling low quality data with a real dataset from an Italian hospital, involving the hospital staff; Martin et al. [130] applied interactive data cleaning in an outpatient clinic's appointment system case study.

Focusing on the temporal perspective, Fischer et al. [65] presented an automated approach to identify and quantify timestamp imperfections, defining 15 related metrics. Dixit et al. [57] proposed an interactive framework to repair ordering imperfections in event logs.

7.2.2 Healthcare Process Simulation

Process simulation is a key methodology for monitoring, analyzing and improving healthcare processes. Several works have focused on developing simulation models specifically for healthcare applications. For instance, Guastalla et al. [76] employed simulation to improve work scheduling in hospital environments, leveraging historical data to enhance operational planning. Van Hulzen et al. [86] developed a process simulation framework to support capacity management decisions within a radiology department, combining data driven model construction with domain expert validation.

Halawa et al. [79] proposed a simulation-based approach to optimize the physical layout and patient flow in a cardiovascular clinic, aiming to improve operational efficiency. Similarly, Tamburis et al. [178] focused on creating simulation models to evaluate performance indicators and benchmark healthcare services.

Table 7.1: Overview of process simulation approaches in the Healthcare domain, and their attributes aligned with key challenges identified in [141]. "+" indicates that the corresponding work fully addresses the challenge, "+/-" indicates partial consideration. Empty cells indicate that the challenge is not addressed. Last row highlights our method.

Reference	Data Quality Improvement (C6)	Accounting for Patient Characteristics (C8)	Model Accuracy Assessment (C4)	Process Optimization (Part of C1)
Guastalla et al. [76]				+
van Hulzen et al. [86]	+	+	+/-	
Halawa et al. [79]		+/-		+
Tamburis et al. [178]			+/-	
Antunes et al. [13]				+
Johnson et al. [88]	+	+	+/-	
Kovalchuck et al. [97]	+/-	+/-	+/-	
Abo-Hamad et al. [6]			+/-	
Mans et al. [126]			+/-	
Ahmed et al. [9]				+
Our Methodology	+	+	+	+

Antunes et al. [13] integrated simulation into an optimization framework, using it to compare baseline and improved scheduling policies in an Emergency Department setting. Johnson et al. [88] introduced the ClearPath methodology, which incorporates simulation to explore care pathways while documenting data quality aspects throughout the modeling process.

Further approaches have emphasized patient-centered simulation. Kovalchuk et al. [97] developed a conceptual framework combining process mining and simulation to model clinical pathways for different groups of patients, thus improving the realism of the simulations. Abohamad et al. [6] applied simulation to identify bottlenecks and explore improvements in emergency department operations.

Mans et al. [126] investigated the impact of IT system introduction on healthcare processes through simulation, providing insights into the technological effects on operational performance. Finally, Ahmed and Alkhamis [9] used simulation modeling based on expert interviews to design optimization strategies for resource allocation in an emergency department context.

7.3 Our Contribution to the Open Challenges

Process mining applications in the healthcare domain face several challenges to be dealt with [141]. Building on these insights, our work contributes to tackling several of the open challenges identified by Munoz-Gama et al. [141], through a methodology tailored to the specific needs and constraints of healthcare envi-

ronments. Our proposed methodology aligns with several of these challenges:

C1: Design Dedicated/Tailored Methodologies and Frameworks. We developed a methodology for healthcare scenarios, which integrates simulation and process mining to automate improvements. Its design reflects key features of organizational healthcare processes, such as urgency-driven workflows, variable patient arrivals, and time-critical resource allocation, identified in collaboration with healthcare stakeholders. Note that the underlying principles of the methodology, namely simulation-driven resource allocation and process improvement, might be valid in other domains with similar organizational and resource constraints, although we do not have a supporting case study that can concretely showcase the methodology's application to other domains.

C2: Discover Beyond Discovery. We used simulation to go beyond traditional discovery by incorporating enhancement and conformance checking.

C4: Deal with Reality. We assessed the realism of generated models using state-of-the-art accuracy metrics [47], ensuring fidelity to observed behavior and compliance with medical protocols.

C5: Do It Yourself (DIY). Healthcare professionals were engaged throughout the methodology design to ensure practical relevance and usability, fostering the adoption of process mining tools in the field.

C6: Pay Attention to Data Quality. Recognizing the critical impact of data quality, especially timestamp granularity and missing values, we implemented a preprocessing phase. This includes a novel method for estimating missing start timestamps, described in Section 2.3.

C8: Look at the Process through the Patient's Eyes. Our simulations incorporate patient characteristics to reflect real-world variability. Optimization efforts emphasize patient-centric improvements, such as reducing waiting times, thus enhancing care pathways.

Table 7.1 compares our approach with existing simulation-based methodologies in healthcare (cf. Section 7.2), focusing on how each of these approaches addresses selected challenges. The comparison is conducted at the conceptual and methodological level, as most existing works differ substantially in purpose and scope, often focusing on process modeling, monitoring, or analysis rather than optimization [86, 178, 88, 97, 6, 126]. Conversely, methods such as those by Guastalla et al. [76], Halawa et al. [79], and Antunes et al. [13] primarily focus on model construction or simulation design, without incorporating data

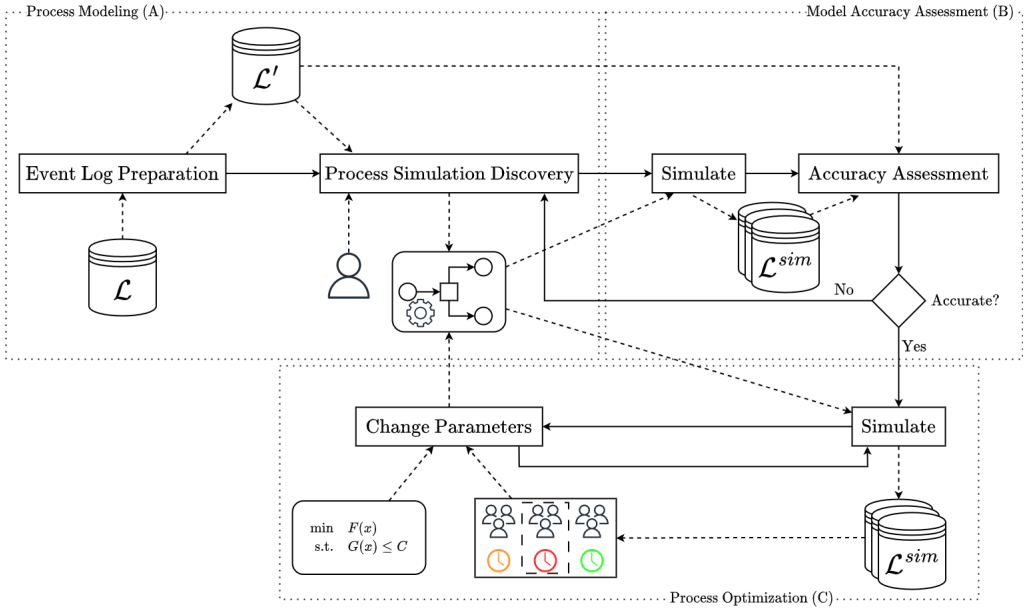


Figure 7.1: Healthcare Process Improvement via Simulations schema. In an initial step the event log is firstly prepared to use it for Process Discovery techniques, and optionally including user validation, for building a process model then enhancing it with multi perspective parameters. Step B assesses the accuracy of the simulations. Finally, the last step uses simulations for providing optimal configurations. Rectangles and solid arrows represent the method workflow, while elements connected with dashed lines indicate input and output objects.

quality improvement, model accuracy validation, or patient-specific variability. Similarly, Ahmed et al. [9] rely on models that are manually built and not extracted from the data: the introduction of this stakeholders' subjectivity can lead to situations in which improvement is made on the supposed processes, which are detached from reality. Given these fundamental differences in objectives and methodological scope, a direct experimental comparison would not yield meaningful insights.

7.4 Methodology

This section presents our data driven improvement methodology for healthcare processes. The methodology is designed mainly for organizational healthcare processes, where resource allocation decisions and predictable control-flows are primary levers for improvement.

The methodology is depicted in Figure 7.1, where the phases are outlined: *Process Modeling (A)*, *Model Accuracy Assessment (B)* and *Process Optimization (C)*.

The input is an initial event log $\mathcal{L}$, that, after an *Event Log Preparation* step, is used for discovering a simulation model, with user support if needed. In the second phase, the accuracy of the simulation model is assessed via measures that compare simulated event logs with the actual one through several perspectives. If the simulation model is not accurate, the previous modeling step is repeated by repairing the discriminated characteristics; otherwise, it is ready to be used in the optimization phase. Finally, we use the simulation model to obtain simulated event logs, from which statistics are derived to guide optimization in changing the simulation parameters. This phase is repeated until a stop criterion is reached or until no further improvement is possible, returning a simulation model that reflects the optimal process configuration.

Process Modeling (A)

The starting point is the creation of a proper event log to use as input to discover a simulation model. The discovery of the distribution of the activity duration requires the logs to contain the events that indicate the starting and completion of each activity execution. This often does not occur, and then we apply our technique discussed in Chapter 2 for estimating missing timestamps.

Once the event log has been prepared, our methodology aims to discover a process simulation model M, from the event log, replacing the human subjectivity in creating model with the objective behavior observed in the event log (cf. Section 1.6).

The simulation model is derived from the event log using the techniques discussed in Part I, whenever relevant information is available, for example, probabilities of activity executions, arrival rates, and temporal distributions. When such information is not present in the event log, it can be provided directly by domain stakeholders. This was the case in the emergency department study presented in Section 7.5, where the information of the resource perspective could not be derived from the available event log. Therefore, healthcare stakeholders provided role definitions (i.e. groups of resources associated with specific activities and their working schedules, such as doctors, nurses, and radiologists) and cost information, which were then integrated into the simulation parameters.

Accuracy Assessment (B)

A successful application of process simulation for process improvement relies on a simulation model that reflects the real process behavior. Conclusions drawn from an unrealistic simulation model lead to process redesigns that may not yield improvements or even may worsen the performances (cf. Section 1.6.3).

As discussed in Section 1.6.3, Chapela-Campa et al. [47] proposed several metrics for evaluating the accuracy of the process simulation model to mimic reality. In this work, we use normalized metrics, obtained by dividing each value by the maximum possible distance, resulting in values ranging from 0 to 1. This enables an easier interpretation of model quality: values near 0 indicate better similarities between the simulated and actual processes, while values that become closer to 1 suggest a poorer similarity. As discussed in [47], normalization methods have been explored, but the authors advise against their use for inter-process comparisons due to inconsistencies introduced by varying log characteristics. However, since our focus is not on comparing different processes, we apply normalization solely to enhance the interpretability of our results.

Process Optimization (C)

Process simulation techniques enable the conduction of *what-if* experiments, avoiding the need for building or disrupting the real-world system, with the scope to analyse and improve processes. By examining various *what-if* scenarios, it is possible to assess the impact of potential changes on the processes and the associated protocols and norms that define the current process.

In the healthcare domain, a main goal is to improve patient journeys while at the same time ensuring that the overall process remains sustainable. An optimal resource allocation may improve the process in several dimensions, for example, reducing patient waiting times, resources overloading, and overall costs. However, some objectives are often conflicting and achieving an effective balance between them is crucial. For example, while reducing patient waiting times can be accomplished by allocating additional resources and/or expanding physical spaces, such improvements inevitably lead to increased operational costs. However, these costs must align with available budgets and policies to ensure the sustainability of the process.

The goal is to improve the balance of multiple KPIs, each modeled as a function $f_i(M)$ that are computed on the behaviors simulated by a given simulation model $M = (N, D, K)$, with N being the Petri net model, $D = (\mathcal{R}, C^{\mathcal{R}}, MT^{\mathcal{R}}, d^{\mathcal{V}})$ the tuple of descriptive parameters, and $K = (p^R, p^{ET}, p^{WT}, p^{AT})$ the tuple of predictive models. In particular, optimization focuses on determining an optimal resource allocation, determined by the resources in $\mathcal{R}$, and their allocation given

by p^R:

$$\begin{aligned} \min_{\mathcal{R},\, p^R} \quad & \mathbb{E}\left[\sum_{i=1}^{n} w_i \cdot f_i(M)\right] \\ \text{s.t.} \quad & g(M) \leq C \end{aligned} \tag{7.1}$$

where $w_1, \ldots, w_n$ are constant weights, and the constraint $g(M) \leq C \in \mathbb{R}^K$, $K > 0$, ensures that certain constraints are satisfied, such as the maximum number of resources or total cost. Weights $w_1, \ldots, w_n$ are set to normalize the KPI values and/or to assign larger or smaller importance to some KPIs over others. In fact, Section 7.5 illustrates how the weights have been determined for the case study presented in this work, so as to normalize the influence of the two KPIs involved.

Note that although optimization targets only resource parameters $\mathcal{R}$ and p^R, the KPI functions $f_i((N, D, K))$ account for all simulation parameters in D and K. In fact, some KPIs, such as waiting time, are influenced not only by resource parameters but also by others, such as activity execution times: shorter activity durations can free resources earlier, reducing waiting times. This distinction ensures that we optimize only the resource-related aspects of the process without ignoring the broader dependencies captured by the simulation.

To solve this optimization problem, we propose a simulation-based algorithm that aims to identify the best trade-off solution. The algorithm begins with an initial resource configuration. Note that this initial configuration could represent a resource allocation where each role has exactly one resource, or it could be an existing configuration from the current system. For each role $\rho \subset \mathcal{R}$, i.e. a set of resources with same activity assignments, the algorithm computes the objective function by only retaining the events that are associated with activity instance performed by resources in ρ.

Specifically, these values can be computed from a simulated event log $\mathcal{L}^{sim}$, generated using the simulation model M, filtering on the events associated with that role. These values reveal the roles where bottlenecks are most likely to occur. The algorithm then selects the role with the highest value and assigns one additional resource to it, ensuring that all constraints are satisfied. Specifically, if $\rho = \{r_1, \ldots, r_k\}$ is the identified bottleneck role, a new resource r is added, thus obtaining the new role $\rho \cup \{r\}$, with $p^R(r)(a) = p^R(r_1)(a) = \cdots = p^R(r_k)(a)$ $\forall a \in \mathcal{A}$, where $p^R \colon \mathcal{R} \to (\mathcal{A} \to \mathbb{R}^+)$.

This approach is based on the hypothesis that allocating additional resources to the most critical role effectively mitigates current capacity constraints, thereby reducing bottlenecks and enhancing overall process performance. The procedure is then iteratively repeated until a certain stopping criterion is reached, e.g.

when the objective function does not improve any further, or the improvement is lower than a given user defined threshold. Finally, the algorithm returns a potential optimal solution, representing a trade-off between the KPIs.

The robustness of the optimal configuration can be evaluated by experimenting with different scenarios, such as an increased patient flow. Further analysis can be conducted on various patient attributes to show the efficiency of the resource configuration across diverse patient profiles.

7.5 Case Study

This section presents the evaluation of our method through a new real-life case study concerning the process of an emergency department of an Italian hospital. The case study is aimed to improve the process at the emergency department and fairly minimize the waiting time of patients in their journeys, while keeping the process sustainable in terms of costs. In particular, the waiting time for a patient is here considered as the sum of waiting times for each activity carried out at the emergency department. The waiting time for an activity for a patient is the delay between when the activity is ready to be performed for the patient in question and when the activity is actually started. This delay is typically caused by capacity constraints (e.g., resources availability), which prevent an immediate start of the activity. In this urgency case scenario, waiting time represents a critical key performance indicator: lower waiting time reduces the patient risks, and increases patient satisfaction because they are treated more promptly. At the same time, managing operational costs related to resource usage is essential for maintaining a sustainable and efficient healthcare service.

We first introduce the case study, describing the dataset and the process (cf. Section 7.5.1). In Section 7.5.2 we discuss the modeling and the accuracy evaluation steps. In Section 7.5.3 we applied our methodology to derive an optimal resource configuration, further evaluating its robustness in variable patient flow scenarios in Section 7.5.4. Finally, Section 7.5.5 reports on the discussion of the results with the emergency department staff.

7.5.1 Introduction to the Event Dataset and Process

The construction of the process simulation model for process improvement has been based on an event log that comprises 37942 traces with a total of 368081 events, covering eight months of process data. Each trace corresponds to the journey of a patient through the Emergency Department, from triage to discharge. The data used in this study cannot be shared due to confidentiality. Table 1.1 illustrates the excerpt of the event log, which covers the events observed for two

Table 7.2: Emergency department event description. First column represents an unique attribute. The second indicates the number of classes of the associated attribute present in the event log. Last column describes the instance attribute.

Attribute	N. Classes	Description
Case Id.	37942	The trace identifier.
Activity	42	Activity label.
Timestamp	-	Associated timestamp (It could be missing).
Lifecycle	2	Associated lifecycle (*start* or *complete*).
Age	-	Integer representing the age of the patient.
Access Type	10	The access type (*Autonomous*, *Ambulance without* or *with doctor*, *Helicopter*, etc.).
Urgency Code	5	A color referring to the urgency case (*White*, *Blue*, *Green*, *Yellow*, *Red*).
Pathology	40	Pathology assigned at the triage phase.

traces, namely for two patient journeys. Table 7.2 provides some statistics and information about the event log and the process. The event log includes information about the process-instance identifier, executed activities with associated timestamps and life-cycles, the age of the patient, the access type, an urgency code (triage color), and the pathology assigned at the triage phase.

According to the regional regulations, five triage colors are introduced [176]:

1. *White code*: non-urgent patients whose conditions are not true accidents or emergencies.
2. *Blue code*: standard patients without danger or distress.
3. *Green code*: urgent patients with serious problems, but apparently stable condition.
4. *Yellow code*: very-urgent patients with serious illness or injured patients whose lives are not in immediate danger.
5. *Red code*: immediate patients in need of immediate treatment for the preservation of life.

Each group can have different treatments and priorities, and patient flow may change accordingly to the emergency case. Table 7.3 reports on the cycle times statistics computed for each triage color, comparing them with those of the complete event log. A notable observation is that an increasing level of urgency leads to higher cycle time and higher standard deviation, due to the variations of more urgent cases. Also, median cycle times differ significantly from the mean, suggesting the presence of several outliers in the case study.

Table 7.3: Average, Standard Deviation and Median cycle time, expressed in minutes, of a process instance per triage color. Last row shows the statics computed in the full event log, i.e. not filtered by triage color.

		Cycle Time (*minutes*)		
Color Group	**Description**	**Avg**	**Std**	**Median**
White - 1	Non-Urgent	191.42	1017.72	104.73
Blue - 2	Standard	255.26	3042.29	121.41
Green - 3	Urgent	355.22	3210.74	145.63
Yellow - 4	Very-Urgent	833.08	5272.11	223.93
Red - 5	Immediate	1215.68	7979.34	162.21
Complete		486.27	3986.25	159.38

Table 7.4: Frequencies of events and traces for train and test event logs per triage color. The column total is the sum of the frequencies of the train and test logs. Last row shows the frequencies in the full event log, i.e. not filtered by triage color.

	Number of Events			**Number of Traces**		
Color Group	**Train**	**Test**	**Total**	**Train**	**Test**	**Total**
White - 1	2372	588	2960	384	96	480
Blue - 2	31685	8482	40167	4436	1220	5656
Green - 3	148054	36600	184654	16454	4122	20576
Yellow - 4	105390	25377	130767	8529	2045	10574
Red - 5	7971	1562	9533	550	106	656
Complete	295472	72609	368081	30353	7589	37942

Many events related to the starting of activities are missing. As mentioned, this would make it impossible to compute the activity durations. Therefore, we preprocessed the event log to estimate the timestamps of the missing events related to the starting of activities. As mentioned, the technique to estimate the start-event timestamp is based on process simulation, thus requires a full-fledged simulation model, except for the activity duration distribution.

The process model was initially discovered via Inductive Miner [105], and later we discussed it with the process experts, who were largely satisfied but asked to apply some changes. These included adjusting the concurrency block after the *Triage* activity to allow the *Visit* to be executed in parallel with other activities, and introducing a loop for the *Lab Service* activity following *Blood Sampling*. Figure 7.2 shows the model using the BPMN notation: in fact, the model is fairly simple and understandable.

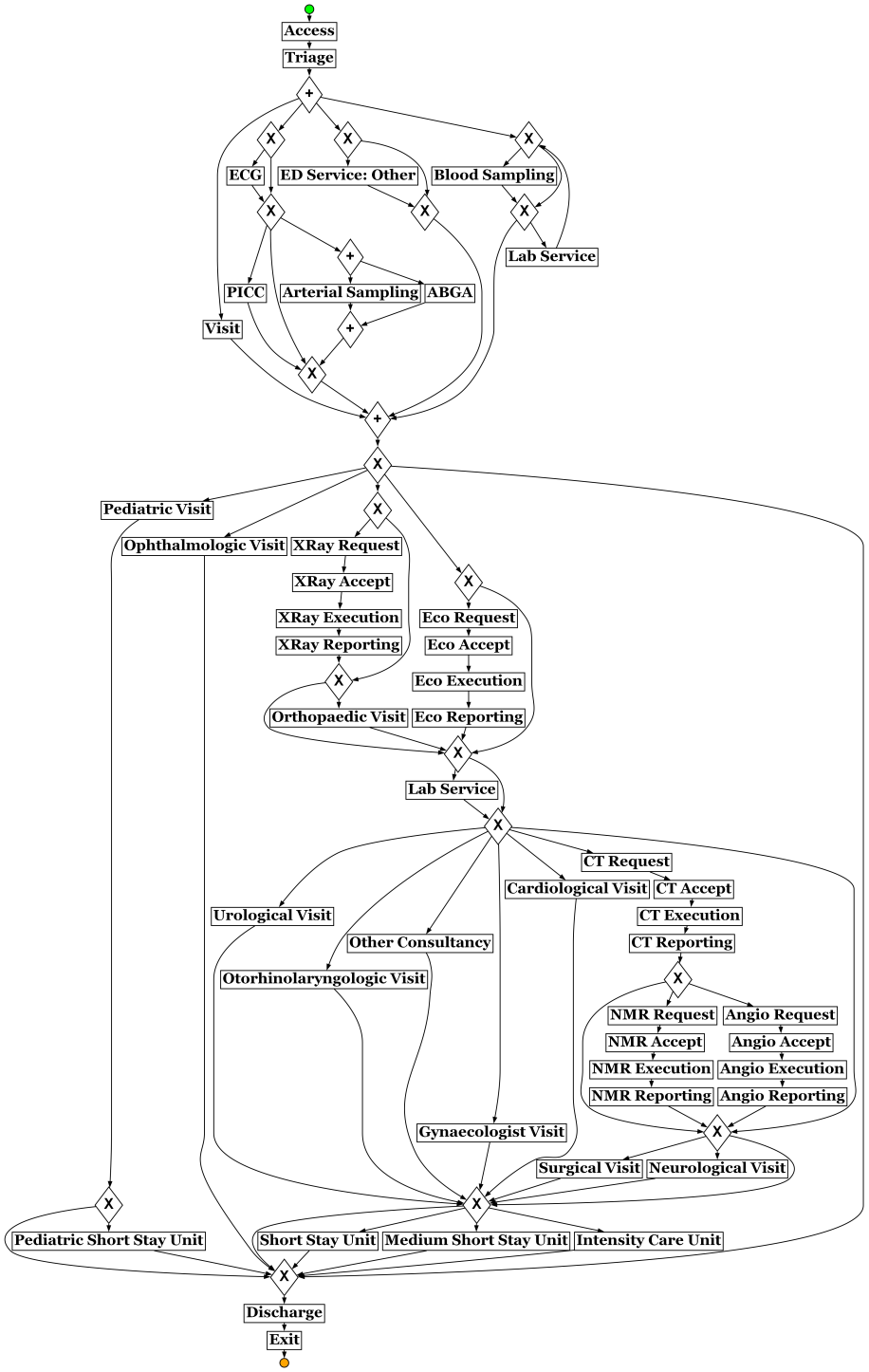


Figure 7.2: Emergency department process model in BPMN notation. The green circle represents the initial state, while the orange represents the final. Labels in the rectangles indicates activity tasks. “+" and “X" represent respectively AND and XOR splits/joins.

Table 7.5: Number of resources per role with their associated activities and cost per hour.

Role	Resources per hour	Cost/h	Activities
Doctors	4	75€	*Visit, Discharge, ED Service Other*
Nurses	5	30€	*Blood Sampling, ECG, PICC, Arterial Sampling, ABGA*
Triage N.	2	30€	*Triage*
Radiologist	1	75€	*XRay, Eco, CT, NMR, Angio Reporting, Eco Execution*
Rad. Tech.	2	30€	*XRay, CT, NMR, Angio Execution,*
Beds	12	30€	*(Pediatric) (Medium) Short Stay Unit, Intensity Care Unit*
Consultant	1 per kind	75€	*Pediatric, Cardiological, Orthopaedic, Surgical, Neurological, Urological, Ophthalmologic, Gynaecologist, Otorhinolaryngologic Visit*

7.5.2 Process Simulator Modeling & Accuracy Assessment

To assess the accuracy of the process model and the simulation model, we temporally divided the event log into a train and a test set. The first 80% of the traces in chronological order were allocated to the training set, while the last 20% to the test set, following a commonly adopted approach when assessing the accuracy of simulation models [113, 41, 42]. Table 7.4 shows the frequencies of events and traces in both sets, also filtering them by urgency code. The training set is used to build the process model and simulation model. The test set is employed to compute the metrics defined in Section 1.6.3, which aim to evaluate the accuracy of the discovered simulation model.

Section 7.5.1 concluded with the discussion of the discovery of the process model in Figure 7.2. This process model is characterized by fitness of 0.97 and precision of 0.82, which were both computed on the test sets. The model also exhibits a high level of generalization with a score of 0.93, highlighting that it is not overly restricted to the behavior observed in the event log. The high values for all of the three metrics indicate that the process model is capable to faithfully represent the process' behavior. It is therefore an excellent candidate for use in simulation.

For the construction of the simulation model, the parameters were trained as discussed in Section 7.4, except for the resources, roles, and their calendars, which were defined separately.

Unfortunately, the event log did not contain resource information for role and calendar discovery. The lack of resource-perspective information was compen-

Table 7.6: Simulation distance measures computed averaging the results obtained across various simulations. The rows with triage colors show results obtained by filtering the test event log and the simulated event logs by the corresponding color. Recall values are normalized between 0 and 1, where 0 indicates no distance.

Event Log	CFLD	2GD	3GD	AED	CED	RED	CAR	CTD
Complete	0.13	0.19	0.25	0.08	0.07	0.01	0.09	0.01
White - 1	0.09	0.15	0.18	0.17	0.15	0.05	0.15	0.10
Blue - 2	0.07	0.16	0.19	0.08	0.08	0.01	0.09	0.01
Green - 3	0.11	0.17	0.22	0.09	0.07	0.01	0.09	0.01
Yellow - 4	0.20	0.26	0.35	0.07	0.07	0.01	0.08	0.01
Red - 5	0.30	0.36	0.47	0.14	0.08	0.01	0.13	0.02

sated by inspecting the hospital information and interviewing process experts: the Emergency Department operates 24 hours a day, 7 days a week. Each role is associated with specific set of activities. Table 7.5 provides a detailed overview of the number of resources available per hour for each role, the associated activities and the cost per hour, which is useful for further analysis.

The fully-fledged simulation model was then used ten times to generate ten simulated event logs, each containing the same number of traces as the test event log. If the simulation model is accurate, the simulated and test event logs are sampled from the same distribution. The metrics introduced in Section 1.6.3 are indeed meant to assess this. These metrics are then averaged across the ten simulated event logs, thus mitigating the intrinsic stochasticity of running a simulation.

These averages are reported in Table 7.6, and they are also filtered by different triage colors. Recall that these metrics are normalized in the range $[0, 1]$, where zero indicates a perfect match of the distribution between the simulated and test event logs (cf. Section 7.4).

CFLD, 2GD, and 3GD metrics generally score relatively low and similarly on average, particularly for the most common colors given at triage, namely white, blue, and green, which together account for 70% of the cases (26,711 out of 37,942 traces, as shown in Table 7.4). This is largely because the discovery of the simulation model is naturally aiming to represent the most frequent case types. Notably, 3GD, which measures similarity across sequences of three events, tends to yield worse scores than 2GD and CFLD: capturing longer activity dependencies is inherently more complex. The model performs less effectively for traces associated with a red color at triage, especially in terms of the 3GD metric. This is likely due to a significant imbalance in the training data, where red-colored traces make up only 1.8% (550 out of 30,353) of the total training traces. Nonetheless,

these instances are rare and have limited impact on the overall performance of the simulation model. As illustrated in Table 7.6, the weighted average remains largely unaffected by the lower scores associated with executions for patients who are assigned a red color at triage. In conclusion, the simulation model can well mimic the control-flow perspective of the emergency-department healthcare management process.

The other metrics are related to the temporal perspectives and assume very low values: this highlights that they are well simulated by the model. The temporal perspective is also tightly connected to the resource perspective: if the resources, with their schedules and assignment to roles and activities, were poorly modeled, this would affect the realism of the waiting queues, namely the queues of activities that are waiting for starting because all resources are not available at that point. Poor waiting-queue realism would consequently degrade the realism of the temporal perspective: for instance, the simulation's waiting times would not match the real waiting times.

In summary, the simulation model shows a high level of realism for most of potential behavior, and thus can be used to improve the process, aiming to improve on the resources allocation.

7.5.3 Resource Allocation Improvement

Once the simulator is built, it can be used for improvement purposes as described in Section 7.4. In the emergency department case study, one of the main objectives is to identify an optimal resource allocation that minimizes patient waiting times while ensuring the process cost remains sustainable.

The case study goal is to minimize the expected **waiting time of patients** into the hospital, thus minimizing the overall times and improving their journeys. The expected patient waiting time can be computed via simulations. Given a simulated event log $\mathcal{L}^{sim}$, obtained from a simulation model M, with $\mathcal{R}$ the set of resources and p^R the resource allocation parameter (cf. Section 7.4), the expected waiting time is computed as follows:

$$\mathbb{E}\left[f_1(M)\right] = \frac{1}{|\mathcal{L}^{sim}|} \sum_{\sigma \in \mathcal{L}^{sim}} \sum_{e \in \sigma} wt(e) \tag{7.2}$$

where $wt(e)$ indicates the waiting time of the activity instance $act(e)$ to which e refers, which is here computed as the difference between the time in which $act(e)$ starts and the time in which the activity instance preceding $act(e)$ completes. The rationale is that high waiting times are due to the congestion of some resources, then increasing the number of the resources related to the distinguish

roles could decrease the waiting times associated with the activities executed by them. However, not adequately increasing the number of resources can lead to unsustainable costs. Then, the **total resource cost** must also be managed during the optimization process. The goal is hence to find a good trade-off between total cost and expected waiting times. The total hourly cost can be computed as follows:

$$f_2(M) = \sum_{r \in \mathcal{R}} Cost(r) \tag{7.3}$$

where $Cost$ refers to the hourly cost function $Cost$ returning the cost per hour of a certain resource. Note that since $f_2(M)$ is constant for fixed M, it follows that $\mathbb{E}[f_2(M)] = f_2(M)$.

The optimization problem in Equation 7.1 is defined here using the two KPI functions defined in Equation 7.2 and Equation 7.3. Specifically, note that:

$$\begin{aligned} \mathbb{E}\left[w_1 \cdot f_1(M) + w_2 \cdot f_2(M)\right] &= w_1 \cdot \mathbb{E}\left[f_1(M)\right] + w_2 \cdot \mathbb{E}\left[f_2(M)\right] \\ &= w_1 \cdot \mathbb{E}\left[f_1(M)\right] + w_2 \cdot f_2(M) \end{aligned} \tag{7.4}$$

where the weights are selected to normalize and assign equal importance to both KPIs, f_1 and f_2. Specifically, w_1 and w_2 are determined based on the maximum possible values that their corresponding KPI functions can achieve. The weight w_1 is set to $1/45.25$, where 45.25 minutes represents the current average waiting time. The weight w_2 is set to $1/(1830 + N \cdot 75)$, where 1830€ is the current hourly cost, and the term $N \cdot 75$ accounts for the potential increase in cost after adding N resources—each contributing to a maximum of 75€ per hour (cf. Table 7.5). This ensures that both performance measures are scaled comparably, allowing for balanced optimization.

For this case study, we defined the constraint where resources from the current configuration must be retained. Removing resources is not permitted, as it could lead to unfair conclusions where certain resources might be laid-off. To satisfy this, we initiated the algorithm by using the current configuration (see Table 7.5), thus ensuring that its resources cannot be removed. The optimization problem is then solved by using the algorithm proposed in Section 7.4.

We experimented with a maximum of 10 iterations. The most substantial improvements occur within the first two iterations, after which the optimization process stabilizes. Figure 7.3 illustrates the average waiting time (y-axis) and total hour cost (x-axis) for each configuration, highlighting the rapid initial gains followed by a plateau in performance. The optimal solution, i.e. the one minimizing the objective function, is highlighted in green. Specifically, it chooses as the best configuration the one obtained after two iterations, leading to a configuration with *one more radiologist and one more orthopedic*, which is expected

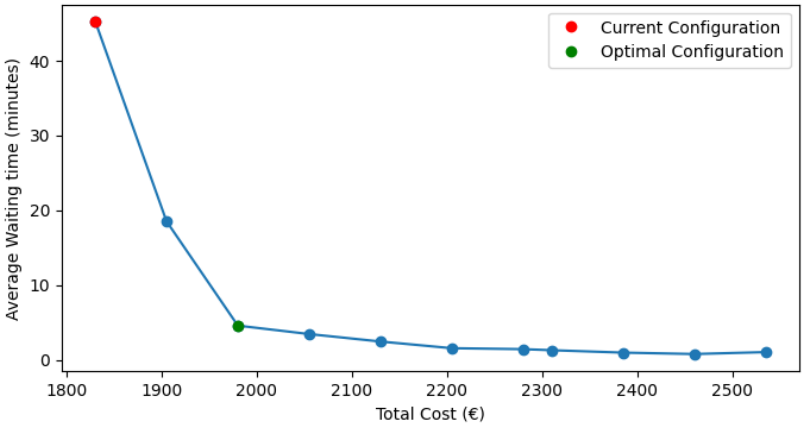


Figure 7.3: Average waiting time (in minutes) per total cost per hour (in €). Each point represent a resource configuration. The red point represents the results using the initial, i.e. the current, resource allocation. The green point indicates the optimal solution.

to decrease the waiting time of ca. 89%, in exchange for a modest hourly-cost increase of ca. 8%.

7.5.4 Readiness Analysis to Sudden Workload Peaks

Once the best configuration was identified, we focused on the ability of the system to tackle sudden peaks in the arrival rate, e.g. due to an emergency scenario. Section 7.1 discussed the importance of healthcare systems to respond to sudden peaks. This readiness analysis adopted the methodology from [97] by adjusting the patient flow. Specifically, we tested arrival rates with a patient-flow ratio set at 0.5, 1.0, 1.5 and 2.0 times the baseline arrival rate of the original configuration. For example, a patient ratio of 0.5 corresponds to a scenario in which the patient flow decreases of 50%, 1 to the current patient flow, 1.5 signifies a 50% increase and 2 a scenario in which patient arrivals double.

For each patient ratio, we simulated 10 event logs and computed the average of the waiting times across these simulations. The results are depicted in Figure 7.4, and reveal a better readiness of the improved process configuration to emergency scenarios, compared to the initial process configuration: waiting times exhibit a significantly faster increase for the initial process configuration. This highlights the efficacy of our optimization approach in managing workflow efficiency and mitigating delays, thereby ensuring more stable and predictable performance under varying patient ratios.

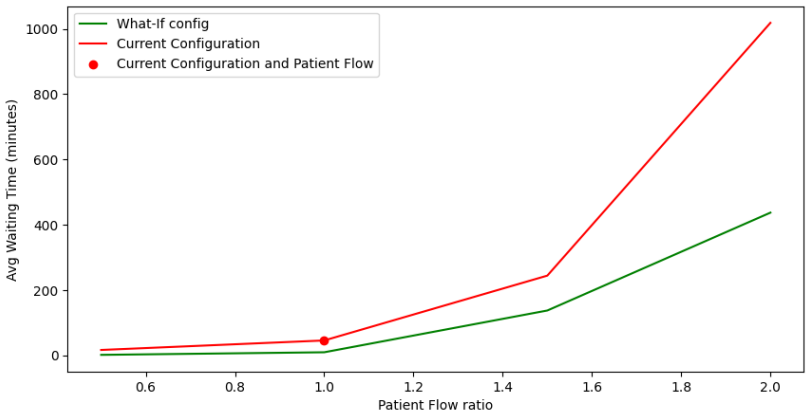


Figure 7.4: Average waiting time (in minutes) per patient flow ratio. Results are obtained averaging values from various simulations. The red line represent results obtained using the current configuration with various patient flow ratio. The green highlights those obtained by using the optimal configuration identified by our method.

7.5.5 Discussion with ED Staff: Model Check and Operational Insights

The simulation model and the corresponding results were presented to ED staff in a dedicated debriefing session. The discussion served two purposes: (i) a plausibility check of model structure and performance realism against day-to-day operations, and (ii) interpretation of scenario results to surface pragmatic improvement actions. Overall, ED clinicians confirmed that the control-flow, resource roles, and performance levels reflected by the model align with how the ED actually works. Specifically, the set of activities and routing logic mirrored their experience of parallel diagnostic and/or treatment paths following triage and visit. This is consistent with the process model previously discovered, which they recognized as a faithful abstraction of their workflows.

A key point of convergence between the simulated evidence and staff perception concerned imaging. Teams emphasized that radiology behaves as a heavily utilized single "server" that periodically attracts a surge of simultaneous requests from multiple pathways (e.g., imaging after the initial visit, or in parallel with blood sampling/lab work), creating queue build-ups during peak windows. This observation is aligned with the model structure and with the resource distribution (cf. Table 7.5), which shows limited radiological reporting/execution capacity relative to the number of upstream activities that can generate demand. In practice, this concurrency produces short periods of high contention even if average loads look manageable, explaining why Radiology appears as the dominant bottleneck in both data and practice. Conversely, the team found the

result concerning the orthopedic specialist less intuitive. While an orthopedist dedicated to the ED is clinically valuable, they noted that orthopedic consults do not arrive at a steady pace; instead, quiet periods alternate with bursts. Under these conditions, a single, fixed consult channel can intermittently inflate waits for the subset of patients requiring orthopedics, even when other resources are momentarily available. This observation does not contradict the *what-if* analyses that identified an additional orthopedist (together with a radiologist) as part of a cost-effective configuration overall; rather, it highlights that consult scheduling and flexibility matter when arrivals are bursty. Staff also recalled that, in prior years, orthopedists remained on their ward and were called in when needed, rather than being assigned as a dedicated ED resource. In that time, they had longer average waiting times for patients, but they were a little more flexible when they had demand peaks.

The staff's appraisal of improvement options was generally positive. There was strong consensus that increasing radiology capacity dedicated to ED demand – for example, via additional radiologist coverage during peak hours (also shared by other EDs since the reporting is possible "online") and/or an extra technician on high-demand shifts – is a sensible and immediately actionable lever. This judgment coheres with the *what-if* evidence showing that targeted additions on the imaging side yield outsized reductions in average waiting time at modest marginal cost. Although the exact configuration will depend on numerous factors, the direction of change was considered realistic and operationally acceptable by the ED leadership. At the same time, rather than simply adding more dedicated orthopedic coverage, the discussion suggested organizational refinements to absorb burstiness without investing too many resources and creating new rigid bottlenecks: e.g., on-call orthopedist responding within set time windows when the ED consult queue exceeds a threshold or pooling ED orthopedic with the ones assigned to the hospital ward for cross-cover during peaks.

7.6 Conclusion

Optimizing processes is a critical and complex challenge in the healthcare domain. Optimal process configuration can enhance patient journeys, reduce costs, and improve the overall quality of care. Process mining techniques offer powerful tools for modeling and analyzing real-life processes [1]. The resulting process models can be used for simulation purposes, enabling stakeholders to observe how potential changes might impact performance without the high costs and risks associated with real-world experimentation. Assessing the accuracy of

these models is crucial. In fact, findings derived from experiments performed using an inaccurate process model could be implausible and, hence,not acceptable.

In this chapter, we proposed a methodology that leverages process simulation to support the optimization of healthcare processes. The proposed methodology begins with the application of process discovery techniques, which are used to extract a process model from real execution data. This model is then enhanced with various perspective parameters, such as time, cost, and resource usage, to provide a more complete and accurate representation of the actual process. The accuracy of the simulation model is assessed using the metrics proposed by Chapela-Campa et al. [47] which consider the different perspectives. Finally, the simulation model can be used for optimization purposes. We specifically introduce an automated optimization approach that uses simulation to identify optimal process configurations. The goal of this optimization is to balance multiple Key Performance Indicators (KPIs), such as minimizing patient waiting time while controlling or reducing operational costs.

We applied our methodology in an emergency department case study. In this context, we formulated a stochastic optimization problem aimed at identifying a resource configuration that would reduce patient waiting times without leading to unsustainable increases in cost.

Finally, we evaluated the robustness of this optimal configuration through a series of *what-if* scenarios in which we varied patient flow conditions to simulate potential crisis situations. Findings show the optimal configuration of resources maintains more stable performance, if compared with the current one, in critical scenarios, suggesting its effectiveness not only under normal conditions but also in more demanding or uncertain operational contexts.

8 Simulating Process Recommendations

Process-aware Recommender (PAR) systems combine predictive monitoring and prescriptive analytics to support operational decision-making by anticipating the future evolution of ongoing process instances and recommending corrective interventions. However, evaluating the effectiveness of such systems remains challenging due to the lack of ground truth in real-life event data. This chapter investigates simulation-based evaluation as a means to address these limitations. We propose an approach that maps the current state of ongoing process instances into a simulation model and then continues execution from this state while injecting activity and resource recommendations generated at runtime. By reproducing realistic process dynamics under controlled and observable conditions, the approach enables systematic assessment of recommendation strategies and provides insights into their operational impact.

8.1 Introduction

Process-aware Recommender (PAR) systems support operational decision-making in business processes by combining **predictive monitoring** and **prescriptive analytics** to anticipate how ongoing cases will evolve and to recommend actions that guide to desirable outcomes, defined through process-specific KPIs such as time or cost. Predictive monitoring techniques forecast future aspects of running cases, such as remaining time, next activities, or final outcomes. Prescriptive analytics goes beyond predictive methods by not only forecasting future states, often undesired, but also recommending or prescribing interventions preventing undesired outcomes and improving overall process performance (cf. Section 1.5).

Despite the growing maturity of prescriptive process analytics, assessing their effectiveness remains challenging. A central limitation is the absence of ground

truth regarding the true causal impact of recommendations or the future evolution of cases had certain recommendations been executed. Real-life event logs only record the actual observed behavior, without revealing unobserved or alternative process paths. Moreover, real logs are often noisy and incomplete, making it difficult to evaluate the reliability of monitoring and recommendation systems.

To overcome these limitations, simulation-based evaluation may be used to reproduce realistic process executions under controlled conditions. In this chapter, we investigate how simulation models can be used to evaluate recommendation strategies at runtime, including both activity and resource recommendations. Specifically, given a set of ongoing process instances, we aim to align the simulation model so that it reflects their current state, and subsequently execute simulations from this state while integrating the recommendations produced at runtime.

The chapter starts giving a brief overview of assessment practices in process mining using synthetic data (cf. Section 8.2). Section 8.3 presents our framework for simulating recommendations from a given process state. Section 8.4 reports on experiments on two different prescriptive methodologies, where our framework has been applied. Section 8.5 concludes the chapter by summarizing findings and discussing possible future directions.

8.2 Related Work

The assessment of process mining techniques using real-life event data is often limited by the lack of ground truth knowledge [175]. Synthetic data offers a promising alternative, as it enables the generation of process behavior, including those that may not be directly observable in real logs. Several tools have been proposed for the generation of synthetic process data. [34] introduced *PLG2*, a tool for the generation of random processes, by producing multiperspective models and logs. [89] presents *PTAndLogGenerator*, a tool that enables the generation of collections of non-structured process models with user-defined probability parameters and supports the subsequent generation of event logs. Burattin et al. [37] proposed *PURPose-Guided Log gEneration (PURPLE)*, a framework for producing event logs with specific properties tailored to different process mining tasks. *AIR-BAGEL* [96] is an interactive tool designed to inject pseudo-real anomalies into event logs by associating them with specific root-causes. All these approaches share a common assumption: a reference process model is considered to represent the true underlying behavior and is first used to generate a "clean" event log, which is then altered through the injection of noise or anoma-

lies. However, this does not allow for generating data containing typical and realistic deviations observed in real-life processes. To address this limitation, Sommers et al. [175] recently proposed an approach that incorporates patterns of behavioral deviations and recording errors, enabling the generation of synthetic yet realistic deviating process models and noisy event logs. However, the aforementioned approaches primarily focuses on supporting the evaluation of process mining algorithms rather than prescriptive.

The application of Business Process Simulation for operational decision-making was first introduced in [155]. In this work, the authors proposed to extract the current state of processes from enterprise information systems and then performing short-term simulations to evaluate decisions affecting process behavior in the near future. Based on this, Wynn et al. [198] and Rozinat et al. [165] proposed approaches to discover BPS models from completed traces and to extract the state of ongoing cases. These studies assume the current process state to be known in advance: which activity instances are enabled, which resource are available, etc.

More recently, Avramenko et al. [17] proposed an approach that initializes the simulation model from a representation of the current state derived from an event log of ongoing cases. In this work, the authors evaluated the ability of the approach to replicate the as-is process, but without investigating the performance of the process after a change. De Moor et al. [54] proposed *SimBank*, a simulator for benchmarking prescriptive process monitoring methods. However, the simulator is limited to a bank's loan process and relies on simplifying assumptions, such as the absence of interdependencies between process instances.

8.3 Framework

This section presents a framework for generating and assessing recommendations based on Business Process Simulation. The goal is to use the current execution state of a running process and evaluate the impact of alternative recommendations by simulating how the ongoing cases would complete under those recommendations. Figure 8.1 illustrates the framework. The core idea is to reconstruct the current process state from ongoing traces, start a simulation from this state injecting recommendations into the simulator, and then measure their impact through key performance indicators (KPIs) computed on the simulated outcomes.

Given a timestamp $t \in \mathbb{N}$, we indicate with $\mathcal{L}^t = \{\sigma_1, \ldots, \sigma_k\}$ a set of ongoing traces at timestamp t: each $\sigma = \langle e_1, \ldots, e_h \rangle \in \mathcal{L}^t$ represents a partial execution

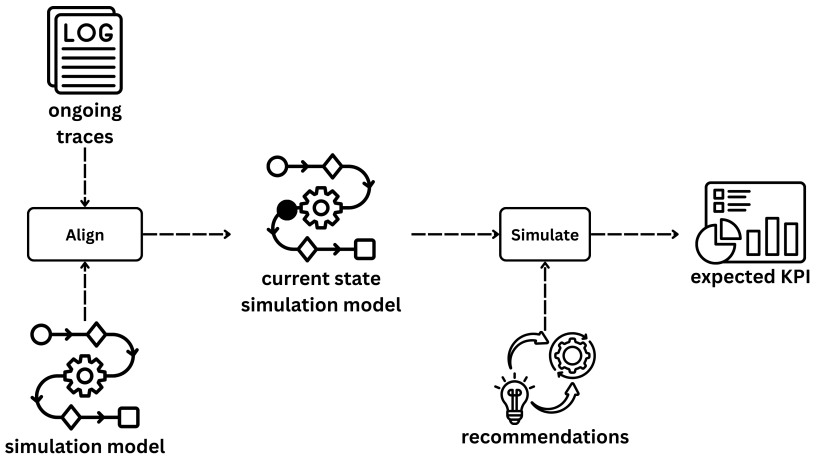


Figure 8.1: Framework for simulating recommendation effects on key performance indicators (KPIs). First an event log of ongoing traces is aligned within a simulation model to reflect the current simulation state. Then, recommendations are injected and traces are simulated thus computing the expected KPI.

with $time(e_h) \leq t$. Note that the traces in $\mathcal{L}^t$ are extracted from a test log, from which it is known that the corresponding process instances continue beyond time t.

The framework relies on a simulation model $M = (N, D, K)$, with N being a Petri net representing the control-flow of activities, D and K the sets of descriptive and predictive parameters, respectively (cf. Section 1.6). The ongoing traces are then mapped to markings in the process model N using token-based replay techniques (cf. Section 1.3.4), specifically the one in [25], which are well suited for mapping prefixes of traces to process models. Through token replay, each ongoing trace σ_i is then assigned a marking m_i. Moreover, the corresponding process state s_i is derived accordingly to Definition 13. Together, these results yield the construction of a set $\{(m_1, s_1), \ldots, (m_k, s_k)\}$ of marking-process-state pairs for each ongoing trace that are used to initialize the simulation model. In this way, the simulation can start from a configuration that accurately reflects the execution state of the process at time t.

Once the simulation model is configured to reflect the current state, a set of recommendations, provided from a Process Aware Recommender system, can be injected into it with the goal of generating their effects on the process. In this work, we instantiate a recommender system as a function $\Psi \colon \mathcal{E}^* \to (\mathcal{A} \times \mathcal{R})$, where $\mathcal{E}$, $\mathcal{A}$, and $\mathcal{R}$ denote the sets of events, activities, and resources, respectively. For each running trace $\sigma \in \mathcal{E}^*$ the function Ψ returns a pair $(a, r) \in \mathcal{A} \times \mathcal{R}$, representing the recommended activity a and the resource r to execute it. The function Ψ is applied to obtain a set of recommendations $\{(a_1, r_1), \ldots, (a_k, r_k)\}$,

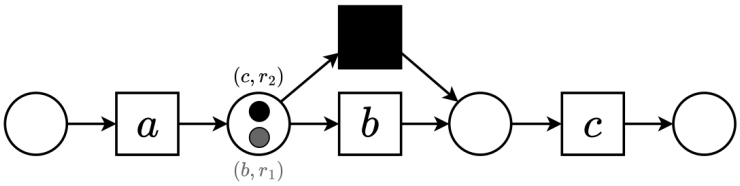


Figure 8.2: Petri net showing the markings obtained after the replay of the set of ongoing traces $\{\langle a \rangle, \langle a \rangle\}$. Each marking is highlighted with a different color and annotated with the corresponding recommended activity and resource.

where each pair $\Psi(\sigma_i) = (a_i, r_i)$ is associated with an ongoing trace σ_i and refers to the next activity and resource to be executed immediately after the current state (m_i, s_i). When injecting a recommendation (a_i, r_i), the general idea is to fire a transition with label a_i and let this activity be performed by r_i. However, the reached marking m_i is not guaranteed to enable a transition with label a_i. To address this we used token replay techniques (cf. Section 1.3.4) for firing silent transitions to reach a marking that enables a transition with label a_i.

Example: Consider the set of ongoing traces $\{\langle a \rangle, \langle a \rangle\}$, and the Petri net in Figure 8.2 with the two reached markings depicted with tokens with different colors. Note that while the gray token can directly fire the recommended transition b, the black token does not enable a transition with label c, i.e. the recommended activity. However, such transition can be reached after firing the silent transition skipping b.

It is important to note that these recommendations may not always be directly executable in the process model. For instance, the recommended activity a_i might not be enabled in the current marking, even after firing silent transitions, or the suggested resource r_i might not be allowed to perform the activity according to model constraints or resource allocation rules. When this happens, the simulation continues without recommendation. Moreover, during the simulation, additional traces may also start in parallel, reflecting the actual arrival of new cases in the system. In this way, the simulator captures both the evolution of the ongoing cases under recommendation-driven decisions and the interaction with other concurrent process instances.

To mitigate the inherent stochasticity of business processes, the simulation is executed multiple times, producing different possible event logs. This allows the framework to capture the variability in process behavior and the potential effects of recommendations under different variable scenarios. The resulting simulated

traces, which incorporate the injected activity–resource recommendations, are then used to compute KPIs such as remaining times. By analyzing these metrics across multiple simulation runs, it becomes possible to evaluate not only the effectiveness of the recommendations, but also to quantify their impact, providing a detailed and quantitative evaluation of how recommendations influence process performance.

8.4 Experiences

This section presents experiences where the proposed framework has been employed in evaluating Process Aware Recommender systems. Specifically, the framework has been used to evaluate two techniques in which the goal is to reduce the remaining time of process traces in online settings: Section 8.4.1 reports on a proposed technique in which resources are allocated to achieve a more balanced distribution of workload among resources, which we published in [146]; in Section 8.4.2, our framework is applied to a technique that leverages counterfactual generation techniques to formulate recommendations, published in [102].

The goal of both techniques is to provide optimal activity and resource recommendations in specific running instances at a certain time. To evaluate the technique, the event-log $\mathcal{L}$ is divided into training and test sets as follows: given a timestamp t_{split}, the set of completed traces before t_{split} is used to train the predictive models and to discover the simulation model, while the set of running traces $\mathcal{L}^{run}$ at time t_{split} has been used for evaluation as discussed in Section 8.3. The timestamp t_{split} is selected as the timestamp such that the event log filtered before that timestamp contains at least the 65% of completed traces and maximizes the number of running traces after it. Figure 8.3 depicts this choice.

Then, the set of ongoing traces $\mathcal{L}^{run}$ at timestamp t_{split} is mapped into the simulation model discovered using the event log $\mathcal{L}^{train}$ containing the completed traces before t_{split}, resulting in a simulation model that reflects the state of the process at timestamp t_{split}. The output of each of the evaluated techniques is a set of activity-resource pairs $\{(a_1, r_1), \ldots, (a_k, r_k)\}$, where k is the size of $\mathcal{L}^{run}$, i.e. the number of ongoing process instances. These are then injected into the simulation model to reproduce their effects. To mitigate the stochasticity of the simulation, we conducted the simulation 10 times, thus obtaining 10 different event logs where the remaining times have been computed.

In the following sections, the two techniques are presented and the results obtained via simulations using our framework are reported, comparing them with benchmark techniques. Note that they are only briefly described here, as a

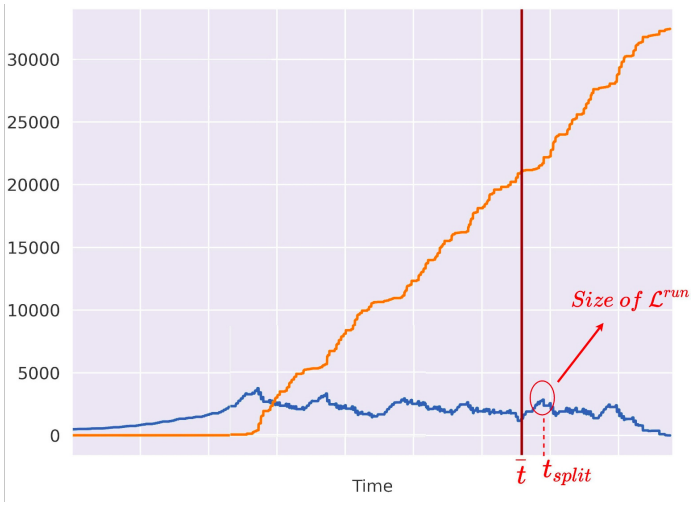


Figure 8.3: Graphical representation of number of completed/running traces. The orange and blue lines represent the number of completed and running traces, respectively. The vertical red bar is the timestamp t when the 65% of traces have completed. Source: [145].

detailed presentation of these techniques is out of the scope of this thesis.

8.4.1 Experience-based Prescriptive Process Analytics

Typical prescriptive interventions in process management involve recommending which activity should be executed next and which resource should perform it, with the objective of optimizing key performance indicators (KPIs) [144, 173, 152]. Building on this foundation, the following section presents a technique that not only aims to optimize immediate process performance but also explicitly considers the evolution of resource experience and workload distribution over time.

Technique

The technique presented in this section moves forward from the literature that just focuses on allocating the best-fitting resources to activities for selected running process executions. In many situations, there is a clear correlation between the performances of activities by resources and the experience of the resources themselves [90]. When recommending based on the historical data on resource performance, this yields to a vicious cycle in which the best-fitting resources are always assigned activities, and they thus become even more experienced and better performing. A harmful consequence of such recommendations may be that the less experienced resources are never given the chance to gain experience

and become better at performing recovery activities: they are always being kept aside. This uneven distribution of activity work is intrinsically unfair because some resources are largely idle, while others are heavily overloaded, increasing the queue of the activities that they have to perform, thus ultimately negatively affecting the overall process performances [3].

This work aims to introduce an approach that balances the necessity for process executions to be improved at their best, on the one hand, and the long-term goal of enlarge the repertoire of experienced resources, on the other. Rather than focusing exclusively on short-term optimal decisions, the proposed approach explicitly accounts for the evolution of resource experience and workload distribution over time.

The overall goal is to enhance the system efficiency by recommending suitable activities and resources to reduce the aggregate remaining time of the (running) traces. The approach starts by ranking the running traces based on their predicted remaining time, as computed from the event log through a dedicated ranking function. This ranking identifies the process instances that are most critical and should be prioritized for intervention.

Next, an oracle function, learned from the experience of the resources encoded in the event log, is used to generate an initial recommendation profile. For each running trace, this profile specifies a candidate activity–resource assignment together with an estimate of the resulting remaining time and the expected execution time of the resource, taking into account how resource performance evolves with experience. While this initial profile optimizes remaining time in the short term, it does not yet consider the overall workload distribution across resources.

To balance the allocation, a time–workload coefficient is introduced to capture short-term performance, workload balance, and resource utilization. This coefficient is then used to iteratively generate and evaluate a sequence of alternative recommendation profiles, progressively improving the trade-off between minimizing the remaining time and achieving a more balanced and sustainable allocation of work. In doing so, the approach seeks a global optimization across all ongoing traces, as proposed in [144], rather than a local, trace-by-trace optimization. Finally, the system provides recommendations derived from the best-performing profiles, offering activity–resource pairs that optimize both immediate process outcomes and long-term organizational efficiency.

Simulation Results

The approach has been evaluated with two different processes for which we could obtain three suitable event logs with rich information on the resource

Table 8.1: Total improvement (in *hours*) and relative total improvement averaged over ten simulations comparing the Workload Distribution technique to the Benchmark. The standard deviation is reported in parenthesis and the percentage of feasible recommendations for the simulation is shown in the last column.

Dataset	Technique	Tot. Impr.	Rel. Tot. Impr.	Feasible Recs
BAC	Workload Dist.	5028.96 h (1150.18 h)	0.07 (0.02)	95.63%
	Benchmark	2156.77 h (2236.04 h)	0.03 (0.04)	96.03%
BPIC17AO-before	Workload Dist.	123050.14 h (5841.34 h)	0.36 (0.02)	95.69%
	Benchmark	111754.49 h (4117.23 h)	0.33 (0.01)	81.43%
BPIC17AO-after	Workload Dist.	20568.03 h (5058.91 h)	0.06 (0.01)	90.71%
	Benchmark	82575.09 h (6317.67 h)	0.24 (0.02)	72.55%

perspective: Bank Account Closure (BAC), BPIC17AO-before and BPIC17AO-after (cf. Section 1.7).

Following the framework introduced in Section 8.3, we evaluated the technique by comparing the results obtained through the recommendations given by the proposed technique with a benchmark where the recommendations assign the most suitable activity to the most proficient resource, without addressing balanced resource utilization.

The results in Table 8.1 show that the workload approach leads to better performance, decreasing the remaining time to complete a trace with respect to what happened in the reality. It outperforms the benchmark in 2 out of 3 cases and results show that always assigning tasks to the best performing resource can result in long queues and increased remaining times.

The table also reports on the suitable recommendations, i.e. the recommendations that are possible to execute accordingly to the simulation model (cf. Section 8.3). It shows a higher percentage of feasible recommendations for the proposed technique compared to those obtained from the benchmark. In fact, recommending the most efficient activity performed by the most efficient resource can lead to recommendations that might be optimal, but not feasible in practice: e.g. a resource may not execute some task, or an activity may not be performed in a certain process state.

8.4.2 Counterfactuals in Prescriptive Process Analytics

Existing prescriptive techniques typically leverage a divide-and-conquer approach: they first select the activity based on its potential to enhance the process efficiency, and then assign it to a suitable resource [100]. While intuitive, this strategy can result in suboptimal outcomes. Conversely, optimizing the activity-resource pair by evaluating all possible combinations is often intractable in practice, especially in organizations with hundreds of resources.

Technique

This section reports on a technique based on counterfactuals [185, 77] to recommend the activity-resource pairs for each running process instance. We use the term *counterfactual* in the context of explainable AI as introduced by [193]: a counterfactual identifies the minimal changes to an input instance that would alter the prediction of the oracle function toward a desired outcome. This extensive employment of counterfactual-based approaches is linked to their nature of defining *what-if* scenarios, such as "*You were denied a loan because your loan duration was 20 years. If your loan duration had been 30 years, you would have been offered a loan*" [193]. This capability makes counterfactual-generation frameworks well-suited for recommending activity-resource pairs. The advantage of prescriptive process analytics is evident: frameworks for counterfactual generation are readily available and can be directly used to generate activity-resource recommendations along with explanations. Also, these frameworks can quickly generate multiple counterfactuals, which translates in our setting to producing several alternative recommendations for the same process instance. Notably, while counterfactuals have been used to explain predictions and recommendations, they have never been used to generate recommendations themselves. This flexibility is beneficial because a rigid resource-to-activity imposition is against the principles of resource-aware recommender systems [52], which emphasize the need to support decision-makers with multiple viable options rather than prescribing fixed assignments.

Process Mining literature proposes a few counterfactual frameworks [33, 27, 83, 84, 87, 140] to explain the predictions, namely the predictive-monitoring outcome. The work presented in this section is the first to introduce a recommendation framework that uses counterfactuals to explicitly suggest the next activities to be executed for the ongoing process instances along with the resources that should carry them out.

Starting from a running trace, the proposed framework exploits a predictive oracle to estimate the outcome associated with alternative next activity–resource

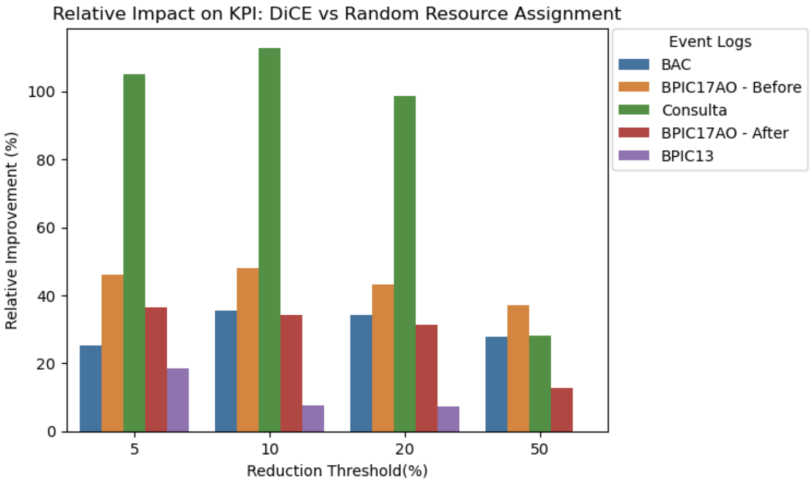
pairs that are feasible according to the transition system and historical resource eligibility. Counterfactual generation is then used to efficiently explore this space by producing candidate pairs that improve the predicted outcome with respect to the currently expected one, without computing all combinations. Among the valid counterfactuals, the framework selects those yielding the largest improvement. The framework is equipped with a parameter, hereafter referred to as **reduction threshold**, that indicates the minimum reduction of process-instance execution times that the recommendation can provide when the activity-resource pair is perturbed. For example, if we set this parameter to 5%, only recommendations that lead to a predicted reduction in total execution time of *at least* 5% are allowed. It is worth noting that the larger reduction thresholds would generally produce fewer counterfactuals, possibly even none. While the proposed approach supports automated decision making, in scenarios involving a human decision-maker, the system could present the top-k ranked counterfactuals, allowing the user to choose the most appropriate one.

Simulation Results

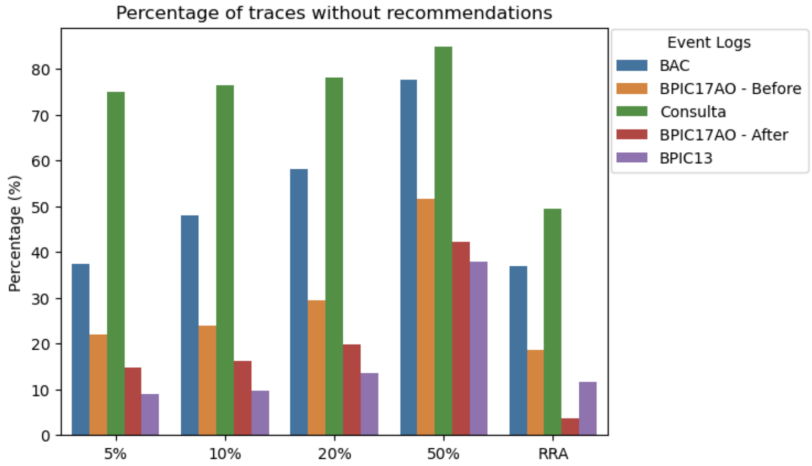
The technique is evaluated leveraging our simulation framework (cf Section 8.3), where the assessment is conducted using four real-life processes, with five event logs: Bank Account Closure (BAC), Consulta, BPIC13, BPIC17AO divided into BPIC17AO-before and BPIC17AO-after (cf. Section 1.7 for further details of the case studies). The counterfactual framework is implemented using DiCE (Diverse Counterfactuals Explanations) [140] with random sampling method to generate counterfactuals. The reduction threshold parameter of the framework directly maps to a corresponding parameter of DiCE. In our experiment, we evaluate thresholds of 5%, 10%, 20%, and 50%. By varying these thresholds, we assess the framework's ability to generate diverse recommendations under different levels of execution time reduction.

To evaluate the effectiveness of the proposed method, we compare it against a baseline, referred hereafter to as *Random Resource Assignment (RRA)*. In this method, given a running trace σ', we first extract the list of possible next activities using the transition system. For each activity, we use the Total Time Oracle function to estimate the execution duration and select the activity that minimizes the predicted total time as the next activity. Once the next activity is determined, a resource is randomly chosen from the list of feasible resources to perform that activity.

Figure 8.4a presents a comparative visualization of the relative improvement achieved by our framework over the RRA method. Overall, the counterfactual framework outperforms the RRA approach, with particularly substantial gains



(a) Relative improvement related to the average total execution time when applying DiCE compared to the Random Resource Assignment method. On the x-axis, there are the reduction thresholds, while on the y-axis, the relative improvement is computed on the whole $\mathcal{L}^{run}$.



(b) Percentage of traces without recommendations when applying DiCE compared to the Random Resource Assignment method. On the x-axis, there are the percentages of reduction thresholds (5%, 10%, 20%, and 50%) when applying DiCE and the RRA column indicating the Random Resource Assignment method.

Figure 8.4: Counterfactuals for Prescriptive Process Analytics results.

observed in the Consulta process, where improvements exceed 100% for the three lowest reduction thresholds. A noticeable trend is observed where the relative improvement for all methods declines at the 50% reduction threshold. Moreover, when looking at the BPIC13 process, we observe that the proposed method is not able to provide any relative improvement at the 50% total execution

time.

Another aspect we observe is the percentage of traces without recommendations illustrated in Figure 8.4b. We can notice that if the percentage of the reduction threshold increases, there will be a decrease in the percentage of traces that receive recommendations. This highlights that *if the threshold is pushed too high, several process instances remain without potential recommendations*, which implies that the overall improvement for all running instances is more limited. Moreover, the RRA approach can consistently generate a higher number of recommendations. This is because RRA does not rely on complex optimization techniques. Instead, it selects the next activity based on the smallest predicted total time and assigns a resource randomly based on the chosen best activity. This random allocation often results in suboptimal resource utilization.

In conclusion, the counterfactual framework can effectively recommend activity-resource pairs that significantly reduce the total execution time, even under strict constraints, resulting in substantial improvements across all five event logs and reduction thresholds. Meanwhile, the random resource assignment prevents the RRA method from achieving equally good results, especially when resource choice critically impacts outcomes.

8.5 Conclusion

Prescriptive process analytics provides techniques aiming at inferring future outcomes of process instances, such as remaining time, resource availability, activity occurrences, etc., and deliver actionable recommendation to prevent undesirable outcomes. A major limitation in this field is the absence of a ground truth for evaluating the actual impact of such recommendations. Several techniques in literature propose the usage of synthetic datasets for the assessment of process mining techniques when the ground truth is missing. However, none of them propose a dedicated framework for reproducing process recommendations in online settings.

This chapter presented a framework where simulation models are used to map a current process state and simulate process recommendations, i.e. couple of activity-resource pairs, at each running process instance. The general idea is to map an event log of running process instance executions to a given simulation model, thus obtaining a simulation state that reflects the situation of process when that set of instances of process executions are running. Then a set of recommendations provided in the form of activity-resource pairs can be injected into the "frozen" simulation model, and the start simulations. The simulation results can then be used to compare the KPI outcomes with different settings,

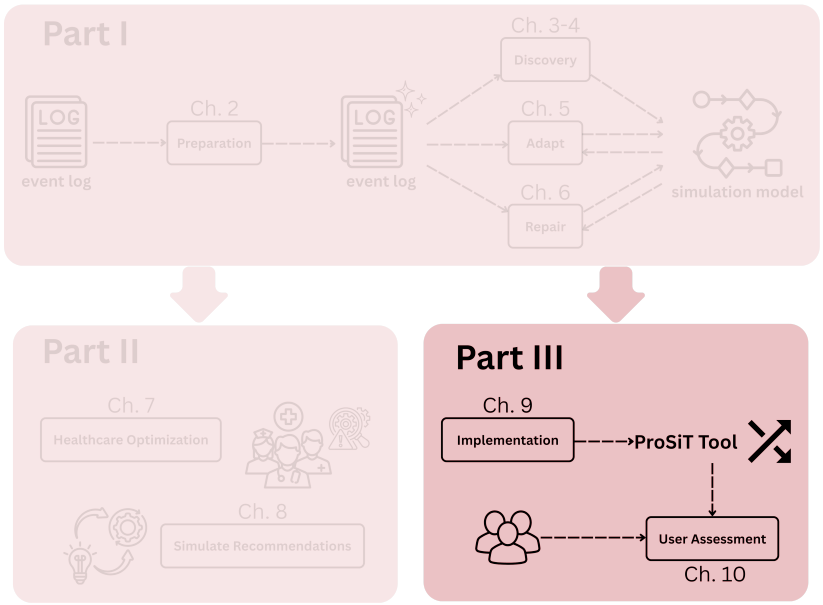
thus assessing where the KPIs are effective.

The proposed framework has been applied to two distinct prescriptive process mining techniques. The first focuses on balancing resource workloads while reducing the remaining execution time of running process instances, whereas the second leverages counterfactual explanations to generate recommendations aimed at minimizing remaining time. Each technique has been evaluated against appropriate benchmarks tailored to its specific scopes. The results demonstrate that the proposed framework effectively supports the simulation and evaluation of process recommendations generated by such techniques, and enables a quantitative assessment of their impact with respect to both the observed reality and alternative benchmarking approaches.

We acknowledge that this chapter does not provide a direct evaluation of the effectiveness of the proposed framework in terms of the accuracy of the generated event data. Nevertheless, the reliability of the framework can reasonably be assumed when the underlying simulation model is. As future work, we plan to explicitly assess the accuracy of the simulator at the current process state, i.e., after the mapping of the set of running traces to the simulation model. In particular, we aim to evaluate the reliability of simulations starting from the observed process state through different simulation accuracy metrics (cf. Section 1.6.3), similarly to what has been proposed in [17].

Part III

Software and User Interactions



The last part of this thesis focuses on the **development and evaluation of a piece of software** for process simulation. In particular, it addresses how the methodological and theoretical contributions introduced in the previous parts can be operationalized into software artifacts.

Chapter 9 It introduces **ProSiT** (*Process Simulation Tool*), a software environment for data-driven process simulation. It consists of **prosit-library**, a modular Python library for programmatic process discovery and simulation, and **prosit-system**, an interactive web-based application. The tool implements the techniques introduced in previous parts of this thesis. In particular, it supports a complete end-to-end workflow, including data ingestion, automated parameter discovery, interactive *what-if* scenario configuration, and quantitative accuracy assessment. This contribution has been presented in [191].

Chapter 10 This chapter presents a structured **user assessment** of ProSiT involving 12 participants with diverse expertise in process management and simulation. The study evaluates the tool usability through tasks performances and the System Usability Scale (SUS). Results indicate a high level of perceived usability confirming that users can effectively interact with tree-based parameter models to perform *what-if* analyses.

Overall, this part consolidates the contributions of the thesis by implementing them into validated software tools, enabling their adoption in real-world process mining and decision-support scenarios.

9 Software

This chapter introduces **ProSiT**, a *Process Simulation Tool* that implements techniques developed in this thesis. It is composed of two complementary artifacts: (i) **Prosit Library**, a modular Python library that implements the core discovery and simulation functionalities, and (ii) **Prosit System**, an interactive web-based tool that supports the configuration, execution, and analysis of business process simulations through a graphical user interface. ProSiT System builds on ProSiT Library to provide an accessible and explainable simulation environment for practitioners, while ProSiT Library can be used independently by researchers and advanced users to integrate ProSiT capabilities into custom analysis pipelines.

9.1 Introduction

Business Process Simulation (BPS) has become a crucial instrument for organizations seeking to understand and improve their operational efficiency. This led to the introduction of BPS tools for the analysis and improvement of business processes. In recent years, both academia and industry have introduced a variety of tools implementing novel techniques from the Business Process Simulation literature. Academic contributions include, among others, BIMP[1], CPN Tools [194], and Prosimos [117], while commercial solutions are offered by vendors such as Celonis[2], SAP Signavio[3], and Apromore[4].

However, the effectiveness of BPS tools hinges on their ability to accurately

[1] https://bimp.cs.ut.ee/

[2] https://www.celonis.com/

[3] https://www.signavio.com/

[4] https://apromore.com/

capture complex process dynamics, while remaining accessible and interpretable to process analysts. Early approaches and many commercial tools still rely on manual model construction, which is time-consuming and error-prone [2]. More recently, several research tools have emerged to automate simulation model discovery from event logs [50, 61, 117, 135]. Despite their potential to improve realism and reduce manual effort, many of these tools still suffer from limited accessibility, configurability, and interpretability, constraining their adoption in practical simulation scenarios.

This chapter introduces **ProSiT** (*Process Simulation Tool*), a software environment that integrates explainable, white-box machine learning techniques for simulation parameter discovery. The novelty of ProSiT lies in the operationalization of the research body developed in this thesis, aiming to increase the accuracy, explainability, and customization of process simulations for *what-if* analyses. ProSiT implements the techniques presented in Chapter 3 and Chapter 4, as well as the incremental discovery technique introduced in Chapter 5, enabling the automatic construction of executable simulation models directly from event logs and process models. The tool supports the full simulation workflow, including the ingestion of the event log and the process model, the automated parameter discovery, the interactive configuration of alternative service scenarios, the visualization of accuracy metrics and the generation and statistical analysis of simulated event logs.

ProSiT is designed as a two-layer framework. At its core, **Prosit Library**, a modular Python library, implements the core discovery and simulation functionalities proposed throughout this thesis. This library enables advanced users and researchers to integrate ProSiT capabilities into custom analysis pipelines, experimental evaluations, or external decision-support systems. On top of this foundation, **Prosit System** exposes these capabilities through an interactive web-based interface, enabling users to configure simulation scenarios, execute experiments, and analyze results without requiring programming expertise.

By combining high simulation accuracy with explainability, ProSiT bridges the gap between data-driven automation and human-centric process analysis. It empowers process analysts to inspect and validate discovered models, experiment with alternative scenarios, and assess service-level impacts through an accessible and integrated simulation environment.

9.2 ProSiT Library

At the core of ProSiT lies a modular Python library, here referred as Prosit Library, that implements the discovery and simulation techniques proposed

throughout this thesis. The library is designed to operate both as the backend of the web-based application (cf. Section 9.3) and as a standalone toolkit for researchers and advanced users who require programmatic access to data-driven simulation models. This design enables the integration of ProSiT capabilities into custom analysis pipelines, experimental workflows, and external decision-support systems. The complete code source and documentation are available in `https://github.com/franvinci/prosit`.

The library builds upon `pm4py` [26], which implements standard process mining algorithms such as event log handling, process discovery, and the computation of alignments between event logs and process models. Discovery-related components are grouped under the `prosit.discovery` package, which contains modules devoted to transition weights discovery, temporal and calendar analysis, resource modeling, and data-aware decision rule extraction (cf. Chapter 3 and Chapter 4). The `prosit.discovery.online_discovery` subpackage contains the online discovery modules (cf. Chapter 5), enabling parameters to be continuously updated as new events arrive. Simulation-specific functionality is encapsulated in the `simulator` module, which defines both the class for simulation parameters and the execution engine responsible for generating simulated event logs. A set of utility modules supports these components by providing shared functionality for distribution fitting, rule evaluation, serialization, and common operations.

A separate branch of the library, available at `https://github.com/franvinci/prosit/tree/recommendations-simulator`, extends the core simulation engine with functionalities for simulating recommendations for running process instances (cf. Chapter 8). In this setting, the simulation can be initialized from a set of ongoing traces where recommendations are included.

This library supports reproducible experimentation, facilitates the integration of novel discovery techniques, enabling both offline and online simulation scenarios. The `prosit` library constitutes a key artifact of this thesis, ensuring that the proposed methods can be readily reproduced, adopted, and extended within broader process mining systems.

9.3 ProSiT System

This section describes the architecture, core functionalities, and API design of the ProSiT web application tool, namely ProSiT System. The modular architecture of the tool (Section 9.3.1) enables a clear integration of the discovery and simulation components developed in this thesis. ProSiT System exposes a REST API (Section 9.3.2) that supports communication with the web-based

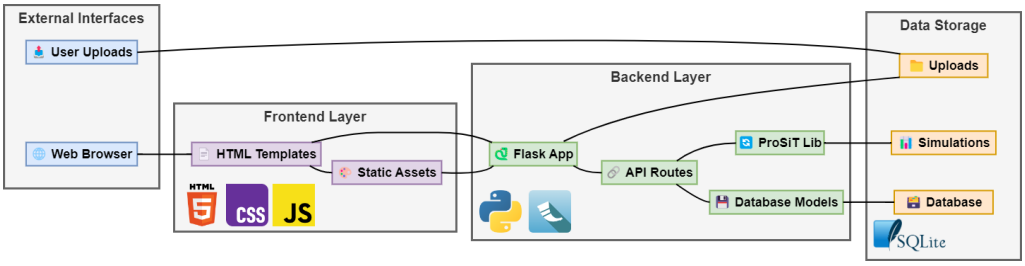


Figure 9.1: High-level architecture of ProSiT System, showing the separation between the frontend layer (HTML/CSS/JS), backend layer (Python/Flask), and data storage (SQLite with SQLAlchemy ORM). The ProSiT library encapsulates the core discovery and simulation logic.

frontend as well as integration with external systems. The end-to-end workflow (Section 9.3.3) guiding users from event log and model ingestion to the configuration and evaluation of alternative simulation scenarios is presented.

9.3.1 System Architecture

ProSiT System follows a client-server architecture implemented as a web application. The overall architecture is depicted in Figure 9.1. The backend is built using Python, integrating ProSiT Library, with the Flask framework, handling all computational operations, including process discovery, parameter learning, and simulation execution.[5] The frontend is implemented with standard web technologies (HTML, CSS, JavaScript) providing a responsive user interface. Communication between frontend and backend occurs through a RESTful API. Data persistence is managed via SQLite database.

For easy deployment and reproducibility, ProSiT System is distributed as a Docker container, ensuring consistent execution across different environments.[6]

9.3.2 API Interface

The backend functionalities of ProSiT System are programmatically accessible through a comprehensive RESTful API, enabling both interactive use via the web interface and integration with external systems. The API follows REST principles and returns JSON responses for all endpoints. Table 9.1 summarizes the core endpoints that support the complete simulation workflow.

The API design follows a sequential workflow: users first upload process data, then discover parameters, optionally modify them, and finally run simula-

[5] https://flask.palletsprojects.com/

[6] https://www.docker.com/

Table 9.1: Core REST API endpoints of ProSiT System, supporting the complete simulation lifecycle from data ingestion to results generation.

Endpoint	Method	Description
/api/upload	POST	Upload an event log or PNML file.
/api/discover	POST	Initiates the automated discovery of simulation parameters from an uploaded event log.
/api/parameters	GET	Retrieve discovered parameters.
/api/parameters	PUT	Updates specific simulation parameters to configure custom scenarios for *what-if* analysis.
/api/simulate	POST	Run a simulation with configured parameters and returns performance metrics and a synthetic event log.

tions. This structured approach ensures reliable state management and supports reproducible *what-if* analyses.

9.3.3 Core Functionalities

The ProSiT System tool supports an end-to-end workflow for data-driven process simulation, guiding users through five key phases:

1. **Data Ingestion:** Users can upload event logs in XES format, and optionally provide process models in PNML format (see Figure 9.2).

2. **Parameter Discovery:** This core functionality uses explainable machine learning models to automatically extract simulation parameters from event logs (cf. Chapter 4). Users can tailor the discovery process by configuring the Inductive Miner algorithm [105] with a specific noise threshold, opting for probabilistic decision trees with controllable depth, and activating incremental discovery with customizable grace periods (cf. Chapter 5). The discovery covers *transition weights* for control-flow decisions; *resource* parameters including assignment probabilities, multitasking capabilities and availability calendars; *arrival time* distributions; *execution time* distributions per activity; *waiting time* distributions per resource; and distributions for *trace attributes* (see Figure 9.3).

3. **Interactive Scenario Configuration:** An intuitive graphical interface allows analysts to modify the discovered parameters, facilitating the creation and comparison of alternative scenarios for comprehensive *what-if* analysis (see Figure 9.4).

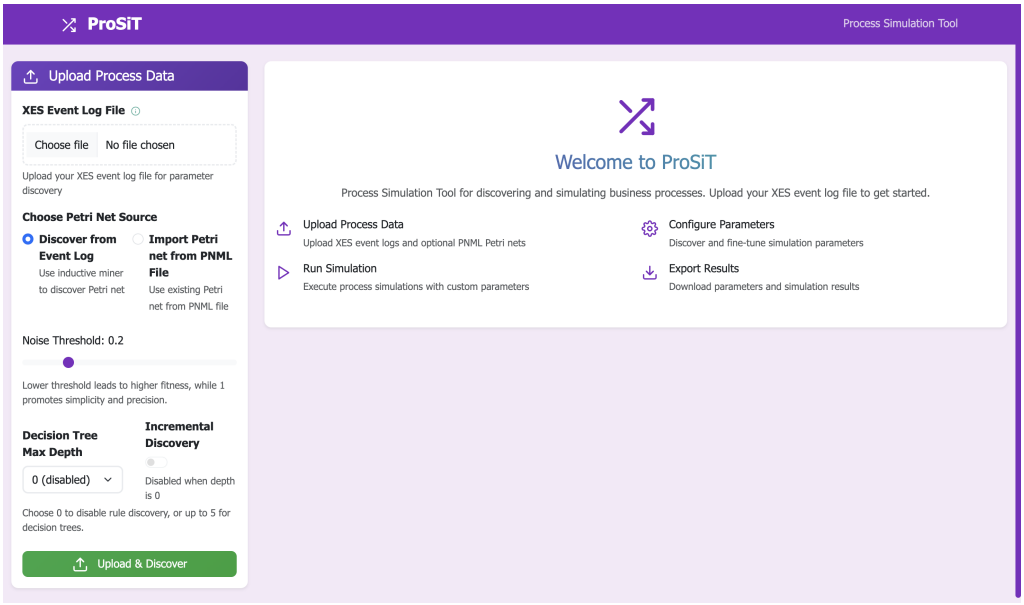


Figure 9.2: ProSiT homepage showing the data upload panel.

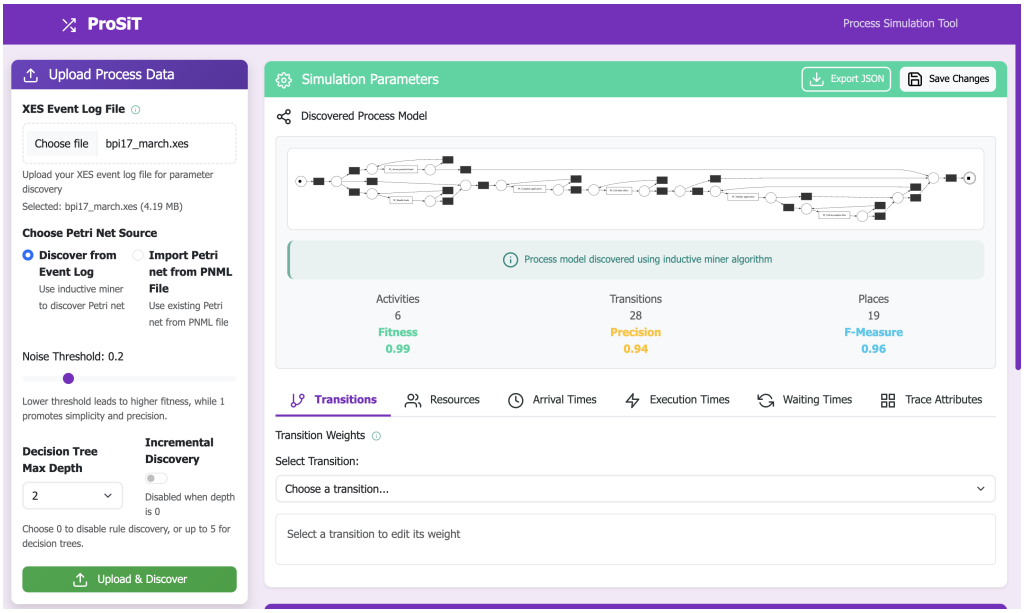


Figure 9.3: ProSiT interface after parameter discovery.

4. **Accuracy Assessment:** The tool evaluates the quality of the simulation by comparing the generated logs with the original event data. Assessment includes control-flow similarity measures (n-gram distance), temporal accuracy through distributional metrics, realism of resource behavior, and

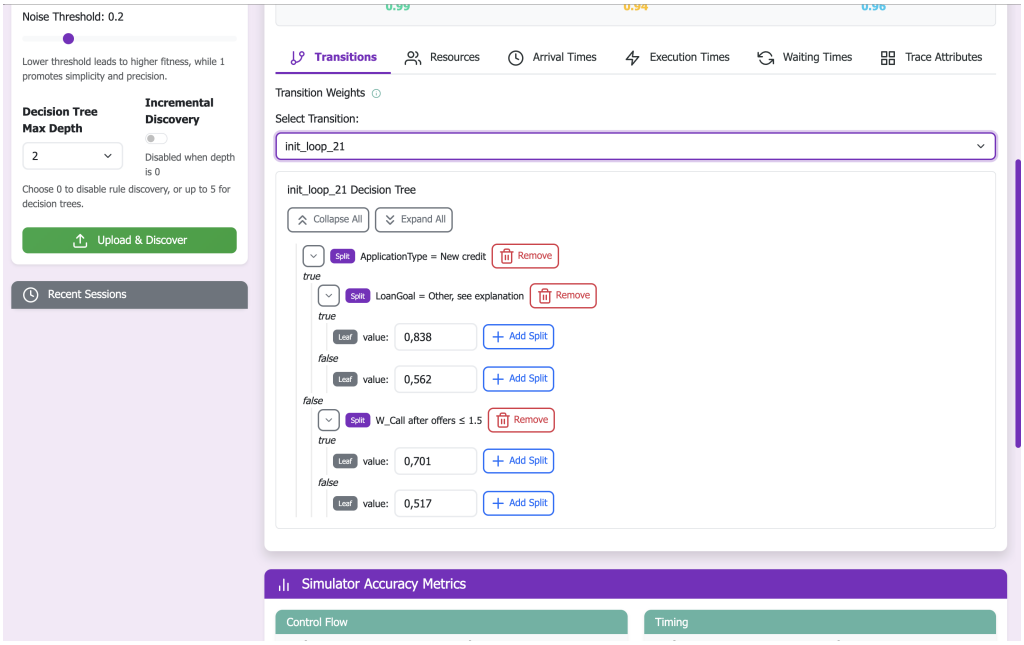


Figure 9.4: Configuration of transition weights parameters in ProSiT.

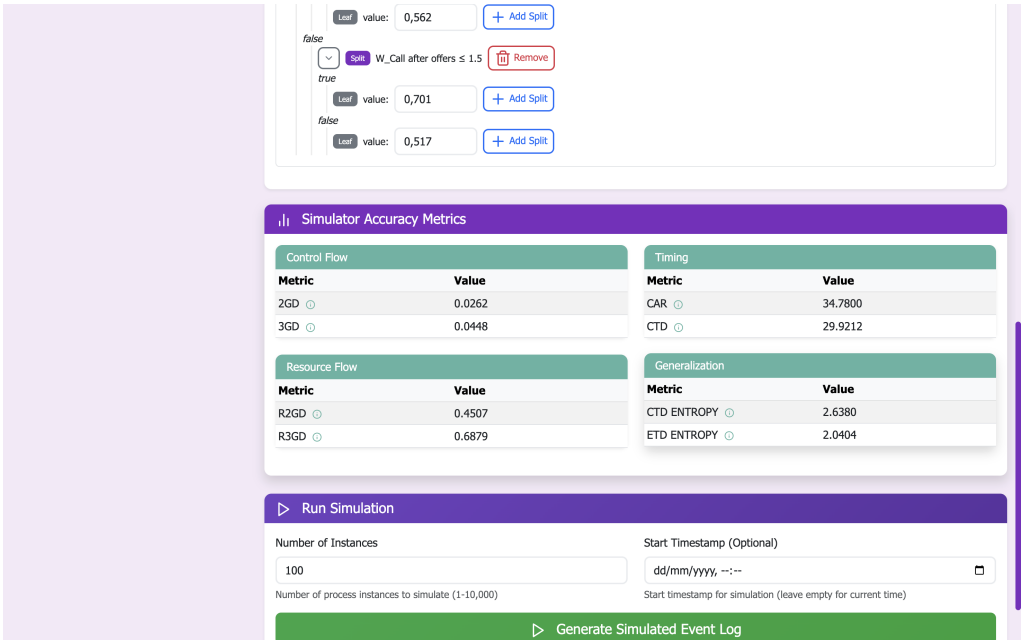


Figure 9.5: Accuracy metrics panel in ProSiT.

generalization capability through entropy analysis (see Figure 9.5).

5. **Simulation Execution:** Users initiate simulations by defining the desired

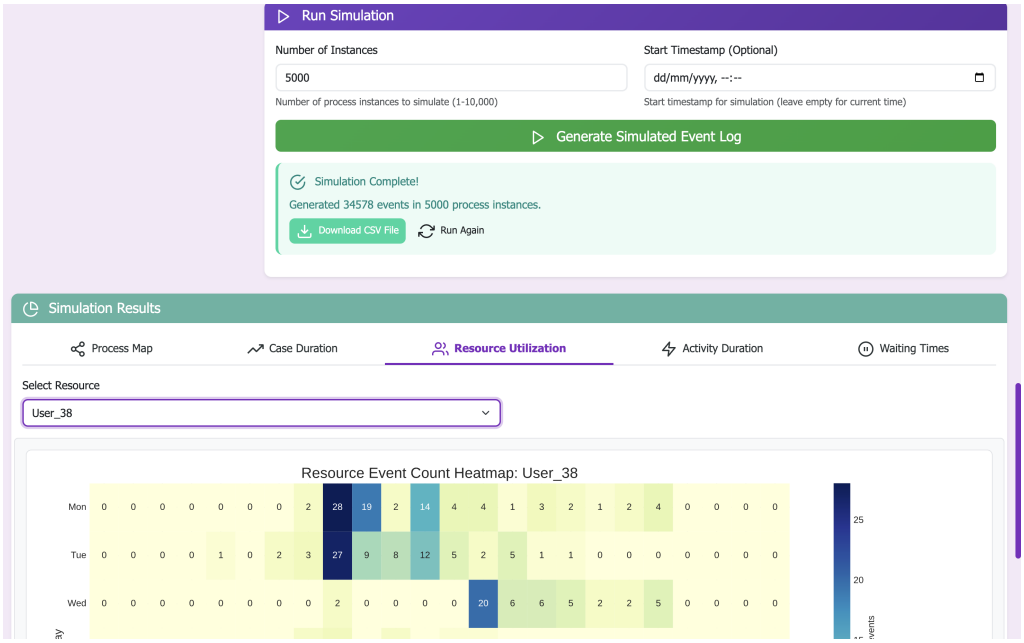


Figure 9.6: ProSiT simulation interface with resource utilization results.

number of traces and a starting timestamp. The execution outputs a downloadable simulated event log, accompanied by visual analytics including process maps with performance data, cycle time distributions, resource utilization heatmaps, and detailed activity execution and waiting time analyses (see Figure 9.6).

This structured workflow enables users to systematically advance from event data to validated simulation insights, with full transparency over simulation parameters and quantitative accuracy assessment.

9.3.4 Resources and Demonstration

ProSiT System is distributed as a Docker container for easy deployment, ensuring consistent execution across different platforms. The Docker image includes all dependencies, including ProSiT Library. Developers can extend the tool by modifying the source code or integrating additional discovery algorithms through the modular API.

A demonstration video showcasing ProSiT System capabilities is online available on `https://youtu.be/ZN7ajI9-2NI`. The video walks through a complete case study using a publicly available event log from the BPI Challenge 2017, demonstrating the entire workflow from log upload through parameter discovery, scenario configuration, simulation execution, and results analysis.

The complete source code, installation instructions, and documentation are available in https://github.com/franvinci/prosit-tool. The repository includes several publicly available event logs that users can directly employ for testing and evaluation purposes.

9.4 Tool Maturity and Conclusion

This chapter has introduced ProSiT, an interactive and explainable tool for data-driven business process simulation. By integrating state-of-the-art machine learning in a white-box form and providing a user-friendly graphical interface, ProSiT addresses a critical gap in existing solutions. Its focus on transparency and configurability allows process analysts and service architects not only to execute simulations, but also to explore, and configure the underlying parameters.

Built on probabilistic decision trees for explainable parameter discovery, ProSiT stands as a mature, fully functional research artifact suitable for both academic and industrial contexts. It demonstrates how white-box models can deliver predictive accuracy while remaining interpretable, thus supporting meaningful *what-if* analyses for service optimization.

The maturity and usability of ProSiT and its functionalities were assessed through a dedicated user study, which is described in detail in the following chapter (cf. Chapter 10). The study evaluates ProSiT System from the perspective of end users, focusing on usability and practical usefulness in realistic analysis scenarios. Building on these findings, future work will focus on refining the user experience and extending the tool in response to user suggestions.

10 User Assessment

This chapter presents a **user assessment** of the ProSiT System introduced in the previous Chapter 9. The study aims to assess the usability of the tool and, consequently, the practical accessibility of the tree-based parameter models introduced in Chapter 4 for process mining users.

We conducted a user study involving 12 participants with varying levels of experience in process management and simulation tools. During the study, participants were asked to perform four predefined tasks reflecting realistic analysis scenarios and to complete a short post-session usability questionnaire.

Final results reveal that the ProSiT System is generally easy to use and well received by users. Participants were able to complete the tasks and the usability scores indicate a high level of perceived usability.

10.1 Introduction

In Chapter 4, we introduced tree-based models to represent parameters in simulation models. This representation allows for configurable and transparent modeling of the process perspectives. We showed that these models are designed to preserve explainability while maintaining a high level of accuracy, supporting advanced *what-if* analyses.

These modeling concepts have been concretely implemented in the **ProSiT** software (cf. Chapter 9), where a GUI integrates such decision-tree models into an interactive process simulation environment. ProSiT allows users not only to explore discovered simulation parameters, but also to actively modify the decision trees as shown in Section 4.3.

However, while tree–based parameter models offer clear theoretical advantages in terms of transparency and configurability, their practical accessibility to

process mining users must be assessed. To address this question, this chapter presents a **user evaluation** of the ProSiT System.

The study involved 12 participants with varying levels of experience in process management and simulation tools, ranging from junior analysts to senior practitioners. Participants were asked to complete a set of four tasks reflecting realistic analysis scenarios. The usability of the tool is assessed through performance metrics, namely the number of errors and the time required to complete a set of predefined tasks, as well as subjective user feedback collected using the **System Usability Scale** (SUS) questionnaire [32]. This methodology allows us to evaluate both the effectiveness of user interaction with the system and perceived usability.

The remainder of this chapter is organized as follows. Section 10.2 reviews related work on user-centered evaluations in process mining. Section 10.3 describes the study design, participants, and experimental setup. The results of the evaluation are presented in Section 10.4.

10.2 Related Work

The evaluation of the ProSiT System is grounded in a growing body of research that investigates usability, explainability, and user interaction in process mining tools. While traditional process mining research has predominantly focused on technical advances, recent work has increasingly emphasized the human aspects of analysis, including trust, interpretability, and user experience.

Several recent studies have examined how users perceive, trust, and struggle with process mining tools. Tentina et al. [179] investigated user trust in process mining systems interviewing process mining users from various industries. A complementary study analyzes why users often struggle to extract actionable insights, deriving factors that influence the use of process mining outputs in order to obtain insights [180].

Understanding the behavior and the needs of the individual analyst is central to improving tool design. Zimmermann et al. [205] identified 23 specific challenges perceived by analysts in practice, highlighting that while mitigation strategies exist, there is a significant need for tailored support at the individual level. Zerbato et al. [202] characterized common analysis strategies and identified the practical factors that influence how analysts apply these techniques in real-world scenarios. In a recent work [203], the authors offered a comprehensive view of the analytical process by combining user interaction data with think-aloud protocols to map the reasoning steps that lead analysts from raw data to valuable findings.

Beyond the analytical process, Ammann et al. [11] investigated how process mining users "act, think and feel", identifying four distinct categories of process mining users categories based on a holistic investigation of their behavior, cognition, and affective states.

Galanti et al. [69] proposed an explainable decision support system for predictive process analytics, including a structured evaluation of tool usability. Kubrak et al. [99] focused on the design and evaluation of a user interface for prescriptive process monitoring, highlighting the role of interface design in supporting effective decision-making. Rizzi et al. [158] evaluated explainable predictive process monitoring techniques from a user perspective, showing how explainability impacts user understanding and trust. Together, these works reinforce the importance of ensuring that advanced analytical systems remain accessible and interpretable to the end user.

Despite the growing attention devoted to user-centered evaluations in process mining, the existing literature has focused primarily on explainable predictive and prescriptive analytics to support decision-making during runtime. However, process simulation tools remain largely underexplored from a user evaluation perspective.

10.3 Methodology

The objective of the user evaluation presented in this chapter is to conduct a structured assessment of the usability of the **ProSiT** tool presented in Chapter 9.

The study aims to understand how effectively and efficiently users can navigate the interface, configure simulations, execute analyses, and interpret outcomes. Beyond observing task performance, the study also intends to capture the subjective perception of usability through the **System Usability Scale (SUS)** [32] and collect qualitative reflections that may reveal both strengths and weak points in the interaction experience.

10.3.1 Participants

We recruited participants by inviting people from our professional network. Participants are individuals who are working or familiar with the domain of process analysis and process management. Their level of experience varies from junior analysts to senior professionals who regularly perform such analyzes as part of their workflow.

Before the session, each participant completed a brief **background questionnaire** capturing contextual information: the number of years they have worked in process management and/or process mining, their experience with process

mining and management tools, and their familiarity with process simulation tools. The questionnaire is available in Section B.1 for consultation.

In particular, we asked them about self-evaluating their familiarity with process management tools and with simulation tools on a 5-point Likert Scale, where 1 means not familiar at all and 5 means expert-level familiarity. Gathering the user-experience is beneficial to help interpret the results, particularly when comparing the perceptions between novice and expert users (cf. Section 10.4).

10.3.2 Ethical Considerations and Consent

The user study complies with the pertinent regulations related to research ethics and professional deontology (the European Convention on Human Rights (1950), the EU Charter on Fundamental Rights (2000), the UNESCO Universal Declaration on Bioethics and Human Rights (2005)). Ethical approval was granted by the HIT Ethical Committee of the University of Padua (protocol no. 2025_282).[1]

All participants were required to read and provide informed consent that describes the purpose of the study, the nature of the data collected, and the extent of the recording before starting the experiment. The form clarifies that participation was voluntary, that recordings were used solely for research purposes, and that participants could withdraw at any point without providing justification.

The study collected only essential personal data. Identification data (name and email address) were used exclusively for participation management and stored separately from research data. All study data were pseudonymized using participant codes and anonymized after the completion of the analysis. No sensitive personal data were collected, and no automated decision-making or profiling was involved.

Data were collected using Google Forms and Zoom; these services may process data according to their own privacy policies, which are outside the researchers' control.[2] All data were stored securely and retained only as long as necessary, with identification data deleted by January 31, 2026.

10.3.3 Experimental Setup and Session Flow

Each session was held online using the Zoom video-conference platform.[3] Participants installed ProSiT on their own machine before the beginning of the

[1] http://hit.unipd.it/comitato_etico_HIT

[2] Zoom privacy policies: https://explore.zoom.us/en/privacy/. Google privacy policies: https://policies.google.com/privacy

[3] https://www.zoom.com/

session, following an installation guide shared in advance.[4]

The sessions were moderated by Francesco Vinci, the author of this thesis. Each session started with the moderator briefly introducing the interface of ProSiT System, describing the general layout and navigation flow. The moderator then initiated screen and audio recording, after asking the participant to close any personal windows or information visible on their screen.

The practical part of the evaluation guided participants through a sequence of four tasks using a publicly available event log, thus ensuring reproducibility and transparency. Specifically, we provided the BPIC17W event log filtered to contain only events performed in one month (March) to reduce computational times during interviews.[5] Note that this does not affect the user assessment methodology.

The tasks were designed to resemble real analytical scenarios and are defined at a high level as follows, while details are provided in the task sheet in Section B.2:

T1 **Upload and Configure Discovery Parameters:** Participants uploaded the provided event log into ProSiT System and configured the parameters required for the discovery of the simulation model. After the configuration the system discovered the process simulation model. This step was meant to assess whether users intuitively locate and understand the import and model configuration options.

T2 **Explore Parameters and Run As-Is Simulation:** After discovery, participants explore the discovered simulation parameters, identify specific parameter groups, and run a first simulation of the *as-is* model. They observe the simulation outcomes and attempt to interpret relevant indicators, such as identifying bottlenecks or performance issues in the current process configuration.

- **What-if Analysis:** Participants conduct two distinct *what-if* analyses:

  T3 Alter transition control-flow probabilities to simulate a change in sequence of activities behavior within the process.

  T4 Add a new resource to the process and modify its associated parameters, specifically allocation, availability and waiting time.

[4] https://github.com/franvinci/prosit-tool/blob/main/README.md

[5] https://github.com/franvinci/prosit-tool/blob/main/example_data/bpi17_march.xes

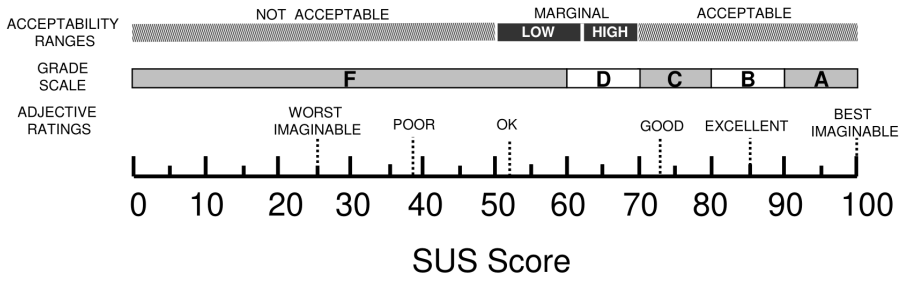


Figure 10.1: A comparison of the adjective ratings, acceptability scores, and school grading scales, in relation to the average SUS score. Source: [21].

> After each *what-if* analysis, they run a new simulation and observe how the modified process parameters affected the outcomes in comparison to the *as-is* model.

During task execution, the moderator did not intervene, in order to preserve the authenticity of user interaction with the system. The intervention was only allowed when a participant requested support to fulfill the task. Specifically, an "error" was recorded whenever the user failed to complete a step of the task or when the moderator intervention was required to ensure its correct execution.

Tasks **T3** and **T4** are of particular importance for this study, as they assess the ability of users to understand and modify decision-tree–based models used to represent simulation parameters (cf. Section 4.3). Supporting interactive, user-driven modification of such white-box models constitutes one of the most novel and distinctive contributions of the ProSiT simulator, as it enables advanced *what-if* analysis while preserving model interpretability and transparency (cf. Chapter 4).

After the completion of the tasks, the participants took part in a short verbal interview focusing on perceived usability, encountered difficulties, and overall interaction experience with the system. The feedback collected was systematically documented to guide the subsequent development of the ProSiT System.

In a post-session evaluation phase, users were asked to complete the System Usability Scale (SUS) questionnaire [32]. The SUS consists of ten statements in which participants indicate their level of agreement on a five-point scale ranging from *Strongly Disagree* to *Strongly Agree*. The statements address aspects such as complexity, consistency, trust in use, and perceived need for technical support. The standard SUS scoring procedure has been applied and the resulting score contributed to the overall usability assessment (see Section B.3). In particular, the SUS provides a global view of subjective usability assessments, with scores ranging from 0 to 100 (cf. Section B.3). To contextualize these numerical values,

Table 10.1: Average completion time, time variability, and error statistics for each task.

Task	Avg Time (sec)	Std Time (sec)	Total Errors	# Users with Errors
T1	28	14	1	1/12
T2	158	34	1	1/12
T3	176	69	4	3/12
T4	329	114	5	3/12

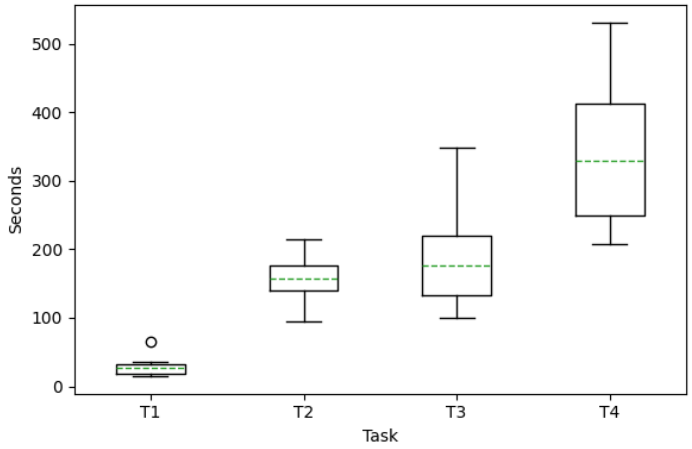


Figure 10.2: Distribution of task completion times across users, shown as boxplots for each task.

the analysis of the evaluation results is based on the interpretation proposed by Brooke [21], which maps SUS scores to grades, adjective ratings, and acceptability ranges (see Figure 10.1).

10.4 Evaluation Results

This section discusses the results of the usability study. Specifically, we focus on the accuracy to perform tasks and usability scores given by the users through the SUS questionnaire. We also analyzed these results in relation to the user experience in process management and/or familiarity in process simulation tools.

10.4.1 Task Accuracy

Task accuracy was measured by counting the number of "errors" defined as discussed in the previous Section 10.3.3, also measuring the time needed to complete each task.

The tasks were designed in increasing order of difficulty, from T1 to T4. This progression is reflected both in the completion times and in the observed error rates (see Table 10.1 and Figure 10.2).

As reported in Table 10.1, T1 was completed quickly and consistently by almost all participants, with an average completion time of 28 seconds and very low variability. T2 required more time, indicating a higher cognitive or interactional load, yet still showed limited time variability and only a single error overall. Notably, the errors observed in these two tasks are imputable to momentary lapses of attention rather than to intrinsic task difficulty.

A similar trend can be observed when considering error counts. T3 and T4 present a higher number of errors compared to T1 and T2, which can be reasonably attributed to the increased complexity and conceptual difficulty of these tasks. Nevertheless, it is important to note that 75% of the participants (9 out of 12) were able to successfully complete both tasks without requiring the intervention of the moderator. Moreover, the errors observed in T3 and T4 were made by participants with less prior experience in BPM and simulation tools.

Although the occurrence of errors in T3 and T4 can be largely accountable for differences in the prior user experience, the completion time does not exhibit the same dependency. Instead, the observed variability in task duration appears to be more strongly influenced by individual user characteristics, such as confidence and exploration style during interaction, rather than by domain expertise.

Despite the inherent difficulty of tasks involving the understanding and modification of decision-tree–based simulation parameters, most users were able to complete the tasks correctly, indicating that the system effectively supports advanced *what-if* analysis while preserving model interpretability and accessibility.

10.4.2 System Usability Scale

System usability was evaluated using the System Usability Scale (SUS) and interpreted according to the grading, adjective ratings, and acceptability ranges proposed by Brooke [21], as discussed in Section B.3.

The system achieved an overall mean SUS score of 81.25 with a 95% confidence interval between 73.9 and 88.6 (see Figure 10.3). This result corresponds to an adjective rating between "Good" and "Excellent", falling firmly within the "Acceptable" range, and in the grade scale between "C" and "B". This indicates a high level of user satisfaction and suggests that the interface is intuitive and effective for the intended tasks.

When analyzing the results in relation to the participants backgrounds we

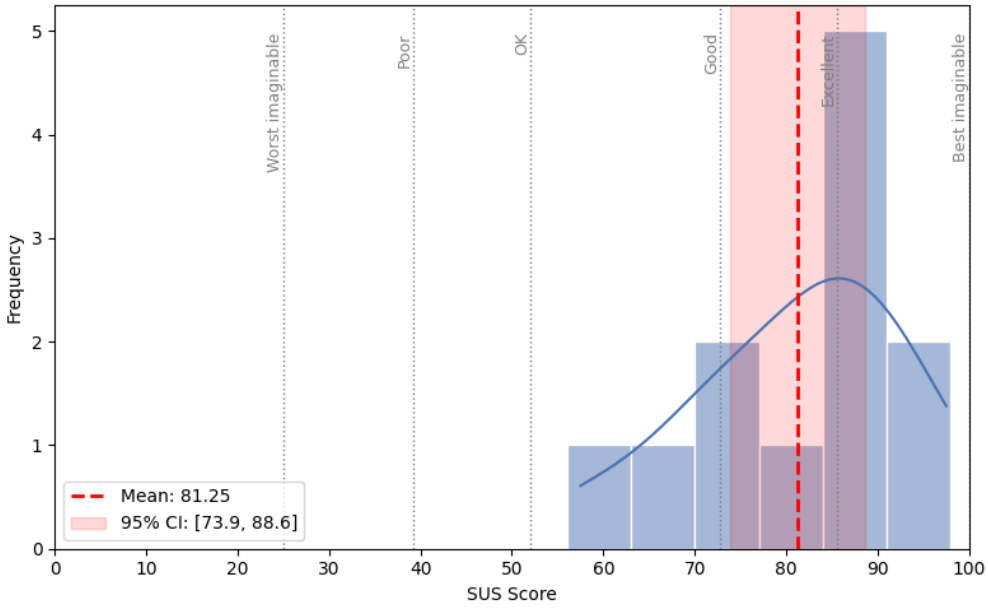


Figure 10.3: Distribution of SUS scores across the 12 participants. The red dashed line indicates the mean score of 81.25, positioning the system in the "Excellent" adjective rating category. The red area represents the 95% Confidence Interval [73.9, 88.6].

observed that users with the highest level of domain expertise (5+ years of BPM experience) tended to provide more critical evaluations. Specifically, the two users with 5+ years of BPM experience provided scores that fell into the "Marginal" category (Grade D/E and "OK" adjective). While all the users with "0-1 year" and "2-3 years" of experience evaluated it as "Excellent". On the other hand, a strong positive correlation was observed among users with high familiarity (4-5 in a Likert scale) with simulation tools. These participants consistently gave the highest scores, ranging between "Good" and "Excellent".

10.4.3 Final Discussion

Figure 10.4 highlights several key insights regarding the interaction between background experience, task performance, and perceived usability.

Participants with extensive BPM experience (i.e. 5+ years) showed a higher level of task accuracy. However, they provided lower SUS scores. This suggests that experienced users may have higher expectations regarding tool features, which influenced their subjective evaluations. This is confirmed by the post-task interviews, during which these users provided more detailed and critical feedback. In particular, they suggested the inclusion of richer statistical infor-

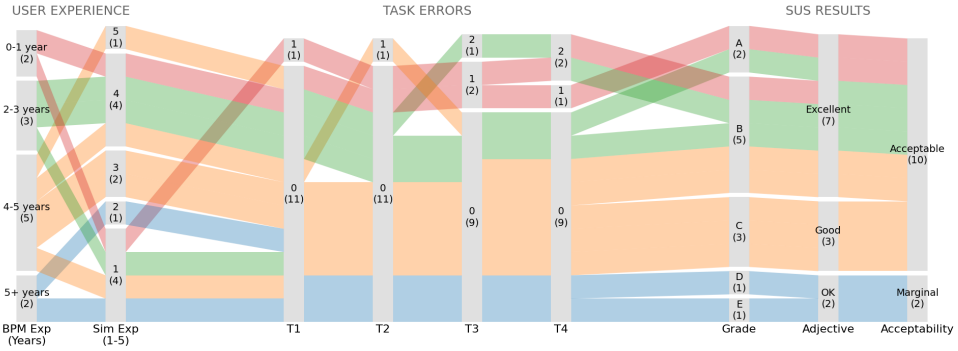


Figure 10.4: Alluvial diagram illustrating the flow between BPM experience (highlighted with different colors), simulation tool familiarity (Sim Exp), task errors across T1–T4, and final SUS results (Grade, Adjective, and Acceptability). The width of the streams represents the number of participants following that specific path.

mation in the post-simulation results page and emphasized the need for more advanced system support during exploratory analysis, such as automated recommendations to guide and structure *what-if* analyses. On the other hand, participants with high simulation tool familiarity demonstrated strong task performance and provided consistently high SUS scores. Note that this statement does not contradict the previous about low SUS scores given by BPM experts: those users with 5+ years of BPM experience did not declare high experience with business process simulation.

Conversely, participants with limited BPM experience or low familiarity with simulation tools generally completed the tasks successfully, making few errors in the more complex T3 and T4, and rated the system highly in terms of usability.

The results in tasks T3 and T4 are particularly relevant, as these tasks assess the user's ability to understand, modify, and reason about tree-based representations of simulation parameters (cf. Chapter 4). The low error rates observed for T3 and T4 indicate that the transparency and structure of the tree-based models effectively support user-driven *what-if* analysis.

In summary, the usability study confirms that the system is generally effective, and well-received by a diverse user population. The average SUS score achieved (81.25) is in line with systems considered to have high usability [21]. The combination of quantitative task metrics and subjective SUS feedback demonstrates that the interface supports a positive user experience, making it suitable for a broad range of BPM professionals, from novices to advanced users.

10.5 Conclusion

This chapter presented a comprehensive user-centered usability assessment of the **ProSiT System** introduced in Chapter 9. The evaluation combined objective performance metrics with subjective user feedback in order to provide a view of how effectively the tool supports process simulation and *what-if* analysis in practice.

The task-based evaluation showed that participants were able to complete all assigned tasks with high accuracy and with minimal need for external assistance. This result suggests that ProSiT successfully balances explainable modeling capabilities with usability. In particular, it demonstrates that the use of probabilistic decision trees for parameter modeling, as proposed in Chapter 4, can be effectively handled by end users.

The subjective usability assessment, measured through the System Usability Scale (SUS), further reinforces these findings. With an average SUS score of 81.25, ProSiT was rated between the *Good* and *Excellent* categories, reflecting strong perceived usability. The analysis also revealed meaningful differences across user profiles: experienced BPM practitioners tended to provide more critical evaluations, likely reflecting higher expectations, whereas users with strong familiarity with simulation tools consistently reported high usability perceptions.

Overall, the results suggest that ProSiT succeeds in making white-box, decision-tree–based simulation models accessible and usable to a broad audience, including both novice and expert users. Moreover, the feedback collected during the study provides valuable guidance for future improvements, including the refinement of existing functionalities, the addition of more contextual information and guided tutorials, enhancements to the visualization components, and the introduction of new features for configuring simulation parameters.

Conclusions

This thesis is focused on providing research advances in Business Process Simulation (BPS). BPS enables organizations to analyze and optimize processes by experimenting *what-if* analyses in a risk-free environment [60]. In recent years, the availability of large volumes of event data, together with advances in artificial intelligence and data science, has opened new opportunities for building more accurate simulation models [164, 128, 42]. However, existing approaches present some limitations, including poor explainability, weak generalization capabilities, and lack of adaptability to evolving process behavior.

To overcome these limitations, this thesis contributes to the advancement of data-driven business process simulation. It proposed novel methods and techniques for the automatic **discovery of simulation models** from event logs, introducing process-state-dependent simulation parameters (cf. Chapter 3 and Chapter 4). By leveraging probabilistic decision tree models, the proposed approaches capture complex conditional dependencies across multiple process perspectives while preserving explainability and configurability. This design enables simulation models that are not only more reliable, but also transparent and accessible to process experts, promoting trust and practical adoption.

The thesis also addressed the problem of **simulation model refinement**, by proposing two different techniques targeting distinct refinement scenarios. The first technique focuses on the **adaptation** of existing simulation models to evolving process behavior (cf. Chapter 5). It introduces an adaptive approach that combines incremental process discovery with online machine learning, enabling simulation models to maintain high accuracy over time and in the presence of concept drift. The second technique addresses the **repair** of simulation models in stable settings (i.e. without observed drifts) by proposing a framework in which inaccuracies in process models are identified through explainable arti-

ficial intelligence techniques and subsequently exploited to repair the affected model components, thus improving simulation accuracy (cf. Chapter 6).

A further contribution of this thesis lies in the **application of process simulation models** in two different contexts. A simulation-driven methodology for optimizing resource allocation in healthcare processes was proposed and evaluated in a real-world case study involving the Emergency Department of an Italian hospital (cf. Chapter 7). The results showed that an optimal resource configuration can potentially lead to reduced waiting times while maintaining sustainable costs and stability in critical scenarios, such as increased patient arrivals. In a second application, a framework for evaluating recommendations provided by Process-Aware Recommender Systems through simulation was introduced (cf. Chapter 8). The framework maps a set of ongoing traces to a simulation model, where recommendations are injected to simulate potential outcomes. This proposal is particularly relevant for supporting researchers that aim to assess prescriptive techniques when the ground truth is not available.

The employment of the proposed techniques and applications is only possible with clean and complete event logs. However, often, such as in the case study presented in Chapter 7, relevant temporal information is missing. To address this, the thesis presents a preliminary step for **estimating timestamps** when they are missing (cf. Chapter 2). The proposed framework reconstructs the missing temporal data, providing event logs with complete information that can be effectively used for process simulation.

Finally, the relevant proposed methods presented in this thesis have been implemented in a **process simulation software**, named *ProSiT* (*Process Simulation Tool*) (cf. Chapter 9). The tool is composed of a Python library that implements the core discovery and simulation engines, and a graphical user interface where users can interact to perform *what-if* analyses. A structured user study demonstrated the usability of the tool and the practical accessibility of the implemented techniques for real-world decision-support scenarios (cf. Chapter 10).

Overall, this thesis contributes to the advancement of state-of-the-art data-driven process simulation techniques and methodologies. We first proposed a coherent path from event-log preparation to discovery and refinement of simulation models, combining process mining, machine learning, and explainable artificial intelligence. The practical relevance of these proposals has been demonstrated through two different applications. We also made them accessible to potential end users through a dedicated process simulation software. These contributions resulted in several peer-reviewed international conferences and journal publications, confirming their relevance in the research domain.

Future Works. Several promising research directions emerge from this work. A first line of future research concerns further improving the accuracy of the tree-based simulation parameters proposed in Chapter 3 and Chapter 4 by integrating more powerful and advanced machine learning techniques. This could include the integration of the distillation mechanism [80], where complex models, like neural networks, are used as "teachers" to "student" models, such as our decision trees. These more sophisticated methods can then be naturally integrated into the refinement techniques introduced in Chapter 5 and Chapter 6.

The integration of these techniques into ProSiT is straightforward. This would further enhance ProSiT as a relevant software for end-to-end data-driven process simulation and decision support, both for researchers and practitioners.

Final future directions regard the extension of domain-specific applications. In particular, the healthcare domain offers significant opportunities for further research, given the complexity and criticality of its processes. Future work may focus on applying the proposed simulation techniques to optimize additional clinical pathways and healthcare settings.

A Online Simulation Discovery: Additional Results

This appendix provides additional plots and detailed experimental results for the online simulation discovery method discussed in Chapter 5. It is divided into two main sections: an evaluation of various distance metrics over time for four different use cases, and a sensitivity analysis exploring the impact of the "grace period" parameter on these metrics.

A.1 Distances Over Time

This section presents detailed visualizations for each evaluation metric in all use cases. Figures A.1–A.8 offer a comprehensive overview of how these metrics evolve over time, comparing the online method against baseline techniques (see Section 5.5.2).

Figure A.1 and Figure A.2 display the Control-Flow Log Distance (CFLD) and 3-Gram Distance (3DG). As discussed in Section 5.5.2 the online and single batch techniques achieve similar results for these measures, with the exception of few temporal windows. In fact, these case studies do not present relevant concept drifts in the control-flow perspectives. However, when comparing the online with the last batch technique, the difference in the performance are evident, showing often increased accuracy for the online proposal.

Figure A.3 and Figure A.4 show the Absolute Event Distribution (AED) and Relative Event Distribution (RED) distances over time. While for AED the differences between the three techniques are minimal, the RED distance shows promising results for the online technique, showing its ability to remain accurate and stable over time.

Figure A.5 and Figure A.6 visualize the Circadian Event Distribution (CED) and Circadian Workload Distribution (CWD). These measures strongly depend

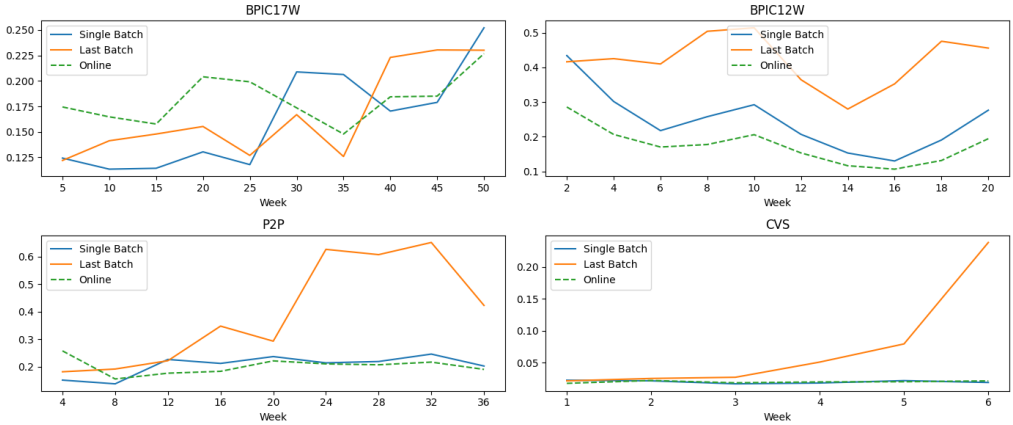


Figure A.1: Control-Flow Log Distance (CFLD) over time for each use case.

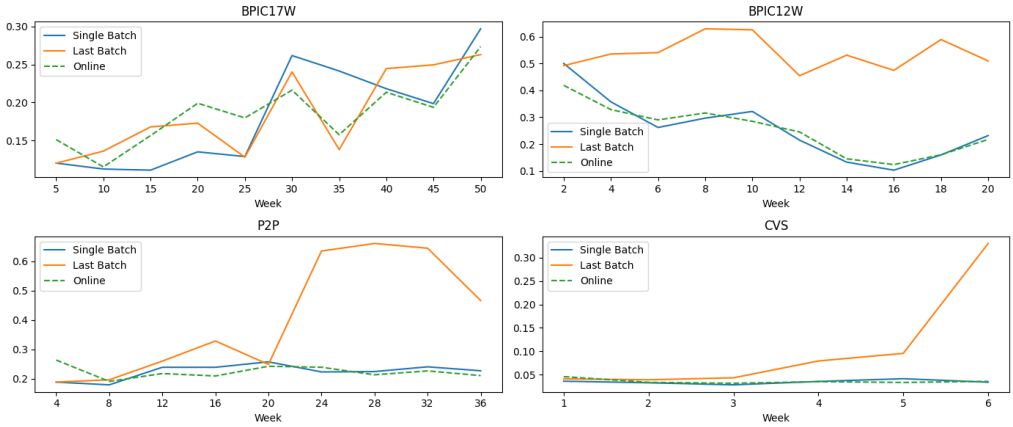


Figure A.2: 3-Gram Distance (3DG) over time for each use case.

on the calendars and behaviors of the resource observed in the event logs. BPIC17W show similar results among the three techniques for both metrics, with the exception of few time windows. The CED measures computed in the BPIC12W case study show instead better results for the online and last bacth techniques. Better results are achieved for the online and the single batch techniques for the CVS case study when compared with the last batch.

Finally, Figure A.7 and Figure A.8 presents the Case Arrival Rate (CAR) and Cycle Time Distribution (CTD) results. While for CAR difference are minimal and csimilar to the AED measures. CTD results, also discussed in Section 5.5.2, are significant especially in the BPIC17W case study where a relevant concept drift has been detected.

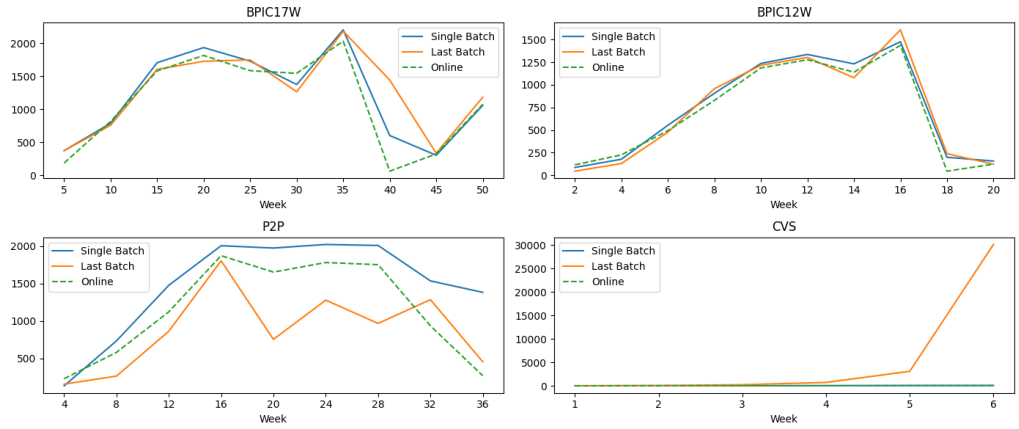


Figure A.3: Absolute Event Distribution (AED) distance over time for each use case.

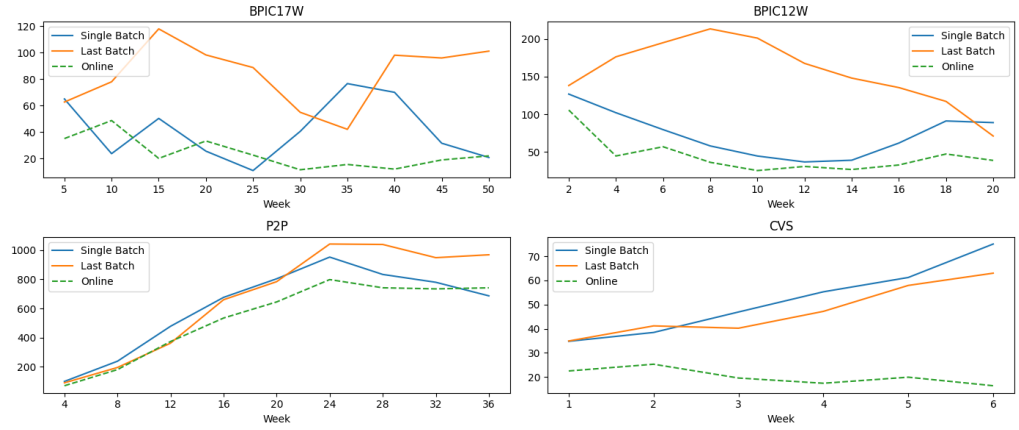


Figure A.4: Relative Event Distribution (RED) distance over time for each use case.

A.2 Distances Over Time per Grace Period

This section provides a detailed analysis of the experiments conducted using fixed grace periods of 100, 500, 1000, 5000, 10000, and 50000. These experiments aimed to evaluate the impact of the grace period parameter on model performance across different scenarios. Figures A.9–A.16 illustrate how varying the grace period affects the balance between adaptability and stability. Particularly, lower grace periods facilitate rapid adaption to sudden concept drifts, while higher grace periods reduce the risk of overfitting and promote more stable decisions. The results guided the selection of the optimal grace period used in the simulation model.

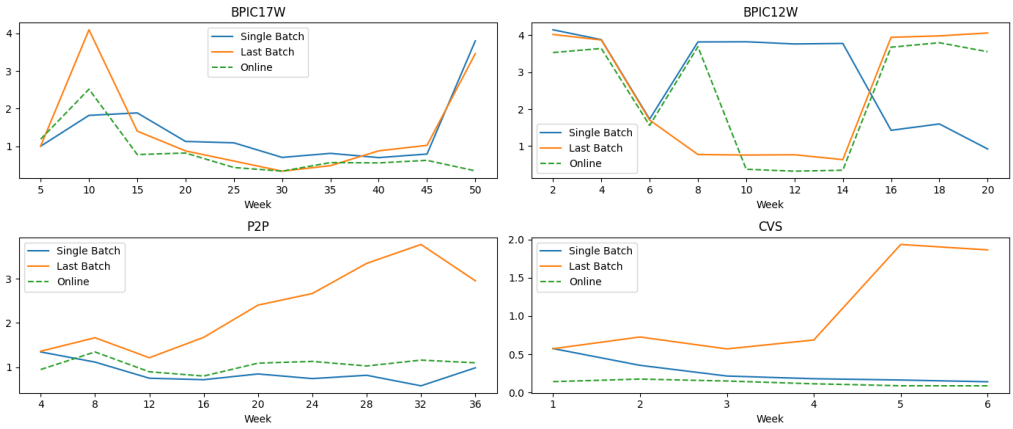


Figure A.5: Circadian Event Distribution (CED) distance over time for each use case.

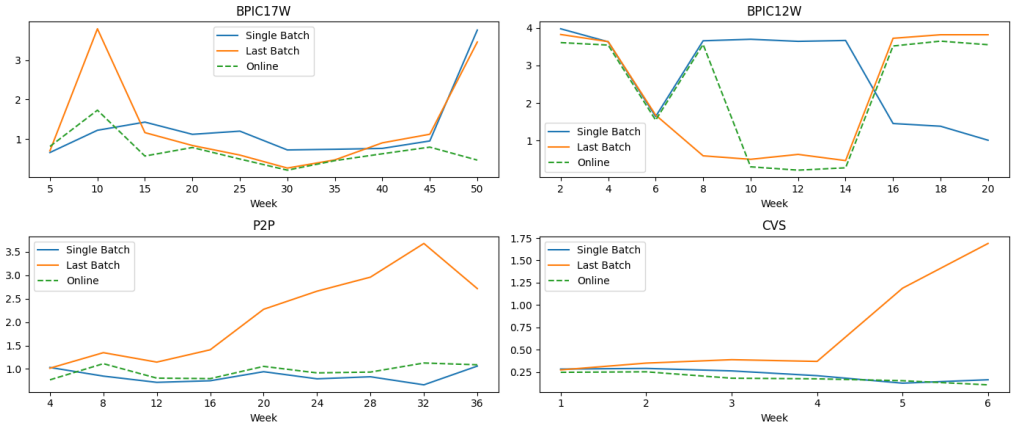


Figure A.6: Circadian Workload Distribution (CWD) distance over time for each use case.

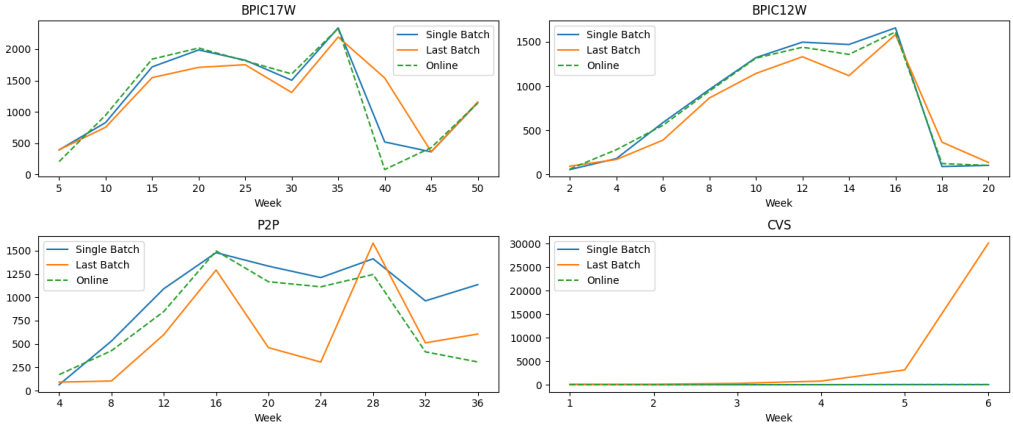


Figure A.7: Case Arrival Rate (CAR) distance over time for each use case.

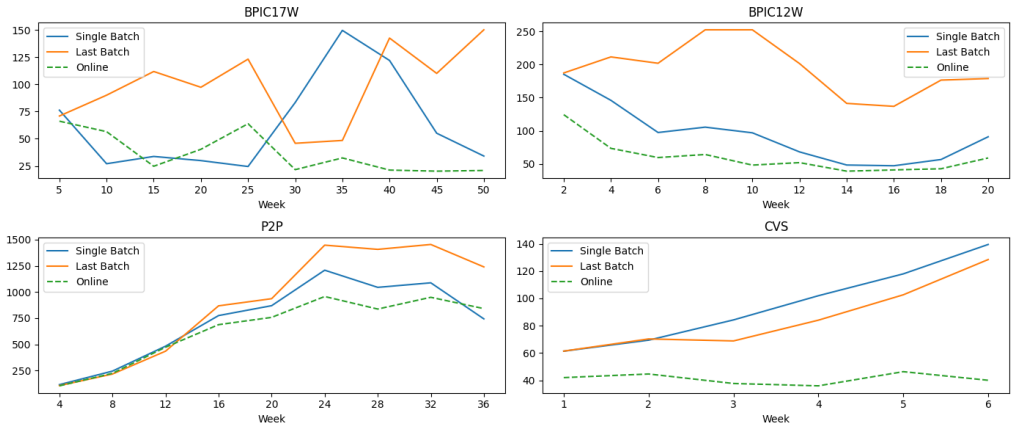


Figure A.8: Cycle Time Distribution (CTD) distance over time for each use case.

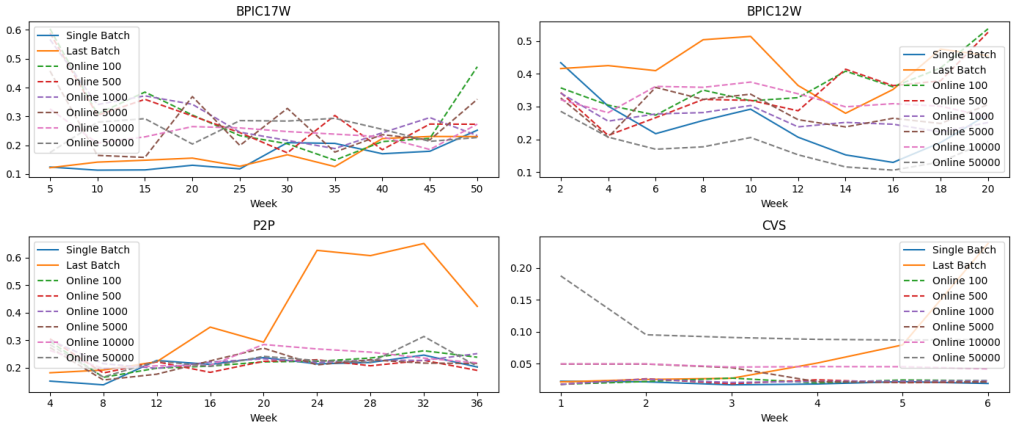


Figure A.9: Control-Flow Log Distance (CFLD) over time for each use case per grace period.

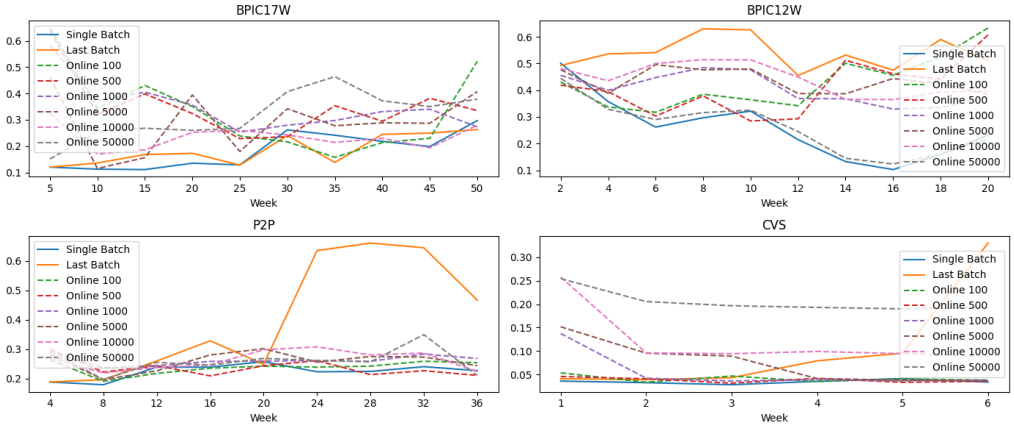


Figure A.10: 3-Gram Distance (3DG) over time for each use case per grace period.

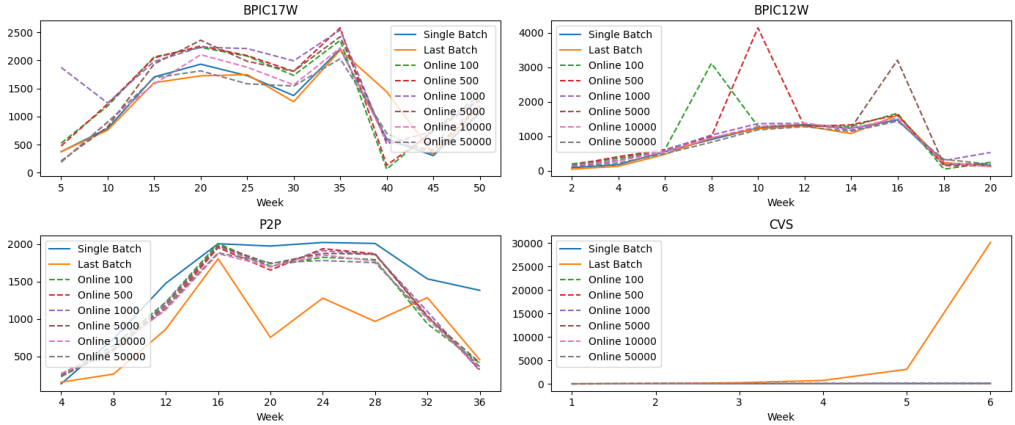


Figure A.11: Absolute Event Distribution (AED) distance over time for each use case per grace period.

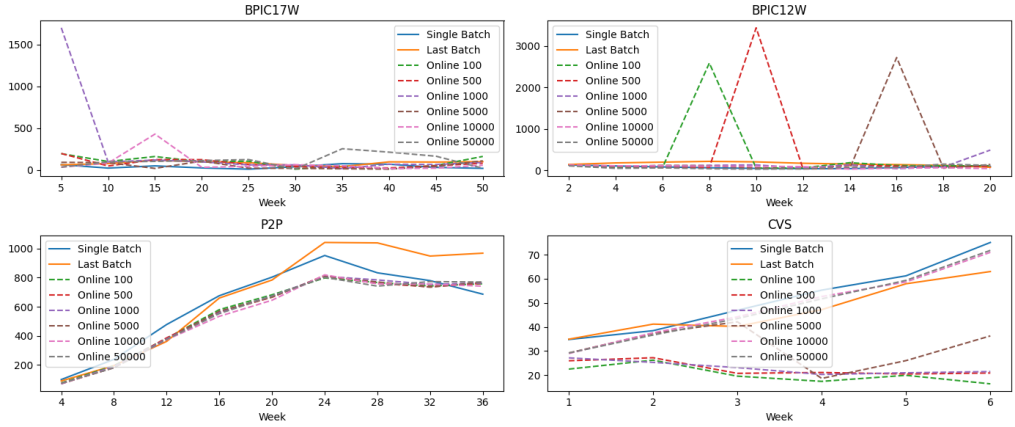


Figure A.12: Relative Event Distribution (RED) distance over time for each use case per grace period.

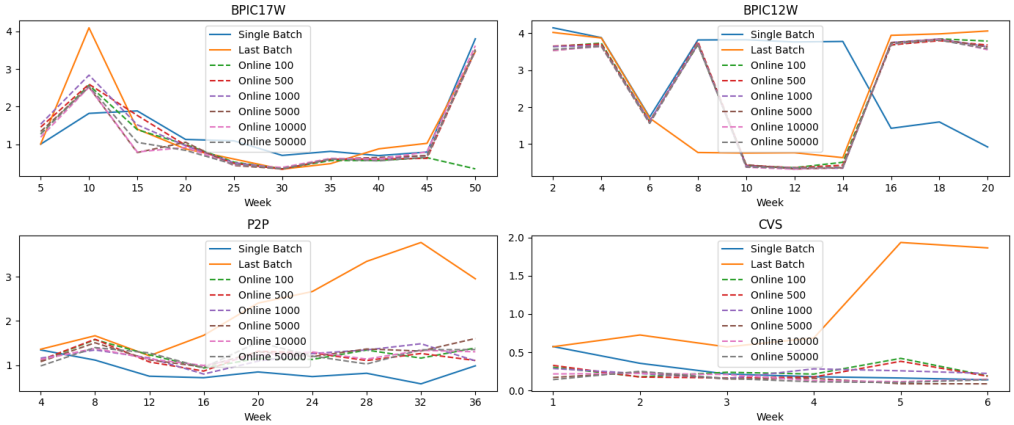


Figure A.13: Circadian Event Distribution (CED) distance over time for each use case per grace period.

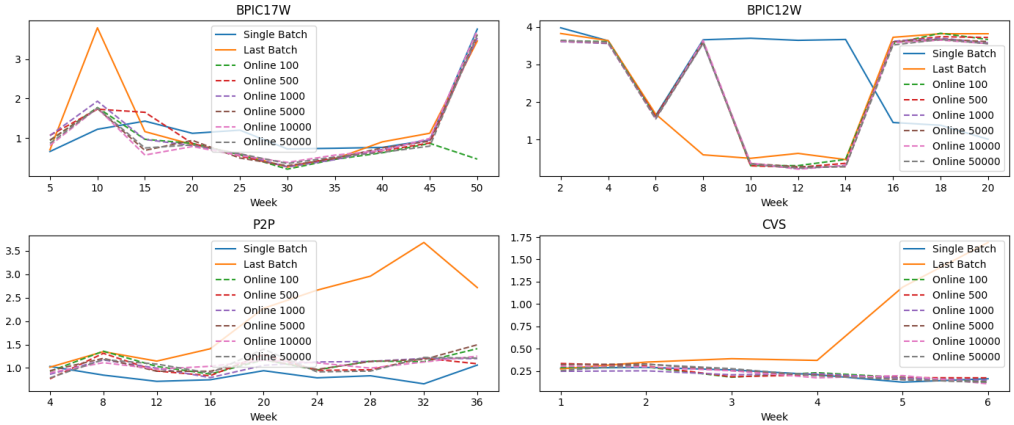


Figure A.14: Circadian Workload Distribution (CWD) distance over time for each use case per grace period.

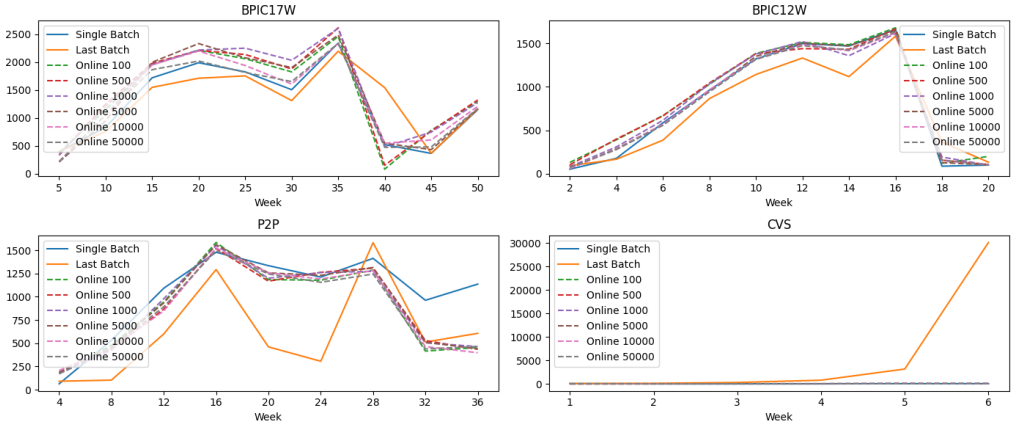


Figure A.15: Case Arrival Rate (CAR) distance over time for each use case per grace period.

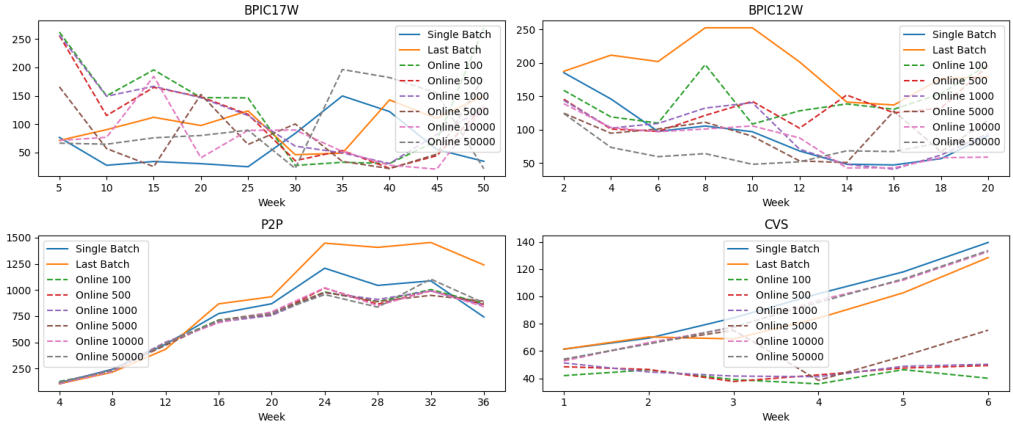


Figure A.16: Cycle Time Distribution (CTD) distance over time for each use case per grace period.

B ProSiT User Evaluation Materials

This appendix provides the relevant documentation to reproduce the user assessment of ProSiT (cf. Chapter 9) presented in Chapter 10.

B.1 Pre-Session Background Questionnaire

This section reports on the background questionnaire provided to the users before starting the task session. The questionnaire consists of the following three questions:

1. How many years have you been involved in process analysis and management?
   (a) 0-1 year
   (b) 2-3 years
   (c) 4-5 years
   (d) 5+ years
2. On a scale from 1 to 5, where 1 means not familiar at all and 5 means expert-level familiarity, how familiar are you with process management tools?
   (a) Not familiar
   (b) Slightly familiar
   (c) Moderately familiar
   (d) Very familiar
   (e) Expert

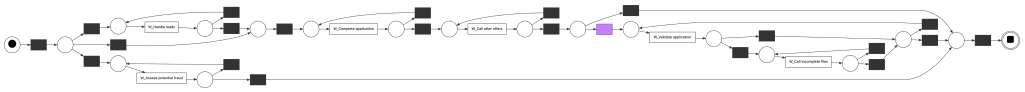


Figure B.1: Resulting Petri net after *Task 1* with the invisible transition *init_loop_21* highlighted in purple.

3. On a scale from 1 to 5, where 1 means not familiar at all and 5 means expert-level familiarity, how familiar are you with process simulation tools?

   (a) Not familiar

   (b) Slightly familiar

   (c) Moderately familiar

   (d) Very familiar

   (e) Expert

B.2 Task Sheet

The tasks provided to the users are described below.

Task 1 – Upload and Configure Discovery Parameters Import the provided event log into ProSiT.[1] Use the **Inductive Miner** for discovering the process model setting the noise threshold to 0.25. Configure the maximum depth for decision parameters to 2. Ensure that incremental discovery is disabled. Once configured, proceed to discover the simulation model.

Task 2 – Explore Parameters and Run an As-Is Simulation Explore the discovered simulation parameters by examining the **execution time** of *W Complete Application* activity, and **trace attributes distributions** focusing on *RequestedAmount*. Then run an initial as-is simulation with 500 instances and *January 1st, 2026, 00:00* as the start timestamp. Once the simulation is completed, analyze the results on the **process map** and **case duration distribution**, and provide a brief interpretation, with a specific focus on identifying potential bottlenecks.

Task 3 – What-if Analysis: Modify Control-Flow Probabilities Modify the firing rule of transition *init_loop_21*, verifying is the one allowing the execution of *W Validate Application* (see Figure B.1), as follows:

- If *RequestedAmount* $\leq$ 15000: set weight = 0

[1] https://github.com/franvinci/prosit-tool/blob/main/example_data/bpi17_march.xes

- If *RequestedAmount* > 15000: set weight = 0.5

Run a new simulation and observe new **case duration** results.

Task 4 – What-if Analysis: Add a Resource and Adjust Parameters Add a **new resource** named *X* with the same behavior as resource *User 3*, ensuring that multitasking is disabled. Assign it exclusively to activity *W Validate Application*. Change the **resource *X* weight** according to the following rule:

- If *W Validate Application* ≤ 0.5 (*W Validate Application* did not occurred yet): availability weight = 0.8
- Otherwise:
  - If *RequestedAmount* ≤ 15000: availability weight = 1.0
  - If *RequestedAmount* > 15000: availability weight = 0.5

Change **its waiting time** as follows:

- If *RequestedAmount* ≤ 15000: set distribution to *fixed* with 15 minutes value.
- If *RequestedAmount* > 15000: set distribution to *exponential* with mean of 20 minutes and minimum and maximum values of respectively 0 and 50 minutes.

Execute a new simulation run and observe **new results on process map**.

This document outlines the objectives of each task without providing hints on how to accomplish them or where specific interface functions are located, ensuring that observations reflect authentic user interaction with the tool design.

B.3 System Usability Scale (SUS)

The System Usability Scale (SUS) questionnaire [32] consists of ten statements (reported in Table B.1) for which participants express their level of agreement on a five-point Likert scale ranging from *Strongly Disagree* to *Strongly Agree*. The items capture multiple aspects of usability, including perceived complexity, consistency, confidence in use, and the need for technical support.

The SUS provides a single, aggregate measure of perceived usability on a scale from 0 to 100 (cf. Section B.3). The score is obtained according to the standard SUS scoring procedure:

- For each *odd-numbered* item, compute:

$$score = response - 1$$

Table B.1: System Usability Scale (SUS) – 1 = Strongly Disagree, 5 = Strongly Agree.

#	**Statement**	**1**	**2**	**3**	**4**	**5**
1.	I think that I would like to use this system frequently.					
2.	I found the system unnecessarily complex.					
3.	I thought the system was easy to use.					
4.	I think that I would need the support of a technical person to be able to use this system.					
5.	I found the various functions in this system were well integrated.					
6.	I thought there was too much inconsistency in this system.					
7.	I would imagine that most people would learn to use this system very quickly.					
8.	I found the system very cumbersome to use.					
9.	I felt very confident using the system.					
10.	I needed to learn a lot of things before I could get going with this system.					

- For each *even-numbered* item, compute:

$$score = 5 - response$$

- Sum the scores of all items
- Multiply the total by 2.5 to obtain the final SUS score, which ranges from 0 to 100.

Bibliography

[1] Wil van der Aalst. *Process Mining: Data Science in Action*. Springer Berlin Heidelberg, 2016. ISBN: 978-3-662-49851-4. DOI: /10.1007/978-3-662-49851-4.

[2] Wil M. P. van der Aalst. "Business Process Simulation Survival Guide". In: *Handbook on Business Process Management 1, Introduction, Methods, and Information Systems, 2nd Ed*. Ed. by Jan vom Brocke and Michael Rosemann. International Handbooks on Information Systems. Springer, 2015, pp. 337–370. DOI: 10.1007/978-3-642-45100-3_15. URL: https://doi.org/10.1007/978-3-642-45100-3%5C_15.

[3] Wil M. P. van der Aalst. "Responsible Data Science: Using Event Data in a "People Friendly" Manner". In: *Enterprise Information Systems - 18th International Conference, ICEIS 2016, Rome, Italy, April 25-28, 2016, Revised Selected Papers*. Ed. by Slimane Hammoudi, Leszek A. Maciaszek, Michele Missikoff, Olivier Camp, and José Cordeiro. Vol. 291. Lecture Notes in Business Information Processing. Springer, 2016, pp. 3–28. DOI: 10.1007/978-3-319-62386-3_1. URL: https://doi.org/10.1007/978-3-319-62386-3%5C_1.

[4] Wil M. P. van der Aalst. "The Application of Petri Nets to Workflow Management". In: *Journal of Circuits, Systems and Computers* 8.1 (1998), pp. 21–66. DOI: 10.1142/S0218126698000043. URL: https://doi.org/10.1142/S0218126698000043.

[5] Wil M. P. van der Aalst, Ton Weijters, and Laura Maruster. "Workflow Mining: Discovering Process Models from Event Logs". In: *IEEE Transactions on Knowledge and Data Engineering* 16.9 (2004), pp. 1128–1142. DOI:

10.1109/TKDE.2004.47. URL: https://doi.org/10.1109/TKDE.2004.47.

[6] Waleed Abo-Hamad, Ahmed Ramy, and Amr Arisha. "A hybrid process-mining approach for simulation modeling". In: *2017 Winter Simulation Conference, WSC 2017, Las Vegas, NV, USA, December 3-6, 2017*. IEEE, 2017, pp. 1527–1538. DOI: 10.1109/WSC.2017.8247894. URL: https://doi.org/10.1109/WSC.2017.8247894.

[7] Jan Niklas Adams, Sebastiaan J. van Zelst, Lara Quack, Kathrin Hausmann, Wil M. P. van der Aalst, and Thomas Rose. "A Framework for Explainable Concept Drift Detection in Process Mining". In: *Business Process Management - 19th International Conference, BPM 2021, Rome, Italy, September 06-10, 2021, Proceedings*. Ed. by Artem Polyvyanyy, Moe Thandar Wynn, Amy Van Looy, and Manfred Reichert. Vol. 12875. Lecture Notes in Computer Science. Springer, 2021, pp. 400–416. DOI: 10.1007/978-3-030-85469-0_25. URL: https://doi.org/10.1007/978-3-030-85469-0_25.

[8] Arya Adriansyah, Jorge Munoz-Gama, Josep Carmona, Boudewijn F. van Dongen, and Wil M. P. van der Aalst. "Measuring precision of modeled behavior". In: *Information Systems and e-Business Management* 13.1 (2015), pp. 37–67. DOI: 10.1007/S10257-014-0234-7. URL: https://doi.org/10.1007/s10257-014-0234-7.

[9] Mohamed A. Ahmed and Talal M. Alkhamis. "Simulation optimization for an emergency department healthcare unit in Kuwait". In: *European Journal of Operational Research* 198.3 (2009), pp. 936–942. DOI: 10.1016/J.EJOR.2008.10.025. URL: https://doi.org/10.1016/j.ejor.2008.10.025.

[10] Ali J. Alaee, Matthias Weidlich, and Arik Senderovich. "Data-Driven Decision Support for Business Processes: Causal Reasoning and Discovery". In: *Business Process Management Forum - BPM 2024 Forum, Krakow, Poland, September 1-6, 2024, Proceedings*. Ed. by Andrea Marrella, Manuel Resinas, Mieke Jans, and Michael Rosemann. Vol. 526. Lecture Notes in Business Information Processing. Springer, 2024, pp. 90–106. DOI: 10.1007/978-3-031-70418-5_6. URL: https://doi.org/10.1007/978-3-031-70418-5_6.

[11] Jana Ammann, Laura Lohoff, Bastian Wurm, and Thomas Hess. "How do Process Mining Users Act, Think, and Feel?" In: *Business & Information Systems Engineering* (2025). DOI: 10.1007/s12599-025-00931-9. URL: https://doi.org/10.1007/s12599-025-00931-9.

[12] Robert Andrews, Kanika Goel, Paul Corry, Robert L. Burdett, Moe Thandar Wynn, and Donna Callow. "Process data analytics for hospital case-mix planning". In: *Journal of Biomedical Informatics* 129 (2022), p. 104056. DOI: 10.1016/J.JBI.2022.104056. URL: https://doi.org/10.1016/j.jbi.2022.104056.

[13] Bianca B. P. Antunes, Adrian Manresa, Leonardo S. L. Bastos, Janaina Figueira Marchesi, and Silvio Hamacher. "A Solution Framework Based on Process Mining, Optimization, and Discrete-Event Simulation to Improve Queue Performance in an Emergency Department". In: *Business Process Management Workshops - BPM 2019 International Workshops, Vienna, Austria, September 1-6, 2019, Revised Selected Papers*. Ed. by Chiara Di Francescomarino, Remco M. Dijkman, and Uwe Zdun. Vol. 362. Lecture Notes in Business Information Processing. Springer, 2019, pp. 583–594. DOI: 10.1007/978-3-030-37453-2_47. URL: https://doi.org/10.1007/978-3-030-37453-2_47.

[14] Abel Armas-Cervantes, Nick R. T. P. van Beest, Marcello La Rosa, Marlon Dumas, and Luciano García-Bañuelos. "Interactive and Incremental Business Process Model Repair". In: *On the Move to Meaningful Internet Systems. OTM 2017 Conferences - Confederated International Conferences: CoopIS, C&TC, and ODBASE 2017, Rhodes, Greece, October 23-27, 2017, Proceedings, Part I*. Ed. by Hervé Panetto, Christophe Debruyne, Walid Gaaloul, Mike P. Papazoglou, Adrian Paschke, Claudio Agostino Ardagna, and Robert Meersman. Vol. 10573. Lecture Notes in Computer Science. Springer, 2017, pp. 53–74. DOI: 10.1007/978-3-319-69462-7_5. URL: https://doi.org/10.1007/978-3-319-69462-7_5.

[15] Adriano Augusto, Raffaele Conforti, Marlon Dumas, and Marcello La Rosa. "Split Miner: Discovering Accurate and Simple Business Process Models from Event Logs". In: *2017 IEEE International Conference on Data Mining (ICDM)*. 2017, pp. 1–10. DOI: 10.1109/ICDM.2017.9.

[16] Adriano Augusto, Timothy Deitz, Noel Faux, Jo-Anne Manski-Nankervis, and Daniel Capurro. "Process mining-driven analysis of COVID-19's impact on vaccination patterns". In: *Journal of Biomedical Informatics* 130 (2022), p. 104081. DOI: 10.1016/J.JBI.2022.104081. URL: https://doi.org/10.1016/j.jbi.2022.104081.

[17] Maksym Avramenko, David Chapela-Campa, Marlon Dumas, and Fredrik Milani. "What's Coming Next? Short-Term Simulation of Business Processes from Current State". In: *7th International Conference on Process Mining, ICPM 2025, Montevideo, Uruguay, October 20-24, 2025*. IEEE, 2025,

pp. 1–8. DOI: 10.1109/ICPM66919.2025.11220647. URL: https://doi.org/10.1109/ICPM66919.2025.11220647.

[18] Christoffer Olling Back, Søren Debois, and Tijs Slaats. "Entropy as a Measure of Log Variability". In: *Journal on Data Semantics* 8.2 (2019), pp. 129–156. DOI: 10.1007/S13740-019-00105-3. URL: https://doi.org/10.1007/s13740-019-00105-3.

[19] Christel Baier and Joost-Pieter Katoen. *Principles of model checking*. MIT Press, 2008. ISBN: 978-0-262-02649-9.

[20] Alireza Bakhshi, Erfan Hassannayebi, and Amir Hossein Sadeghi. "Optimizing sepsis care through heuristics methods in process mining: A trajectory analysis". In: *Healthcare Analytics* 3 (2023), p. 100187. ISSN: 2772-4425. DOI: 10.1016/j.health.2023.100187.

[21] Aaron Bangor, Philip Kortum, and James Miller. "Determining what individual SUS scores mean: adding an adjective rating scale". In: *Journal of Usability Studies* 4.3 (May 2009), pp. 114–123.

[22] Elisabetta Benevento, Davide Aloini, and Wil M. P. van der Aalst. "How Can Interactive Process Discovery Address Data Quality Issues in Real Business Settings? Evidence from a Case Study in Healthcare". In: *Journal of Biomedical Informatics* 130 (2022), p. 104083. DOI: 10.1016/J.JBI.2022.104083. URL: https://doi.org/10.1016/j.jbi.2022.104083.

[23] Elisabetta Benevento, Prabhakar M. Dixit, Mohammadreza Fani Sani, Davide Aloini, and Wil M. P. van der Aalst. "Evaluating the Effectiveness of Interactive Process Discovery in Healthcare: A Case Study". In: *Business Process Management Workshops - BPM 2019 International Workshops, Vienna, Austria, September 1-6, 2019, Revised Selected Papers*. Ed. by Chiara Di Francescomarino, Remco M. Dijkman, and Uwe Zdun. Vol. 362. Lecture Notes in Business Information Processing. Springer, 2019, pp. 508–519. DOI: 10.1007/978-3-030-37453-2_41. URL: https://doi.org/10.1007/978-3-030-37453-2_41.

[24] Guy Berkenstadt, Avigdor Gal, Arik Senderovich, Roee Shraga, and Matthias Weidlich. "Queueing Inference for Process Performance Analysis with Missing Life-Cycle Data". In: *2nd International Conference on Process Mining, ICPM 2020, Padua, Italy, October 4-9, 2020*. Ed. by Boudewijn F. van Dongen, Marco Montali, and Moe Thandar Wynn. IEEE, 2020, pp. 57–64. DOI: 10.1109/ICPM49681.2020.00019. URL: https://doi.org/10.1109/ICPM49681.2020.00019.

[25] Alessandro Berti and Wil M. P. van der Aalst. "Reviving Token-based Replay: Increasing Speed While Improving Diagnostics". In: *Proceedings of the International Workshop on Algorithms & Theories for the Analysis of Event Data 2019 Satellite event of the conferences: 40th International Conference on Application and Theory of Petri Nets and Concurrency Petri Nets 2019 and 19th International Conference on Application of Concurrency to System Design ACSD 2019, ATAED@Petri Nets/ACSD 2019, Aachen, Germany, June 25, 2019*. Ed. by Wil M. P. van der Aalst, Robin Bergenthum, and Josep Carmona. Vol. 2371. CEUR Workshop Proceedings. CEUR-WS.org, 2019, pp. 87–103. URL: https://ceur-ws.org/Vol-2371/ATAED2019-87-103.pdf.

[26] Alessandro Berti, Sebastiaan J. van Zelst, and Daniel Schuster. "PM4Py: A process mining library for Python". In: *Software Impacts* 17 (2023), p. 100556. DOI: 10.1016/J.SIMPA.2023.100556. URL: https://doi.org/10.1016/j.simpa.2023.100556.

[27] Aditya Bhattacharya, Jeroen Ooge, Gregor Stiglic, and Katrien Verbert. "Directive Explanations for Monitoring the Risk of Diabetes Onset: Introducing Directive Data-Centric Explanations and Combinations to Support What-If Explorations". In: *Proceedings of the 28th International Conference on Intelligent User Interfaces, IUI 2023, Sydney, NSW, Australia, March 27-31, 2023*. ACM, 2023, pp. 204–219. DOI: 10.1145/3581641.3584075. URL: https://doi.org/10.1145/3581641.3584075.

[28] Albert Bifet and Ricard Gavaldà. "Adaptive Learning from Evolving Data Streams". In: *Advances in Intelligent Data Analysis VIII, 8th International Symposium on Intelligent Data Analysis, IDA 2009, Lyon, France, August 31 - September 2, 2009. Proceedings*. Ed. by Niall M. Adams, Céline Robardet, Arno Siebes, and Jean-François Boulicaut. Vol. 5772. Lecture Notes in Computer Science. Springer, 2009, pp. 249–260. DOI: 10.1007/978-3-642-03915-7_22. URL: https://doi.org/10.1007/978-3-642-03915-7_22.

[29] Christopher M. Bishop. *Pattern recognition and machine learning, 5th Edition*. Information science and statistics. Springer, 2007. ISBN: 9780387310732. URL: https://www.worldcat.org/oclc/71008143.

[30] Jagadeesh Chandra J. C. Bose, R. S. Mans, and Wil M. P. van der Aalst. "Wanna improve process mining results?" In: *IEEE Symposium on Computational Intelligence and Data Mining, CIDM 2013, Singapore, 16-19 April, 2013*. IEEE, 2013, pp. 127–134. DOI: 10.1109/CIDM.2013.6597227. URL: https://doi.org/10.1109/CIDM.2013.6597227.

[31] R. P. Jagadeesh Chandra Bose, Wil M. P. van der Aalst, Indre Zliobaite, and Mykola Pechenizkiy. "Dealing With Concept Drifts in Process Mining". In: *IEEE Transactions on Neural Networks and Learning Systems* 25.1 (2014), pp. 154–171. DOI: 10.1109/TNNLS.2013.2278313. URL: https://doi.org/10.1109/TNNLS.2013.2278313.

[32] John Brooke. "SUS: A 'Quick' and 'Dirty' Usability Scale". In: *Usability Evaluation in Industry*. Ed. by Patrick W. Jordan, Bruce Thomas, Bernard A. Weerdmeester, and Ian Lyall McClelland. Taylor and Francis, June 1996. Chap. 21, pp. 189–194. ISBN: 9780748404605.

[33] Andrei Buliga, Chiara Di Francescomarino, Chiara Ghidini, Marco Montali, and Massimiliano Ronzani. "Generating Counterfactual Explanations Under Temporal Constraints". In: *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*. Ed. by Toby Walsh, Julie Shah, and Zico Kolter. AAAI Press, 2025, pp. 15622–15631. DOI: 10.1609/AAAI.V39I15.33715. URL: https://doi.org/10.1609/aaai.v39i15.33715.

[34] Andrea Burattin. "PLG2: Multiperspective Process Randomization with Online and Offline Simulations". In: *Proceedings of the BPM Demo Track 2016 Co-located with the 14th International Conference on Business Process Management (BPM 2016), Rio de Janeiro, Brazil, September 21, 2016*. Ed. by Leonardo Azevedo and Cristina Cabanillas. Vol. 1789. CEUR Workshop Proceedings. CEUR-WS.org, 2016, pp. 1–6. URL: https://ceur-ws.org/Vol-1789/bpm-demo-2016-paper1.pdf.

[35] Andrea Burattin. "Streaming Process Mining". In: *Process Mining Handbook*. Ed. by Wil M. P. van der Aalst and Josep Carmona. Vol. 448. Lecture Notes in Business Information Processing. Springer, 2022, pp. 349–372. DOI: 10.1007/978-3-031-08848-3_11. URL: https://doi.org/10.1007/978-3-031-08848-3_11.

[36] Andrea Burattin, Alessandro Sperduti, and Wil M. P. van der Aalst. "Control-flow discovery from event streams". In: *Proceedings of the IEEE Congress on Evolutionary Computation, CEC 2014, Beijing, China, July 6-11, 2014*. IEEE, 2014, pp. 2420–2427. DOI: 10.1109/CEC.2014.6900341. URL: https://doi.org/10.1109/CEC.2014.6900341.

[37] Andrea Burattin, Alessandro Sperduti, and Marco Veluscek. "Business models enhancement through discovery of roles". In: *2013 IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*. 2013, pp. 103–110. DOI: 10.1109/CIDM.2013.6597224.

[38] Adam Burke, Sander J. J. Leemans, and Moe Thandar Wynn. "Discovering Stochastic Process Models by Reduction and Abstraction". In: *Application and Theory of Petri Nets and Concurrency - 42nd International Conference, PETRI NETS 2021, Virtual Event, June 23-25, 2021, Proceedings*. Ed. by Didier Buchs and Josep Carmona. Vol. 12734. Lecture Notes in Computer Science. Springer, 2021, pp. 312–336. DOI: 10.1007/978-3-030-76983-3_16. URL: https://doi.org/10.1007/978-3-030-76983-3_16.

[39] Adam Burke, Sander J. J. Leemans, and Moe Thandar Wynn. "Stochastic Process Discovery by Weight Estimation". In: *Process Mining Workshops - ICPM 2020 International Workshops, Padua, Italy, October 5-8, 2020, Revised Selected Papers*. Ed. by Sander J. J. Leemans and Henrik Leopold. Vol. 406. Lecture Notes in Business Information Processing. Springer, 2020, pp. 260–272. DOI: 10.1007/978-3-030-72693-5_20. URL: https://doi.org/10.1007/978-3-030-72693-5_20.

[40] Manuel Camargo. "Automated discovery of business process simulation models from event logs: a hybrid process mining and deep learning approach". PhD thesis. University of los Andes, Colombia, 2021. URL: http://hdl.handle.net/1992/54943.

[41] Manuel Camargo, Daniel Báron, Marlon Dumas, and Oscar González Rojas. "Learning business process simulation models: A Hybrid process mining and deep learning approach". In: *Information Systems* 117 (2023), p. 102248. DOI: 10.1016/J.IS.2023.102248. URL: https://doi.org/10.1016/j.is.2023.102248.

[42] Manuel Camargo, Marlon Dumas, and Oscar González. "Automated discovery of business process simulation models from event logs". In: *Decision Support Systems* 134 (2020), p. 113284. DOI: 10.1016/J.DSS.2020.113284. URL: https://doi.org/10.1016/j.dss.2020.113284.

[43] Manuel Camargo, Marlon Dumas, and Oscar González Rojas. "Discovering generative models from event logs: data-driven simulation vs deep learning". In: *PeerJ Computer Science* 7 (2021), e577. DOI: 10.7717/PEERJ-CS.577. URL: https://doi.org/10.7717/peerj-cs.577.

[44] Manuel Camargo, Marlon Dumas, and Oscar González Rojas. "Learning Accurate LSTM Models of Business Processes". In: *Business Process Management - 17th International Conference, BPM 2019, Vienna, Austria, September 1-6, 2019, Proceedings*. Ed. by Thomas T. Hildebrandt, Boudewijn F. van Dongen, Maximilian Röglinger, and Jan Mendling. Vol. 11675. Lecture Notes in Computer Science. Springer, 2019, pp. 286–302. DOI: 10.

1007/978-3-030-26619-6_19. URL: https://doi.org/10.1007/978-3-030-26619-6_19.

[45] Josep Carmona, Boudewijn F. van Dongen, Andreas Solti, and Matthias Weidlich. *Conformance Checking - Relating Processes and Models*. Springer, 2018. ISBN: 978-3-319-99413-0. DOI: 10.1007/978-3-319-99414-7. URL: https://doi.org/10.1007/978-3-319-99414-7.

[46] Josep Carmona and Ricard Gavaldà. "Online Techniques for Dealing with Concept Drift in Process Mining". In: *Advances in Intelligent Data Analysis XI - 11th International Symposium, IDA 2012, Helsinki, Finland, October 25-27, 2012. Proceedings*. Ed. by Jaakko Hollmén, Frank Klawonn, and Allan Tucker. Vol. 7619. Lecture Notes in Computer Science. Springer, 2012, pp. 90–102. DOI: 10.1007/978-3-642-34156-4_10. URL: https://doi.org/10.1007/978-3-642-34156-4_10.

[47] David Chapela-Campa, Ismail Benchekroun, Opher Baron, Marlon Dumas, Dmitry Krass, and Arik Senderovich. "A framework for measuring the quality of business process simulation models". In: *Information Systems* 127 (2025), p. 102447. DOI: 10.1016/J.IS.2024.102447. URL: https://doi.org/10.1016/j.is.2024.102447.

[48] David Chapela-Campa, Ismail Benchekroun, Opher Baron, Marlon Dumas, Dmitry Krass, and Arik Senderovich. "Can I Trust My Simulation Model? Measuring the Quality of Business Process Simulation Models". In: *Business Process Management - 21st International Conference, BPM 2023, Utrecht, The Netherlands, September 11-15, 2023, Proceedings*. Ed. by Chiara Di Francescomarino, Andrea Burattin, Christian Janiesch, and Shazia Sadiq. Vol. 14159. Lecture Notes in Computer Science. Springer, 2023, pp. 20–37. DOI: 10.1007/978-3-031-41620-0_2. URL: https://doi.org/10.1007/978-3-031-41620-0_2.

[49] David Chapela-Campa and Marlon Dumas. "Enhancing business process simulation models with extraneous activity delays". In: *Information Systems* 122 (2024), p. 102346. DOI: 10.1016/J.IS.2024.102346. URL: https://doi.org/10.1016/j.is.2024.102346.

[50] David Chapela-Campa, Orlenys López-Pintado, Ihar Suvorau, and Marlon Dumas. "SIMOD: Automated discovery of business process simulation models". In: *SoftwareX* 30 (2025), p. 102157. DOI: 10.1016/J.SOFTX.2025.102157. URL: https://doi.org/10.1016/j.softx.2025.102157.

[51] Carlo Combi, Beatrice Amico, Riccardo Bellazzi, Andreas Holzinger, Jason H. Moore, Marinka Zitnik, and John H. Holmes. "A manifesto on explainability for artificial intelligence in medicine". In: *Artificial Intelligence*

in Medicine 133 (2022), p. 102423. DOI: 10.1016/J.ARTMED.2022.102423. URL: https://doi.org/10.1016/j.artmed.2022.102423.

[52] Marco Comuzzi. "Ant-Colony Optimisation for Path Recommendation in Business Process Execution". In: *Journal on Data Semantics* 8.2 (2019), pp. 113–128. DOI: 10.1007/S13740-018-0099-X. URL: https://doi.org/10.1007/s13740-018-0099-x.

[53] Ariyam Das, Jin Wang, Sahil M. Gandhi, Jae Lee, Wei Wang, and Carlo Zaniolo. "Learn Smart with Less: Building Better Online Decision Trees with Fewer Training Examples". In: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*. Ed. by Sarit Kraus. ijcai.org, 2019, pp. 2209–2215. DOI: 10.24963/IJCAI.2019/306. URL: https://doi.org/10.24963/ijcai.2019/306.

[54] Jakob De Moor, Hans Weytjens, Johannes De Smedt, and Jochen De Weerdt. "SimBank: From Simulation to Solution in Prescriptive Process Monitoring". In: *Business Process Management Forum*. Ed. by Arik Senderovich, Cristina Cabanillas, Irene Vanderfeesten, and Hajo A. Reijers. Cham: Springer Nature Switzerland, 2026, pp. 165–182. ISBN: 978-3-032-02929-4. DOI: 10.1007/978-3-032-02929-4_10.

[55] Haowen Deng, Youyou Zhou, Lin Wang, and Cheng Zhang. "Ensemble learning for the early prediction of neonatal jaundice with genetic features". In: *BMC Medical Informatics Decision Making* 21.1 (2021), p. 338. DOI: 10.1186/S12911-021-01701-9. URL: https://doi.org/10.1186/s12911-021-01701-9.

[56] Vadim Denisov, Dirk Fahland, and Wil M. P. van der Aalst. "Repairing Event Logs with Missing Events to Support Performance Analysis of Systems with Shared Resources". In: *Application and Theory of Petri Nets and Concurrency - 41st International Conference, PETRI NETS 2020, Paris, France, June 24-25, 2020, Proceedings*. Ed. by Ryszard Janicki, Natalia Sidorova, and Thomas Chatain. Vol. 12152. Lecture Notes in Computer Science. Springer, 2020, pp. 239–259. DOI: 10.1007/978-3-030-51831-8_12. URL: https://doi.org/10.1007/978-3-030-51831-8_12.

[57] Prabhakar M. Dixit, Suriadi Suriadi, Robert Andrews, Moe Thandar Wynn, Arthur H. M. ter Hofstede, Joos C. A. M. Buijs, and Wil M. P. van der Aalst. "Detection and Interactive Repair of Event Ordering Imperfection in Process Logs". In: *Advanced Information Systems Engineering - 30th International Conference, CAiSE 2018, Tallinn, Estonia, June 11-15, 2018, Proceedings*. Ed. by John Krogstie and Hajo A. Reijers. Vol. 10816.

Lecture Notes in Computer Science. Springer, 2018, pp. 274–290. DOI: 10.1007/978-3-319-91563-0_17. URL: https://doi.org/10.1007/978-3-319-91563-0_17.

[58] Pedro M. Domingos and Geoff Hulten. "Mining high-speed data streams". In: *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining, Boston, MA, USA, August 20-23, 2000*. Ed. by Raghu Ramakrishnan, Salvatore J. Stolfo, Roberto J. Bayardo, and Ismail Parsa. ACM, 2000, pp. 71–80. DOI: 10.1145/347090.347107. URL: https://doi.org/10.1145/347090.347107.

[59] Boudewijn F. van Dongen, Josep Carmona, and Thomas Chatain. "A Unified Approach for Measuring Precision and Generalization Based on Anti-alignments". In: *Business Process Management - 14th International Conference, BPM 2016, Rio de Janeiro, Brazil, September 18-22, 2016. Proceedings*. Ed. by Marcello La Rosa, Peter Loos, and Oscar Pastor. Vol. 9850. Lecture Notes in Computer Science. Springer, 2016, pp. 39–56. DOI: 10.1007/978-3-319-45348-4_3. URL: https://doi.org/10.1007/978-3-319-45348-4_3.

[60] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. *Fundamentals of Business Process Management*. 2nd. Springer Publishing Company, Incorporated, 2018. ISBN: 3662565080. DOI: 10.1007/978-3-662-56509-4.

[61] Jack Edh, Tuva Falk, Filip Kanon, Erik Simonsson, Hugo Sjödin, Jonatan Wincent, Elisabeth Barar, Robert Blümel, and Lukas Kirchdorfer. "A Business Process Simulation Tool Bridging Control-Flow and Resource-Centric Paradigms". In: *Proceedings of the Best Dissertation Award, Doctoral Consortium, and Demonstration & Resources Forum at BPM 2025*. CEUR-WS.org, 2025. URL: https://ceur-ws.org/Vol-4032/paper-31.pdf.

[62] Jeffrey L. Elman. "Finding Structure in Time". In: *Cognitive Science* 14.2 (1990), pp. 179–211. DOI: 10.1207/S15516709COG1402_1. URL: https://doi.org/10.1207/s15516709cog1402%5C_1.

[63] Bedilia Estrada-Torres, Manuel Camargo, Marlon Dumas, Luciano García-Bañuelos, Ibrahim Mahdy, and Maksym Yerokhin. "Discovering business process simulation models in the presence of multitasking and availability constraints". In: *Data & Knowledge Engineering* 134 (2021), p. 101897. DOI: 10.1016/J.DATAK.2021.101897. URL: https://doi.org/10.1016/j.datak.2021.101897.

[64] Dirk Fahland and Wil M. P. van der Aalst. "Model repair - aligning process models to reality". In: *Information Systems* 47 (2015), pp. 220–243. DOI: 10.1016/J.IS.2013.12.007. URL: https://doi.org/10.1016/j.is.2013.12.007.

[65] Dominik Andreas Fischer, Kanika Goel, Robert Andrews, Christopher G. J. van Dun, Moe Thandar Wynn, and Maximilian Roglinger. "Enhancing Event Log Quality: Detecting and Quantifying Timestamp Imperfections". In: *Business Process Management - 18th International Conference, BPM 2020, Seville, Spain, September 13-18, 2020, Proceedings*. Ed. by Dirk Fahland, Chiara Ghidini, Jorg Becker, and Marlon Dumas. Vol. 12168. Lecture Notes in Computer Science. Springer, 2020, pp. 309–326. DOI: 10.1007/978-3-030-58666-9_18. URL: https://doi.org/10.1007/978-3-030-58666-9_18.

[66] Claudia Fracca, Massimiliano de Leoni, Fabio Asnicar, and Alessandro Turco. "Estimating Activity Start Timestamps in the Presence of Waiting Times via Process Simulation". In: *Advanced Information Systems Engineering - 34th International Conference, CAiSE 2022, Leuven, Belgium, June 6-10, 2022, Proceedings*. Ed. by Xavier Franch, Geert Poels, Frederik Gailly, and Monique Snoeck. Vol. 13295. Lecture Notes in Computer Science. Springer, 2022, pp. 287–303. DOI: 10.1007/978-3-031-07472-1_17. URL: https://doi.org/10.1007/978-3-031-07472-1_17.

[67] Chiara Di Francescomarino and Chiara Ghidini. "Predictive Process Monitoring". In: *Process Mining Handbook*. Ed. by Wil M. P. van der Aalst and Josep Carmona. Vol. 448. Lecture Notes in Business Information Processing. Springer, 2022, pp. 320–346. DOI: 10.1007/978-3-031-08848-3_10. URL: https://doi.org/10.1007/978-3-031-08848-3_10.

[68] Riccardo Galanti, Bernat Coma-Puig, Massimiliano de Leoni, Josep Carmona, and Nicolò Navarin. "Explainable Predictive Process Monitoring". In: *2nd International Conference on Process Mining, ICPM 2020, Padua, Italy, October 4-9, 2020*. Ed. by Boudewijn F. van Dongen, Marco Montali, and Moe Thandar Wynn. IEEE, 2020, pp. 1–8. DOI: 10.1109/ICPM49681.2020.00012. URL: https://doi.org/10.1109/ICPM49681.2020.00012.

[69] Riccardo Galanti, Massimiliano de Leoni, Merylin Monaro, Nicolò Navarin, Alan Marazzi, Brigida Di Stasi, and Stéphanie Maldera. "An explainable decision support system for predictive process analytics". In: *Engineering Applications of Artificial Intelligence* 120 (2023), p. 105904. DOI: 10.1016/J.ENGAPPAI.2023.105904. URL: https://doi.org/10.1016/j.engappai.2023.105904.

[70] João Gama. *Knowledge Discovery from Data Streams*. Chapman and Hall / CRC Data Mining and Knowledge Discovery Series. CRC Press, 2010. ISBN: 978-1-4398-2611-9. URL: http://www.crcpress.com/product/isbn/9781439826119.

[71] João Gama, Indre Zliobaite, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. "A survey on concept drift adaptation". In: *ACM Computing Surveys* 46.4 (2014), 44:1–44:37. DOI: 10.1145/2523813. URL: https://doi.org/10.1145/2523813.

[72] Mauro Gambini, Marcello La Rosa, Sara Migliorini, and Arthur H. M. ter Hofstede. "Automated Error Correction of Business Process Models". In: *Business Process Management - 9th International Conference, BPM 2011, Clermont-Ferrand, France, August 30 - September 2, 2011. Proceedings*. Ed. by Stefanie Rinderle-Ma, Farouk Toumani, and Karsten Wolf. Vol. 6896. Lecture Notes in Computer Science. Springer, 2011, pp. 148–165. DOI: 10.1007/978-3-642-23059-2_14. URL: https://doi.org/10.1007/978-3-642-23059-2_14.

[73] Roberto Gatta, Stefania Orini, and Mauro Vallati. "Process Mining in Healthcare: Challenges and Promising Directions". In: *Artificial Intelligence in Healthcare: Recent Applications and Developments*. Ed. by Tianhua Chen, Jenny Carter, Mufti Mahmud, and Arjab Singh Khuman. Springer Nature Singapore, 2022, pp. 47–61. ISBN: 978-981-19-5272-2. DOI: 10.1007/978-981-19-5272-2_2.

[74] Laura Genga, Fabio Rossi, Claudia Diamantini, Emanuele Storti, and Domenico Potena. "Model repair supported by frequent anomalous local instance graphs". In: *Information Systems* 122 (2024), p. 102349. DOI: 10.1016/J.IS.2024.102349. URL: https://doi.org/10.1016/j.is.2024.102349.

[75] Fernanda Gonzalez-Lopez, Niels Martin, Rene de la Fuente, Victor Galvez-Yanjari, Javiera Guzmán, Eduardo Kattan, Marcos Sepúlveda, and Jorge Munoz-Gama. "ProDeM: A Process-Oriented Delphi Method for systematic asynchronous and consensual surgical process modelling". In: *Artificial Intelligence in Medicine* 135 (2023), p. 102426. DOI: 10.1016/J.ARTMED.2022.102426. URL: https://doi.org/10.1016/j.artmed.2022.102426.

[76] Alberto Guastalla, Emilio Sulis, Roberto Aringhieri, Stefano Branchi, Chiara Di Francescomarino, and Chiara Ghidini. "Workshift Scheduling Using Optimization and Process Mining Techniques: An Application in Healthcare". In: *Winter Simulation Conference, WSC 2022, Singapore, Decem-*

ber 11-14, 2022. IEEE, 2022, pp. 1116–1127. DOI: 10.1109/WSC57314.2022.10015405. URL: https://doi.org/10.1109/WSC57314.2022.10015405.

[77] Riccardo Guidotti. "Counterfactual explanations and how to find them: literature review and benchmarking". In: *Data Mining and Knowledge Discovery* 38.5 (2024), pp. 2770–2824. DOI: 10.1007/S10618-022-00831-6. URL: https://doi.org/10.1007/s10618-022-00831-6.

[78] Antonella Guzzo, Antonino Rullo, and Eugenio Vocaturo. "Process mining applications in the healthcare domain: A comprehensive review". In: *WIREs Data Mining and Knowledge Discovery* 12.2 (2022). DOI: 10.1002/WIDM.1442. URL: https://doi.org/10.1002/widm.1442.

[79] Farouq Halawa, Sreenath Chalil Madathil, and Mohammad T. Khasawneh. "Integrated framework of process mining and simulation-optimization for pod structured clinical layout design". In: *Expert Systems with Applications* 185 (2021), p. 115696. DOI: 10.1016/J.ESWA.2021.115696. URL: https://doi.org/10.1016/j.eswa.2021.115696.

[80] Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. "Distilling the Knowledge in a Neural Network". In: *CoRR* abs/1503.02531 (2015). arXiv: 1503.02531. URL: http://arxiv.org/abs/1503.02531.

[81] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/NECO.1997.9.8.1735. URL: https://doi.org/10.1162/neco.1997.9.8.1735.

[82] Wassily Hoeffding. "Probability Inequalities for Sums of Bounded Random Variables". In: *Journal of the American Statistical Association* 58.301 (1963), pp. 13–30. DOI: 10.1080/01621459.1963.10500830. eprint: https://www.tandfonline.com/doi/pdf/10.1080/01621459.1963.10500830. URL: https://www.tandfonline.com/doi/abs/10.1080/01621459.1963.10500830.

[83] Chihcheng Hsieh, Catarina Moreira, and Chun Ouyang. "DiCE4EL: Interpreting Process Predictions using a Milestone-Aware Counterfactual Approach". In: *3rd International Conference on Process Mining, ICPM 2021, Eindhoven, The Netherlands, October 31 - Nov. 4, 2021*. Ed. by Claudio Di Ciccio, Chiara Di Francescomarino, and Pnina Soffer. IEEE, 2021, pp. 88–95. DOI: 10.1109/ICPM53251.2021.9576881. URL: https://doi.org/10.1109/ICPM53251.2021.9576881.

[84] Tsung-Hao Huang, Andreas Metzger, and Klaus Pohl. "Counterfactual Explanations for Predictive Business Process Monitoring". In: *Information Systems - 18th European, Mediterranean, and Middle Eastern Conference, EM-*

CIS 2021, Virtual Event, December 8-9, 2021, Proceedings. Ed. by Marinos Themistocleous and Maria Papadaki. Vol. 437. Lecture Notes in Business Information Processing. Springer, 2021, pp. 399–413. DOI: 10.1007/978-3-030-95947-0_28. URL: https://doi.org/10.1007/978-3-030-95947-0%5C_28.

[85] Gerhardus van Hulzen, Niels Martin, and Benoit Depaire. "The Need for Interactive Data-Driven Process Simulation in Healthcare: A Case Study". In: *Process Mining Workshops - ICPM 2020 International Workshops, Padua, Italy, October 5-8, 2020, Revised Selected Papers*. Ed. by Sander J. J. Leemans and Henrik Leopold. Vol. 406. Lecture Notes in Business Information Processing. Springer, 2020, pp. 317–329. DOI: 10.1007/978-3-030-72693-5_24. URL: https://doi.org/10.1007/978-3-030-72693-5_24.

[86] Gerhardus van Hulzen, Niels Martin, Benoit Depaire, and Geert Souverijns. "Supporting capacity management decisions in healthcare using data-driven process simulation". In: *Journal of Biomedical Informatics* 129 (2022), p. 104060. DOI: 10.1016/J.JBI.2022.104060. URL: https://doi.org/10.1016/j.jbi.2022.104060.

[87] Olusanmi Hundogan, Xixi Lu, Yupei Du, and Hajo A. Reijers. "CREATED: Generating Viable Counterfactual Sequences for Predictive Process Analytics". In: *Advanced Information Systems Engineering - 35th International Conference, CAiSE 2023, Zaragoza, Spain, June 12-16, 2023, Proceedings*. Ed. by Marta Indulska, Iris Reinhartz-Berger, Carlos Cetina, and Oscar Pastor. Vol. 13901. Lecture Notes in Computer Science. Springer, 2023, pp. 541–557. DOI: 10.1007/978-3-031-34560-9_32. URL: https://doi.org/10.1007/978-3-031-34560-9%5C_32.

[88] Owen A. Johnson, Thamer Ba Dhafari, Angelina Kurniati, Frank Fox, and Eric Rojas. "The ClearPath Method for Care Pathway Process Mining and Simulation". In: *Business Process Management Workshops - BPM 2018 International Workshops, Sydney, NSW, Australia, September 9-14, 2018, Revised Papers*. Ed. by Florian Daniel, Quan Z. Sheng, and Hamid Motahari. Vol. 342. Lecture Notes in Business Information Processing. Springer, 2018, pp. 239–250. DOI: 10.1007/978-3-030-11641-5_19. URL: https://doi.org/10.1007/978-3-030-11641-5_19.

[89] Toon Jouck and Benoit Depaire. "Generating Artificial Data for Empirical Analysis of Control-flow Discovery Algorithms - A Process Tree and Log Generator". In: *Business & Information Systems Engineering* 61.6 (2019), pp. 695–712. DOI: 10.1007/S12599-018-0541-5. URL: https://doi.org/10.1007/s12599-018-0541-5.

[90] Sonja Kabicher-Fuchs, Jürgen Mangler, and Stefanie Rinderle-Ma. "Experience Breeding in Process-Aware Information Systems". In: *Advanced Information Systems Engineering - 25th International Conference, CAiSE 2013, Valencia, Spain, June 17-21, 2013. Proceedings*. Ed. by Camille Salinesi, Moira C. Norrie, and Oscar Pastor. Vol. 7908. Lecture Notes in Computer Science. Springer, 2013, pp. 594–609. DOI: 10.1007/978-3-642-38709-8_38. URL: https://doi.org/10.1007/978-3-642-38709-8%5C_38.

[91] Anna A. Kalenkova, Josep Carmona, Artem Polyvyanyy, and Marcello La Rosa. "Automated Repair of Process Models Using Non-local Constraints". In: *Application and Theory of Petri Nets and Concurrency - 41st International Conference, PETRI NETS 2020, Paris, France, June 24-25, 2020, Proceedings*. Ed. by Ryszard Janicki, Natalia Sidorova, and Thomas Chatain. Vol. 12152. Lecture Notes in Computer Science. Springer, 2020, pp. 280–300. DOI: 10.1007/978-3-030-51831-8_14. URL: https://doi.org/10.1007/978-3-030-51831-8_14.

[92] Anandi Karunaratne, Artem Polyvyanyy, and Alistair Moffat. "The Role of Log Representativeness in Estimating Generalization in Process Mining". In: *6th International Conference on Process Mining, ICPM 2024, Kgs. Lyngby, Denmark, October 14-18, 2024*. IEEE, 2024, pp. 33–40. DOI: 10.1109/ICPM63005.2024.10680679. URL: https://doi.org/10.1109/ICPM63005.2024.10680679.

[93] Lukas Kirchdorfer, Robert Blümel, Timotheus Kampik, Han van der Aa, and Heiner Stuckenschmidt. "AgentSimulator: An Agent-based Approach for Data-driven Business Process Simulation". In: *6th International Conference on Process Mining, ICPM 2024, Kgs. Lyngby, Denmark, October 14-18, 2024*. IEEE, 2024, pp. 97–104. DOI: 10.1109/ICPM63005.2024.10680660. URL: https://doi.org/10.1109/ICPM63005.2024.10680660.

[94] Lukas Kirchdorfer, Robert Blümel, Timotheus Kampik, Han van der Aa, and Heiner Stuckenschmidt. "Discovering multi-agent systems for resource-centric business process simulation". In: *Process Science* 2.4 (2025). DOI: 10.1007/s44311-025-00009-5. URL: https://doi.org/10.1007/s44311-025-00009-5.

[95] Eva L. Klijn, Felix Mannhardt, and Dirk Fahland. "Multi-perspective Concept Drift Detection: Including the Actor Perspective". In: *Advanced Information Systems Engineering - 36th International Conference, CAiSE 2024, Limassol, Cyprus, June 3-7, 2024, Proceedings*. Ed. by Giancarlo Guizzardi, Flávia Maria Santoro, Haralambos Mouratidis, and Pnina Soffer. Vol. 14663. Lecture Notes in Computer Science. Springer, 2024, pp. 141–157. DOI:

10.1007/978-3-031-61057-8_9. URL: https://doi.org/10.1007/978-3-031-61057-8_9.

[96] Jonghyeon Ko, Jongyup Lee, and Marco Comuzzi. "AIR-BAGEL: An Interactive Root cause-Based Anomaly Generator for Event Logs". In: *Proceedings of the ICPM Doctoral Consortium and Tool Demonstration Track 2020 co-located with the 2nd International Conference on Process Mining (ICPM 2020), Padua, Italy, October 4-9, 2020*. Ed. by Claudio Di Ciccio, Benoit Depaire, Jochen De Weerdt, Chiara Di Francescomarino, and Jorge Munoz-Gama. Vol. 2703. CEUR Workshop Proceedings. CEUR-WS.org, 2020, pp. 35–38. URL: https://ceur-ws.org/Vol-2703/paperTD5.pdf.

[97] Sergey V. Kovalchuk, Anastasia A. Funkner, Oleg G. Metsker, and Aleksey N. Yakovlev. "Simulation of patient flow in multiple healthcare units using process and data mining techniques for model identification". In: *Journal of Biomedical Informatics* 82 (2018), pp. 128–142. DOI: 10.1016/J.JBI.2018.05.004. URL: https://doi.org/10.1016/j.jbi.2018.05.004.

[98] Alexander Kraus and Han van der Aa. "Looking for Change: A Computer Vision Approach for Concept Drift Detection in Process Mining". In: *Business Process Management - 22nd International Conference, BPM 2024, Krakow, Poland, September 1-6, 2024, Proceedings*. Ed. by Andrea Marrella, Manuel Resinas, Mieke Jans, and Michael Rosemann. Vol. 14940. Lecture Notes in Computer Science. Springer, 2024, pp. 273–290. DOI: 10.1007/978-3-031-70396-6_16. URL: https://doi.org/10.1007/978-3-031-70396-6_16.

[99] Kateryna Kubrak, Fredrik Milani, Alexander Nolte, and Marlon Dumas. "Design and Evaluation of a User Interface Concept for Prescriptive Process Monitoring". In: *Advanced Information Systems Engineering - 35th International Conference, CAiSE 2023, Zaragoza, Spain, June 12-16, 2023, Proceedings*. Ed. by Marta Indulska, Iris Reinhartz-Berger, Carlos Cetina, and Oscar Pastor. Vol. 13901. Lecture Notes in Computer Science. Springer, 2023, pp. 347–363. DOI: 10.1007/978-3-031-34560-9_21. URL: https://doi.org/10.1007/978-3-031-34560-9%5C_21.

[100] Kateryna Kubrak, Fredrik Milani, Alexander Nolte, and Marlon Dumas. "Prescriptive process monitoring: *Quo vadis*?" In: *PeerJ Computer Science* 8 (2022), e1097. DOI: 10.7717/PEERJ-CS.1097. URL: https://doi.org/10.7717/peerj-cs.1097.

[101] Manuel Laguna and Johan Marklund. *Business Process Modeling, Simulation and Design*. Upper Saddle River, NJ: Pearson Prentice Hall, 2004. DOI: 10.1201/9781315162119. URL: https://doi.org/10.1201/9781315162119.

[102] Ngoc-Diem Le, Alessandro Padella, Francesco Vinci, and Massimiliano de Leoni. "Leveraging Counterfactuals for Prescriptive Process Analytics". In: *Business Process Management: Responsible BPM Forum, Process Technology Forum, Educators Forum*. Ed. by Mahendrawathi ER, Avigdor Gal, Thomas Grisold, Flavia Santoro, Mathias Weske, Remco M. Dijkman, Dimka Karastoyanova, Banu Aysolmaz, Wasana Bandara, and Kate Revoredo. Cham: Springer Nature Switzerland, 2026, pp. 200–215. ISBN: 978-3-032-02936-2. DOI: 10.1007/978-3-032-02936-2_15. URL: https://doi.org/10.1007/978-3-032-02936-2%5C_15.

[103] Yann LeCun, Yoshua Bengio, and Geoffrey E. Hinton. "Deep learning". In: *Nature* 521.7553 (2015), pp. 436–444. DOI: 10.1038/NATURE14539. URL: https://doi.org/10.1038/nature14539.

[104] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: 10.1109/5.726791. URL: https://doi.org/10.1109/5.726791.

[105] Sander J. J. Leemans, Dirk Fahland, and Wil M. P. van der Aalst. "Discovering Block-Structured Process Models from Event Logs - A Constructive Approach". In: *Application and Theory of Petri Nets and Concurrency*. Ed. by José-Manuel Colom and Jörg Desel. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 311–329. ISBN: 978-3-642-38697-8. DOI: 10.1007/978-3-642-38697-8_17.

[106] Sander J. J. Leemans, Andrew Partington, Jonathan Karnon, and Moe Thandar Wynn. "Process mining for healthcare decision analytics with micro-costing estimations". In: *Artificial Intelligence in Medicine* 135 (2023), p. 102473. DOI: 10.1016/J.ARTMED.2022.102473. URL: https://doi.org/10.1016/j.artmed.2022.102473.

[107] Sander J. J. Leemans, Anja F. Syring, and Wil M. P. van der Aalst. "Earth Movers' Stochastic Conformance Checking". In: *Business Process Management Forum - BPM Forum 2019, Vienna, Austria, September 1-6, 2019, Proceedings*. Ed. by Thomas T. Hildebrandt, Boudewijn F. van Dongen, Maximilian Röglinger, and Jan Mendling. Vol. 360. Lecture Notes in Business Information Processing. Springer, 2019, pp. 127–143. DOI: 10.1007/978-

3-030-26643-1_8. URL: https://doi.org/10.1007/978-3-030-26643-1_8.

[108] Richard Lenz and Manfred Reichert. "IT support for healthcare processes - premises, challenges, perspectives". In: *Data & Knowledge Engineering* 61.1 (2007), pp. 39–58. DOI: 10.1016/J.DATAK.2006.04.007. URL: https://doi.org/10.1016/j.datak.2006.04.007.

[109] Giorgio Leonardi, Stefania Montani, and Manuel Striani. "Explainable process trace classification: An application to stroke". In: *Journal of Biomedical Informatics* 126 (2022), p. 103981. DOI: 10.1016/J.JBI.2021.103981. URL: https://doi.org/10.1016/j.jbi.2021.103981.

[110] Massimiliano de Leoni. "Foundations of Process Enhancement". In: *Process Mining Handbook*. Ed. by Wil M. P. van der Aalst and Josep Carmona. Cham: Springer International Publishing, 2022, pp. 243–273. ISBN: 978-3-031-08848-3. DOI: 10.1007/978-3-031-08848-3_8.

[111] Massimiliano de Leoni, Marcus Dees, and Laurens Reulink. "Design and Evaluation of a Process-aware Recommender System based on Prescriptive Analytics". In: *2nd International Conference on Process Mining, ICPM 2020, Padua, Italy, October 4-9, 2020*. Ed. by Boudewijn F. van Dongen, Marco Montali, and Moe Thandar Wynn. IEEE, 2020, pp. 9–16. DOI: 10.1109/ICPM49681.2020.00013. URL: https://doi.org/10.1109/ICPM49681.2020.00013.

[112] Massimiliano de Leoni and Felix Mannhardt. "Decision Discovery in Business Processes". In: *Encyclopedia of Big Data Technologies*. Ed. by Sherif Sakr and Albert Y. Zomaya. Springer, 2019. DOI: 10.1007/978-3-319-63962-8_96-1. URL: https://doi.org/10.1007/978-3-319-63962-8_96-1.

[113] Massimiliano de Leoni, Francesco Vinci, Sander J. J. Leemans, and Felix Mannhardt. "Investigating the Influence of Data-Aware Process States on Activity Probabilities in Simulation Models: Does Accuracy Improve?" In: *Business Process Management*. Ed. by Chiara Di Francescomarino, Andrea Burattin, Christian Janiesch, and Shazia Sadiq. Springer Nature Switzerland, 2023, pp. 129–145. ISBN: 978-3-031-41620-0. DOI: 10.1007/978-3-031-41620-0_8.

[114] Keyi Li, Ivan Marsic, Aleksandra Sarcevic, Sen Yang, Travis M. Sullivan, Peyton E. Tempel, Zachary P. Milestone, Karen J. O'Connell, and Randall S. Burd. "Discovering interpretable medical process models: A case study in trauma resuscitation". In: *Journal of Biomedical Informatics* 140 (2023),

p. 104344. DOI: 10.1016/J.JBI.2023.104344. URL: https://doi.org/10.1016/j.jbi.2023.104344.

[115] Niels Lohmann and Dirk Fahland. "Where Did I Go Wrong? - Explaining Errors in Business Process Models". In: *Business Process Management - 12th International Conference, BPM 2014, Haifa, Israel, September 7-11, 2014. Proceedings*. Ed. by Shazia Sadiq, Pnina Soffer, and Hagen Völzer. Vol. 8659. Lecture Notes in Computer Science. Springer, 2014, pp. 283–300. DOI: 10.1007/978-3-319-10172-9_18. URL: https://doi.org/10.1007/978-3-319-10172-9_18.

[116] Orlenys López-Pintado and Marlon Dumas. "Business process simulation: Probabilistic modeling of intermittent resource availability and multitasking behavior". In: *Information Systems* 127 (2025), p. 102471. DOI: 10.1016/J.IS.2024.102471. URL: https://doi.org/10.1016/j.is.2024.102471.

[117] Orlenys López-Pintado, Iryna Halenok, and Marlon Dumas. "Prosimos: Discovering and Simulating Business Processes with Differentiated Resources". In: *Enterprise Design, Operations, and Computing. EDOC 2022 Workshops - IDAMS, SoEA4EE, TEAR, EDOC Forum, Demonstrations Track and Doctoral Consortium, Bozen-Bolzano, Italy, October 4-7, 2022, Revised Selected Papers*. Ed. by Tiago Prince Sales, Henderik A. Proper, Giancarlo Guizzardi, Marco Montali, Fabrizio Maria Maggi, and Claudenir M. Fonseca. Vol. 466. Lecture Notes in Business Information Processing. Springer, 2022, pp. 346–352. DOI: 10.1007/978-3-031-26886-1_23. URL: https://doi.org/10.1007/978-3-031-26886-1_23.

[118] Orlenys López-Pintado, Serhii Murashko, and Marlon Dumas. "Discovery and Simulation of Data-Aware Business Processes". In: *6th International Conference on Process Mining, ICPM 2024, Kgs. Lyngby, Denmark, October 14-18, 2024*. IEEE, 2024, pp. 105–112. DOI: 10.1109/ICPM63005.2024.10680675. URL: https://doi.org/10.1109/ICPM63005.2024.10680675.

[119] Yang Lu, Qifan Chen, and Simon K. Poon. "A Robust and Accurate Approach to Detect Process Drifts from Event Streams". In: *Business Process Management - 19th International Conference, BPM 2021, Rome, Italy, September 06-10, 2021, Proceedings*. Ed. by Artem Polyvyanyy, Moe Thandar Wynn, Amy Van Looy, and Manfred Reichert. Vol. 12875. Lecture Notes in Computer Science. Springer, 2021, pp. 383–399. DOI: 10.1007/978-3-030-85469-0_24. URL: https://doi.org/10.1007/978-3-030-85469-0_24.

[120] Scott M. Lundberg and Su-In Lee. "A Unified Approach to Interpreting Model Predictions". In: *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*. Ed. by Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett. 2017, pp. 4765–4774. URL: https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html.

[121] Fabrizio Maria Maggi, Chiara Di Francescomarino, Marlon Dumas, and Chiara Ghidini. "Predictive Monitoring of Business Processes". In: *Advanced Information Systems Engineering - 26th International Conference, CAiSE 2014, Thessaloniki, Greece, June 16-20, 2014. Proceedings*. Ed. by Matthias Jarke, John Mylopoulos, Christoph Quix, Colette Rolland, Yannis Manolopoulos, Haralambos Mouratidis, and Jennifer Horkoff. Vol. 8484. Lecture Notes in Computer Science. Springer, 2014, pp. 457–472. DOI: 10.1007/978-3-319-07881-6_31. URL: https://doi.org/10.1007/978-3-319-07881-6_31.

[122] Felix Mannhardt. "Multi-perspective process mining". English. Proefschrift. PhD thesis. Mathematics and Computer Science, Feb. 2018. ISBN: 978-90-386-4438-7. URL: https://research.tue.nl/en/publications/multi-perspective-process-mining.

[123] Felix Mannhardt, Sander J. J. Leemans, Christopher T. Schwanen, and Massimiliano de Leoni. "Modelling Data-Aware Stochastic Processes - Discovery and Conformance Checking". In: *Application and Theory of Petri Nets and Concurrency - 44th International Conference, PETRI NETS 2023, Lisbon, Portugal, June 25-30, 2023, Proceedings*. Ed. by Luís Gomes and Robert Lorenz. Vol. 13929. Lecture Notes in Computer Science. Springer, 2023, pp. 77–98. DOI: 10.1007/978-3-031-33620-1_5. URL: https://doi.org/10.1007/978-3-031-33620-1_5.

[124] R. S. Mans, Helen Schonenberg, Minseok Song, Wil M. P. van der Aalst, and Piet J. M. Bakker. "Application of Process Mining in Healthcare - A Case Study in a Dutch Hospital". In: *Biomedical Engineering Systems and Technologies, International Joint Conference, BIOSTEC 2008, Funchal, Madeira, Portugal, January 28-31, 2008, Revised Selected Papers*. Ed. by Ana L. N. Fred, Joaquim Filipe, and Hugo Gamboa. Vol. 25. Communications in Computer and Information Science. Springer, 2008, pp. 425–438. DOI: 10.1007/978-3-540-92219-3_32. URL: https://doi.org/10.1007/978-3-540-92219-3_32.

[125] Ronny Mans, Wil M. P. van der Aalst, Rob J. B. Vanwersch, and Arnold J. Moleman. "Process Mining in Healthcare: Data Challenges When Answering Frequently Posed Questions". In: *Process Support and Knowledge Representation in Health Care - BPM 2012 Joint Workshop, ProHealth 2012/KR4HC 2012, Tallinn, Estonia, September 3, 2012, Revised Selected Papers*. Ed. by Richard Lenz, Silvia Miksch, Mor Peleg, Manfred Reichert, David Riaño, and Annette ten Teije. Vol. 7738. Lecture Notes in Computer Science. Springer, 2012, pp. 140–153. DOI: 10.1007/978-3-642-36438-9_10. URL: https://doi.org/10.1007/978-3-642-36438-9_10.

[126] Ronny Mans, Hajo A. Reijers, Daniel Wismeijer, and Michiel van Genuchten. "A process-oriented methodology for evaluating the impact of IT: A proposal and an application in healthcare". In: *Information Systems* 38.8 (2013), pp. 1097–1115. DOI: 10.1016/J.IS.2013.06.005. URL: https://doi.org/10.1016/j.is.2013.06.005.

[127] Marco Ajmone Marsan, Gianfranco Balbo, Gianni Conte, Susanna Donatelli, and Giuliana Franceschinis. "Modelling with Generalized Stochastic Petri Nets". In: *SIGMETRICS Performance Evaluation Review* 26.2 (1998), p. 2. DOI: 10.1145/288197.581193. URL: https://doi.org/10.1145/288197.581193.

[128] Niels Martin, Benoit Depaire, and An Caris. "The Use of Process Mining in Business Process Simulation Model Construction - Structuring the Field". In: *Business & Information Systems Engineering* 58.1 (2016), pp. 73–87. DOI: 10.1007/S12599-015-0410-4. URL: https://doi.org/10.1007/s12599-015-0410-4.

[129] Niels Martin, Benoit Depaire, An Caris, and Dimitri Schepers. "Retrieving the resource availability calendars of a process from an event log". In: *Information Systems* 88 (2020). DOI: 10.1016/J.IS.2019.101463. URL: https://doi.org/10.1016/j.is.2019.101463.

[130] Niels Martin, Antonio Martinez-Millana, Bernardo Valdivieso, and Carlos Fernández-Llatas. "Interactive Data Cleaning for Process Mining: A Case Study of an Outpatient Clinic's Appointment System". In: *Business Process Management Workshops - BPM 2019 International Workshops, Vienna, Austria, September 1-6, 2019, Revised Selected Papers*. Ed. by Chiara Di Francescomarino, Remco M. Dijkman, and Uwe Zdun. Vol. 362. Lecture Notes in Business Information Processing. Springer, 2019, pp. 532–544. DOI: 10.1007/978-3-030-37453-2_43. URL: https://doi.org/10.1007/978-3-030-37453-2_43.

[131] Niels Martin, Jochen De Weerdt, Carlos Fernández-Llatas, Avigdor Gal, Roberto Gatta, Gema Ibáñez, Owen A. Johnson, Felix Mannhardt, Luis Marco-Ruiz, Steven Mertens, Jorge Munoz-Gama, Fernando Seoane, Jan Vanthienen, Moe Thandar Wynn, David Baltar Boilève, Jochen Bergs, Mieke Joosten-Melis, Stijn Schretlen, and Bram B. Van Acker. "Recommendations for enhancing the usability and understandability of process mining in healthcare". In: *Artificial Intellingence in Medicine* 109 (2020), p. 101962. DOI: 10.1016/J.ARTMED.2020.101962. URL: https://doi.org/10.1016/j.artmed.2020.101962.

[132] Niels Martin, Nils Wittig, and Jorge Munoz-Gama. "Using Process Mining in Healthcare". In: *Process Mining Handbook*. Ed. by Wil M. P. van der Aalst and Josep Carmona. Vol. 448. Lecture Notes in Business Information Processing. Springer, 2022, pp. 416–444. DOI: 10.1007/978-3-031-08848-3_14. URL: https://doi.org/10.1007/978-3-031-08848-3_14.

[133] Tabib Ibne Mazhar, Asad Tariq, Sander J. J. Leemans, Kanika Goel, Moe Thandar Wynn, and Andrew Staib. "Stochastic-Aware Comparative Process Mining in Healthcare". In: *Business Process Management - 21st International Conference, BPM 2023, Utrecht, The Netherlands, September 11-15, 2023, Proceedings*. Ed. by Chiara Di Francescomarino, Andrea Burattin, Christian Janiesch, and Shazia Sadiq. Vol. 14159. Lecture Notes in Computer Science. Springer, 2023, pp. 341–358. DOI: 10.1007/978-3-031-41620-0_20. URL: https://doi.org/10.1007/978-3-031-41620-0_20.

[134] Francesca Meneghello, Claudia Fracca, Massimiliano de Leoni, Fabio Asnicar, and Alessandro Turco. "A Framework to Improve the Accuracy of Process Simulation Models". In: *Research Challenges in Information Science - 16th International Conference, RCIS 2022, Barcelona, Spain, May 17-20, 2022, Proceedings*. Ed. by Renata S. S. Guizzardi, Jolita Ralyté, and Xavier Franch. Vol. 446. Lecture Notes in Business Information Processing. Springer, 2022, pp. 142–158. DOI: 10.1007/978-3-031-05760-1_9. URL: https://doi.org/10.1007/978-3-031-05760-1_9.

[135] Francesca Meneghello, Chiara Di Francescomarino, and Chiara Ghidini. "RIMS_tool: a Hybrid Simulator for Business Processes". In: *Doctoral Consortium and Demo Track 2023 at the International Conference on Process Mining 2023 co-located with the 5th International Conference on Process Mining (ICPM 2023), Rome, Italy, October 27, 2023*. Ed. by Jan Martijn E. M. van der Werf, Cristina Cabanillas, Francesco Leotta, and Laura Genga. Vol. 3648. CEUR Workshop Proceedings. CEUR-WS.org, 2023. URL: https://ceur-ws.org/Vol-3648/paper%5C_2654.pdf.

[136] Francesca Meneghello, Chiara Di Francescomarino, Chiara Ghidini, and Massimiliano Ronzani. "Runtime integration of machine learning and simulation for business processes: Time and decision mining predictions". In: *Information Systems* 128 (2025), p. 102472. DOI: 10.1016/J.IS.2024.102472. URL: https://doi.org/10.1016/j.is.2024.102472.

[137] Alexey A. Mitsyuk, Irina A. Lomazova, Ivan S. Shugurov, and Wil M. P. van der Aalst. "Process Model Repair by Detecting Unfitting Fragments". In: *Supplementary Proceedings of the Sixth International Conference on Analysis of Images, Social Networks and Texts (AIST 2017), Moscow, Russia, July 27 - 29, 2017*. Ed. by Wil M. P. van der Aalst, Mikhail Yu. Khachay, Sergei O. Kuznetsov, Victor S. Lempitsky, Irina A. Lomazova, Natalia V. Loukachevitch, Amedeo Napoli, Alexander Panchenko, Panos M. Pardalos, Andrey V. Savchenko, Stanley Wasserman, and Dmitry I. Ignatov. Vol. 1975. CEUR Workshop Proceedings. CEUR-WS.org, 2017, pp. 301–313. URL: https://ceur-ws.org/Vol-1975/paper32.pdf.

[138] Christoph Molnar. *Interpretable Machine Learning. A Guide for Making Black Box Models Explainable*. 3rd ed. 2025. ISBN: 978-3-911578-03-5. URL: https://christophm.github.io/interpretable-ml-book.

[139] Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphaël Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdessalem, and Albert Bifet. "River: machine learning for streaming data in Python". In: *Journal of Machine Learning Research* 22 (2021), 110:1–110:8. URL: https://jmlr.org/papers/v22/20-1380.html.

[140] Ramaravind Kommiya Mothilal, Amit Sharma, and Chenhao Tan. "Explaining machine learning classifiers through diverse counterfactual explanations". In: *FAT* '20: Conference on Fairness, Accountability, and Transparency, Barcelona, Spain, January 27-30, 2020*. Ed. by Mireille Hildebrandt, Carlos Castillo, L. Elisa Celis, Salvatore Ruggieri, Linnet Taylor, and Gabriela Zanfir-Fortuna. ACM, 2020, pp. 607–617. DOI: 10.1145/3351095.3372850. URL: https://doi.org/10.1145/3351095.3372850.

[141] Jorge Munoz-Gama, Niels Martin, Carlos Fernández-Llatas, Owen A. Johnson, Marcos Sepúlveda, Emmanuel Helm, Victor Galvez-Yanjari, Eric Rojas, Antonio Martinez-Millana, Davide Aloini, Ilaria Angela Amantea, Robert Andrews, Michael Arias, Iris Beerepoot, Elisabetta Benevento, Andrea Burattin, Daniel Capurro, Josep Carmona, Marco Comuzzi, Benjamin Dalmas, Rene de la Fuente, Chiara Di Francescomarino, Claudio Di Ciccio, Roberto Gatta, Chiara Ghidini, Fernanda Gonzalez-Lopez, Gema Ibáñez-Sánchez, Hilda B. Klasky, Angelina Prima Kurniati, Xixi

Lu, Felix Mannhardt, Ronny Mans, Mar Marcos, Renata Medeiros de Carvalho, Marco Pegoraro, Simon K. Poon, Luise Pufahl, Hajo A. Reijers, Simon Remy, Stefanie Rinderle-Ma, Lucia Sacchi, Fernando Seoane, Minseok Song, Alessandro Stefanini, Emilio Sulis, Arthur H. M. ter Hofstede, Pieter J. Toussaint, Vicente Traver, Zoe Valero-Ramon, Inge van de Weerd, Wil M. P. van der Aalst, Rob J. B. Vanwersch, Mathias Weske, Moe Thandar Wynn, and Francesca Zerbato. "Process mining for healthcare: Characteristics and challenges". In: *Journal of Biomedical Informatics* 127 (2022), p. 103994. DOI: 10.1016/J.JBI.2022.103994. URL: https://doi.org/10.1016/j.jbi.2022.103994.

[142] Nicolò Navarin, Matteo Cambiaso, Andrea Burattin, Fabrizio Maria Maggi, Luca Oneto, and Alessandro Sperduti. "Towards Online Discovery of Data-Aware Declarative Process Models from Event Streams". In: *2020 International Joint Conference on Neural Networks, IJCNN 2020, Glasgow, United Kingdom, July 19-24, 2020*. IEEE, 2020, pp. 1–8. DOI: 10.1109/IJCNN48605.2020.9207500. URL: https://doi.org/10.1109/IJCNN48605.2020.9207500.

[143] Nicolò Navarin, Beatrice Vincenzi, Mirko Polato, and Alessandro Sperduti. "LSTM networks for data-aware remaining time prediction of business process instances". In: *2017 IEEE Symposium Series on Computational Intelligence, SSCI 2017, Honolulu, HI, USA, November 27 - Dec. 1, 2017*. IEEE, 2017, pp. 1–7. DOI: 10.1109/SSCI.2017.8285184. URL: https://doi.org/10.1109/SSCI.2017.8285184.

[144] Alessandro Padella and Massimiliano de Leoni. "Resource Allocation in Recommender Systems for Global KPI Improvement". In: *Business Process Management Forum - BPM 2023 Forum, Utrecht, The Netherlands, September 11-15, 2023, Proceedings*. Ed. by Chiara Di Francescomarino, Andrea Burattin, Christian Janiesch, and Shazia Sadiq. Vol. 490. Lecture Notes in Business Information Processing. Springer, 2023, pp. 249–266. DOI: 10.1007/978-3-031-41623-1_15. URL: https://doi.org/10.1007/978-3-031-41623-1%5C_15.

[145] Alessandro Padella, Massimiliano de Leoni, Onur Dogan, and Riccardo Galanti. "Explainable Process Prescriptive Analytics". In: *4th International Conference on Process Mining, ICPM 2022, Bolzano, Italy, October 23-28, 2022*. Ed. by Andrea Burattin, Artem Polyvyanyy, and Barbara Weber. IEEE, 2022, pp. 16–23. DOI: 10.1109/ICPM57379.2022.9980535. URL: https://doi.org/10.1109/ICPM57379.2022.9980535.

[146] Alessandro Padella, Felix Mannhardt, Francesco Vinci, Massimiliano de Leoni, and Irene Vanderfeesten. "Experience-Based Resource Allocation for Remaining Time Optimization". In: *Business Process Management - 22nd International Conference, BPM 2024, Krakow, Poland, September 1-6, 2024, Proceedings*. Ed. by Andrea Marrella, Manuel Resinas, Mieke Jans, and Michael Rosemann. Vol. 14940. Lecture Notes in Computer Science. Springer, 2024, pp. 345–362. DOI: 10.1007/978-3-031-70396-6_20. URL: https://doi.org/10.1007/978-3-031-70396-6%5C_20.

[147] Marco Pegoraro and Wil M. P. van der Aalst. "Mining Uncertain Event Data in Process Mining". In: *International Conference on Process Mining, ICPM 2019, Aachen, Germany, June 24-26, 2019*. IEEE, 2019, pp. 89–96. DOI: 10.1109/ICPM.2019.00023. URL: https://doi.org/10.1109/ICPM.2019.00023.

[148] Marco Pegoraro, Merih Seran Uysal, and Wil M. P. van der Aalst. "Discovering Process Models from Uncertain Event Data". In: *Business Process Management Workshops - BPM 2019 International Workshops, Vienna, Austria, September 1-6, 2019, Revised Selected Papers*. Ed. by Chiara Di Francescomarino, Remco M. Dijkman, and Uwe Zdun. Vol. 362. Lecture Notes in Business Information Processing. Springer, 2019, pp. 238–249. DOI: 10.1007/978-3-030-37453-2_20. URL: https://doi.org/10.1007/978-3-030-37453-2_20.

[149] Artem Polyvyanyy, Wil M. P. van der Aalst, Arthur H. M. ter Hofstede, and Moe Thandar Wynn. "Impact-Driven Process Model Repair". In: *ACM Transactions on Software Engineering and Methodology* 25.4 (2017), 28:1–28:60. DOI: 10.1145/2980764. URL: https://doi.org/10.1145/2980764.

[150] Mahsa Pourbafrani and Wil M. P. van der Aalst. "Interactive Process Improvement Using Simulation of Enriched Process Trees". In: *Service-Oriented Computing - ICSOC 2021 Workshops - AIOps, STRAPS, AI-PA and Satellite Events, Dubai, United Arab Emirates, November 22-25, 2021, Proceedings*. Ed. by Hakim Hacid, Monther Aldwairi, Mohamed Reda Bouadjenek, Marinella Petrocchi, Noura Faci, Fatma Outay, Amin Beheshti, Lauritz Thamsen, and Hai Dong. Vol. 13236. Lecture Notes in Computer Science. Springer, 2021, pp. 61–76. DOI: 10.1007/978-3-031-14135-5_5. URL: https://doi.org/10.1007/978-3-031-14135-5_5.

[151] Liudmila Ostroumova Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. "CatBoost: unbiased boosting with categorical features". In: *Advances in Neural Information Process-*

ing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada. Ed. by Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett. 2018, pp. 6639–6649. URL: https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html.

[152] Luise Pufahl, Fabian Stiehle, Sven Ihde, Mathias Weske, and Ingo Weber. "Resource allocation in business process executions - A systematic literature study". In: *Information Systems* 132 (2025), p. 102541. DOI: 10.1016/J.IS.2025.102541. URL: https://doi.org/10.1016/j.is.2025.102541.

[153] Fazla Rabbi, Debapriya Banik, Niamat Ullah Ibne Hossain, and Alexandr Sokolov. "Using process mining algorithms for process improvement in healthcare". In: *Healthcare Analytics* 5 (2024), p. 100305. ISSN: 2772-4425. DOI: 10.1016/j.health.2024.100305.

[154] Aaditya Ramdas, Nicolás García Trillos, and Marco Cuturi. "On Wasserstein Two-Sample Testing and Related Families of Nonparametric Tests". In: *Entropy* 19.2 (2017), p. 47. DOI: 10.3390/E19020047. URL: https://doi.org/10.3390/e19020047.

[155] H.A. Reijers and W.M.P. van der Aalst. "Short-term simulation: Bridging the gap between operational control and strategic decision making". English. In: *Proceedings of the IASTED International conference on modelling and simulation*. Ed. by M.H. Hamza. IASTED/Acta Press, 1999, pp. 417–421.

[156] Christian Rennert and Wil M. P. van der Aalst. "Improving Precision in Process Trees Using Subprocess Tree Logs". In: *Process Mining Workshops - ICPM 2023 International Workshops, Rome, Italy, October 23-27, 2023, Revised Selected Papers*. Ed. by Johannes De Smedt and Pnina Soffer. Vol. 503. Lecture Notes in Business Information Processing. Springer, 2023, pp. 110–122. DOI: 10.1007/978-3-031-56107-8_9. URL: https://doi.org/10.1007/978-3-031-56107-8_9.

[157] Kate Revoredo. "On the use of domain knowledge for process model repair". In: *Software and Systems Modeling* 22.4 (2023), pp. 1099–1111. DOI: 10.1007/S10270-022-01067-0. URL: https://doi.org/10.1007/s10270-022-01067-0.

[158] Williams Rizzi, Marco Comuzzi, Chiara Di Francescomarino, Chiara Ghidini, Suhwan Lee, Fabrizio Maria Maggi, and Alexander Nolte. "Explainable predictive process monitoring: a user evaluation". In: *Process Science* 1.1 (Oct. 2024), p. 3. ISSN: 2948-2178. DOI: 10.1007/s44311-024-00003-3. URL: https://doi.org/10.1007/s44311-024-00003-3.

[159] Williams Rizzi, Chiara Di Francescomarino, Chiara Ghidini, and Fabrizio Maria Maggi. "How do I update my model? On the resilience of Predictive Process Monitoring models to change". In: *Knowledge and Information Systems* 64.5 (2022), pp. 1385–1416. DOI: 10.1007/S10115-022-01666-9. URL: https://doi.org/10.1007/s10115-022-01666-9.

[160] Andreas Rogge-Solti, Wil M. P. van der Aalst, and Mathias Weske. "Discovering Stochastic Petri Nets with Arbitrary Delay Distributions from Event Logs". In: *Business Process Management Workshops - BPM 2013 International Workshops, Beijing, China, August 26, 2013, Revised Papers*. Ed. by Niels Lohmann, Minseok Song, and Petia Wohed. Vol. 171. Lecture Notes in Business Information Processing. Springer, 2013, pp. 15–27. DOI: 10.1007/978-3-319-06257-0_2. URL: https://doi.org/10.1007/978-3-319-06257-0_2.

[161] Andreas Rogge-Solti and Mathias Weske. "Prediction of business process durations using non-Markovian stochastic Petri nets". In: *Information Systems* 54 (2015), pp. 1–14. DOI: 10.1016/J.IS.2015.04.004. URL: https://doi.org/10.1016/j.is.2015.04.004.

[162] Emmelien De Roock and Niels Martin. "Process mining in healthcare - An updated perspective on the state of the art". In: *Journal of Biomedical Informatics* 127 (2022), p. 103995. DOI: 10.1016/J.JBI.2022.103995. URL: https://doi.org/10.1016/j.jbi.2022.103995.

[163] Anne Rozinat and Wil M. P. van der Aalst. "Conformance checking of processes based on monitoring real behavior". In: *Information Systems* 33.1 (2008), pp. 64–95. DOI: 10.1016/J.IS.2007.07.001. URL: https://doi.org/10.1016/j.is.2007.07.001.

[164] Anne Rozinat, R. S. Mans, Minseok Song, and Wil M. P. van der Aalst. "Discovering simulation models". In: *Information Systems* 34.3 (2009), pp. 305–327. DOI: 10.1016/J.IS.2008.09.002. URL: https://doi.org/10.1016/j.is.2008.09.002.

[165] Anne Rozinat, Moe Thandar Wynn, Wil M. P. van der Aalst, Arthur H. M. ter Hofstede, and Colin J. Fidge. "Workflow simulation for operational decision support". In: *Data & Knowledge Engineering* 68.9 (2009), pp. 834–850. DOI: 10.1016/J.DATAK.2009.02.014. URL: https://doi.org/10.1016/j.datak.2009.02.014.

[166] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020. ISBN: 9780134610993. URL: http://aima.cs.berkeley.edu/.

[167] Denise Maria Vecino Sato, Sheila Cristiana De Freitas, Jean Paul Barddal, and Edson Emílio Scalabrin. "A Survey on Concept Drift in Process Mining". In: *ACM Computing Surveys* 54.9 (2022), 189:1–189:38. DOI: 10.1145/3472752. URL: https://doi.org/10.1145/3472752.

[168] Beate Scheibel and Stefanie Rinderle-Ma. "An End-to-End Approach for Online Decision Mining and Decision Drift Analysis in Process-Aware Information Systems". In: *Intelligent Information Systems - CAiSE Forum 2023, Zaragoza, Spain, June 12-16, 2023, Proceedings*. Ed. by Cristina Cabanillas and Francisca Pérez. Vol. 477. Lecture Notes in Business Information Processing. Springer, 2023, pp. 17–25. DOI: 10.1007/978-3-031-34674-3_3. URL: https://doi.org/10.1007/978-3-031-34674-3_3.

[169] Daniel Schuster. *Incremental Process Discovery*. Vol. 540. Lecture Notes in Business Information Processing. Springer, 2025. ISBN: 978-3-031-80564-6. DOI: 10.1007/978-3-031-80565-3. URL: https://doi.org/10.1007/978-3-031-80565-3.

[170] Daniel Schuster, Niklas Föcking, Sebastiaan J. van Zelst, and Wil M. P. van der Aalst. "Incremental Discovery of Process Models Using Trace Fragments". In: *Business Process Management - 21st International Conference, BPM 2023, Utrecht, The Netherlands, September 11-15, 2023, Proceedings*. Ed. by Chiara Di Francescomarino, Andrea Burattin, Christian Janiesch, and Shazia Sadiq. Vol. 14159. Lecture Notes in Computer Science. Springer, 2023, pp. 55–73. DOI: 10.1007/978-3-031-41620-0_4. URL: https://doi.org/10.1007/978-3-031-41620-0_4.

[171] Daniel Schuster, Sebastiaan J. van Zelst, and Wil M. P. van der Aalst. "Cortado: A dedicated process mining tool for interactive process discovery". In: *SoftwareX* 22 (2023), p. 101373. DOI: 10.1016/J.SOFTX.2023.101373. URL: https://doi.org/10.1016/j.softx.2023.101373.

[172] Arik Senderovich, Sander J. J. Leemans, Shahar Harel, Avigdor Gal, Avishai Mandelbaum, and Wil M. P. van der Aalst. "Discovering Queues from Event Logs with Varying Levels of Information". In: *Business Process Management Workshops - BPM 2015, 13th International Workshops, Innsbruck, Austria, August 31 - September 3, 2015, Revised Papers*. Ed. by Manfred Reichert and Hajo A. Reijers. Vol. 256. Lecture Notes in Business Information Processing. Springer, 2015, pp. 154–166. DOI: 10.1007/978-3-319-42887-1_13. URL: https://doi.org/10.1007/978-3-319-42887-1_13.

[173] Mahmoud Shoush and Marlon Dumas. "When to Intervene? Prescriptive Process Monitoring Under Uncertainty and Resource Constraints". In: *Business Process Management Forum - BPM 2022 Forum, Münster, Germany,*

September 11-16, 2022, Proceedings. Ed. by Claudio Di Ciccio, Remco M. Dijkman, Adela del-Río-Ortega, and Stefanie Rinderle-Ma. Vol. 458. Lecture Notes in Business Information Processing. Springer, 2022, pp. 207–223. DOI: 10.1007/978-3-031-16171-1_13. URL: https://doi.org/10.1007/978-3-031-16171-1%5C_13.

[174] Alex Smola and S. V. N. Vishwanathan. *An Introduction to Machine Learning*. Cambridge University Press, 2008. URL: https://alex.smola.org/drafts/thebook.pdf.

[175] Dominique Sommers, Natalia Sidorova, and Boudewijn F. van Dongen. "Assessing Process Mining Techniques: a Ground Truth Approach". In: *6th International Conference on Process Mining, ICPM 2024, Kgs. Lyngby, Denmark, October 14-18, 2024*. IEEE, 2024, pp. 41–48. DOI: 10.1109/ICPM63005.2024.10680619. URL: https://doi.org/10.1109/ICPM63005.2024.10680619.

[176] Alessandro Stefanini, Davide Aloini, Elisabetta Benevento, Riccardo Dulmin, and Valeria Mininno. "Performance analysis in emergency departments: a data-driven approach". In: *Measuring Business Excellence* 22.2 (2018), pp. 130–145. DOI: 10.1108/MBE-07-2017-0040.

[177] Suriadi Suriadi, Robert Andrews, Arthur H. M. ter Hofstede, and Moe Thandar Wynn. "Event log imperfection patterns for process mining: Towards a systematic approach to cleaning event logs". In: *Information Systems* 64 (2017), pp. 132–150. DOI: 10.1016/J.IS.2016.07.011. URL: https://doi.org/10.1016/j.is.2016.07.011.

[178] Oscar Tamburis and Christian Esposito. "Process mining as support to simulation modeling: A hospital-based case study". In: *Simulation Modelling Practice and Theory* 104 (2020), p. 102149. DOI: 10.1016/J.SIMPAT.2020.102149. URL: https://doi.org/10.1016/j.simpat.2020.102149.

[179] Irina Tentina, Felix Mannhardt, Francesca Zerbato, and Boudewijn F. van Dongen. "Can Users Trust Process Mining?" In: *Business Information Systems - 25th International Conference, BIS 2025, Poznań, Poland, June 25-27, 2025, Proceedings*. Ed. by Krzysztof Wecel. Vol. 554. Lecture Notes in Business Information Processing. Springer, 2025, pp. 137–151. DOI: 10.1007/978-3-031-94193-1_11. URL: https://doi.org/10.1007/978-3-031-94193-1%5C_11.

[180] Irina Tentina, Francesca Zerbato, Felix Mannhardt, and Boudewijn F. van Dongen. "Why Do Users Struggle to Get Insights from Process Mining?" In: *7th International Conference on Process Mining, ICPM 2025, Montevideo, Uruguay, October 20-24, 2025*. IEEE, 2025, pp. 1–8. DOI: 10.

1109/ICPM66919.2025.11220617. URL: https://doi.org/10.1109/ICPM66919.2025.11220617.

[181] Wil M. P. van der Aalst, Arya Adriansyah, and Boudewijn van Dongen. "Replaying history on process models for conformance checking and performance analysis". In: *WIREs Data Mining and Knowledge Discovery* (2012). DOI: doi.org/10.1002/widm.1045.

[182] Lien Vanbrabant, Niels Martin, Katrien Ramaekers, and Kris Braekers. "Quality of input data in emergency department simulations: Framework and assessment techniques". In: *Simulation Modelling Practice and Theory* 91 (2019), pp. 83–101. DOI: 10.1016/J.SIMPAT.2018.12.002. URL: https://doi.org/10.1016/j.simpat.2018.12.002.

[183] Ágnes Vathy-Fogarassy, István Vassányi, and István Kósa. "Multi-level process mining methodology for exploring disease-specific care processes". In: *Journal of Biomedical Informatics* 125 (2022), p. 103979. DOI: 10.1016/J.JBI.2021.103979. URL: https://doi.org/10.1016/j.jbi.2021.103979.

[184] Borja Vázquez-Barreiros, Manuel Mucientes, and Manuel Lama. "ProDiGen: Mining complete, precise and minimal structure process models with a genetic algorithm". In: *Information Sciences* 294 (2015), pp. 315–333. DOI: 10.1016/J.INS.2014.09.057. URL: https://doi.org/10.1016/j.ins.2014.09.057.

[185] Sahil Verma, Varich Boonsanong, Minh Hoang, Keegan Hines, John Dickerson, and Chirag Shah. "Counterfactual Explanations and Algorithmic Recourses for Machine Learning: A Review". In: *ACM Computing Surveys* 56.12 (2024), 312:1–312:42. DOI: 10.1145/3677119. URL: https://doi.org/10.1145/3677119.

[186] Cédric Villani. "Optimal transport: old and new". In: *Grundlehren der mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences]* 338 (2009). DOI: 10.1007/978-3-540-71050-9.

[187] Francesco Vinci, Davide Aloini, Elisabetta Benevento, Alessandro Stefanini, Francesca Zen, and Massimiliano de Leoni. "Improving organizational processes in healthcare through simulation-driven resource allocation: Methodology and real-world case study". In: *Artificial Intelligence in Medicine* 176 (2026), p. 103391. ISSN: 0933-3657. DOI: 10.1016/j.artmed.2026.103391. URL: https://www.sciencedirect.com/science/article/pii/S0933365726000436.

[188] Francesco Vinci and Massimiliano de Leoni. "Balancing fitness and precision in process model repair: framework formalization, assessment, and benefits for simulation". In: *Process Science* 2.3 (2025). DOI: 10.1007/s44311-025-00007-7. URL: https://doi.org/10.1007/s44311-025-00007-7.

[189] Francesco Vinci and Massimiliano de Leoni. "Repairing Process Models Through Simulation and Explainable AI". In: *Business Process Management - 22nd International Conference, BPM 2024, Krakow, Poland, September 1-6, 2024, Proceedings*. Ed. by Andrea Marrella, Manuel Resinas, Mieke Jans, and Michael Rosemann. Vol. 14940. Lecture Notes in Computer Science. Springer, 2024, pp. 129–145. DOI: 10.1007/978-3-031-70396-6_8. URL: https://doi.org/10.1007/978-3-031-70396-6_8.

[190] Francesco Vinci, Gyunam Park, Wil M. P. van der Aalst, and Massimiliano de Leoni. "Online Discovery of Simulation Models for Evolving Business Processes". In: *Business Process Management - 23rd International Conference, BPM 2025, Seville, Spain, August 31 - September 5, 2025, Proceedings*. Ed. by Arik Senderovich, Cristina Cabanillas, Irene Vanderfeesten, and Hajo A. Reijers. Vol. 16044. Lecture Notes in Computer Science. Springer, 2025, pp. 451–468. DOI: 10.1007/978-3-032-02867-9_27. URL: https://doi.org/10.1007/978-3-032-02867-9_27.

[191] Francesco Vinci, Gyunam Park, Wil M. P. van der Aalst, and Massimiliano de Leoni. "ProSiT: A Tool for Interactive and Transparent Process Simulations". In: *Proceedings of the International Conference on Service Oriented Computing (ICSOC): Demonstrations and Resources*. Lecture Notes in Computer Science. Springer, 2026.

[192] Francesco Vinci, Gyunam Park, Wil M. P. van der Aalst, and Massimiliano de Leoni. "Reliable and Configurable Process Simulations via Probabilistic White-Box Models". In: *Service-Oriented Computing*. Ed. by Marco Aiello, Shuiguang Deng, Juan-Manuel Murillo, Ilche Georgievski, Boualem Benatallah, and Zhongjie Wang. Singapore: Springer Nature Singapore, 2026, pp. 337–352. DOI: 10.1007/978-981-95-5015-9_24. URL: https://doi.org/10.1007/978-981-95-5015-9%5C_24.

[193] Sandra Wachter, Brent D. Mittelstadt, and Chris Russell. "Counterfactual Explanations without Opening the Black Box: Automated Decisions and the GDPR". In: *Harvard Journal of Law & Technology* 31 (2017), pp. 842–887. arXiv: 1711.00399. URL: http://arxiv.org/abs/1711.00399.

[194] Michael Westergaard and Lars Michael Kristensen. "The Access/CPN Framework: A Tool for Interacting with the CPN Tools Simulator". In: *Applications and Theory of Petri Nets, 30th International Conference, PETRI NETS 2009, Paris, France, June 22-26, 2009. Proceedings*. Ed. by Giuliana Franceschinis and Karsten Wolf. Vol. 5606. Lecture Notes in Computer Science. Springer, 2009, pp. 313–322. DOI: 10.1007/978-3-642-02424-5_19. URL: https://doi.org/10.1007/978-3-642-02424-5%5C_19.

[195] Darrell Whitley. "A genetic algorithm tutorial". In: *Statistics and Computing* 4.2 (1994), pp. 65–85. DOI: 10.1007/BF00175354. URL: https://doi.org/10.1007/BF00175354.

[196] World Bank. *Societal Aging*. Accessed: April 4, 2025. 2024. URL: https://www.worldbank.org/en/topic/pensions/brief/societal-aging.

[197] World Health Organization. *Ageing and health*. Accessed: April 4, 2025. 2024. URL: https://www.who.int/news-room/fact-sheets/detail/ageing-and-health.

[198] Moe Thandar Wynn, Marlon Dumas, Colin J. Fidge, Arthur H. M. ter Hofstede, and Wil M. P. van der Aalst. "Business Process Simulation for Operational Decision Support". In: *Business Process Management Workshops, BPM 2007 International Workshops, BPI, BPD, CBP, ProHealth, RefMod, semantics4ws, Brisbane, Australia, September 24, 2007, Revised Selected Papers*. Ed. by Arthur H. M. ter Hofstede, Boualem Benatallah, and Hye-Young Paik. Vol. 4928. Lecture Notes in Computer Science. Springer, 2007, pp. 66–77. DOI: 10.1007/978-3-540-78238-4_8. URL: https://doi.org/10.1007/978-3-540-78238-4_8.

[199] H. Yang, B.F. van Dongen, A.H.M. ter Hofstede, M.T. Wynn, and J. Wang. *Estimating completeness of event logs*. Tech. rep. BPM Center, 2012.

[200] S.J. van Zelst. "Process mining with streaming data". English. Proefschrift. Phd Thesis 1 (Research TU/e / Graduation TU/e). Mathematics and Computer Science, Mar. 2019. ISBN: 978-90-386-4699-2. URL: https://research.tue.nl/en/publications/process-mining-with-streaming-data/.

[201] Sebastiaan J. van Zelst, Boudewijn F. van Dongen, and Wil M. P. van der Aalst. "Event stream-based process discovery using abstract representations". In: *Knowledge and Information Systems* 54.2 (2018), pp. 407–435. DOI: 10.1007/S10115-017-1060-2. URL: https://doi.org/10.1007/s10115-017-1060-2.

[202] Francesca Zerbato, Pnina Soffer, and Barbara Weber. "Process Mining Practices: Evidence from Interviews". In: *Business Process Management - 20th International Conference, BPM 2022, Münster, Germany, September 11-16, 2022, Proceedings*. Ed. by Claudio Di Ciccio, Remco M. Dijkman, Adela del-Río-Ortega, and Stefanie Rinderle-Ma. Vol. 13420. Lecture Notes in Computer Science. Springer, 2022, pp. 268–285. DOI: 10.1007/978-3-031-16103-2_19. URL: https://doi.org/10.1007/978-3-031-16103-2%5C_19.

[203] Francesca Zerbato, Lisa Zimmermann, Katerina Vrotsou, and Barbara Weber. "From analysis to findings: How do process mining analysts discover results?" In: *Information Systems* 135 (2026), p. 102596. DOI: 10.1016/J.IS.2025.102596. URL: https://doi.org/10.1016/j.is.2025.102596.

[204] Chunchun Zhao and Sartaj Sahni. "Linear space string correction algorithm using the Damerau-Levenshtein distance". In: *BMC Bioinformatics* 21-S.1 (2020), p. 4. DOI: 10.1186/S12859-019-3184-8. URL: https://doi.org/10.1186/s12859-019-3184-8.

[205] Lisa Zimmermann, Francesca Zerbato, and Barbara Weber. "What makes life for process mining analysts difficult? A reflection of challenges". In: *Software and Systems Modeling* 23.6 (2024), pp. 1345–1373. DOI: 10.1007/S10270-023-01134-0. URL: https://doi.org/10.1007/s10270-023-01134-0.

Acknowledgments

The Ph.D. journey has undoubtedly been the most challenging experience I have ever undertaken. Yet it has also been the most rewarding and transformative of my life so far. It has been the realization of the dream to contribute, even in a small way, to the advancement of scientific knowledge. None of this would have been possible without the support and guidance of the many people who have walked this path with me.

First and foremost, I want to express my deepest gratitude to my supervisor, Massimiliano, whose mentorship has shaped the researcher I am today. From the very beginning, he instilled in me not only the rigor and methodology of research but also the passion and curiosity that drive it. He introduced me to the scientific community and allowed me to seize exceptional opportunities. I carry his teachings with immense pride and gratitude.

A special thank you goes to Wil and Gyunam for welcoming me during my visits at Fraunhofer FIT and RWTH Aachen University. I am grateful to Gyunam for his constant feedback and for the richness of our collaboration, which guided my work throughout my year in Aachen. I thank Wil for his sharp and valuable insights and for the inspiring way he approaches science. The time spent in Aachen and the people I met in that stimulating research environment will stay with me forever.

I am deeply honored that this thesis was reviewed by Prof. Marlon Dumas and Prof. Andrea Marrella. Receiving such thorough and positive evaluations from them is a privilege that I do not take for granted, and I sincerely thank them for the time and care they devoted to this work.

This thesis is also the result of fruitful and enriching collaborations, and I am grateful to all my co-authors. It has been a privilege to work with each of you. I also extend my sincere thanks to all the researchers and colleagues who,

through conference discussions and beyond, contributed directly or indirectly to the ideas presented in this work.

I thank my colleagues at the University of Padua and in particular the Process Science research group, for the moments we shared along the way. A special thanks goes to Alessandro, who has been a steady presence throughout this journey.

To all my dearest friends, from my beloved Messina to Padova and beyond, thank you for your support and warmth throughout this journey. Thank you for being there, in big and small ways, both in bright moments and in difficult ones. I extend my thanks to my relatives, especially my grandmother Carmela and my aunt Maria.

My final and most heartfelt thanks go to my family. To my mother Mariacristina, my father Giuseppe, and my brothers Davide and Mattia. You supported me unconditionally, celebrated every small victory with me, and stood by me in every moment of doubt. You have always believed in me, and it is to you that I owe the deepest part of who I am.

Padova, March 2026 *Francesco Vinci*